Lecturas recomendadas

Cybersecurity Engineering Labs

Lab 4 — Network Discovery and Port Scanning

Manuel Nevado Blanco

November 3, 2026

Abstract

Este documento recopila, para estudio, las *lecturas recomendadas* (*preparatory readings*) del laboratorio 4 (Network Discovery and Port Scanning), tal como las lista la guía del estudiante: *"Read chapters 1, 3, 4, 5, 7 and 8 of the nmap book."* Se incluye el contenido íntegro de esos capítulos de *Nmap Network Scanning* (Gordon "Fyodor" Lyon), tomado de la edición online oficial (https://nmap.org/book/), con la URL de origen de cada sección. La numeración de capítulos coincide con la del libro.

Contents

1 Getting Started with Nmap **3**

Introduction . . . 3

Nmap Overview and Demonstration . . . 3

The Phases of an Nmap Scan . . . 12

Legal Issues . . . 14

The History and Future of Nmap . . . 20

3 Host Discovery ("Ping Scanning") **27**

Introduction . . . 27

Specifying Target Hosts and Networks . . . 27

Finding an Organization's IP Addresses . . . 29

DNS Resolution . . . 35

Host Discovery Controls . . . 36

Host Discovery Techniques . . . 39

Putting It All Together: Host Discovery Strategies . . . 44

Host Discovery Code Algorithms . . . 51

4 Port Scanning Overview **53**

Introduction to Port Scanning . . . 53

A Quick Port Scanning Tutorial . . . 59

Command-line Flags . . . 61

IPv6 Scanning (-6) . . . 67

SOLUTION: Scan a Large Network for a Certain Open TCP Port . . . 68

5 Port Scanning Techniques and Algorithms **73**

Introduction . . . 73

TCP SYN (Stealth) Scan (-sS) . . . 74

TCP Connect Scan (-sT) . . . 77

UDP Scan (-sU) . . . 78

TCP FIN, NULL, and Xmas Scans (-sF, -sN, -sX) . . . 83

Custom Scan Types with –scanflags . . . 86

TCP ACK Scan (-sA) . . . 88

TCP Window Scan (-sW) . . . 89

TCP Maimon Scan (-sM) . . . 91

TCP Idle Scan (-sI) . . . 91

IP Protocol Scan (-sO) . . . 99
TCP FTP Bounce Scan (-b) . . . 101
Scan Code and Algorithms . . . 102

7 Service and Application Version Detection 107

Introduction . . . 107
Usage and Examples . . . 109
Technique Described . . . 110
Technique Demonstrated . . . 114
Post-processors . . . 116
nmap-service-probes File Format . . . 119
Community Contributions . . . 125
SOLUTION: Find All Servers Running an Insecure or Nonstandard Application Version 128
SOLUTION: Hack Version Detection to Suit Custom Needs, such as Open Proxy Detection . . . 130

8 Remote OS Detection 132

Introduction . . . 132
Usage and Examples . . . 133
TCP/IP Fingerprinting Methods Supported by Nmap . . . 137
IPv6 fingerprinting . . . 148
Fingerprinting Methods Avoided by Nmap . . . 152
Understanding an Nmap Fingerprint . . . 154
Device Types . . . 162
OS Matching Algorithms . . . 164
Dealing with Misidentified and Unidentified Hosts . . . 166
SOLUTION: Detect Rogue Wireless Access Points on an Enterprise Network . . . 169

Chapter 1

Getting Started with Nmap

Fuente original: https://nmap.org/book/intro.html

Introduction

Nmap ("Network Mapper") is a free and open source utility for network exploration and security auditing. Many systems and network administrators also find it useful for tasks such as network inventory, managing service upgrade schedules, and monitoring host or service uptime. Nmap uses raw IP packets in novel ways to determine what hosts are available on the network, what services (application name and version) those hosts are offering, what operating systems (and OS versions) they are running, what type of packet filters/firewalls are in use, and dozens of other characteristics. It was designed to rapidly scan large networks, but works fine against single hosts. Nmap runs on all major computer operating systems, and both console and graphical versions are available.

This chapter uses fictional stories to provide a broad overview of Nmap and how it is typically used. An important legal section helps users avoid (or at least be aware of) controversial usage that could lead to ISP account cancellation or even civil and criminal charges. It also discusses the risks of crashing remote machines as well as miscellaneous issues such as the open source Nmap license (based on the GNU GPL), and copyright.

Nmap Overview and Demonstration

Fuente original: https://nmap.org/book/nmap-overview-and-demos.html

Sometimes the best way to understand something is to see it in action. This section includes examples of Nmap used in (mostly) fictional yet typical circumstances. Nmap newbies should not expect to understand everything at once. This is simply a broad overview of features that are described in depth in later chapters. The "solutions" included throughout this book demonstrate many other common Nmap tasks for security auditors and network administrators.

Avatar Online

Felix dutifully arrives at work on December 15th, although he does not expect many structured tasks. The small San Francisco penetration-testing firm he works for has been quiet lately due to impending holidays. Felix spends business hours pursuing his latest hobby of building powerful Wi-Fi antennas for wireless assessments and war driving exploration. Nevertheless, Felix is hoping for more business. Hacking has been his hobby and fascination since a childhood spent learning everything he could about networking, security, Unix, and phone systems. Occasionally his curiosity took him too far, and Felix was almost swept up in the 1990 Operation Sundevil prosecutions. Fortunately Felix emerged from adolescence without a criminal record, while retaining his expert knowledge of security weaknesses. As a professional, he is able to perform the same types of network intrusions as before, but with the added benefit of contractual immunity from prosecution and even a paycheck! Rather than keeping his creative exploits secret, he can

brag about them to client management when presenting his reports. So Felix was not disappointed when his boss interrupted his antenna soldering to announce that the sales department closed a pen-testing deal with the Avatar Online gaming company.

Avatar Online (AO) is a small company working to create the next generation of massive multiplayer online role-playing games (MMORPGs). Their product, inspired by the Metaverse envisioned in Neil Stevenson's *Snow Crash*, is fascinating but still highly confidential. After witnessing the high-profile leak of Valve Software's upcoming game source code, AO quickly hired the security consultants. Felix's task is to initiate an external (from outside the firewall) vulnerability assessment while his partners work on physical security, source code auditing, social engineering, and so forth. Felix is permitted to exploit any vulnerabilities found.

The first step in a vulnerability assessment is network discovery. This reconnaissance stage determines what IP address ranges the target is using, what hosts are available, what services those hosts are offering, general network topology details, and what firewall/filtering policies are in effect.

Determining the IP ranges to scan would normally be an elaborate process involving ARIN (or another geographical registry) lookups, DNS queries and zone transfer attempts, various web sleuthing techniques, and more. But in this case, Avatar Online explicitly specified what networks they want tested: the corporate network on 6.209.24.0/24 and their production/DMZ systems residing on 6.207.0.0/22. Felix checks the IP whois records anyway and confirms that these IP ranges are allocated to AO[1]. Felix subconsciously decodes the CIDR notation[2] and recognizes this as 1,280 IP addresses. No problem.

Being the careful type, Felix first starts out with what is known as an Nmap list scan (`-sL` option). This feature simply enumerates every IP address in the given target netblock(s) and does a reverse-DNS lookup (unless `-n` was specified) on each. One reason to do this first is stealth. The names of the hosts can hint at potential vulnerabilities and allow for a better understanding of the target network, all without raising alarm bells[3]. Felix is doing this for another reason—to double-check that the IP ranges are correct. The systems administrator who provided the IPs might have made a mistake, and scanning the wrong company would be a disaster. The contract signed with Avatar Online may act as a get-out-of-jail-free card for penetrating their networks, but will not help if Felix accidentally compromises another company's server! The command he uses and an excerpt of the results are shown in Example 1.1.

Example 1.1. Nmap list scan against Avatar Online IP addresses

```
felix> nmap -sL 6.209.24.0/24 6.207.0.0/22

Starting Nmap ( https://nmap.org )
Nmap scan report for 6.209.24.0
Nmap scan report for fw.corp.avataronline.com (6.209.24.1)
Nmap scan report for dev2.corp.avataronline.com (6.209.24.2)
Nmap scan report for 6.209.24.3
Nmap scan report for 6.209.24.4
...
Nmap scan report for dhcp-21.corp.avataronline.com (6.209.24.21)
Nmap scan report for dhcp-22.corp.avataronline.com (6.209.24.22)
Nmap scan report for dhcp-23.corp.avataronline.com (6.209.24.23)
...
Nmap scan report for 6.207.0.0
Nmap scan report for gw.avataronline.com (6.207.0.1)
Nmap scan report for ns1.avataronline.com (6.207.0.2)
Nmap scan report for ns2.avataronline.com (6.207.0.3)
Nmap scan report for ftp.avataronline.com (6.207.0.4)
Nmap scan report for 6.207.0.5
Nmap scan report for 6.207.0.6
Nmap scan report for www.avataronline.com (6.207.0.7)
Nmap scan report for 6.207.0.8
...
Nmap scan report for cluster-c120.avataronline.com (6.207.2.120)
```

```
Nmap scan report for cluster-c121.avataronline.com (6.207.2.121)
Nmap scan report for cluster-c122.avataronline.com (6.207.2.122)
...
Nmap scan report for 6.207.3.255
Nmap done: 1280 IP addresses (0 hosts up) scanned in 331.49 seconds
felix>
```

Reading over the results, Felix finds that all of the machines with reverse-DNS entries resolve to Avatar Online. No other businesses seem to share the IP space. Moreover, these results give Felix a rough idea of how many machines are in use and a good idea of what many are used for. He is now ready to get a bit more intrusive and try a port scan. He uses Nmap features that try to determine the application and version number of each service listening on the network. He also requests that Nmap try to guess the remote operating system via a series of low-level TCP/IP probes known as OS fingerprinting. This sort of scan is not at all stealthy, but that does not concern Felix. He is interested in whether the administrators of AO even notice these blatant scans. After a bit of consideration, Felix settles on the following command:

```
nmap -sS -p- -PE -PP -PS80,443 -PA3389 -PU40125 -A -T4 -oA avatartcpscan-%D
6.209.24.0/24 6.207.0.0/22
```

These options are described in later chapters, but here is a quick summary of them.

`-sS`

Enables the efficient TCP port scanning technique known as SYN scan. Felix would have added a U at the end if he also wanted to do a UDP scan, but he is saving that for later. SYN scan is the default scan type, but stating it explicitly does not hurt.

`-p-`

Requests that Nmap scan every port from 1-65535. This is more comprehensive than the default, which is to scan only the 1,000 ports which we've found to be most commonly accessible in large-scale Internet testing. This option format is simply a short cut for `-p1-65535`. Felix could have specified `-p0-65535` if he wanted to scan the rather illegitimate port zero as well. The `-p` option has a very flexible syntax, even allowing the specification of a differing set of UDP and TCP ports.

`-PE -PP -PS80,443 -PA3389 -PU40125`

These are all host discovery techniques (ping types) used in combination to determine which targets on a network are really available and avoid wasting a lot of time scanning IP addresses that are not in use. This particular incantation sends ICMP echo request and timestamp request packets; TCP SYN packet to ports 80 and 443; TCP ACK packets to port 3389; and a UDP packet to port 40,125. If Nmap receives a response from a target host to any of these probes, it considers the host to be up and available for scanning. This is the most effective six-probe combination that we've found in large-scale empirical testing for host discovery against targets over the Internet. It is more extensive than the Nmap default, which is `-PE -PS443 -PA80 -PP`. In a pen-testing situation, you often want to scan every host even if they do not seem to be up. After all, they could just be heavily filtered in such a way that the probes you selected are ignored but some other obscure port may be available. To scan every IP whether it shows an available host or not, specify the `-Pn` option instead of all of the above. Felix starts such a scan in the background, though it may take many hours to complete.

`-A`

This shortcut option turns on Advanced and Aggressive features such as OS and service detection. At the time of this writing it is equivalent to `-sV -sC -O --traceroute` (version

detection, Nmap Scripting Engine with the default set of scripts, remote OS detection, and traceroute). More features may be added to `-A` later.

`-T4`

Adjusts timing to the `aggressive` level (#4 of 5). This is the same as specifying `-T aggressive`, but is easier to type and spell. In general, the `-T4` option is recommended if the connection between you and the target networks is reasonably fast and reliable.

`-oA avatartcpscan-%D`

Outputs results in every format (normal, XML, grepable) to files named `avatartcpscan-`*`<date>`*`.`*`<extension>` where the extensions are .nmap, .xml, and .gnmap respectively. The *`<date>`* gives the month, day, and year in a format like `073010`. All of the output formats include the start date and time, but Felix likes to note the date explicitly in the filename. Normal output and errors are still sent to stdout[4] as well.

`6.209.24.0/24 6.207.0.0/22`

These are the Avatar Online netblocks discussed above. They are given in CIDR notation, but Nmap allows them to be specified in many other formats. For example, `6.209.24.0/24` could instead be specified as `6.209.24.0-255`.

Since such a comprehensive scan against more than a thousand IP addresses could take a while, Felix simply starts it executing and resumes work on his Yagi antenna. An couple hours later he notices that it has finished and takes a peek at the results. Example 1.2 shows one of the machines discovered.

Example 1.2. Nmap results against an AO firewall

```
Nmap scan report for fw.corp.avataronline.com (6.209.24.1)
(The 65530 ports scanned but not shown below are in state: filtered)
PORT     STATE  SERVICE    VERSION
22/tcp   open   ssh        OpenSSH 3.7.1p2 (protocol 1.99)
| ssh-hostkey: 1024 7c:14:2f:92:ca:61:90:a4:11:3c:47:82:d5:8e:a9:6b (DSA)
|_2048 41:cf7d:839d:7f66:0ae1:8331:7fd4:5a97:5a (RSA)
|_sshv1: Server supports SSHv1
53/tcp   open   domain     ISC BIND 9.2.1
110/tcp  open   pop3       Courier pop3d
113/tcp  closed auth
143/tcp  open   imap       Courier Imap 1.6.X - 1.7.X
3128/tcp open   http-proxy Squid webproxy 2.2.STABLE5
Device type: general purpose
Running: Linux 2.4.X|2.5.X
OS details: Linux Kernel 2.4.0 - 2.5.20
Uptime 3.134 days
```

To the trained eye, this conveys substantial information about AO's security posture. Felix first notes the reverse DNS name—this machine is apparently meant to be a firewall for their corporate network. The next line is important, but all too often ignored. It states that the vast majority of the ports on this machine are in the `filtered` state. This means that Nmap is unable to reach the port because it is blocked by firewall rules. The fact that all ports except for a few chosen ones are in this state is a sign of security competence. Deny-by-default is a security mantra for good reasons—it means that even if someone accidentally left SunRPC (port 111) open on this machine, the firewall rules would prevent attackers from communicating with it.

Felix then looks at every port line in turn. The first port is Secure Shell (OpenSSH). Version 3.7.1p2 is common, as many administrators upgraded to this version due to potentially exploitable buffer management bugs affecting previous versions. Nmap also notes that the `sshv1` NSE script reports that the less secure SSHv1 protocol is supported. A truly paranoid sysadmin would only allow SSH connections from certain trusted IP addresses, but one can argue for

open access in case the administrator needs emergency access while far from home. Security often involves trade-offs, and this one may be justifiable. Felix makes a note to try the Ncrack brute force network authentication cracker and especially his private timing-based SSH user enumeration tool against the server.

Felix also notes port 53. It is running ISC BIND, which has a long history of remotely exploitable security holes. Visit the BIND security page for further details. BIND 9.2.1 even has a potentially exploitable buffer overflow, although the default build is not vulnerable. Felix checks and finds that this server is not vulnerable to the libbind issue, but that is beside the point. This server almost certainly should not be running an externally-accessible nameserver. A firewall should only run the bare essentials to minimize the risk of a disastrous compromise. Besides, this server is not authoritative for any domains—the real nameservers are on the production network. An administrator probably only meant for clients within the firewall to contact this nameserver, but did not bother locking it down to only the internal interface. Felix will later try to gather important information from this unnecessary server using zone transfer requests and intrusive queries. He may attempt cache poisoning as well. By spoofing the IP of `windowsupdate.microsoft.com` or another important download server, Felix may be able to trick unsuspecting internal client users into running a trojan-horse program that provides him with full network access behind the firewall.

The next two open ports are 110 (POP3) and 143 (IMAP). Note that 113 (auth) between them is `closed` instead of `open`. POP3 and IMAP are mail retrieval services which, like BIND, have no legitimate place on this server. They are also a security risk in that they generally transfer the mail and (even worse) authentication credentials unencrypted. Users should probably VPN in and check their mail from an internal server. These ports could also be wrapped in SSL encryption. Nmap would have then listed the services as `ssl/pop3` and `ssl/imap`. Felix will try his user enumeration and password guessing attacks on these services, which will probably be much more effective than against SSH.

The final open port is a Squid proxy. This is another service that may have been intended for internal client use and should not be accessible from the outside (and particularly not on the firewall). Felix's initially positive opinion of the AO security administrators drops further. Felix will test whether he can abuse this proxy to connect to other sites on the Internet. Spammers and malicious hackers often use proxies in this way to hide their tracks. Even more critical, Felix will try to proxy his way into the internal network. This common attack is how Adrian Lamo broke into the New York Times internal network in 2002. Lamo was caught after he called reporters to brag about his exploits against the NY Times and other companies in detail.

The following lines disclose that this is a Linux box, which is valuable information when attempting exploitation. The low three-day uptime was detected during OS fingerprinting by sending several probes for the TCP timestamp option value and extrapolating the line back to zero.

Felix then examines the Nmap output for another machine, as shown in Example 1.3.

Example 1.3. Another interesting AO machine

```
Nmap scan report for dhcp-23.corp.avataronline.com (6.209.24.23)
(The 65526 ports scanned but not shown below are in state: closed)
PORT      STATE    SERVICE       VERSION
135/tcp   filtered msrpc
136/tcp   filtered profile
137/tcp   filtered netbios-ns
138/tcp   filtered netbios-dgm
139/tcp   filtered netbios-ssn
445/tcp   open     microsoft-ds  Microsoft Windows XP microsoft-ds
1002/tcp  open     windows-icfw?
1025/tcp  open     msrpc         Microsoft Windows msrpc
16552/tcp open     unknown
Device type: general purpose
```

```
Running: Microsoft Windows NT/2K/XP
OS details: Microsoft Windows XP Professional RC1+ through final release

Host script results:
|_nbstat: NetBIOS name: TRACYD, NetBIOS user: <unknown>,  NetBIOS MAC: 00:20:35:00:29:a2:7f (IBM)
|_smbv2-enabled: Server doesn't support SMBv2 protocol
| smb-os-discovery:
|   OS: Windows XP (Windows 2000 LAN Manager)
|_  Name: WORKGROUP\JOHND
```

Felix smiles when he spies this Windows XP box on the Network. Thanks to a spate of MS RPC vulnerabilities, those machines are trivial to compromise if the OS patches aren't up-to-date. The second line shows that the default state is `closed`, meaning the firewall does not have the same deny-by-default policy for this machine as for itself. Instead they tried to specifically block the Windows ports they consider dangerous on 135-139. This filter is woefully inadequate, as MS exports MS RPC functionality on many other ports in Windows XP. TCP ports 445 and 1025 are two examples from this scan. While Nmap failed to recognize 16552, Felix has seen this pattern enough to know that it is probably the MS Messenger Service. If AO had been using deny-by-default filtering, port 16552 would not be accessible in the first place. Looking through the results page, Felix sees several other Windows machines on this DHCP network. Felix cannot wait to try his favorite DCOM RPC exploit against them. It was written by HD Moore and is available at `http://www.metasploit.com/tools/dcom.c`. If that fails, there are a couple newer MS RPC vulnerabilities he will try.

Felix continues poring over the results for vulnerabilities he can leverage to compromise the network. On the production network, he sees that `gw.avataronline.com` is a Cisco router that also acts as a rudimentary firewall for the systems. They fall into the trap of only blocking privileged ports (those under 1024), which leaves a bunch of vulnerable SunRPC and other services accessible on that network. The machines with names like `clust-*` each have dozens of ports open that Nmap does not recognize. They are probably custom daemons running the AO game engine. `www.avataronline.com` is a Linux box with an open Apache server on the HTTP and HTTPS ports. Unfortunately, it is linked with an exploitable version of the OpenSSL library. Oops! Before the sun sets, Felix has gained privileged access to hosts on both the corporate and production networks.

As Felix has demonstrated, Nmap is frequently used by security auditors and network administrators to help locate vulnerabilities on client/corporate networks. Subsequent chapters describe the techniques used by Felix, as well as many other Nmap features, in much greater detail.

Saving the Human Race

Figure 1.1. Trinity begins her assault



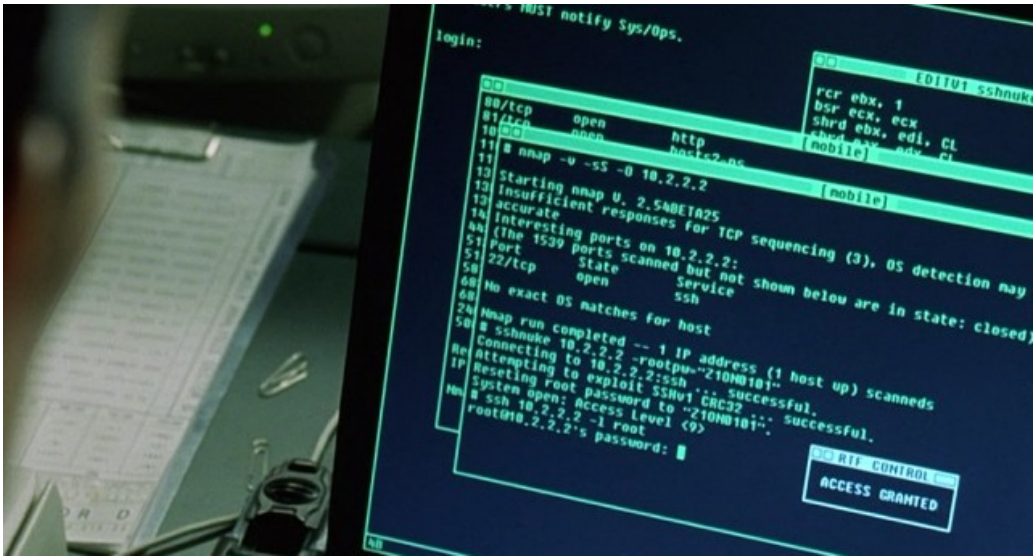
Trinity is in quite a pickle! Having discovered that the world we take for granted is really a virtual "Matrix" run by machine overlords, Trinity decides to fight back and free the human race from this mental slavery. Making matters worse, her underground colony of freed humans (Zion) is under attack by 250,000 powerful alien sentinels. Her only hope involves deactivating the emergency power system for 27 city blocks in less than five minutes. The previous team died trying. In life's bleakest moments when all hope seems to be lost, what should you turn to? Nmap, of course! But not quite yet.

She first must defeat the perimeter security, which on many networks involves firewalls and intrusion detection systems (IDS). She is well aware of advanced techniques for circumventing these devices (covered later in this book). Unfortunately, the emergency power system administrators knew better than to connect such a critical system to the Internet, even indirectly. No amount of source routing or IP ID spoofed scanning will help Trinity overcome this "air gap" security. Thinking fast, she devises a clever plan that involves jumping her motorcycle off the rooftop of a nearby building, landing on the power station guard post, and then beating up all of the security guards. This advanced technique is not covered in any physical security manual, but proves highly effective. It demonstrates how clever hackers research and devise their own attacks, rather than always utilizing the script-kiddie approach of canned exploits.

Trinity fights her way to the computer room and sits down at a terminal. She quickly determines that the network is using the private 10.0.0.0/8 network address space. A ping to the network address generates responses from dozens of machines. An Nmap ping scan would have provided a more comprehensive list of available machines, but using the broadcast technique saved precious seconds. Then she whips out Nmap[5]. The terminal has version 2.54BETA25 installed. This version is ancient (2001) and less efficient than newer releases, but Trinity had no time to install a better version from the future. This job will not take long anyway. She runs the command `nmap -v -sS -O 10.2.1.3`. This executes a TCP SYN scan and OS detection against 10.2.1.3 and provides verbose output. The host appears to be a security disaster—AIX 3.2 with well over a dozen ports open. Unfortunately, this is not the machine she needs to compromise. So she runs the same command against 10.2.2.2. This time the target OS is unrecognized (she should have upgraded Nmap!) and only has port 22 open. This is the Secure Shell encrypted administration service. As any sexy latex-clad hacker goddess knows, many SSH servers from around that time (2001) have an exploitable vulnerability in the CRC32 compensation attack detector. Trinity

whips out an all-assembly-code exploit and utilizes it to change the root password of the target box to `Z1ON0101`. Trinity uses much more secure passwords under normal circumstances. She logs in as root and issues a command to disable the emergency backup power system for 27 city blocks, finishing just in time! Here is a shot of the action:

Figure 1.2. Trinity scans the Matrix



A terminal-view video showing the whole hack is available at `https://nmap.org/movies.html`, unless the MPAA finds out and sends sentinels or lawyers after us.

MadHat in Wonderland

This story differs from the previous ones in that it is actually true. Written by frequent Nmap user and contributor Lee "MadHat" Heath, it describes how he enhanced and customized Nmap for daily use in a large enterprise. In true open source spirit, he has released these valuable scripts on his Web site. IP addresses have been changed to protect the corporate identity. The remainder of this section is in his own words.

After spending the past couple of decades learning computers and working my way up from tech support through sysadmin and into my dream job of Information Security Officer for a major Internet company, I found myself with a problem. I was handed the sole responsibility of security monitoring for our entire IP space. This was almost 50,000 hosts worldwide when I started several years ago, and it has doubled since then.

Scanning all of these machines for potential vulnerabilities as part of monthly or quarterly assessments would be tough enough, but management wanted it done daily. Attackers will not wait a week or month to exploit a newly exposed vulnerability, so I can't wait that long to find and patch it either.

Looking around for tools, I quickly chose Nmap as my port scanner. It is widely considered to be the best scanner, and I had already been using it for years to troubleshoot networks and test security. Next I needed software to aggregate Nmap output and print differences between runs. I considered several existing tools, including HD Moore's Nlog. Unfortunately none of these monitored changes in the way I desired. I had to know whenever a router or firewall access control list was misconfigured or a host was publicly sharing inappropriate content. I also worried about the scalability of these other solutions, so I decided to tackle the problem myself.

The first issue to come up was speed. Our networks are located worldwide, yet I was provided with only a single U.S.-based host to do the scanning. In many cases, firewalls between the sites slowed the scanning down significantly. Scanning all 100,000 hosts took over 30 hours, which is unacceptable for a daily scan. So I wrote a script called nmap-wrapper which runs dozens of Nmap processes in parallel, reducing the scan time to fifteen hours, even including OS detection.

The next problem was dealing with so much data. A SQL database seemed like the best approach for scalability and data-mining reasons, but I had to abandon that idea due to time pressures. A future version may add this support. Instead, I used a flat file to store the results of each class C address range for each day. The most powerful and extensible way to parse and store this information was the Nmap XML format, but I chose the "grepable" (`-oG` option) format because it is so easy to parse from simple scripts. Per-host timestamps are also stored for reporting purposes. These have proven quite helpful when administrators try to blame machine or service crashes on the scanner. They cannot credibly claim a service crash at 7:12AM when I have proof that the scan ran at 9:45AM.

The scan produces copious data, with no convenient access method. The standard Unix **`diff`** tool is not smart enough to report only the changes I care about, so I wrote a Perl script named nmap-diff to provide daily change reports. A typical output report is shown in Example 1.4.

Example 1.4. nmap-diff typical output

```
> nmap-diff.pl -c3
  5 IPs showed changes

  10.12.4.8 (ftp-box.foocompany.biz)
         21/tcp   open   ftp
         80/tcp   open   http
        443/tcp   open   https
       1027/tcp   open   IIS
     + 1029/tcp   open   ms-lsa
      38292/tcp   open   landesk-cba
  OS: Microsoft Windows Millennium Edition (Me)
      Windows 2000 Professional or Advanced Server
      or Windows XP

  10.16.234.3 (media.foocompany.biz)
         80/tcp   open   http
     +  554/tcp   open   rtsp
     + 7070/tcp   open   realserver

  192.168.10.186 (testbox.foocompany.biz)
    + 8082/tcp   open   blackice-alerts
  OS: Linux Kernel 2.4.0 - 2.5.20

  172.24.12.58 (mtafoocompany.biz)
    +   25/tcp   open   smtp
  OS: FreeBSD 4.3 - 4.4PRERELEASE

  172.23.76.22 (media2.foocorp.biz)
         80/tcp   open   http
       1027/tcp   open   IIS
     + 1040/tcp   open   netsaint
       1755/tcp   open   wms
       3372/tcp   open   msdtc
       6666/tcp   open   irc-serv
       7007/tcp   open   afs3-bos
  OS: Microsoft Windows Millennium Edition (Me)
      Windows 2000 Professional or Advanced Server
      or Windows XP
```

Management and staff were impressed when I demonstrated this new system at an internal company security symposium. But instead of allowing me to rest on my laurels, they began asking for new features. They wanted counts of mail and web servers, growth estimates, and more. This data was all available from the scans, but was difficult to access. So I created

yet another Perl script, nmap-report, which made querying the data much easier. It takes specifications such as open ports or operating systems and finds all the systems that matched on a given day.

One problem with this approach to security monitoring is that employees do not always place services on their IANA-registered official ports. For example, they might put a web server on port 22 (SSH) or vice versa. Just as I was debating how to address this problem, Nmap came out with an advanced service and version detection system (see Chapter 7, *Service and Application Version Detection*). nmap-report now has a rescan feature that uses version scanning to report the true services rather than guessing based on port number. I hope to further integrate version detection in future versions. Example 1.5 shows nmap-report listing FTP servers.

Example 1.5. nmap-report execution

```
> nmap-report -p21 -rV
[...]
172.21.199.76 (ftp1.foocorp.biz)
   21/tcp   open   ssl|ftp Serv-U ftpd 4.0

192.168.12.56 (ftp2.foocorp.biz)
   21/tcp   open   ftp     NcFTPd

 192.168.13.130 (dropbox.foocorp.biz)
   21/tcp   open   ftp     WU-FTPD 6.00LS
```

While being far from perfect, these scripts have proven themselves quite valuable at monitoring large networks for security-impacting changes. Since Nmap itself is open source, it only seemed fair to release my scripts to the public as well. I have made them freely available at http://www.unspecific.com/nmap.

---

[1] These IP addresses are actually registered to the United States Army Yuma Proving Ground, which is used to test a wide variety of artillery, missiles, tanks, and other deadly weapons. The moral is to be very careful about who you scan, lest you accidentally hit a highly sensitive network. The scan results in this story are not actually from this IP range.

[2] Classless Inter-Domain Routing (CIDR) notation is a method for describing networks with more granularity than class A (CIDR /8), class B (CIDR /16), or class C (CIDR /24) notation. See https://en.wikipedia.org/wiki/Classless_Inter-Domain_Routing.

[3] It is possible that the target nameserver will log a suspicious bunch of reverse-DNS queries from Felix's nameserver, but most organizations don't even keep such logs, much less analyze them.

[4] stdout is the "C" notation for representing the standard output mechanism for a system, such as to the Unix xterm or Windows command window in which Nmap was initiated.

[5] A sexy leather-clad attacker from the previous team actually started the session. It is unclear at what point she died and left the remaining tasks to Trinity.

The Phases of an Nmap Scan

Fuente original: https://nmap.org/book/nmap-phases.html

Now that we've seen some applications of Nmap, let's look at what happens when an Nmap scan runs. Scans proceed in phases, with each phase finishing before the next one begins. As you can see from the phase descriptions below, there is far more to Nmap than just port scanning.

Script pre-scanning. The Nmap Scripting Engine (NSE) uses a collection of special-purpose scripts to gain more information about remote systems. NSE is not executed unless you re-

quest it with options such as `--script` or `-sC`, and the pre-scanning phase only happens when scripts which need it are selected. This phase is for scripts which only have to be run once per Nmap execution rather than running separately against individual targets. Examples include `dhcp-discover` and `broadcast-dns-service-discovery`, which use broadcast queries to obtain information from common network services. NSE is fully documented in Chapter 9, and the different phases are covered in the section called "Script Types and Phases".

Target enumeration. In this phase, Nmap researches the host specifiers provided by the user, which may be a combination of host DNS names, IP addresses, CIDR network notations, and more. You can even use (`-iR`) to ask Nmap to choose your targets for you! Nmap resolves these specifiers into a list of IPv4 or IPv6 addresses for scanning. This phase cannot be skipped since it is essential for further scanning, but you can simplify the processing by passing just IP addresses so Nmap doesn't have to do forward resolution. If you pass the `-sL -n` options (list scan with no reverse-DNS resolution), Nmap will print out the targets and perform no further scanning. This phase is discussed in the section called "Specifying Target Hosts and Networks" and the section called "List Scan (`-sL`)".

Host discovery (ping scanning). Network scans usually begin by discovering which targets on the network are online and thus worth deeper investigation. This process is called *host discovery* or *ping scanning*. Nmap offers many host discovery techniques, ranging from quick ARP requests to elaborate combinations of TCP, ICMP, and other types of probes. This phase is run by default, though you can skip it (simply assume all target IPs are online) using the `-Pn` (no ping) option. To quit after host discovery, specify `-sn -n`. Host discovery is the subject of Chapter 3.

Reverse-DNS resolution. Once Nmap has determined which hosts to scan, it looks up the reverse-DNS names of all hosts found online by the ping scan. Sometimes a host's name provides clues to its function, and names make reports more readable than providing only IP numbers. This step may be skipped with the `-n` (no resolution) option, or expanded to cover all target IPs (even down ones) with `-R` (resolve all). Name resolution is covered in the section called "DNS Resolution".

Port scanning. This is Nmap's core operation. Probes are sent, and the responses (or non-responses) to those probes are used to classify remote ports into states such as `open`, `closed`, or `filtered`. That brief description doesn't begin to encompass Nmap's many scan types, configurability of scans, and algorithms for improving speed and accuracy. An overview of port scanning is in Chapter 4. Detailed information on algorithms and command-line options are in Chapter 5. Port scanning is performed by default, though you can skip it with the `-sn` option and still perform some of the later traceroute and partial Nmap Scripting Engine phases by specifying their particular command-line options (such as `--traceroute` and `--script`).

Version detection. If any ports are found to be open, Nmap may be able to determine what server software is running on the remote system. It does this by sending a variety of probes to the open ports and matching any responses against a database of thousands of more than 6,500 known service signatures. Version detection is enabled with the `-sV` option and is fully described in Chapter 7.

OS detection. If requested with the `-O` option, Nmap proceeds to OS detection. Different operating systems implement network standards in subtly different ways. By measuring these differences it is often possible to determine the operating system running on a remote host. Nmap matches responses to a standard set of probes against a database of more than a thousand known operating system responses. OS detection is covered in Chapter 8.

Traceroute. Nmap contains an optimized traceroute implementation, enabled by the `--traceroute` option. It can find the network routes to many hosts in parallel, using the best available probe packets as determined by Nmap's previous discovery phases. Traceroute usually

involves another round of reverse-DNS resolution for the intermediate hosts. More information is found in the section called "Host Discovery".

Script scanning. Most Nmap Scripting Engine (NSE) scripts run during this main script scanning phase, rather than the prescan and postscan phases. NSE is powered by the Lua programming language and a standard library designed for network information gathering. Scripts running during this phase generally run once for each target host and port number that they interact with. They commonly perform tasks such as detecting service vulnerabilities, malware discovery, collecting more information from databases and other network services, and advanced version detection. NSE is not executed unless you request it with options such as `--script` or `-sC`.

Output. Finally, Nmap collects all the information it has gathered and writes it to the screen or to a file. Nmap can write output in several formats. Its default, human-readable format (interactive format) is usually presented in this book. Nmap also offers an XML-based output format, among others. The ins and outs of output are the subject of Chapter 13.

As already discussed, Nmap offers many options for controlling which of these phases are run. For scans of large networks, each phase is repeated many times since Nmap deals with the hosts in smaller groups. It scans each group completely and outputs those results, then moves on to the next batch of hosts.

Script post-scanning. After Nmap has completed its scanning and normal output, scripts in this phase can process results and deliver final reports and statistics. See the section called "Script Types and Phases". Nmap does not yet include any scripts in this phase, so it only runs if the user includes and executes their own post-scanning scripts.

Legal Issues

Fuente original: https://nmap.org/book/legal-issues.html

When used properly, Nmap helps protect your network from invaders. But when used improperly, Nmap can (in rare cases) get you sued, fired, expelled, jailed, or banned by your ISP. Reduce your risk by reading this legal guide before launching Nmap.

Is Unauthorized Port Scanning a Crime?

The legal ramifications of scanning networks with Nmap are complex and so controversial that third-party organizations have even printed T-shirts and bumper stickers promulgating opinions on the matter[6], as shown in Figure 1.3. The topic also draws many passionate but often unproductive debates and flame wars. If you ever participate in such discussions, try to avoid the overused and ill-fitting analogies to knocking on someone's home door or testing whether his door and windows are locked.

Figure 1.3. Strong opinions on port scanning legality and morality





While I agree with the sentiment that port scanning should not be illegal, it is rarely wise to take legal advice from a T-shirt. Indeed, taking it from a software engineer and author is only slightly better. Speak to a competent lawyer within your jurisdiction for a better understanding of how the law applies to your particular situation. With that important disclaimer out of the way, I'll provide some general information that may prove helpful.

The best way to avoid controversy when using Nmap is to always secure written authorization from the target network representatives before initiating any scanning. There is still a chance that your ISP will give you trouble if they notice it (or if the target administrators accidentally send them an abuse report), but this is usually easy to resolve. When you are performing a penetration test, this authorization should be in the Statement of Work. When testing your own company, make certain that this activity clearly falls within your job description. Security consultants should be familiar with the excellent Open Source Security Testing Methodology Manual (OSSTMM), which provides best practices for these situations.

While civil and (especially) criminal court cases are the nightmare scenario for Nmap users, these are very rare. After all, no United States federal laws explicitly criminalize port scanning. A much more frequent occurrence is that the target network will notice a scan and send a complaint to the network service provider where the scan initiated (your ISP). Most network administrators do not seem to care or notice the many scans bouncing off their networks daily, but a few complain. The scan source ISP may track down the user corresponding to the reported IP address and time, then chide the user or even kick him off the service. Port scanning without authorization is sometimes against the provider's acceptable use policy (AUP). For example, the AUP for the huge cable-modem ISP Comcast says:

> *Network probing or port scanning tools are only permitted when used in conjunction with a residential home network, or if explicitly authorized by the destination host and/or network. Unauthorized port scanning, for any reason, is strictly prohibited.*

Even if an ISP does not explicitly ban unauthorized port scanning, they might claim that some "anti-hacking" provision applies. Of course this does not make port scanning illegal. Many perfectly legal and (in the United States) constitutionally protected activities are banned by ISPs. For example, the AUP quoted above also prohibits users from transmitting, storing, or

posting "any information or material which a reasonable person could deem to be objectionable, offensive, indecent, pornographic, ... embarrassing, distressing, vulgar, hateful, racially or ethnically offensive, or otherwise inappropriate, regardless of whether this material or its dissemination is unlawful". In other words, some ISPs ban any behavior that could possibly offend or annoy someone[7]. Indiscriminate scanning of other people's networks does have that potential. If you decide to perform such controversial scanning anyway, never do it from work, school, or any other service provider that has substantial control over your well-being. Use a commercial broadband or wireless provider instead. Losing your DSL connection and having to change providers is a slight nuisance, but it is immeasurably preferable to being expelled or fired.

While legal cases involving port scanning (without follow-up hacking attacks) are rare, they do happen. One of the most notable cases involved a man named Scott Moulton who had an ongoing consulting contract to maintain the Cherokee County, Georgia emergency 911 system. In December 1999, he was tasked with setting up a router connecting the Canton, Georgia Police Department with the E911 Center. Concerned that this might jeopardize the E911 Center security, Scott initiated some preliminary port scanning of the networks involved. In the process he scanned a Cherokee County web server that was owned and maintained by a competing consulting firm named VC3. They noticed the scan and emailed Scott, who replied that he worked for the 911 Center and was testing security. VC3 then reported the activity to the police. Scott lost his E911 maintenance contract and was arrested for allegedly violating the Computer Fraud and Abuse Act of America Section 1030(a)(5)(B). This act applies against anyone who "intentionally accesses a protected computer without authorization, and as a result of such conduct, causes damage" (and meets other requirements). The damage claimed by VC3 involved time spent investigating the port scan and related activity. Scott sued VC3 for defamation, and VC3 countersued for violation of the Computer Fraud and Abuse Act as well as the Georgia Computer Systems Protection Act.

The civil case against Scott was dismissed before trial, implying a complete lack of merit. The ruling made many Nmap users smile:

> *"Court holds that plaintiff's act of conducting an unauthorized port scan and throughput test of defendant's servers does not constitute a violation of either the Georgia Computer Systems Protection Act or the Computer Fraud and Abuse Act."—Civ. Act. No. 1:00-CV-434-TWT (N.D. Ga. November 6, 2000)*

This was an exciting victory in the civil case, but Scott still had the criminal charges pending. Fortunately he kept his spirits high, sending the following note to the *nmap-hackers* mailing list:

> *I am proud that I could be of some benefit to the computer society in defending and protecting the rights of specialists in the computer field, however it is EXTREMELY costly to support such an effort, of which I am not happy about. But I will continue to fight and prove that there is nothing illegal about port scanning especially when I was just doing my job.*

Eventually, the criminal court came to the same conclusion and all charges were dropped. While Scott was vindicated in the end, he suffered six-figure legal bills and endured stressful years battling through the court system. The silver lining is that after spending so much time educating his lawyers about the technical issues involved, Scott started a successful forensics services company.

While the Moulton case sets a good example (if not legal precedent), different courts or situations could still lead to worse outcomes. Remember that many states have their own computer abuse laws, some of which can arguably make even pinging a remote machine without authorization illegal[8].

Laws in other nations obviously differ as well. For example, A 17-year-old youth was convicted in Finland of attempted computer intrusion for simply port scanning a bank. He was fined to cover the target's investigation expenses. The Moulton ruling might have differed if the VC3 machine had actually crashed and they were able to justify the $5,000 damage figure required by the act.

At the other extreme, an Israeli judge acquitted Avi Mizrahi in early 2004 for vulnerability scanning the Mossad secret service. Judge Abraham Tennenbaum even commended Avi in his ruling:

> *In a way, Internet surfers who check the vulnerabilities of Web sites are acting in the public good. If their intentions are not malicious and they do not cause any damage, they should even be praised.*

In 2007 and 2008, broad new cybercrime laws took effect in Germany and England. These laws are meant to ban the distribution, use, and even possession of "hacking tools". For example, the UK amendment to the Computer Misuse Act makes it illegal to "supply or offer to supply [a program], believing that it is likely to be used to commit, or to assist in the commission of [a Computer Misuse Act violation]". These laws have already led some security tool authors to close shop or move their projects to other countries. The problem is that most security tools can be used by both ethical professionals (white-hats) to defend their networks and black-hats to attack. These dangerous laws are based on the tool author or user's intent, which is subjective and hard to divine. Nmap was designed to help secure the Internet, but I'd hate to be arrested and forced to defend my intentions to a judge and jury, especially in a foreign country like Germany where I don't even speak the language. These laws are unlikely to affect tools as widespread and popular as Nmap, but they have had a chilling effect on smaller tools and those which are more commonly abused by computer criminals (such as exploitation frameworks).

Regardless of the legal status of port scanning, ISP accounts will continue to be terminated if many complaints are generated. The best way to avoid ISP abuse reports or civil/criminal charges is to avoid annoying the target network administrators in the first place. Here are some practical suggestions:

- Ensure that you have permission to scan. Probably at least 90% of network scanning is non-controversial. You are rarely badgered for scanning your own machine or the networks you administer. The controversy comes when scanning other networks. There are many reasons (good and bad) for doing this sort of network exploration. Perhaps you are scanning the other systems in your dorm or department to look for publicly shared files (FTP, SMB, WWW, etc.). Or maybe you are just trying to find the IP address of a certain printer. You might have scanned your favorite web site to see if they are offering any other services, or because you were curious what OS they run. Perhaps you are just trying to test connectivity, or maybe you wanted to do a quick security sanity check before handing off your credit card details to that e-commerce company. You might be conducting Internet research. Or are you performing initial reconnaissance in preparation for a break-in attempt? The remote administrators rarely know your true intentions, and do sometimes get suspicious. The best approach is to get permission first. I have seen a few people with non-administrative roles land in hot water after deciding to "prove" network insecurity by launching an intrusive scan of the entire company or campus. Administrators

tend to be more cooperative when asked in advance than when woken up at 3:00 AM by an IDS alarm claiming they are under massive attack. So whenever possible, obtain written authorization before scanning a network. Adrian Lamo would probably have avoided jail if he had asked the New York Times to test their security rather than telling reporters about the flaws afterward. Unfortunately they might have said no. Be prepared for this answer.

- Target your scan as tightly as possible. Any machine connected to the Internet is scanned regularly enough that most administrators ignore such Internet background noise. But scanning enough networks or executing very noisy/intrusive scans increases the probability of generating complaints. So if you are only looking for web servers, specify `-p80` rather than scanning all 65,536 TCP ports on each machine. If you are only trying to find available hosts, do an Nmap ping scan rather than full port scan. Do not scan a CIDR /16 (65K hosts) when a /24 netblock suffices. The random scan mode now takes an argument specifying the number of hosts, rather than running forever. So consider `-iR 1000` rather than `-iR 10000` if the former is sufficient. Use the default timing (or even `-T polite`) rather than `-T insane`. Avoid noisy and relatively intrusive scans such as version detection (`-sV`) or NSE (`--script`). Similarly, a SYN scan (`-sS`) is quieter than a connect scan (`-sT`) while providing the same information and often being faster.
- As noted previously, do not do anything controversial from your work or school connections. Even though your intentions may be good, you have too much to lose if someone in power (e.g. boss, dean) decides you are a malicious cracker. Do you really want to explain your actions to someone who may not even know what port scanning means? Spend $40 a month for a shell, cell data, or residential broadband account. Not only are the repercussions less severe if you offend someone from such an account, but target network administrators are less likely to even bother complaining to mass-market providers. Also read the relevant AUP and choose a provider accordingly. If your provider (like Comcast discussed above) bans any unauthorized port scanning and posting of "offensive" material, do not be surprised if you are kicked off for this activity. In general, the more you pay to a service provider the more accommodating they are. A T1 provider is highly unlikely to yank your connection without notice because someone reported being port scanned. A dialup or residential DSL/cable provider very well might. This can happen even when the scan was forged by someone else.
- Nmap offers many options for stealthy scans, including source-IP spoofing, decoy scanning, and the more recent idle scan technique. These are discussed in the IDS evasion chapter. But remember that there is always a trade-off. You are harder to find if you launch scans from an open WAP far from your house, with 17 decoys, while doing subsequent probes through a chain of nine open proxies. But if anyone does track you down, they will be mighty suspicious of your intentions.
- Always have a legitimate reason for performing scans. An offended administrator might write to you first (or your ISP might forward his complaint to you) expecting some sort of justification for the activity. In the Scott Moulton case discussed above, VC3 first emailed Scott to ask what was going on. If they had been satisfied with his answer, matters might have stopped there rather than escalating into civil and criminal litigation. When I scan large portions of the Internet for research purposes, I use a reverse-DNS name that describes the project and run a web server on that IP address with detailed information and opt-out instructions.

Also remember that ancillary and subsequent actions are often used as evidence of intent. A port scan by itself does not always signify an attack. A port scan followed closely by an IIS exploit, however, broadcasts the intention loud and clear. This is important because decisions

to prosecute (or fire, expel, complain, etc.) are often based on the whole event and not just one component (such as a port scan).

One dramatic case involved a Canadian man named Walter Nowakowski, who was apparently the first person to be charged in Canada with theft of communications (Canadian Criminal Code Section S.342.1) for accessing the Internet through someone's unsecured Wi-Fi network. Thousands of Canadian "war drivers" do this every day, so why was he singled out? Because of ancillary actions and intent. He was allegedly caught driving the wrong way on a one-way street, naked from the waist down, with laptop in hand, while downloading child pornography through the aforementioned unsecured wireless access point. The police apparently considered his activity egregious enough that they brainstormed for relevant charges and tacked on theft of communications to the many child pornography-related charges.

Similarly, charges involving port scanning are usually reserved for the most egregious cases. Even when paranoid administrators notify the police that they have been scanned, prosecution (or any further action) is exceedingly rare. The fact that a 911 emergency service was involved is likely what motivated prosecutors in the Moulton case. Your author scanned millions of Internet hosts while writing this book and received fewer than ten complaints.

To summarize this whole section, the question of whether port scanning is legal does not have a simple answer. I cannot unequivocally say "port scanning is never a crime", as much as I would like to. Laws differ dramatically between jurisdictions, and cases hinge on their particular details. Even when facts are nearly identical, different judges and prosecutors do not always interpret them the same way. I can only urge caution and reiterate the suggestions above.

For testing purposes, you have permission to scan the host `scanme.nmap.org`. You may have noticed it used in several examples already. Note that this permission only includes scanning with Nmap and not testing exploits or denial of service attacks. To conserve bandwidth, please do not initiate more than a dozen scans against that host per day. If this free scanning target service is abused, it will be taken down and Nmap will report `Failed to resolve given hostname/IP: scanme.nmap.org`.

Can Port Scanning Crash the Target Computer/Networks?

Nmap does not have any features designed to crash target networks. It usually tries to tread lightly. For example, Nmap detects dropped packets and slows down when they occur in order to avoid overloading the network. Nmap also does not send any corrupt packets. The IP, TCP, UDP, and ICMP headers are always appropriate, though the destination host is not necessarily expecting the packets. For these reasons, no application, host, or network component should ever crash based on an Nmap scan. If they do, that is a bug in the system which should be repaired by the vendor.

Reports of systems being crashed by Nmap are rare, but they do happen. Many of these systems were probably unstable in the first place and Nmap either pushed them over the top or they crashed at the same time as an Nmap scan by pure coincidence. In other cases, poorly written applications, TCP/IP stacks, and even operating systems have been demonstrated to crash reproducibly given a certain Nmap command. These are usually older legacy devices, as newer equipment is rarely released with these problems. Smart companies use Nmap and many other common network tools to test devices prior to shipment. Those who omit such pre-release testing often find out about the problem in early beta tests when a box is first deployed on the Internet. It rarely takes long for a given IP to be scanned as part of Internet background noise. Keeping systems and devices up-to-date with the latest vendor patches and firmware should reduce the susceptibility of your machines to these problems, while also improving the security and usability of your network.

In many cases, finding that a machine crashes from a certain scan is valuable information. After all, attackers can do anything Nmap can do by using Nmap itself or their own custom scripts. Devices should not crash from being scanned and if they do, vendors should be pressured to provide a patch. In some usage scenarios, detecting fragile machines by crashing them is undesirable. In those cases you may want to perform very light scanning to reduce the risk of adverse effects. Here are a few suggestions:

- Use SYN scan (`-sS`) instead of connect scan (`-sT`). User-mode applications such as web servers can rarely even detect the former because it is all handled in kernel space and thus the services have no excuse to crash.
- Version scanning (`-sV`) and some of our NSE scripts (`-sC` or `--script`) risk crashing poorly written applications. Similarly, some buggy operating systems have been reported to crash when OS fingerprinted (`-O`). Omit these options for particularly sensitive environments or where you do not need the results.
- Using `-T2` or slower (`-T1`, `-T0`) timing modes can reduce the chances that a port scan will harm a system, though they slow your scan dramatically. Older Linux boxes had an identd daemon that would block services temporarily if they were accessed too frequently. This could happen in a port scan, as well as during legitimate high-load situations. Slower timing might help here. These slow timing modes should only be used as a last resort because they can slow scans by an order of magnitude or more.
- Limit the number of ports and machines scanned to the fewest that are required. Every machine scanned has a minuscule chance of crashing, and so cutting the number of machines down improves your odds. Reducing the number of ports scanned reduces the risks to end hosts as well as network devices. Many NAT/firewall devices keep a state entry for every port probe. Most of them expire old entries when the table fills up, but occasional (pathetic) implementations crash instead. Reducing the ports and hosts scanned reduces the number of state entries and thus might help those fragile and defective devices stay up.

Nmap Copyright

While Nmap is open source, it still has a copyright license that must be respected. As free software, Nmap also carries no warranty. These issues are covered in much greater detail in the section called "Legal Notices". Companies wishing to bundle and use Nmap within proprietary software and appliances are especially encouraged to read this section so they don't inadvertently violate the Nmap license. Fortunately the Nmap Project sells commercial redistribution licenses for companies which need one.

---

[6] These are from the now-defunct AmericanSushi.Com.

[7] The Comcast AUP was improved after this was first published. The latest version is available at http://www.comcast.net/terms/use/

[8] An excellent paper on this topic by lawyer Ethan Preston is available at http://grove.ufl.edu/~techlaw/vol6/issue1/preston.html. He has also written an excellent paper relating to the legal risks of publishing security information and exploits at http://www.mcandl.com/computer-security.html.

The History and Future of Nmap

Fuente original: https://nmap.org/book/history-future.html

Many ancient and well loved security tools, such as Netcat, tcpdump, and John the Ripper, haven't changed much over the years. Others, including Wireshark, Metasploit, Cain and Abel, and Snort, have been under constant development since the day they were released. Nmap is in that second category. It was released as a simple Linux-only port scanner in 1997. Over the next 16+ years it sprouted a myriad of valuable features, including OS detection, version detection, the Nmap Scripting Engine, a Windows port, a graphical user interface, Ncat, Nping, Ndiff, and more. We plan to continue this rapid development pace in the future as well!

The History of Nmap

This section provides a timeline of the most important milestones over 16 years of Nmap history. For all significant Nmap changes (thousands of them), read the Nmap Changelog. Old releases of Nmap can be found at `https://nmap.org/dist/`, and ancient versions at `https://nmap.org/dist-old/`.

- **September 1, 1997** — Nmap is first released in *Phrack* magazine Issue 51, Article 11. It doesn't have a version number because new releases aren't planned. Nmap is about 2,000 lines long, and compilation is as simple as `gcc -O6 -o nmap nmap.c -lm`.
- **September 5, 1997** — Due to popular demand, a slightly improved version of the *Phrack* code is released, calling itself version 1.25. The gzipped tarball is 28KB. Version 1.26 (48KB) is released 19 days later.
- **January 11, 1998** — Insecure.Org is registered and Nmap moves there from its previous home at the DataHaven Project ISP.
- **March 14, 1998** — Renaud Deraison writes to inform me that he is writing a security scanner, and asks if he can use some Nmap source code. Of course I say yes. Nine days later he sends me a pre-release version of Nessus, noting that it "is designed for sysadmins, not 3l33t H4ck3rZ".
- **December 12, 1998** — Nmap version 2.00 is publicly released, introducing Nmap OS detection for the first time after several months of private development. An paper describing the techniques was released in *Phrack* 54, Article 9. By this point Nmap is broken up into many files, consists of about 8,000 lines of code, is kept in a private CVS revision control system, and the tarball size is 275KB. The *nmap-hackers* mailing list is started, and later grows to more than 80,000 members.
- **April 11, 1999** — Nmap 2.11BETA1 is released. This is the first version to contain a graphical user interface as an alternative to the traditional command-line usage. The bundled Unix-only NmapFE GUI was originally written by Zach Smith. Some people like it, but most prefer command-line execution.
- **April 28, 2000** — Nmap 2.50 is released. By this point the tarball has grown to 461KB. This release includes timing modes such as `-T aggressive`, direct SunRPC scanning, and the Window and ACK scan methods.
- **May 28, 2000** — Gerhard Rieger sends a message to the *nmap-dev* list describing a new "protocol scan" he has developed for Nmap, and he even includes a patch. This is so cool that I release Nmap 2.54BETA1 with his patch less than 12 hours later.
- **December 7, 2000** — Nmap 2.54BETA16 is released as the first official version to compile and run on Microsoft Windows. The Windows porting work was done by Ryan Permeh and Andy Lutomirski.

- **July 9, 2001** — The Nmap IP ID idle scan is introduced with Nmap 2.54BETA26. A paper describing the technique is released concurrently. This extremely cool (though not always practical) scan technique is described in the section called "TCP Idle Scan (`-sI`)".
- **July 25, 2002** — I quit my job at Netscape/AOL and start my dream job working on Nmap full time.
- **July 31, 2002** — Nmap 3.00 is released. The tarball is 922K. This release includes Mac OS X support, XML output, and uptime detection.
- **August 28, 2002** — Nmap is converted from C to C++ and IPv6 supported is added as part of the Nmap 3.10ALPHA1 release.
- **May 15, 2003** — Nmap is featured in the movie *The Matrix Reloaded*, where Trinity uses it (followed by a real SSH exploit) to hack a power station and save the world. This leads to more publicity for Nmap than it had ever seen before. Hollywood takes notice, and Nmap becomes a standard prop for hacking scenes in movies. Nmap is later seen in *The Bourne Ultimatum*, *Die Hard 4*, *The Girl with the Dragon Tattoo*, *Dredd*, and many others. Details and screen shots from all these movies are available at `https://nmap.org/movies/`.
- **July 21, 2003** — I finish a first implementation of Nmap service/version detection (Chapter 7, *Service and Application Version Detection*) and release it to a couple dozen top Nmap developers and users as Nmap 3.40PVT1. That is followed up by 16 more private releases over the next couple months as we improve the system and add signatures.
- **September 16, 2003** — Nmap service detection is publicly released as part of Nmap 3.45.
- **February 20, 2004** — Nmap 3.50 is released. The tarball is now 1,571KB. SCO Corporation is banned from redistributing Nmap because they refuse to comply with the GPL. They have to rebuild their Caldera release ISOs to remove Nmap. This release includes the packet tracing and UDP ping options, as well as an OS classification system.
- **August 31, 2004** — The core Nmap port scanning engine is rewritten for Nmap 3.70. The new engine, named `ultra_scan` features dramatically improved algorithms and parallelization support to improve both accuracy and speed. The differences are particularly dramatic for hosts behind strict firewalls.
- **June 25, 2005** — Google sponsors 10 college and graduate students to work on Nmap full time for the summer as part of Google's Summer of Code initiative. Projects include a modern reimplementation of Netcat named Ncat (Chris Gibson), a second generation OS detection system (Zhao Lei), a new cross-platform GUI which later becomes Zenmap (Adriano Monteiro Marques), and many other cool projects described at `https://seclists.org/nmap-hackers/2005/8`.
- **September 8, 2005** — Nmap gains raw ethernet frame sending support with the release of version 3.90. This allows for ARP scanning (see the section called "ARP Scan (`-PR`)") and MAC address spoofing as well as evading the raw IP packet ban introduced by Microsoft in Windows XP SP2.
- **January 31, 2006** — Nmap 4.00 is released. The tarball is now 2,388KB. This release includes runtime interaction to provide on-demand completion estimates, a Windows executable installer, NmapFE updates to support GTK2, and much more.
- **May 24, 2006** — Google sponsors 10 more Nmap summer developers as part of their SoC program. Zhao and Adriano return as part of 2006 SoC to further develop their respective projects. Diman Todorov is sponsored to help develop the Nmap Scripting

Engine (Chapter 9, *Nmap Scripting Engine*). These and seven other talented students and their projects are described at https://seclists.org/nmap-hackers/2006/9.

- **June 24, 2006** — After two years of development and testing, the 2nd generation OS detection system is integrated into Nmap 4.20ALPHA1. This new system is based on everything we've learned and the new ideas we've conceived since the first generation system debuted eight years earlier. The new system, described in Chapter 8, *Remote OS Detection*, proves much more accurate and granular than the first generation, although it takes us 2.5 years to reach the size of the old DB.
- **December 10, 2006** — The Nmap Scripting Engine is released as part of Nmap 4.21ALPHA1. NSE allows users to write (and share) simple scripts to automate a wide variety of networking tasks. This initial version includes 23 scripts for simple tasks such as detecting the insecure SSHv1 and SSLv2 protocols, obtaining service owner information from identd, and testing whether DNS servers support recursive queries.
- **December 20, 2006** — Nmap's Subversion source code repository opens to the public. Until this time, only a handful of developers had access to the private source repository. Everyone else had to wait for releases. Now everyone can follow Nmap development day to day. A new `nmap-svn` mailing list provides real-time change notification by email. For details, see the section called "Obtaining Nmap from the Subversion (SVN) Repository".
- **May 28, 2007** — Google sponsors six summer Nmap developers as part of their SoC program. Among the sponsored students was David Fifield, who continued long after the summer ended and became Nmap's co-maintainer. He even helped greatly in producing this book! The Nmap students and their projects are listed at https://seclists.org/nmap-hackers/2007/3.
- **July 8, 2007** — The Zenmap graphical front end (back then it was called Umit) is improved and integrated into the Nmap 4.22SOC1 release for testing. The venerable NmapFE GUI is removed. Zenmap is covered in Chapter 12, *Zenmap GUI Users' Guide*.
- **December 13, 2007** — Nmap 4.50 is released to celebrate Nmap's 10th anniversary!
- **June 1, 2008** — Nmap 4.65 is released and includes, for the first time, an executable Mac OS X installer. The Nmap source tarball is now four megabytes. This release includes 41 NSE scripts, 1,307 OS fingerprints, and 4,706 version detection signatures.
- **August 18, 2008** — The Nmap project completes its fourth Summer of Code, with our highest success percentage ever (six out of seven sponsored students succeeded). They greatly improved Zenmap, the Nmap Scripting Engine, OS detection, and Ncat, as described at https://seclists.org/nmap-dev/2008/q4/193.
- **September 8, 2008** — Nmap 4.75 is released with almost 100 significant improvements over 4.68. These include the Zenmap network topology viewer, which draws an interactive diagram of discovered network devices. After 11 years of distributing Nmap ("the Network Mapper"), it can now actually draw network maps. This version also includes scan aggregation, allowing several scans to be combined into one unified view. Both of these features are described in Chapter 12, *Zenmap GUI Users' Guide*). This release also includes port-frequency data from my Worldscan project, which I presented at Black Hat and Defcon the previous month. This allows Nmap to focus on the ports which have been empirically shown to be the most popular.
- **October 4, 2008** — The Nmap Documentation Portal is launched at https://nmap.org/nsedoc/. This allows users to brows all of Nmap's NSE script and library documentation in one place. The documentation is generated from specially-formed comments in the source code (see the section called "Writing Script Documentation (NSEDoc)").

- **January 1, 2009** — The first version of this book, *Nmap Network Scanning*, is released. The answer to whether anyone besides me would be interested in a 450+ page book on port scanning turns out to be a resounding yes! More than 10,000 copies are sold in the first 18 months. It is also released by other publishers in the German, Korean, and Brazilian Portuguese languages.
- **January 23, 2009** — Added Ncat, a feature-packed networking utility which reads and writes data across a network from the command line. It is a reimplementation of the classic Netcat tool which adds support for modern networking features such as SSL, connection redirection, and proxying. See Chapter 17, *Ncat Reference Guide*.
- **January 23, 2009** — Added the Ndiff utility which compares the results of two Nmap scans, making it easy to scan networks on a regular basis and create a report (XML or text) showing any changes. See Chapter 16, *Ndiff Reference Guide*.
- **March 30, 2009** — A special Nmap release (4.85BETA5) is produced to remotely detect the Conficker worm which has infected millions of machines on the Internet. Demand is so high that Nmap is booted from Dreamhost's "unlimited bandwidth" web hosting plan for using too much bandwidth. The University of California, San Diego steps up and provides a fast mirror server. The next version of the Conficker worm blocks infected users from reaching Insecure.Org and Nmap.Org. We take that as a compliment to Nmap's effectiveness.
- **June 12, 2009** — Added SCTP port scanning and host discovery support to Nmap. SCTP is a layer 4 protocol used mostly for telephony related applications.
- **July 16, 2009** — Released Nmap 5.00. The tarball is more than 27MB and contains 2,003 OS fingerprints, 5,512 version detection signatures, and 59 NSE scripts.
- **August 24, 2009** — The Nmap project completes its fifth Summer of Code. For the first time, every student passed! They created Nping as well as the network authentication cracker Ncrack and made dramatic improvements to the Nmap Scripting Engine, Ncat, and Zenmap.
- **March 29, 2010** — Added Nping, a handy tool for network packet generation, response analysis and response time measurement (see Chapter 18, *Nping Reference Guide*).
- **July 28, 2010** — David Fifield and I present on Mastering the Nmap Scripting Engine at the Black Hat and Defcon conferences, including live demos (video: `https://nmap.org/presentations/`).
- **August 17, 2010** — Scanning the top million web sites for fun leads to a spinoff art project as we release Icons of the Web, a mosaic of web site icons scaled by their popularity. It is written up in the New York Times, The Telegraph, Slashdot, Gizmodo, Engadget, and many others. The Newseum in Washington D.C. exhibits a giant version and it was even featured in the Guinness World Records book.
- **January 28, 2011** — Released Nmap 5.50. We added support for the ancient Gopher protocol for just fun, but serious improvements included tripling the NSE scripts to 177 and adding a new Zenmap script selection interface. We also added a novel echo mode feature to Nping which can ease network analysis and debugging (see the section called "Echo Mode"). Performance was a high priority too, with certain scan times reduced from hours to minutes and memory consumption reduced 90% in our benchmark scan.
- **June 8, 2011** — We made a commermorative World IPv6 day release with numerous new IPv6 features, plus we ensured that all of our websites were IPv6-enabled. While Nmap has supported basic IPv6 scanning since 2002, we added advanced host discovery and raw

port scanning. We also soon added a new machine language based IPv6 OS detection system (see the section called "IPv6 fingerprinting").

- **November 4, 2011** — After surveying more than 3,000 Nmap users for their favorite security tools, we releaunched our SecTools.org web site with the new data. Our previous sites were just static HTML files, but this new release includes interactive rating and review features to better share information. 49 of the the top 125 tools profiled on the site are new to the list. Of course the Nmap family of tools are excluded from the site due to potential conflicts of interest.
- **May 21, 2012** — Nmap 6 was released! The tarball is more than 54MB and contains 3,572 OS fingerprints, 8,165 version detection signatures, and 348 NSE scripts.

The Future of Nmap

While it is easy to catalogue the history of Nmap, the future is uncertain. Nmap didn't start off with any grand development plan, and most of the milestones in the preceding timeline were not planned more than a year or two in advance. Instead of trying to predict the shape of the Internet and networking way out in the future, I closely study where it is now and decide what will be most useful for Nmap in the near term. So I have no idea where Nmap will be 10 years from now, though I expect it to be as popular and vibrant as ever. The Nmap community is large enough to guide Nmap wherever it needs to go. Nmap has faced curve balls before, such as the sudden removal of raw packet support in Windows XP SP2, dramatic changes in network filtering practices and technology, and the slow emergence of IPv6. Each of those required significant changes to Nmap, and we will have to do the same to embrace or at least cope with future networking changes.

While the 10-year plan is up in the air, we do have plans and guiding priorities for the next several years:

- **Nmap Scripting Engine** — NSE has exploded in capabilities and popularity. As of mid-2010, it contains 131 scripts—68% higher than the year before. NSE scripts are also growing more powerful thanks to the ever-increasing library and infrastructure feature support. I would like to see NSE grow to 500+ scripts in the next few years. Nmap will soon support `prescan` and `postscan` scripts which are not tied to an individual host or open port like the existing `hostrule` and `portrule` scripts are.
- **Scanning web sites** — Nmap will also grow in its ability to handle web scanning. When Nmap was first developed, different services were often provided as separate daemons identified by the port number they listen on. Now, many new services simply run over HTTP and are identified by a URL path name rather than port number. Scanning for known URL paths is similar in many ways to port scanning (and to the SunRPC scanning which Nmap has also done for many years). The Nmap Scripting Engine already supports many web scanning features, but I'd like to see http brute force password cracking, better spidering support, an HTML/XML parser, better web application fingerprinting, and the ability to proxy certain requests through HTTP and SOCKS proxies. Nmap should be able to port scan through those proxies too.
- **Web infrastructure improvements** — Nmap is a cooperative project developed by many individuals. As the number of people involved increases, we also need to scale the tools and infrastructure resources to facilitate that cooperation. We're planning an Nmap wiki, an improved Subversion source control server, and possibly a web discussion forum. We are also moving the infrastructure to virtual machines for security isolation and to

better share administration tasks. The web sites desperately need a redesign as well, unless 1990's table-based layouts come back in favor.

- **Online scanning web service** — We're working on a service which allows users to scan their own machines and networks "from the cloud" to see what is exposed from an outside perspective. Repeat scans can be scheduled with changes sent in email alerts or browsed online.

In addition to those high-level priorities, we have some concrete tasks in mind:

- Internationalize Nmap to support many different languages rather than just English. The Zenmap front-end already does this, supporting five languages, as described in the section called "Zenmap in Your Language".
- Add a testing harness system to more easily catch regressions (bugs). Ncat has already implemented such a system and it has proven extremely useful.
- The OS and Version detection systems may be augmented to support the NIST Common Platform Enumeration (CPE) standard for better compatibility with other tools.
- Zenmap will soon offer an advanced script selection interface, making it easy to browse and choose scripts and script-arguments supported by the version of Nmap installed on a machine.
- We plan to produce, encourage the production of, and distribute more Nmap-related videos to introduce people to Nmap and teach them tricks and techniques for using it effectively.

For a rougher and lower-level list of what we're working on right now and planned for the near future, you can always read the Nmap todo list at `https://nmap.org/svn/todo/nmap.txt`.

Some of the coolest Nmap features in the past, such as OS and version detection, were developed in secret and given a surprise release. You can expect more of these in coming years because they are so much fun!

Chapter 3

Host Discovery ("Ping Scanning")

Fuente original: https://nmap.org/book/host-discovery.html

Introduction

One of the very first steps in any network reconnaissance mission is to reduce a (sometimes huge) set of IP ranges into a list of active or interesting hosts. Scanning every port of every single IP address is slow and usually unnecessary. Of course what makes a host interesting depends greatly on the scan purposes. Network administrators may only be interested in hosts running a certain service, while security auditors may care about every single device with an IP address. An administrator may be comfortable using just an ICMP ping to locate hosts on his internal network, while an external penetration tester may use a diverse set of dozens of probes in an attempt to evade firewall restrictions.

Because host discovery needs are so diverse, Nmap offers a wide variety of options for customizing the techniques used. Despite the name ping scan, this goes well beyond the simple ICMP echo request packets associated with the ubiquitous ping tool. Users can skip the ping step entirely with a list scan (`-sL`) or by disabling ping (`-Pn`), or engage the network with arbitrary combinations of multi-port TCP SYN/ACK, UDP, and ICMP probes. The goal of these probes is to solicit responses which demonstrate that an IP address is actually active (is being used by a host or network device). On many networks, only a small percentage of IP addresses are active at any given time. This is particularly common with private address space such as 10.0.0.0/8. That network has 16.8 million IPs, but I have seen it used by companies with fewer than a thousand machines. Host discovery can find those machines in a sparsely allocated sea of IP addresses.

This chapter first discusses how Nmap ping scanning works overall, with high-level control options. Then specific techniques are covered, including how they work and when each is most appropriate. Nmap offers many ping techniques because it often takes carefully crafted combinations to get through a series of firewalls and router filters leading to a target network. Effective overall ping scanning strategies are discussed, followed by a low-level look at the algorithms used.

Specifying Target Hosts and Networks

Fuente original: https://nmap.org/book/host-discovery-specify-targets.html

Everything on the Nmap command-line that isn't an option (or option argument) is treated as a target host specification. The simplest case is to specify a target IP address or hostname for scanning.

Sometimes you wish to scan a whole network of adjacent hosts. For this, Nmap supports CIDR-style addressing. You can append `/<numbits>` to an IPv4 address or hostname and Nmap will scan every IP address for which the first `<numbits>` are the same as for the reference IP or hostname given. For example, `192.168.10.0/24` would scan the 256 hosts between 192.168.10.0 (binary: `11000000 10101000 00001010 00000000`) and 192.168.10.255 (binary:

11000000 10101000 00001010 11111111), inclusive. 192.168.10.40/24 would scan exactly the same targets. Given that the host scanme.nmap.org is at the IP address 64.13.134.52, the specification scanme.nmap.org/16 would scan the 65,536 IP addresses between 64.13.0.0 and 64.13.255.255. The smallest allowed value is /0, which targets the whole Internet. The largest value is /32, which scans just the named host or IP address because all address bits are fixed.

CIDR notation is short but not always flexible enough. For example, you might want to scan 192.168.0.0/16 but skip any IPs ending with .0 or .255 because they may be used as subnet network and broadcast addresses. Nmap supports this through octet range addressing. Rather than specify a normal IP address, you can specify a comma-separated list of numbers or ranges for each octet. For example, 192.168.0-255.1-254 will skip all addresses in the range that end in .0 or .255, and 192.168.3-5,7.1 will scan the four addresses 192.168.3.1, 192.168.4.1, 192.168.5.1, and 192.168.7.1. Either side of a range may be omitted; the default values are 0 on the left and 255 on the right. Using - by itself is the same as 0-255, but remember to use 0- in the first octet so the target specification doesn't look like a command-line option. Ranges need not be limited to the final octets: the specifier 0-255.0-255.13.37 will perform an Internet-wide scan for all IP addresses ending in 13.37. This sort of broad sampling can be useful for Internet surveys and research.

IPv6 addresses can only be specified by their fully qualified IPv6 address or hostname. CIDR and octet ranges aren't supported for IPv6 because they are rarely useful.

Nmap accepts multiple host specifications on the command line, and they don't need to be the same type. The command **nmap scanme.nmap.org 192.168.0.0/8 10.0.0,1,3-7.-** does what you would expect.

Input From List (-iL)

Passing a huge list of hosts is often awkward on the command line, yet it is a common need. For example, your DHCP server might export a list of 10,000 current leases that you wish to scan. Or maybe you want to scan all IP addresses except for those ones to locate hosts using unauthorized static IP addresses. Simply generate the list of hosts to scan and pass that filename to Nmap as an argument to the -iL option. Entries can be in any of the formats accepted by Nmap on the command line (IP address, hostname, CIDR, IPv6, or octet ranges). Each entry must be separated by one or more spaces, tabs, or newlines. You can specify a hyphen (-) as the filename if you want Nmap to read hosts from standard input rather than an actual file.

Choose Targets at Random (-iR *<numtargets>*)

For Internet-wide surveys and other research, you may want to choose targets at random. This is done with the -iR option, which takes as an argument the number of IPs to generate. Nmap automatically skips certain undesirable IPs, such as those in private, multicast, or unallocated address ranges. The argument 0 can be specified for a never-ending scan. Keep in mind that some network administrators bristle at unauthorized scans of their networks. Carefully read the section called "Legal Issues" before using -iR.

If you find yourself really bored one rainy afternoon, try the command **nmap -sS -PS80 -iR 0 -p 80** to locate random web servers for browsing.

Excluding Targets (--exclude, --excludefile *<filename>*)

It is common to have machines which you don't want to scan under any circumstances. Machines can be so critical that you won't take any risk of an adverse reaction. You might be blamed

for a coincidental outage even if the Nmap scan had nothing to do with it. Or perhaps you have legacy hardware that is known to crash when scanned, but you haven't been able to fix or replace it yet. Or maybe certain IP ranges represent subsidiary companies, customers, or partners that you aren't authorized to scan. Consultants often don't want their own machine included in a scan of their client's networks. Whatever the reason, you can exclude hosts or entire networks with the `--exclude` option. Simply pass the option a comma-separated list of excluded targets and netblocks using the normal Nmap syntax. Alternatively, you can create a file of excluded hosts/networks and pass that to Nmap with the `--excludefile` option. The `--exclude` option doesn't mix with IP ranges that use commas (`192.168.0.10,20,30`) because `--exclude` itself uses commas. Use `--excludefile` in these cases.

Practical Examples

While some tools have simple interfaces that only allow a list of hosts or maybe let you specify the start and end IP addresses for a range, Nmap is much more powerful and flexible. But Nmap can also be more difficult to learn—and scanning the wrong IP addresses is occasionally disastrous. Fortunately, Nmap offers a dry run using the list scan (`-sL` option). Simply execute **`nmap -sL -n <targets>`** to see which IPs would be scanned before you actually do it.

Examples may be the most effective way to teach the Nmap host specification syntax. This section provides some, starting with the simplest.

`nmap scanme.nmap.org`, `nmap scanme.nmap.org/32`, `nmap 64.13.134.52`

These three commands all do the same thing, assuming that scanme.nmap.org resolves to 64.13.134.52. They scan that one IP and then exit.

`nmap scanme.nmap.org/24`, `nmap 64.13.134.52/24`, `nmap 64.13.134.-`, `nmap 64.13.134.0-255`

These four commands all ask Nmap to scan the 256 IP addresses from 64.13.134.0 through 64.13.134.255. In other words, they ask to scan the class C sized address space surrounding scanme.nmap.org.

`nmap 64.13.134.52/24 --exclude scanme.nmap.org,insecure.org`

Tells Nmap to scan the class C around 64.13.134.52, but to skip scanme.nmap.org and insecure.org if they are found within that address range.

`nmap 10.0.0.0/8 --exclude 10.6.0.0/16,ultra-sensitive-host.company.com`

Tells Nmap to scan the whole private 10 range except that it must skip anything starting with 10.6 as well as ultra-sensitive-host.company.com.

`egrep '^lease' /var/lib/dhcp/dhcpd.leases | awk '{print $2}' | nmap -iL -`

Obtain the list of assigned DHCP IP addresses and feed them directly to Nmap for scanning. Note that a hyphen is passed to `-iL` to read from standard input.

`nmap -6 2001:800:40:2a03::3`

Scans the IPv6 host at address 2001:800:40:2a03::3.

Finding an Organization's IP Addresses

Fuente original: https://nmap.org/book/host-discovery-find-ips.html

Nmap automates many aspects of network scanning, but you still must tell it which networks to scan. I suppose you could specify `-iR` and hope Nmap hits your target company randomly, or you could try the brute force method of specifying `0.0.0.0/0` to scan the whole Internet. But either of those options could take months or years, and possibly get you into trouble. So it is important to carefully research target netblocks before scanning them. Even if you are conducting a legitimate penetration test and the client gave you a list of their netblocks, it is important to double check them. Clients sometimes have out-of-date records or simply write them down wrong. An authorization letter signed by your client won't help if you accidentally break into the wrong company.

In many cases, you start with only a company's domain name. This section demonstrates a few of the most common and effective ways to turn that into a list of netblocks owned, operated by, or affiliated with the target company. Typical Linux command-line utilities are demonstrated, but similar tools are available for other platforms.

At the ShmooCon conference in 2006, a fellow came up to me and complained that Nmap documentation specified many example ways to scan `target.com`. He noted that ICANN had reserved the domain name `example.com` for this purpose, and pressured me to revise the man page accordingly. While he was technically right, it was a strange thing to obsess about. His motivation became clear when he handed me his business card:

Figure 3.1. A business card explains everything



Apparently, many Nmap users copied examples straight from the man page and ran them without changing the target specifier. So target.com was flooded with scans and corresponding IDS alerts. In honor of that incident, the goal of this section is to determine IP ranges assigned to and used by Target Corporation.

DNS Tricks

The primary purpose of DNS is to resolve domain names into IP addresses, so it is a logical place to start. In Example 3.1, I use the Linux **`host`** command to query some common DNS record types.

Example 3.1. Using the `host` command to query common DNS record types

```
> host -t ns target.com
target.com name server ns4.target.com.
target.com name server ns3.target.com.
target.com name server ns1-auth.sprintlink.net.
target.com name server ns2-auth.sprintlink.net.
target.com name server ns3-auth.sprintlink.net.
> host -t a target.com
target.com has address 161.225.130.163
target.com has address 161.225.136.0
> host -t aaaa target.com
target.com has no AAAA record
> host -t mx target.com
target.com mail is handled by 50 smtp02.target.com.
target.com mail is handled by 5 smtp01.target.com.
> host -t soa target.com
target.com has SOA record extdns02.target.com. hostmaster.target.com.
```

Next I resolve the IP addresses for the hostnames above (using host again) and I try a few common subdomain names such as `www.target.com` and `ftp.target.com`. Starting with names like `ns3.target.com` and `smtp01.target.com`, I try changing the digits to find new machines. All of this leaves me with the following target.com names and addresses:

Table 3.1. First pass at listing target.com IPs

Hostname	IP Addresses
ns3.target.com	161.225.130.130
ns4.target.com	161.225.136.136
ns5.target.com	161.225.130.150
target.com	161.225.136.0, 161.225.130.163
smtp01.target.com	161.225.140.120
smtp02.target.com	198.70.53.234, 198.70.53.235
extdns02.target.com	172.17.14.69
www.target.com	207.171.166.49

While a substantial hostname list can be generated in this manner, the mother lode of hostnames comes from a zone transfer. Most DNS servers now reject zone transfer requests, but it is worth a try because many still allow it. Be sure to try every DNS server you have found through domain NS records and port scanning corporate IP ranges. So far we have found seven Target nameservers: `ns3.target.com`, `ns4.target.com`, `ns5.target.com`, `ns1-auth.sprintlink.net`, `ns2-auth.sprintlink.net`, `ns3-auth.sprintlink.net`, and `extdns02.target.com`. Unfortunately, all of those servers either refused the transfer or did not support the TCP DNS connections required for a zone transfer. Example 3.2 shows a failed `target.com` zone transfer attempt using the common `dig` (domain information groper) tool[9], followed by a successful one against an unrelated organization (`cpsr.org`).

Example 3.2. Zone transfer failure and success

```
> dig @ns2-auth.sprintlink.net -t AXFR target.com
; <<>> DiG 9.5.0b3 <<>> @ns2-auth.sprintlink.net -t AXFR target.com

; Transfer failed.

> dig @ns2.eppi.com -t AXFR cpsr.org
; <<>> DiG 9.5.0b1 <<>> @ns2.eppi.com -t AXFR cpsr.org

cpsr.org             10800   IN      SOA   ns1.findpage.com. root.cpsr.org.
cpsr.org.            10800   IN      NS    ns.stimpy.net.
cpsr.org.            10800   IN      NS    ns1.findpage.com.
cpsr.org.            10800   IN      NS    ns2.eppi.com.
cpsr.org.            10800   IN      A     208.96.55.202
cpsr.org.            10800   IN      MX    0 smtp.electricembers.net.
```

```
diac.cpsr.org.       10800   IN      A     64.147.163.10
groups.cpsr.org.     10800   IN      NS    ns1.electricembers.net.
localhost.cpsr.org.  10800   IN      A     127.0.0.1
mail.cpsr.org.       10800   IN      A     209.209.81.73
peru.cpsr.org.       10800   IN      A     208.96.55.202
www.peru.cpsr.org.   10800   IN      A     208.96.55.202
[...]
```

A common mistake when gathering forward DNS results like these is assuming that all systems found under a domain name must be part of that organization's network and safe to scan. In fact, nothing prevents an organization from adding records pointing anywhere on the Internet. This is commonly done to outsource services to third parties while keeping the source domain name for branding. For example, `www.target.com` resolves to `207.171.166.49`. Is this part of Target's network, or is it managed by a third party we might not want to scan? Three quick and easy tests are DNS reverse-resolution, traceroute, and whois against the relevant IP address registry. The first two steps can be done by Nmap, while the Linux whois command works well for the third. These tests against `target.com` are shown in Example 3.3 and Example 3.4.

Example 3.3. Nmap reverse-DNS and traceroute scan against www.target.com

```
# nmap -Pn -T4 --traceroute www.target.com

Starting Nmap ( https://nmap.org )
Nmap scan report for 166-49.amazon.com (207.171.166.49)
Not shown: 998 filtered ports
PORT    STATE SERVICE
80/tcp  open  http
443/tcp open  https

TRACEROUTE (using port 80/tcp)
HOP RTT    ADDRESS
[cut]
9   84.94  ae-2.ebr4.NewYork1.Level3.net (4.69.135.186)
10  87.91  ae-3.ebr4.Washington1.Level3.net (4.69.132.93)
11  94.80  ae-94-94.csw4.Washington1.Level3.net (4.69.134.190)
12  86.40  ae-21-69.car1.Washington3.Level3.net (4.68.17.7)
13  185.10 AMAZONCOM.car1.Washington3.Level3.net (4.71.204.18)
14  84.70  72.21.209.38
15  85.73  72.21.193.37
16  85.68  166-49.amazon.com (207.171.166.49)

Nmap done: 1 IP address (1 host up) scanned in 20.57 seconds
```

Example 3.4. Using whois to find owner of www.target.com IP address

```
> whois 207.171.166.49
[Querying whois.arin.net]
[whois.arin.net]

OrgName:    Amazon.com, Inc.
OrgID:      AMAZON-4
Address:    605 5th Ave S
City:       SEATTLE
StateProv:  WA
PostalCode: 98104
Country:    US
[...]
```

In Example 3.3, the reverse DNS (two places) and interesting traceroute results are bolded. The Amazon.com domain name makes it highly likely that the web site is run by Amazon rather than Target itself. Then the whois results showing "Amazon.com, Inc." as the IP space owner removes all doubt. The web site is Target branded, but displays "Powered by Amazon.com" at

the bottom. If we were hired by Target to test their security, we would need separate permission from Amazon to touch this address space.

Web databases can also be used to find hostnames under a given domain. For example, Netcraft has a web site DNS search feature at `http://searchdns.netcraft.com/?host`. Typing `.target.com` in to the form brings 36 results, as shown in Figure 3.2. Their handy table shows the netblock owner too, which catches cases such as Amazon running `www.target.com`. We already knew about some of the discovered hosts, but we would have been unlikely to guess names such as `sendasmoochie.target.com`.

Figure 3.2. Netcraft finds 36 Target web servers

Found 36 sites

	Site	Site Report	First seen	Netblock	OS
1.	www.target.com		October 1995	Amazon.com, Inc.	unknown
2.	weeklyad.target.com		January 2005	Akamai Technologies	Linux
3.	sites.target.com		August 2005	Target Corporation	F5 Big-IP
4.	redcard.target.com		November 2005	Target Corporation	F5 Big-IP
5.	www.target.com.au		June 2000	APNIC	Windows 2000
6.	targetrewards.target.com		August 2005	Target Corporation	F5 Big-IP
7.	cinemared.target.com		August 2005	Target Corporation	F5 Big-IP
8.	recipes.target.com		November 2005	Allrecipes.com, Inc.	Windows Server 2003
9.	bookmarked.target.com		September	Implex.net	Linux

Google can also be used for this purpose with queries such as `site:target.com`.

Whois Queries Against IP Registries

After a set of initial "seed" IPs are discovered, they must be researched to ensure they belong to the company you expect and to determine what netblocks they are part of. A small company might have a tiny allocation of 1–16 IP addresses, while larger corporations often have thousands. This information is kept in regional databases, such as ARIN (American Registry for Internet Numbers) for North America and RIPE for Europe and the Middle East. Modern whois tools take an IP address and automatically query the appropriate registry.

Small and mid-sized companies normally don't have IP space allocated by the likes of ARIN. Instead, they are delegated netblocks from their ISPs. Sometimes you get this ISP information from IP queries. This generally leaves you with a big netblock and you don't know which portion of it is allocated to your target. Fortunately, many ISPs now subdelegate customer ranges using Shared Whois (SWIP) or Referral Whois (RWhois). If the ISP has done this, you learn the customer's exact netblock size.

One of the IP addresses previously discovered for target.com was `161.225.130.163`. Example 3.5 demonstrates a whois query (automatically directed against ARIN) to determine the owner and IP allocation information for this IP.

Example 3.5. Using whois to find netblock containing 161.225.130.163

```
> whois 161.225.130.163
[Querying whois.arin.net]
```

```
[whois.arin.net]

OrgName:    Target Corporation
OrgID:      TARGET-14
Address:    1000 Nicollet TPS 3165
City:       Minneapolis
StateProv:  MN
PostalCode: 55403
Country:    US

NetRange:   161.225.0.0 - 161.225.255.255
CIDR:       161.225.0.0/16
NetName:    TARGETNET
NetHandle:  NET-161-225-0-0-1
Parent:     NET-161-0-0-0-0
NetType:    Direct Assignment
NameServer: NS3.TARGET.COM
NameServer: NS4.TARGET.COM
Comment:
RegDate:    1993-03-04
Updated:    2005-11-02

OrgTechHandle: DOMAI45-ARIN
OrgTechName:   Domainnames admin
OrgTechPhone:  +1-612-696-2525
OrgTechEmail:  Domainnames.admin@target.com
```

Not surprisingly, Target owns a huge Class B netblock, covering all 65,536 IPs from 161.225.0.0 through 161.225.255.255. Since the OrgName is Target, this isn't a case where we are seeing results from their ISP.

The next step is to similarly look up all previously discovered IPs which don't fall within this range. Then you can begin with more advanced queries. The command **`whois -h whois.arin.net \?`** gives the ARIN query syntax. It would be nice if you could search for all netblocks matching a given address, OrgID, or OrgTechEmail, but IP registries generally don't allow that. However, many other helpful queries are allowed. For example, **`whois -h whois.arin.net @target.com`** shows all the ARIN contacts with email addresses at target.com. The query **`whois -h whois.arin.net "n target*"`** shows all the netblock handles starting with `target`. It is not case sensitive. Similarly, **`whois -h whois.arin.net "o target*"`** shows all of the organizational names starting with `target`. You can look up the address, phone number, and contact email associated with each entry to determine whether they are part of the company you wish to scan. Often they are 3rd parties who happen to have a similar name.

Internet Routing Information

The core routing protocol of the Internet is the Border Gateway Protocol (BGP). When scanning mid-sized and large organizations, BGP routing tables can help you find their IP subnets all over the world. For example, suppose you want to scan IP addresses belonging to Microsoft Corporation. A DNS lookup for `microsoft.com` provides the IP address `207.46.196.115`. A whois query as discussed in the previous section shows that the whole 207.46.0.0/16 block belongs to Microsoft at their appropriate "One Microsoft Way" address in Redmond. That provides 65,536 IP addresses to scan, but BGP tables expose many more.

Entities such as Microsoft are assigned autonomous system (AS) numbers for routing purposes. A handy tool for determining the AS number advertised for a given IP address is available at http://asn.cymru.com/. Typing `207.46.0.0` into this form provides Microsoft's AS number 8075. Next, I want to find all of the IP prefixes which route to this AS. A handy tool for doing so is available at http://www.robtex.com/as/. Typing in `AS8075` and hitting `Go` on that page leads to a summary screen showing 42 prefixes found. Those prefixes represent 339,456 IP addresses and can be enumerated by clicking the `BGP` tab.

While obtaining BGP information from canned web forms such as these is convenient, obtaining routing data from actual routers is more fun and may allow more powerful custom queries. Several organizations provide such a service. For an example, telnet to `route-views.routeviews.org` or visit http://routeviews.org. Of course these services provide read-only access to the data. If you need to manipulate global routing tables as part of a diabolical plan to take over the Internet, that is beyond the scope of this book.

---

[9] Nmap's `dns-zone-transfer` NSE script could have been used instead (see Chapter 9, *Nmap Scripting Engine*).

DNS Resolution

Fuente original: https://nmap.org/book/host-discovery-dns.html

The key focus of Nmap host discovery is determining which hosts are up and responsive on the network. That narrows down the field of targets, since you can't hack a host which doesn't exist. But don't let discovery end there. You wouldn't date someone just because they're breathing, and selecting boxes on the network to penetrate deserves special care too. A great source of information (about networked hosts, not potential dates) is DNS, the domain name system. Even security conscious organizations often assign names which disclose the function of their systems. It's not uncommon to see wireless access points named `wap` or `wireless`, firewalls named `fw`, `firewall`, or `fw-1`, and development web servers with not-yet-published content named `dev`, `staging`, `www-int`, or `beta`. Locations or department names are also often disclosed, as in the company whose Chicago office firewall is named `fw.chi`.

By default, Nmap performs reverse-DNS resolution for every IP which responds to host discovery probes (i.e. those that are online). If host discovery is skipped with `-Pn`, resolution is performed for all IPs. Rather than use the slow standard DNS resolution libraries, Nmap uses a custom stub resolver which performs dozens of requests in parallel.

While the defaults generally work well, Nmap offers four options for controlling DNS resolution. They can substantially affect scanning speed and the amount of information gathered.

`-n` (No DNS resolution)

Tells Nmap to never do reverse DNS resolution on the active IP addresses it finds. Since DNS can be slow even with Nmap's built-in parallel stub resolver, this option reduces scanning times.

`-R` (DNS resolution for all targets)

Tells Nmap to always do reverse DNS resolution on the target IP addresses. Normally reverse DNS is only performed against responsive (online) hosts.

`--system-dns` (Use system DNS resolver)

By default, Nmap resolves IP addresses by sending queries directly to the name servers configured on your host and then listening for responses. Many requests (often dozens) are performed in parallel to improve performance. Specify this option to use your system resolver instead (one IP at a time via the `getnameinfo` call). This is slow and rarely useful unless you find a bug in the Nmap parallel resolver (please let us know if you do). The system resolver is always used for IPv6 scans.

`--dns-servers <server1>[,<server2>[,...]]` (Servers to use for reverse DNS

queries)

By default, Nmap determines your DNS servers (for rDNS resolution) from your resolv.conf file (Unix) or the Registry (Win32). Alternatively, you may use this option to specify alternate servers. This option is not honored if you are using `--system-dns` or an IPv6 scan. Using multiple DNS servers is often faster, especially if you choose authoritative servers for your target IP space. This option can also improve stealth, as your requests can be bounced off just about any recursive DNS server on the Internet.

This option also comes in handy when scanning private networks. Sometimes only a few name servers provide proper rDNS information, and you may not even know where they are. You can scan the network for port 53 (perhaps with version detection), then try Nmap list scans (`-sL`) specifying each name server one at a time with `--dns-servers` until you find one which works.

Host Discovery Controls

Fuente original: https://nmap.org/book/host-discovery-controls.html

By default, Nmap will include a ping scanning stage prior to more intrusive probes such as port scans, OS detection, Nmap Scripting Engine, or version detection. Nmap usually only performs intrusive scans on machines that are shown to be available during the ping scan stage. This saves substantial time and bandwidth compared to performing full scans against every single IP address. Yet this approach is not ideal for all circumstances. There are times when you do want to scan every IP (`-Pn`), and other times when you want to perform host discovery without a port scan (`-sn`). There are even times when you want to print out the target hosts and exit prior to even sending ping probes (`-sL`). Nmap offers several high-level options to control this behavior.

List Scan (`-sL`)

List scan is a degenerate form of host discovery that simply lists each host on the network(s) specified, without sending any packets to the target hosts. By default, Nmap still performs reverse-DNS resolution on the hosts to learn their names. Nmap also reports the total number of IP addresses at the end. List scan is a good sanity check to ensure that you have proper IP addresses for your targets. If the hosts display domain names you do not recognize, it is worth investigating further to prevent scanning the wrong company's network.

There are many reasons target IP ranges can be incorrect. Even network administrators can mistype their own netblocks, and pen-testers have even more to worry about. In some cases, security consultants are given the wrong addresses. In others, they try to find proper IP ranges through resources such as whois databases and routing tables. The databases can be out of date, or the company could be loaning IP space to other organizations. Whether to scan corporate parents, siblings, service providers, and subsidiaries is an important issue that should be worked out with the customer in advance. A preliminary list scan helps confirm exactly what targets are being scanned.

Another reason for an advance list scan is stealth. In some cases, you do not want to begin with a full-scale assault on the target network that is likely to trigger IDS alerts and bring unwanted attention. A list scan is unobtrusive and provides information that may be useful in choosing which individual machines to target. It is possible, though highly unlikely, that the target will notice all of the reverse-DNS requests. When that is a concern, you can bounce

through anonymous recursive DNS servers using the `--dns-servers` option as described in the section called "DNS proxying".

A list scan is specified with the `-sL` command-line option. Since the idea is to simply print a list of target hosts, options for higher level functionality such as port scanning, OS detection, or ping scanning cannot be combined with `-sL`. If you wish to disable ping scanning while still performing such higher level functionality, read up on the `-Pn` option. Example 3.6 shows list scan being used to enumerate the CIDR /28 network range (16 IP addresses) surrounding the main Stanford University web server.

Example 3.6. Enumerating hosts surrounding www.stanford.edu with list scan

```
felix~> nmap -sL www.stanford.edu/28

Starting Nmap ( https://nmap.org )
Host www9.Stanford.EDU (171.67.16.80) not scanned
Host www10.Stanford.EDU (171.67.16.81) not scanned
Host scriptorium.Stanford.EDU (171.67.16.82) not scanned
Host coursework-a.Stanford.EDU (171.67.16.83) not scanned
Host coursework-e.Stanford.EDU (171.67.16.84) not scanned
Host www3.Stanford.EDU (171.67.16.85) not scanned
Host leland-dev.Stanford.EDU (171.67.16.86) not scanned
Host coursework-preprod.Stanford.EDU (171.67.16.87) not scanned
Host stanfordwho-dev.Stanford.EDU (171.67.16.88) not scanned
Host workgroup-dev.Stanford.EDU (171.67.16.89) not scanned
Host courseworkbeta.Stanford.EDU (171.67.16.90) not scanned
Host www4.Stanford.EDU (171.67.16.91) not scanned
Host coursework-i.Stanford.EDU (171.67.16.92) not scanned
Host leland2.Stanford.EDU (171.67.16.93) not scanned
Host coursework-j.Stanford.EDU (171.67.16.94) not scanned
Host 171.67.16.95 not scanned
Nmap done: 16 IP addresses (0 hosts up) scanned in 0.38 seconds
```

Disable Port Scan (`-sn`)

This option tells Nmap not to run a port scan after host discovery. When used by itself, it makes Nmap do host discovery, then print out the available hosts that responded to the scan. This is often called a "ping scan". Even though no port scanning is done, you can still request Nmap Scripting Engine (`--script`) host scripts and traceroute probing (`--traceroute`). A ping-only scan is one step more intrusive than a list scan, and can often be used for the same purposes. It performs light reconnaissance of a target network quickly and without attracting much attention. Knowing how many hosts are up is more valuable to attackers than the list of every single IP and host name provided by list scan.

Systems administrators often find this option valuable as well. It can easily be used to count available machines on a network or monitor server availability. This is often called a ping sweep, and is more reliable than pinging the broadcast address because many hosts do not reply to broadcast queries.

Example 3.7 shows a quick ping scan against the CIDR /24 (256 IPs) surrounding one of my favorite web sites, Linux Weekly News.

Example 3.7. Discovering hosts surrounding `www.lwn.net` with a ping scan

```
# nmap -sn -T4 www.lwn.net/24

Starting Nmap ( https://nmap.org )
Host 66.216.68.0 seems to be a subnet broadcast address (returned 1 extra ping)
Host 66.216.68.1 appears to be up.
Host 66.216.68.2 appears to be up.
Host 66.216.68.3 appears to be up.
Host server1.camnetsec.com (66.216.68.10) appears to be up.
```

```
Host akqa.com (66.216.68.15) appears to be up.
Host asria.org (66.216.68.18) appears to be up.
Host webcubic.net (66.216.68.19) appears to be up.
Host dizzy.yellowdog.com (66.216.68.22) appears to be up.
Host www.outdoorwire.com (66.216.68.23) appears to be up.
Host www.inspectorhosting.com (66.216.68.24) appears to be up.
Host jwebmedia.com (66.216.68.25) appears to be up.
[...]
Host rs.lwn.net (66.216.68.48) appears to be up.
Host 66.216.68.52 appears to be up.
Host cuttlefish.laughingsquid.net (66.216.68.53) appears to be up.
[...]
Nmap done: 256 IP addresses (105 hosts up) scanned in 12.69 seconds
```

This example only took 13 seconds, but provides valuable information. In that class C sized address range, 105 hosts are online. From the unrelated domain names all packed into such a small IP space, it is clear that LWN uses a colocation or dedicated server provider. If the LWN machines turn out to be highly secure, an attacker might go after one of those neighbor machines and then perform a local ethernet attack with tools such as Ettercap or Dsniff. An ethical use of this data would be a network administrator considering moving machines to this provider. He might e-mail a few of the listed organizations and ask their opinion of the service before signing a long-term contract or making the expensive and disruptive datacenter move.

The `-sn` option sends an ICMP echo request, a TCP SYN packet to port 443, a TCP ACK packet to port 80, and an ICMP timestamp request by default. Since unprivileged Unix users (or Windows users without Npcap installed) cannot send these raw packets, only SYN packets are sent instead in those cases. The SYN packet is sent using a TCP `connect` system call to ports 80 and 443 of the target host. When a privileged user tries to scan targets on a local ethernet network, ARP requests (`-PR`) are used unless the `--send-ip` option is specified.

The `-sn` option can be combined with any of the techniques discussed in the section called "Host Discovery Techniques" for greater flexibility. If any of those probe type and port number options are used, the default probes are overridden. When strict firewalls are in place between the source host running Nmap and the target network, using those advanced techniques is recommended. Otherwise hosts could be missed when the firewall drops probes or their responses.

Disable Ping (`-Pn`)

Another option is to skip the Nmap discovery stage altogether. Normally, Nmap uses this stage to determine active machines for heavier scanning. By default, Nmap only performs heavy probing such as port scans, version detection, or OS detection against hosts that are found to be up. Disabling host discovery with the `-Pn` option causes Nmap to attempt the requested scanning functions against every target IP address specified. So if a class B sized target address space (/16) is specified on the command line, all 65,536 IP addresses are scanned. Proper host discovery is skipped as with a list scan, but instead of stopping and printing the target list, Nmap continues to perform requested functions as if each target IP is active.

There are many reasons for disabling the Nmap ping tests. One of the most common is intrusive vulnerability assessments. One can specify dozens of different ping probes in an attempt to elicit a response from all available hosts, but it is still possible that an active yet heavily firewalled machine might not reply to any of those probes. So to avoid missing anything, auditors frequently perform intense scans, such as for all 65,536 TCP ports, against every IP on the target network. It may seem wasteful to send hundreds of thousands of packets to IP addresses that probably have no host listening, and it can slow scan times by an order of magnitude or more. Nmap must send retransmissions to every port in case the original probe was dropped in transit, and Nmap must spend substantial time waiting for responses because it has no round-trip-time (RTT)

estimate for these non-responsive IP addresses. But serious penetration testers are willing to pay this price to avoid even a slight risk of missing active machines. They can always do a quick scan as well, leaving the massive `-Pn` scan to run in the background while they work. Chapter 6, *Optimizing Nmap Performance* provides more performance tuning advice.

Another frequent reason given for using `-Pn` is that the tester has a list of machines that are already known to be up. So the user sees no point in wasting time with the host discovery stage. The user creates their own list of active hosts and then passes it to Nmap using the `-iL` (take input from list) option. This strategy is rarely beneficial from a time-saving perspective. Due to the retransmission and RTT estimate issues discussed in the previous paragraph, even one unresponsive IP address in a large list will often take more time to scan than a whole ping scanning stage would have. In addition, the ping stage allows Nmap to gather RTT samples that can speed up the following port scan, particularly if the target host has strict firewall rules. While specifying `-Pn` is rarely helpful as a time saver, it is important if some of the machines on your list block all of the discovery techniques that would otherwise be specified. Users must strike a balance between scan speed and the possibility of missing heavily cloaked machines.

Host Discovery Techniques

Fuente original: https://nmap.org/book/host-discovery-techniques.html

There was a day when finding whether an IP address was registered to an active host was easy. Simply send an ICMP echo request (ping) packet and wait for a response. Firewalls rarely blocked these requests, and the vast majority of hosts obediently responded. Such a response has been required since 1989 by RFC 1122, which clearly states that "Every host MUST implement an ICMP Echo server function that receives Echo Requests and sends corresponding Echo Replies".

Unfortunately for network explorers, many administrators have decided that security concerns trump RFC requirements and have blocked ICMP ping messages. Example 3.8 uses an ICMP-only Nmap ping scan against six popular Web sites, but receives only two responses. This demonstrates that hosts can no longer be assumed unavailable based on failure to reply to ICMP ping probes. The "`-sn -PE`" options in this example specify an ICMP-only ping scan. The `-R` option tells Nmap to perform reverse-DNS resolution against all hosts, even down ones.

Example 3.8. Attempts to ping popular Internet hosts

```
# nmap -sn -PE -R -v microsoft.com ebay.com citibank.com google.com \
                     slashdot.org yahoo.com

Starting Nmap ( https://nmap.org )
Host origin2.microsoft.com (207.46.250.252) appears to be down.
Host pages.ebay.com (66.135.192.87) appears to be down.
Host ld1-www.citicorp.com (192.193.195.132) appears to be down.
Host 216.239.57.99 appears to be up.
Host slashdot.org (66.35.250.150) appears to be down.
Host w3.rc.dcn.yahoo.com (216.109.127.30) appears to be up.
Nmap done: 6 IP addresses (2 hosts up) scanned in 3.76 seconds
```

Fortunately, Nmap offers a wide variety of host discovery techniques beyond the standard ICMP echo request. They are described in the following sections. Note that if you specify any of the `-P` options discussed in this section, they replace the default discovery probes rather than adding to them.

TCP SYN Ping (-PS<*port list*>)

The -PS option sends an empty TCP packet with the SYN flag set. The default destination port is 80 (configurable at compile time by changing DEFAULT_TCP_PROBE_PORT_SPEC in nmap.h), but an alternate port can be specified as a parameter. A list of ports may be specified (e.g. -PS22-25,80,113,1050,35000), in which case probes will be attempted against each port in parallel.

The SYN flag suggests to the remote system that you are attempting to establish a connection. Normally the destination port will be closed, and a RST (reset) packet will be sent back. If the port happens to be open, the target will take the second step of a TCP three-way-handshake by responding with a SYN/ACK TCP packet. The machine running Nmap then tears down the nascent connection by responding with a RST rather than sending an ACK packet which would complete the three-way-handshake and establish a full connection.[10]

Nmap does not care whether the port is open or closed. Either the RST or SYN/ACK response discussed previously tell Nmap that the host is available and responsive.

On Unix boxes, only the privileged user root is generally able to send and receive raw TCP packets. For unprivileged users, a workaround is automatically employed whereby the connect system call is initiated against each target port. This has the effect of sending a SYN packet to the target host, in an attempt to establish a connection. If connect returns with a quick success or an ECONNREFUSED failure, the underlying TCP stack must have received a SYN/ACK or RST and the host is marked available. If the connection attempt is left hanging until a timeout is reached, the host is marked as down. This workaround is also used for IPv6 connections, as raw IPv6 packet building support is not yet available in Nmap.

Example 3.8 failed to detect four out of six machines because they did not respond to ICMP echo requests. Repeating the experiment using a SYN probe to port 80 (HTTP) garners responses from all six, as shown in Example 3.9.

Example 3.9. Retry host discovery using port 80 SYN probes

```
# nmap -sn -PS80 -R -v microsoft.com ebay.com citibank.com google.com \
                       slashdot.org yahoo.com

Starting Nmap ( https://nmap.org )
Host origin2.microsoft.com (207.46.249.252) appears to be up.
Host pages.ebay.com (66.135.192.87) appears to be up.
Host ld1-www.citicorp.com (192.193.195.132) appears to be up.
Host 216.239.57.99 appears to be up.
Host slashdot.org (66.35.250.150) appears to be up.
Host w3.rc.dcn.yahoo.com (216.109.127.30) appears to be up.
Nmap done: 6 IP addresses (6 hosts up) scanned in 0.48 seconds
```

In addition to detecting all six machines, the second run is much faster. It takes less than half a second because the machines are scanned in parallel and the scan never times out waiting for a response. This test is not entirely fair because these are all popular web servers and thus can be expected to listen on TCP port 80. However, it still demonstrates the point that different types of hosts respond to different probe types. Nmap supports the usage of many scan types in parallel to enable effective scanning of diverse networks.

TCP ACK Ping (-PA<*port list*>)

The TCP ACK ping is quite similar to the SYN ping. The difference, as you could likely guess, is that the TCP ACK flag is set instead of the SYN flag. Such an ACK packet purports to

be acknowledging data over an established TCP connection, but no such connection exists. So remote hosts should always respond with a RST packet, disclosing their existence in the process.

The `-PA` option uses the same default port as the SYN probe (80) and can also take a list of destination ports in the same format. If an unprivileged user tries this, or an IPv6 target is specified, the `connect` workaround discussed previously is used. This workaround is imperfect because `connect` is actually sending a SYN packet rather than an ACK.

The reason for offering both SYN and ACK ping probes is to maximize the chances of bypassing firewalls. Many administrators configure routers and other simple firewalls to block incoming SYN packets except for those destined for public services like the company web site or mail server. This prevents other incoming connections to the organization, while allowing users to make unobstructed outgoing connections to the Internet. This non-stateful approach takes up few resources on the firewall/router and is widely supported by hardware and software filters. As just one example of the prevalence of this method, the Linux Netfilter/iptables firewall software offers the `--syn` convenience option, which the man page describes as follows.

> *Only match TCP packets with the SYN bit set and the ACK and RST bits cleared. Such packets are used to request TCP connection initiation; for example, blocking such packets coming in an interface will prevent incoming TCP connections, but outgoing TCP connections will be unaffected. It is equivalent to –tcp-flags SYN,RST,ACK SYN.*

When firewall rules such as this are in place, SYN ping probes (`-PS`) are likely to be blocked when sent to closed target ports. In such cases, the ACK probe excels by cutting right through these rules.

Another common type of firewall uses stateful rules that drop unexpected packets. This feature was initially found mostly on high-end firewalls, though it has become much more common over the years. The Linux Netfilter/iptables system supports this through the `--state` option, which categorizes packets based on connection state as described in the following man page excerpt:

> *Possible states are INVALID meaning that the packet is associated with no known connection, ESTABLISHED meaning that the packet is associated with a connection which has seen packets in both directions, NEW meaning that the packet has started a new connection, or otherwise associated with a connection which has not seen packets in both directions, and RELATED meaning that the packet is starting a new connection, but is associated with an existing connection, such as an FTP data transfer, or an ICMP error.*

The ACK probe is unlikely to work against firewalls taking this approach, as such an unexpected packet will be classified in the INVALID state and probably dropped. Example 3.10 shows an attempted ACK ping against Microsoft. Their stateful firewall drops the packet, leading Nmap to wrongly conclude that the host is down. The SYN probe has a much better chance of working in such cases. This raises the question of which technique to use when the firewall rules of the target networks are unknown or inconsistent. The proper answer is usually both. Nmap can send SYN and ACK probes to many ports in parallel, as well as performing other host discovery techniques at the same time. This is further discussed in the section called "Putting It All Together: Host Discovery Strategies".

Example 3.10. Attempted ACK ping against Microsoft

```
# nmap -sn -PA www.microsoft.com
```

```
Starting Nmap ( https://nmap.org )
Warning: Hostname www.microsoft.com resolves to 5 IPs. Using 207.46.192.254.
Note: Host seems down. If it is really up, but blocking ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 2.22 seconds
```

UDP Ping (`-PU<port list>`)

Another host discovery option is the UDP ping, which sends a UDP packet to the given ports. The port list takes the same format as with the previously discussed `-PS` and `-PA` options. If no ports are specified, the default is 40,125. This default can be configured at compile-time by changing `DEFAULT_UDP_PROBE_PORT_SPEC` in `nmap.h`. A highly uncommon port is used by default because sending to open ports is often undesirable for this particular scan type.

For most ports, the packet will be empty, though for a few common ports like 53 and 161, a protocol-specific payload will be sent that is more likely to get a response. The `--data-length` option sends a fixed-length random payload for all ports.

Upon hitting a closed port on the target machine, the UDP probe should elicit an ICMP port unreachable packet in return. This signifies to Nmap that the machine is up and available. Many other types of ICMP errors, such as host/network unreachables or TTL exceeded are indicative of a down or unreachable host. A lack of response is also interpreted this way. If an open port is reached, most services simply ignore the empty packet and fail to return any response. This is why the default probe port is 40,125, which is highly unlikely to be in use. A few services, such as the Character Generator (chargen) protocol, will respond to an empty UDP packet, and thus disclose to Nmap that the machine is available. Custom payloads, for the ports that have them, make it more likely that a probe will get a response.

The primary advantage of this scan type is that it bypasses firewalls and filters that only screen TCP. For example, I once owned a Linksys BEFW11S4 wireless broadband router. The external interface of this device filtered all TCP ports by default, but UDP probes would still elicit port unreachable messages and thus give away the device.

ICMP Ping Types (`-PE`, `-PP`, and `-PM`)

In addition to the unusual TCP and UDP host discovery types discussed previously, Nmap can send the standard packets sent by the ubiquitous ping program. Nmap sends an ICMP type 8 (echo request) packet to the target IP addresses, expecting a type 0 (echo reply) in return from available hosts. As noted at the beginning of this chapter, many hosts and firewalls now block these packets, rather than responding as required by RFC 1122. For this reason, ICMP-only scans are rarely reliable enough against unknown targets over the Internet. But for system administrators monitoring an internal network, this can be a practical and efficient approach. Use the `-PE` option to enable this echo request behavior.

While echo request is the standard ICMP ping query, Nmap does not stop there. The ICMP standards (RFC 792 and RFC 950) also specify timestamp request, information request, and address mask request packets as codes 13, 15, and 17, respectively. While the ostensible purpose for these queries is to learn information such as address masks and current times, they can easily be used for host discovery. Nmap does not currently implement information request packets, as they are not widely supported (RFC 1122 insists that "a host SHOULD NOT implement these messages"). Timestamp and address mask queries can be sent with the `-PP` and `-PM` options, respectively. A timestamp reply (ICMP code 14) or address mask reply (code 18) discloses that the host is available. These two queries can be valuable when administrators specifically block echo request packets, but forget that other ICMP queries can be used for the same purpose.

IP Protocol Ping (`-PO<protocol list>`)

The newest host discovery option is the IP protocol ping, which sends IP packets with the specified protocol number set in their IP header. The protocol list takes the same format as do port lists in the previously discussed TCP and UDP host discovery options. If no protocols are specified, the default is to send multiple IP packets for ICMP (protocol 1), IGMP (protocol 2), and IP-in-IP (protocol 4). The default protocols can be configured at compile-time by changing `DEFAULT_PROTO_PROBE_PORT_SPEC` in `nmap.h`. Note that for the ICMP, IGMP, TCP (protocol 6), and UDP (protocol 17), the packets are sent with the proper protocol headers while other protocols are sent with no additional data beyond the IP header (unless the `--data-length` option is specified).

This host discovery method looks for either responses using the same protocol as a probe, or ICMP protocol unreachable messages which signify that the given protocol isn't supported by the destination host. Either type of response signifies that the target host is alive.

ARP Scan (`-PR`)

One of the most common Nmap usage scenarios is to scan an ethernet LAN. On most LANs, especially those using private address ranges granted by RFC 1918, the vast majority of IP addresses are unused at any given time. When Nmap tries to send a raw IP packet such as an ICMP echo request, the operating system must determine the destination hardware (ARP) address corresponding to the target IP so that it can address the ethernet frame properly. This requires it to issue a series of ARP requests. This is shown in Example 3.11, where a ping scan is attempted against a local ethernet host. The `--send-ip` option tells Nmap to send IP level packets (rather than raw ethernet) even though it is a local network. Wireshark output of the three ARP requests and their timing has been pasted into the session.

Example 3.11. Raw IP ping scan of an offline target

```
# nmap -n -sn --send-ip 192.168.33.37

Starting Nmap ( https://nmap.org )
  0.000000 00:01:29:f5:27:f2 -> ff:ff:ff:ff:ff:ff ARP Who has 192.168.33.37?
  0.999836 00:01:29:f5:27:f2 -> ff:ff:ff:ff:ff:ff ARP Who has 192.168.33.37?
  1.999684 00:01:29:f5:27:f2 -> ff:ff:ff:ff:ff:ff ARP Who has 192.168.33.37?
Note: Host seems down. If it is really up, but blocking ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 2.04 seconds
```

This example took more than two seconds to finish because the (Linux) OS sent three ARP requests, one second apart, before giving up on the host. Given that ARP replies usually come within a couple milliseconds, multi-second waits are excessive. Decreasing this timeout period is no priority for OS vendors because the vast majority of packets are sent to hosts that actually exist. Nmap, on the other hand, must send packets to 16 million IPs when given a target such as 10.0.0.0/8. A two second wait for each becomes a huge delay even though many targets are pinged in parallel.

There is another problem with raw IP ping scans on LANs. When a destination host is found to be unresponsive as in the previous example, the source host generally adds an incomplete entry for that destination IP in its kernel ARP table. ARP table space is finite, and some operating systems react badly when it fills up. When Nmap is used in raw IP mode (`--send-ip`), Nmap sometimes has to wait several minutes for ARP cache entries to expire before it can continue with host discovery.

ARP scanning resolves both problems by putting Nmap in control. Nmap issues the raw ARP requests and handles retransmission and timeout periods at its own discretion. The system ARP

cache is bypassed. Example 3.12 shows the difference. This ARP scan takes just over a tenth of the time taken by its IP equivalent.

Example 3.12. ARP ping scan of an offline target

```
# nmap -n -sn -PR --packet-trace --send-eth 192.168.33.37

Starting Nmap ( https://nmap.org )
SENT (0.0060s) ARP who-has 192.168.33.37 tell 192.168.0.100
SENT (0.1180s) ARP who-has 192.168.33.37 tell 192.168.0.100
Note: Host seems down. If it is really up, but blocking ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 0.23 seconds
```

In Example 3.12, neither the `-PR` or `--send-eth` options have any effect. This is because ARP is the default scan type when scanning ethernet hosts that Nmap detects are on a local ethernet network. This includes traditional wired ethernet as well as 802.11 wireless networks. Not only is ARP scanning more efficient as discussed above, it is also more accurate. Hosts frequently block IP-based ping packets, but they generally cannot block ARP requests or responses and still communicate on the network. Even if different ping types (such as `-PE` or `-PS`) are specified, Nmap uses ARP instead for any of the targets which are on the same LAN. If you absolutely don't want to do an ARP scan, specify `--send-ip` as shown in Example 3.11, "Raw IP ping scan of an offline target".

Giving Nmap control to send raw ethernet frames also allows Nmap to control the source MAC address. If you have the only PowerBook in the room at a security conference and a massive ARP scan is initiated from a MAC address registered to Apple, heads may turn in your direction. You can spoof your MAC address with the `--spoof-mac` option, as discussed in the section called "MAC Address Spoofing".

Default Combination

If none of these host discovery techniques are chosen, Nmap uses a default which is equivalent to the `-PE -PS443 -PA80 -PP` arguments for Windows or privileged (root) Unix users. Attentive readers know that this means an ICMP echo request, a TCP SYN packet, a TCP ACK packet, and an ICMP timestamp request are sent to each machine. An exception to this is that an ARP scan is used for any targets which are on a local ethernet network. For unprivileged Unix shell users, the default is equivalent to `-PS80,443` (a TCP `connect` call against ports 80 and 443 of the target hosts). For security auditing, I recommend using a more comprehensive set of ping types, such as those discussed in the section called "Designing the ideal combinations of probes".

---

[10] The RST packet is sent by the kernel of the machine running Nmap in response to the unexpected SYN/ACK, not by Nmap itself.

Putting It All Together: Host Discovery Strategies

Fuente original: https://nmap.org/book/host-discovery-strategies.html

Related Options

Previous sections describe the major options used to control the Nmap host discovery phase and customize the techniques used. However, there are many more general Nmap options which are

relevant here. This section provides a brief description of how these option flags relate to ping scanning. See Chapter 15, *Nmap Reference Guide* for complete descriptions of each option.

`-v` (same as `--verbose`)

By default, Nmap usually only prints active, responsive hosts. Verbose mode causes Nmap to print down hosts, as well as extra information about active ones.

`--source-port <portnum>` (same as `-g`)

Setting a constant source port works for ping scanning (TCP and UDP) as it does with other Nmap features. Some naive firewall administrators make a ruleset exception in order to keep DNS (port 53) or FTP-DATA (port 20) working. Of course this opens a hole big enough to drive an Nmap ping scan through. the section called "Source Port Manipulation" provides further details on this technique.

`-n`, `-R`

The `-n` option disables all DNS resolution, while the `-R` option enables DNS queries for all hosts, even down ones. The default behavior is to limit DNS resolution to active hosts. These options are particularly important for ping scanning because DNS resolution can greatly affect scan times.

`--dns-servers <server1>[,<server2>[,...]]` (Servers to use for reverse DNS queries)

By default Nmap will try to determine your DNS servers (for rDNS resolution) from your resolv.conf file (Unix) or the Registry (Win32). Alternatively, you may use this option to specify alternate servers. This option is not honored if you are using `--system-dns` or an IPv6 scan. Using multiple DNS servers is often faster and more stealthy than querying just one. The best performance is often obtained by specifying all of the authoritative servers for the target IP space.

`--data-length <length>`

This option adds *`<length>`* random bytes of data to every packet, and works with the TCP, UDP, and ICMP ping scan types (for privileged users scanning IPv4). This helps make the scan less conspicuous and more like the packets generated by the ubiquitous ping diagnostics program. Several intrusion detection systems (IDS), including Snort, have alerts for zero-byte ping packets. This option evades those alerts. An option value of 32 makes an echo request look more like it came from Windows, while 56 simulates the default Linux ping.

`--ttl <value>`

Setting the outgoing TTL is supported for privileged users doing IPv4 ping scans. This can be useful as a safety precaution to ensure a scan does not propagate beyond the local network. It can also be used to simulate a native ping program even more convincingly. Some enterprise networks suffer from known routing loops which they can't easily fix. Reducing the outgoing TTL with `--ttl` helps to reduce router CPU load when loops are encountered.

Canned timing options (`-T3`, `-T4`, `-T5`, etc.)

Higher `-T` values speed up ping scanning, just as they speed other Nmap features. With a moderately fast and reliable connection between the source and target networks (i.e. anything more than a dial-up modem), the `-T4` option is recommended.

`--max-parallelism`, `--min-parallelism <value>`

These affect how many probes may be outstanding at once. With the default ping type (two probes), the parallelism value is roughly the number of machines scanned in parallel. Reducing the ping techniques to one probe per host (e.g. `-PE`) will double the number of

hosts scanned at once for a given parallelism level, while increasing to four probes per host (e.g. `-PE -PS22,113,50000`) halves it. Most users simply stick to the canned timing options such as `-T4`.

`--min-rtt-timeout`, `--max-rtt-timeout`, `--initial-rtt-timeout` *<time>*

These options control how long Nmap waits for a ping response.

Input options (`-iL` *<filename>*, `-iR` *<number>*)

Host input options are supported as in the rest of Nmap. Users often combine the input-from-list (`-iL`) option with `-Pn` to avoid ping-scanning hosts that are already known to be up. Before doing this in an attempt to save time, read the section called "Disable Ping (`-Pn`)". The `-iR` option chooses hosts at random from allocated Internet IP space. It takes as an argument the number of random hosts you wish to scan. Use zero for a never-ending (until you abort or kill the Nmap process) scan.

Output options (`-oA`, `-oN`, `-oG`, `-oX`, etc.)

All of the Nmap output types (normal, grepable, and XML) support ping scanning. Chapter 13, *Nmap Output Formats* further describes how they work.

`--randomize-hosts`

Shuffling the host scan order with this option may make the scan less conspicuous, though it also can make the scan output a bit more difficult to follow.

`--reason`

The normal Nmap output indicates whether a host is up or not, but does not describe which discovery test(s) the host responded to. For this detail, add the `--reason` option. The results can be confusing for host discovery since Nmap does not always try every probe. It stops as soon as it gets a first response. So Nmap might report an ICMP echo response from a host during the run, but then a RST response might be received first during a second run and lead Nmap to report that.

`--packet-trace`

When you want even more details than `--reason` provides, try `--packet-trace`. This option shows every packet send and received by Nmap, including details such as sequence numbers, TTL values, and TCP flags.

`-D` *<decoy1,decoy2,...>*

Decoys are fully supported for privileged IPv4 ping scans, camouflaging the true attacker.

`-6`

The TCP `connect`-based ping scans (`-PS`) support the IPv6 protocol, including multi-port mode (such as `-PS22,80,113`.

`-S` *<source IP address>*, `-e` *<sending device name>*

As with other functions of Nmap, the source address and sending device can be specified with these options.

General options

By default, unless `-sn` or `-sL` are specified, Nmap moves onto more intrusive scanning after the host discovery stage. Thus many dozens of general port scanning, OS detection, and version detection options can be used. See the reference guide or relevant chapters for further information.

Choosing and Combining Ping Options

Effective scanning requires more than knowing all of the options described in this and previous sections. Users must understand how and when to use them to suit the target network topology and scanning goals.

Most valuable probes

In May 2009 we did some research to find the most effective ping probes. We tested dozens of different probes and host discovery options against thousands of Internet hosts. The results of this research are shown in Table 3.2, which ranks the different probes by effectiveness. Percentages are out of the number of hosts that responded to any of the probes. We tested 90 different probes; this is a subset.

Table 3.2. Best host discovery probes

Hosts found	Probe
62.47%	`-PE`
44.17%	`-PS443`
43.28%	`-PA80`
43.01%	`-PA443`
42.47%	`-PS80`
40.65%	`-PA110`
40.42%	`-PA3389`
40.41%	`-PS110`
39.89%	`-PA22`
39.62%	`-PS21`
39.62%	`-PA21`
38.75%	`-PS22`
37.50%	`-PS3389`
36.66%	`-PP`
31.17%	`-PU40125 --source-port 53 --data-length 24`
29.96%	`-PU31338 --source-port 53 --data-length 24`
29.05%	`-PU631 --source-port 53 --data-length 24`
26.38%	`-PU40125`
26.09%	`-PS25`
25.69%	`-PA25`
25.35%	`-PU31338`
24.71%	`-PU631`
24.15%	`-PU53 --source-port 53 --data-length 24`
22.20%	`-PU53`
9.09%	`-PO2`
9.03%	`-PO150`
7.20%	`-PO4`
4.21%	`-PM`

We can pick out some interesting features from Table 3.2. The best single probe by far is `-PE`, ICMP echo. Following that are various SYN and ACK probes (`-PS` and `-PA`) to common ports. ICMP timestamp, `-PP`, does quite well. The best UDP probes (`-PU`) are those to random high ports, and they are more effective when combined with `--source-port 53 --data-length 24`. SYN and ACK to port 25 (SMTP) are notably worse than other TCP probes, perhaps because that port is commonly filtered. Various IP protocol pings (`-PO`) are not very effective, and `-PM`, the ICMP address mask request, comes in last.

The best multi-probe combinations are not found simply by picking entries from the top of Table 3.2. That's because some probes find many of the same hosts as another, so using both of them is little better than using just one. We had a computer program find the optimum combination for different numbers of probes, by examining how many hosts each probe found and how much they overlapped with other probes. The best combinations are in Table 3.3. Nmap's default four-probe combination was selected from this list as a compromise between comprehensiveness and speed.

Table 3.3. Best host discovery probe combinations

	Hosts found	**Probe combination**
1 probe	62.47%	`-PE`
2 probes	77.61%	`-PE -PA80`
3 probes	83.83%	`-PE -PA80 -PS443`
4 probes	88.64%	`-PE -PA80 -PS443 -PP`
5 probes	91.12%	`-PE -PA80 -PS443 -PP -PU40125 --source-port 53`
6 probes	92.42%	`-PE -PS80 -PS443 -PP -PU40125 -PA3389 --source-port 53`
7 probes	93.10%	`-PE -PS80 -PS443 -PP -PU40125 -PS3389 -PA21 --source-port 53`
8 probes	93.69%	`-PE -PS80 -PS443 -PP -PU40125 -PS3389 -PA21 -PU161 --source-port 53`

TCP probe and port selection

The TCP ping options are some of the most powerful discovery techniques in Nmap. An administrator may be able to get away with blocking ICMP echo request packets without affecting most users, but a server absolutely must respond to SYN packets sent to the public services it provides. Meanwhile, ACK packets often get through non-stateful firewalls. I would recommend using both of SYN and ACK probes, using lists of ports based on any knowledge you might have of the target networks as well as more generally popular ports. Table 3.4 lists the ports that are most likely to be responsive (open or closed) based on empirical testing (see the section called "What Are the Most Popular Ports?"). On hosts with a default-drop filter (the hardest type to reach), these are the ports most likely to elicit a response.

Table 3.4. Most valuable TCP probe ports, in descending order of accessibility.

Port number / Service	**Reasoning**
`80/http`	The prevalence of Web servers on the Internet leads many newbies to believe that the Web is the Internet.
`443/https`	SSL is a popular way for web sites to protect confidential directory information.

113/auth	The auth (identd) service allows servers (usually mail or IRC) to request the username of clients connected to them. Administrators often leave this port unfiltered to avoid long timeouts that can occur when firewall rules prevent servers from connecting back to port 113. Using this port for ping scanning can sometimes lead to false positives, as some administrators have been known to configure their firewalls to forge RST packets back in response to auth queries to any IP on their network, even when no machine exists at that IP. Administrators do this to avoid server timeouts while still preventing the ports from being accessed.
21/ftp	This file transfer protocol lives on, though many firewall administrators would not mourn its passing.
23/telnet	Many devices still offer this administrative interface, though it is a security nightmare.
25/smtp	Mail is another Internet "killer app" that companies allow through their firewalls.
53/domain	Domain name servers are extremely widespread.
22/ssh	SSH seems to have finally surpassed Telnet as the standard for remote terminal administration.
110/pop3	POP3 is a common method of remotely reading email.
3389/ms-term-server	Microsoft Terminal Services allows users (and sometimes hackers) to access applications and data on a remote computer.
1723/pptp	Point-to-Point Tunneling Protocol is often used to implement VPN solutions on Microsoft Windows.
8080/http-proxy	It is common to run a web proxy on port 8080. The port is also sometimes used as an alternate HTTP port when port 80 is not available.

In addition to popular ports such as the ones in the list above, choosing at least one high-numbered port is recommended. Many poorly configured firewalls only have default-deny for the privileged ports, meaning those below 1,024. I usually pick a high numbered port out of the air, such as 40,000 or 10,042, to catch machines behind this sort of firewall.

In choosing the ports to probe, remember to emphasize platform diversity. If you are limiting your ping scan to two ports, HTTP (80) and SSH (22) are probably better than HTTP (80) and HTTPS (443) because the latter two are related web services, and many machines that have HTTPS will often have HTTP available anyway. Finding two accessible ports on the same machine is no better for ping scanning purposes than finding one. The goal is to choose ports so that a broad set of hosts will match at least one of them.

Note that the valuable port table does not include many client-oriented ports such as the ubiquitous Windows SMB port 135. The primary reason is that this table only looked at hosts behind default-deny firewalls, where the vast majority of ports are filtered. In those situations, Windows ports such as 135–139 and 445 are usually blocked. When these machines are not behind a firewall, the open ports are unimportant for ping scanning because the thousands of closed ports work just as well.

UDP port selection

In selecting UDP ports, remember that an open port is unlikely to respond to the probes. Unfiltered ports are desired. To avoid open ports, you might consider excluding common UDP services like DNS (port 53) and SNMP (161). On the other hand, firewall rules are often so broad that those probes (particularly to port 53) might get through and hit a closed port. So I would recommend choosing at least port 53 and an arbitrarily selected high-numbered port such as 37,452.

ICMP probe selection

For ICMP, the standard ping (echo request) is usually worth trying. Many administrators specifically allow this because it is useful for debugging or because RFC 1122 requires it. I would also use at least one of the address mask or timestamp requests. These are valuable for networks where administrators intentionally block echo request packets, but forget about other ICMP queries.

Designing the ideal combinations of probes

How all of these ping types are combined into a ping scan strategy depends on characteristics of the target network and on the scan goals. For internal networks, the default ping type usually works well. The default is also fine for most casual scanning, where missing an occasional host is no big deal. Adding more probes can help catch those occasional stealthy machines, at the expense of making the ping scan take a bit longer. Time taken is roughly proportional to the number of probes sent to each machine. For security scans of target networks over the Internet, adding more probes is usually advisable. Try to include a diverse set of the techniques discussed previously. Here is a set of ping options that should catch the vast majority of hosts: `-PE -PP -PS21,22,23,25,80,113,443,31339 -PA80,113,443,10042`. Adding in `--source-port 53` might be worthwhile as well. How much better will the results be, and how much longer will it take? That depends on the target network, of course, but the Nmap random target selection option (`-iR`) makes it easy to perform a quick test. Example 3.13 shows Nmap generating 50,000 random IP addresses and then performing a default ping scan. You should remember that the default is four probes: an ICMP echo request, a TCP SYN to port 443, a TCP ACK to port 80, and an ICMP timestamp request.

Example 3.13. Generating 50,000 IP addresses, then ping scanning with default options

```
# nmap -n -sL -iR 50000 | awk '/^Host / {print $2}' | sort -n > 50K_IPs
# head -5 50K_IPs
3.0.47.195
3.100.112.251
3.100.98.39
3.10.89.120
3.101.183.79
# nmap -n -sn -T4 -iL 50K_IPs -oA 50KHosts_DefaultPing

Starting Nmap ( https://nmap.org )
Host 4.178.9.27 is up (0.27s latency).
Host 12.135.202.138 is up (0.20s latency).
Host 12.151.172.161 is up (0.089s latency).
[thousands of lines cut]
Host 222.99.77.130 is up (0.20s latency).
Nmap done: 50000 IP addresses (3927 hosts up) scanned in 2532.05 seconds
```

Scanning the 50,000 addresses took just over 42 minutes, and 3,927 hosts were detected. To de-

termine the effects of using a wider range of ping techniques, the same 50K hosts were rescanned with 14 probes per port rather than the default of four. As shown in Example 3.14, Nmap was able to detect 785 (20%) more hosts. It took about 147 minutes, which is almost 3.5 times as long. Given all the new hosts detected, that extra time was well spent. Note that not all of the new hosts will be legitimate. Increasing the number of ping probes increases the chances that Nmap will hit network artifacts that make a non-existent host appear to be active. Firewalls that return a RST for SYN or ACK packets to port 113 are one example of this.

Example 3.14. Repeating ping scan with extra probes

```
# nmap -n -sn -PE -PP -PS21,22,23,25,80,113,443,31339 -PA80,113,443,10042 \
  -T4 --source-port 53 -iL 50K_IPs -oA 50KHosts_ExtendedPing
Starting Nmap ( https://nmap.org )
Host 4.238.177.186 is up (0.44s latency).
Host 12.135.202.138 is up (0.13s latency).
Host 12.151.172.161 is up (0.092s latency).
[thousands of hosts cut]
Host 222.94.94.113 is up (0.23s latency).
Nmap done: 50000 IP addresses (4712 hosts up) scanned in 8842.31 seconds
```

When performing security audits for clients, I normally start TCP analysis with a port scan against the most common 1,000 ports (the default) with comprehensive ping scan options like those shown in Example 3.14, "Repeating ping scan with extra probes". Such a scan does not take particularly long, allowing me to quickly start working. I also launch `-Pn` (ping disabled) scans against all 65K TCP ports in the background while I work. When they finish, which may be days later, I compare them to my initial quick scan and investigate any new ports or machines found.

Host Discovery Code Algorithms

Fuente original: https://nmap.org/book/host-discovery-algorithms.html

One of the greatest benefits of open source software like Nmap is that curious users are always able to study the source code when they want answers about its operation. The highest level ping scanning function is `nexthost` (in `targets.cc`, which calls `massping` to initialize a list of targets. `Massping` in turn passes the list off to `ultra_scan` (in `scan_engine.cc`). Ultra_scan is Nmap's general-purpose scanning function and does all the hard work of sending, receiving, and interpreting packets. For more on `ultra_scan` see the section called "Scan Code and Algorithms".

While source code analysis is the only way to truly get the complete picture of Nmap operation down to every trivial detail, it is not always the easiest approach to understanding Nmap. In many cases, the most effective way to explore Nmap's behavior given a set of command-line options is to add the `--packet-trace` option, which prints out all of the packets sent and received by Nmap.

Because the source code and the `--packet-trace` option are excellent resources for learning the nitty-gritty details of Nmap operation, I'll only discuss how host discovery works at a high level here. When Nmap is executed, it may be passed networks containing hundreds of thousands or even millions of hosts. So Nmap breaks them into blocks that are small enough to deal with at one time (dozens up to a few thousand hosts). `ultra_scan` then works its way through the block, sending packets as fast as its congestion controls allow. Rather than sending all the probes requested by the user to each host all at once, Nmap sends the first probe to all the targets, then the second probe, and so on. When a conclusive response to a probe is received,

that host is marked as up or down as appropriate and no further probes are sent to it. A target host which fails to respond to any probes, even after retransmissions, is marked as down. Nmap waits until every host has either received a conclusive response or has timed out. Eventually, Nmap runs out of new hosts in the block and the number of outstanding probes dwindles to zero as retransmissions complete. The ping scanning subsystem returns the results so that Nmap can begin port scanning or any other requested probing of the target machines. When Nmap finishes completely with a block of hosts, it prints the results and passes the next block to the ping scanner.

Multiple hosts, usually with multiple probes per host, are handled in parallel. The number of outstanding probes and timeout periods are modified in real-time based on network latency and reliability. The `ultra_scan` performance algorithms are further described in the section called "Scan Code and Algorithms".

Chapter 4

Port Scanning Overview

Fuente original: https://nmap.org/book/port-scanning.html

Introduction to Port Scanning

While Nmap has grown in functionality over the years, it began as an efficient port scanner, and that remains its core function. The simple command **nmap** ***<target>*** scans the most commonly used 1,000 TCP ports on the host ***<target>***, classifying each port into the state `open`, `closed`, `filtered`, `unfiltered`, `open|filtered`, or `closed|filtered`.

What Exactly is a Port?

Ports are simply a software abstraction, used to distinguish between communication channels. Similar to the way IP addresses are used to identify machines on networks, ports identify specific applications in use on a single machine. For example, your web browser will by default connect to TCP port 80 of machines in HTTP URLs. If you specify the secure HTTPS protocol instead, the browser will try port 443 by default.

Nmap works with two protocols that use ports: TCP and UDP. A connection for each protocol is uniquely identified by four elements: source and destination IP addresses and corresponding source and destination ports. All of these elements are simply numbers placed in the headers of each packet sent between hosts. The protocol is an eight-bit field, which specifies what type of packet is contained in the IP data (payload) section. For example, TCP is protocol number six, and UDP is 17. IPv4 addresses have a length of 32-bits, while ports are 16-bits long. IPv6 addresses are 128-bits in length. Further IP, TCP, and UDP header layout details can be found in the section called "TCP/IP Reference".

Because most popular services are registered to a well-known port number, one can often guess what services open ports represent. Nmap includes an `nmap-services` file, containing the well-known service for registered port and protocol numbers, as well as common ports for trojan backdoors and other applications that don't bother registering with the Internet Assigned Numbers Authority (IANA). Nmap prints this service name for reference along with the port number.

Because the port number field is 16-bits wide, values can reach 65,535. The lowest possible value, zero, is invalid. The Berkeley sockets API, which defines how programs are usually written for network communication, does not allow port zero to be used as such. Instead, it interprets a port zero request as a wildcard, meaning that the programmer does not care which is used. The system then chooses an available port number. For example, programmers rarely care what source port number is used for an outgoing connection. So they set it to zero and let the operating system choose one.

While port zero is invalid, nothing stops someone from specifying it in the header field. Some malicious trojan backdoors listen on port zero of compromised systems as a stealthy way to offer illegitimate access without appearing on most port scans. To combat this, Nmap does allow scanning of port zero when it is specified explicitly (e.g. `-p0-65535`).

The first class of valid ports, numbers one through 1,023, are known as reserved ports. Unix systems (unlike Windows) require that applications have special (root) privileges in order to

bind to and listen on these ports. The idea is to allow remote users to trust that they are connecting to a valid service started by an administrator and not by some wicked, unprivileged user. If the registered port for SSH was 2,222 instead of 22, a malicious user could start up a rogue SSH daemon on that port, collecting passwords from anyone who connects. As most common server applications listen on reserved ports, these are often the most fruitful to scan.

The ephemeral port range is another class of ports. This pool of ports is made available by the system for allocation as needed. When an application specifies port zero (meaning "any port"), the system chooses a port from this range. The range varies by operating system, and is usually configurable. It should contain at least a couple thousand ports to avoid running out when many concurrent connections are open. The Nmap connect scan can use hundreds at a time as it scans every specified port on each target machine. On Linux, you can view or set the range using the file `/proc/sys/net/ipv4/ip_local_port_range`. Example 4.1 shows that on my Linux system, the range is 32,768 to 61,000. Such a large range should be sufficient in almost all cases, but I expand it just to demonstrate how to do so.

Example 4.1. Viewing and increasing the ephemeral port range on Linux

```
felix/# cat /proc/sys/net/ipv4/ip_local_port_range
32768   61000
felix/# echo "10000 65000" > /proc/sys/net/ipv4/ip_local_port_range
felix/# cat /proc/sys/net/ipv4/ip_local_port_range
10000   65000
felix/#
```

SunRPC ports are often found in the ephemeral range. Other applications open ephemeral ports temporarily for a file transfer or other event. FTP clients often do this when requesting an active mode transfer. Some P2P and instant messaging clients do so as well.

The IANA has their own port classification scheme, which differs slightly from the vernacular of this book. Their authoritative port list at http://www.iana.org/assignments/port-numbers divides the space into the following three classes:

well-known ports

These are reserved ports (within the range of 1 to 1,023, as discussed above) which have been registered with the IANA for a certain service. Familiar examples are ports 22, 25, and 80 for the services SSH, SMTP, and HTTP, respectively.

registered ports

These ports fall within the range 1,024 to 49,151 and have been registered with the IANA in the same way the well known ports have. Most of these are not as commonly used as the well-known ports. The key difference is that unprivileged users can bind to these ports and thus run the services on their registered port. Users cannot do so on most platforms for well-known ports, since they reside in the reserved port range.

dynamic and/or private ports

The IANA reserves the port numbers from 49152 through 65535 for dynamic uses such as those discussed in the ephemeral ports section. Proprietary services that are only used within a company may also use these ports.

When this book mentions registered or well-known ports without any reference to the IANA, it usually means ports registered with Nmap in the `nmap-services` file, regardless of whether they fall in the reserved port range.

Nmap's port registration file (`nmap-services`) contains empirical data about how frequently each TCP or UDP port is found to be open. By default, Nmap scans the 1,000 most popular

ports of each protocol it is asked to scan. There are many options for specifying an alternate set of ports (by frequency or by listing them explicitly), as described in the section called "Selecting Ports to Scan".

What Are the Most Popular Ports?

I spent the Summer of 2008 scanning tens of millions of Internet hosts and collecting data from enterprises to determine how frequently each port number is found open. It is important to be familiar with the most common service ports, and also interesting to see which ones made the list. The following two lists provide the top TCP and UDP ports as determined by our empirical scan data. The listed service is the one found in our `nmap-services` file. We try to list the most common service for each port there, though of course it is possible for a port to be used for different things.

Top 20 (most commonly open) TCP ports

1. Port 80 (HTTP)—If you don't even know this service, you're reading the wrong book. This accounted for more than 14% of the open ports we discovered.
2. Port 23 (Telnet)—Telnet lives on (particularly as an administration port on devices such as routers and smart switches) even though it is insecure (unencrypted).
3. Port 443 (HTTPS)—SSL-encrypted web servers use this port by default.
4. Port 21 (FTP)—FTP, like Telnet, is another insecure protocol which should die. Even with anonymous FTP (avoiding the authentication sniffing worry), data transfer is still subject to tampering.
5. Port 22 (SSH)—Secure Shell, an encrypted replacement for Telnet (and, in some cases, FTP).
6. Port 25 (SMTP)—Simple Mail Transfer Protocol (also insecure).
7. Port 3389 (ms-term-server)—Microsoft Terminal Services administration port.
8. Port 110 (POP3)—Post Office Protocol version 3 for email retrieval (insecure).
9. Port 445 (Microsoft-DS)—For SMB communication over IP with MS Windows services (such as file/printer sharing).
10. Port 139 (NetBIOS-SSN)—NetBIOS Session Service for communication with MS Windows services (such as file/printer sharing). This has been supported on Windows machines longer than 445 has.
11. Port 143 (IMAP)—Internet Message Access Protocol version 2. An insecure email retrieval protocol.
12. Port 53 (Domain)—Domain Name System (DNS), an insecure system for conversion between host/domain names and IP addresses.
13. Port 135 (MSRPC)—Another common port for MS Windows services.
14. Port 3306 (MySQL)—For communication with MySQL databases.
15. Port 8080 (HTTP-Proxy)—Commonly used for HTTP proxies or as an alternate port for normal web servers (e.g. when another server is already listening on port 80, or when run by unprivileged UNIX users who can only bind to high ports).
16. Port 1723 (PPTP)—Point-to-point tunneling protocol (a method of implementing VPNs which is often required for broadband connections to ISPs).

17. Port 111 (RPCBind)—Maps SunRPC program numbers to their current TCP or UDP port numbers.
18. Port 995 (POP3S)—POP3 with SSL added for security.
19. Port 993 (IMAPS)—IMAPv2 with SSL added for security.
20. Port 5900 (VNC)—A graphical desktop sharing system (insecure).

Top 20 (most commonly open) UDP ports

1. Port 631 (IPP)—Internet Printing Protocol.
2. Port 161 (SNMP)—Simple Network Management Protocol.
3. Port 137 (NETBIOS-NS)—One of many UDP ports for Windows services such as file and printer sharing.
4. Port 123 (NTP)—Network Time Protocol.
5. Port 138 (NETBIOS-DGM)—Another Windows service.
6. Port 1434 (MS-SQL-DS)—Microsoft SQL Server.
7. Port 445 (Microsoft-DS)—Another Windows Services port.
8. Port 135 (MSRPC)—Yet Another Windows Services port.
9. Port 67 (DHCPS)—Dynamic Host Configuration Protocol Server (gives out IP addresses to clients when they join the network).
10. Port 53 (Domain)—Domain Name System (DNS) server.
11. Port 139 (NETBIOS-SSN)—Another Windows Services port.
12. Port 500 (ISAKMP)—The Internet Security Association and Key Management Protocol is used to set up IPsec VPNs.
13. Port 68 (DHCPC)—DHCP client port.
14. Port 520 (Route)—Routing Information Protocol (RIP).
15. Port 1900 (UPNP)—Microsoft Simple Service Discovery Protocol, which enables discovery of Universal plug-and-play devices.
16. Port 4500 (nat-t-ike)—For negotiating Network Address Translation traversal while initiating IPsec connections (during Internet Key Exchange).
17. Port 514 (Syslog)—The standard UNIX log daemon.
18. Port 49152 (Varies)—The first of the IANA-specified dynamic/private ports. No official ports may be registered from here up until the end of the port range (65536). Some systems use this range for their ephemeral ports, so services which bind a port without requesting a specific number are often allocated 49152 if they are the first program to do so.
19. Port 162 (SNMPTrap)—Simple Network Management Protocol trap port (An SNMP agent typically uses 161 while an SNMP manager typically uses 162).
20. Port 69 (TFTP)—Trivial File Transfer Protocol.

What is Port Scanning?

Port scanning is the act of remotely testing numerous ports to determine what state they are in. The most interesting state is usually open, meaning that an application is listening and

accepting connections on the port. Many techniques are available for conducting such a scan. Chapter 5, *Port Scanning Techniques and Algorithms* explains the circumstances under which each is most appropriate.

While many port scanners have traditionally lumped all ports into the open or closed states, Nmap is much more granular. It divides ports into six states. These states are not intrinsic properties of the port itself, but describe how Nmap sees them. For example, an Nmap scan from the same network as the target may show port `135/tcp` as open, while a scan at the same time with the same options from across the Internet might show that port as filtered.

The six port states recognized by Nmap

open

An application is actively accepting TCP connections or UDP packets on this port. Finding these is often the primary goal of port scanning. Security-minded people know that each open port is an avenue for attack. Attackers and pen-testers want to exploit the open ports, while administrators try to close or protect them with firewalls without thwarting legitimate users. Open ports are also interesting for non-security scans because they show services available for use on the network. Before you get too excited about an open port, note that it is possible that the application is protected with a TCP wrapper (tcpd) or that the application itself is configured to only service approved client IP addresses. Such cases still leave more attack surface than a closed port.

closed

A closed port is accessible (it receives and responds to Nmap probe packets), but there is no application listening on it. They can be helpful in showing that a host is online and using an IP address (host discovery, or ping scanning), and as part of OS detection. Because closed ports are reachable, they may be worth scanning later in case some open up. Administrators may want to consider blocking such ports with a firewall so they appear in the filtered state, discussed next.

filtered

Nmap cannot determine whether the port is open because packet filtering prevents its probes from reaching the port. The filtering could be from a dedicated firewall device, router rules, or host-based firewall software. These ports frustrate attackers because they provide so little information. Sometimes they respond with ICMP error messages such as type 3 code 13 (destination unreachable: communication administratively prohibited), but filters that simply drop probes without responding are far more common. This forces Nmap to retry several times just in case the probe was dropped due to network congestion rather than filtering. This sort of filtering slows scans down dramatically.

unfiltered

The unfiltered state means that a port is accessible, but Nmap is unable to determine whether it is open or closed. Only the ACK scan, which is used to map firewall rulesets, classifies ports into this state. Scanning unfiltered ports with other scan types such as Window scan, SYN scan, or FIN scan, may help resolve whether the port is open.

open|filtered

Nmap places ports in this state when it is unable to determine whether a port is open or filtered. This occurs for scan types in which open ports give no response. The lack of response could also mean that a packet filter dropped the probe or any response it elicited. So Nmap does not know for sure whether the port is open or being filtered. The UDP, IP protocol, FIN, NULL, and Xmas scans classify ports this way.

closed|filtered

This state is used when Nmap is unable to determine whether a port is closed or filtered. It is only used for the IP ID Idle scan discussed in the section called "TCP Idle Scan (`-sI`)".

While Nmap attempts to produce accurate results, keep in mind that all of its insights are based on packets returned by the target machines (or firewalls in front of them). Such hosts may be untrustworthy and send responses intended to confuse or mislead Nmap. Much more common are non-RFC-compliant hosts that do not respond as they should to Nmap probes. FIN, NULL, and Xmas scans are particularly susceptible to this problem. Such issues are specific to certain scan types and so are discussed in the relevant sections of Chapter 5, *Port Scanning Techniques and Algorithms*.

Why Scan Ports?

Port scanning is not only performed for fun and amusement. There are numerous practical benefits to regularly scanning your networks. Foremost among these is security. One of the central tenets of network security is that reducing the number and complexity of services offered reduces the opportunity for attackers to break in. Most remote network compromises come from exploiting a server application listening on a TCP or UDP port. In many cases, the exploited application is not even used by the targeted organization, but was enabled by default when the machine was set up. Had that service been disabled, or protected by a firewall, the attack would have been thwarted.

Realizing that every open port is an opportunity for compromise, attackers regularly scan targets, taking an inventory of all open ports. They compare this list of listening services with their list of favorite exploits for vulnerable software. It takes just one match to compromise a machine, creating a foothold that is often used to infest the whole network. Attackers who are less discriminate about who they target will often scan for just the default port of an exploitable application. This is much faster than scanning every port, though the service will be missed when running on a non-default port. Such attackers are often derided as "script kiddies", because they often know little more about security than how to run an exploit script written by someone more skilled. Across many organizations, such attackers are bound to find vulnerable hosts. They can be quite a nuisance, though their sheer numbers and relentless pounding against Internet-accessible machines often drive people to patch systems quickly. This reduces the likelihood of more serious, targeted attacks succeeding.

An important defense against these crackers is for systems administrators to scan their own networks regularly with tools such as Nmap. Take the list of open ports, and shut down any services that aren't used. Ensure that those which must remain available are fully patched and that you are on the vendor's security notification list. Firewall rules should be added where possible, limiting access to only legitimate users. Hardening instructions are available on the Web for most popular applications, reducing the cracker's opportunity even further. Nmap cannot do most of this for you, but it creates the list of available services to start out with. Some administrators try to use **`netstat`** instead, but that doesn't scale well. It requires access to every machine, and some mobile machines are easy to miss. Plus, you can't run **`netstat`** on your average wireless access point, VoIP phone, or printer. In addition, there is always the risk that a compromised machine will have a trojaned **`netstat`** which gives out false information. Most of the modern rootkits installed by attackers include this functionality. Relying solely on Nmap is a mistake too. A combination of careful design, configuration auditing, and regular scanning is well advised.

While security is the most common reason for port scanning, administrators often find that it suits other purposes as well. Creating an inventory of machines and the services they offer can

be useful for asset tracking, network design, policy compliance checks, software license tracking, availability testing, network debugging, and more.

A Quick Port Scanning Tutorial

Fuente original: https://nmap.org/book/port-scanning-tutorial.html

One of my goals in developing Nmap is to keep the most common usage simple, while retaining the flexibility for custom and advanced scans. This is accomplished with the command-line interface by offering dozens of options, but choosing sane defaults when they are not specified. A newbie can start out with a command as simple as **`nmap <target>`**. Meanwhile, advanced users sometimes specify so many options that their terminal line wraps around.

A similar balance must be struck with command output. The most important results should stick out even to the occasional user who hasn't even read the man page. Yet the output should be comprehensive and concise enough to suit professional penetration testers who run Nmap against thousands of machines daily. Users smart enough to read this book or the Nmap source code benefit from greater control of the scanner and insights into what Nmap output really means.

This tutorial demonstrates some common Nmap port scanning scenarios and explains the output. Rather than attempt to be comprehensive, the goal is simply to acquaint new users well enough to understand the rest of this chapter.

The simplest Nmap command is just **`nmap`** by itself. This prints a cheat sheet of common Nmap options and syntax. A more interesting command is **`nmap <target>`**, which does the following:

1. Converts *`<target>`* from a hostname into an IPv4 address using DNS. If an IP address is specified instead of a hostname this lookup is skipped.
2. Pings the host, by default with an ICMP echo request packet and a TCP ACK packet to port 80, to determine whether it is up and running. If not, Nmap reports that fact and exits. I could have specified `-Pn` to skip this test. See Chapter 3, *Host Discovery ("Ping Scanning")*.
3. Converts the target IP address back to the name using a reverse-DNS query. Because of the way DNS works, the reverse name may not be the same as the *`<target>`* specified on the command-line. This query can be skipped with the `-n` option to improve speed and stealthiness.
4. Launches a TCP port scan of the most popular 1,000 ports listed in `nmap-services`. A SYN stealth scan is usually used, but connect scan is substituted instead for non-root Unix users who lack the privileges necessary to send raw packets.
5. Prints the results to standard output in normal human-readable format, and exits. Other output formats and locations (files) can be specified, as described in Chapter 13, *Nmap Output Formats*. Example 4.2 displays the results when scanme.nmap.org is used as *`<target>`*.

Example 4.2. Simple scan: nmap scanme.nmap.org

```
# nmap scanme.nmap.org

Starting Nmap ( https://nmap.org )
Nmap scan report for scanme.nmap.org (64.13.134.52)
Not shown: 994 filtered ports
PORT    STATE  SERVICE
```

```
22/tcp  open   ssh
25/tcp  closed smtp
53/tcp  open   domain
70/tcp  closed gopher
80/tcp  open   http
113/tcp closed auth

Nmap done: 1 IP address (1 host up) scanned in 4.99 seconds
```

The first output line in Example 4.2 simply gives the URL for downloading Nmap. The time Nmap started and version number are normally provided as well, though these were generally removed from this book for consistency and to avoid line wrapping.

The next line provides the target IP address (IPv4 in this case), and reverse DNS name (also known as the PTR record) if it is available. Nmap promises to show the "interesting ports", though all ports scanned are accounted for. The ports considered most interesting because they are open or in a rarely-seen state for that host are itemized individually. When many ports are in a single non-open state, they are considered a default state, and aggregated onto a single line to avoid diluting the results with thousands of uninteresting entries. In this case, Nmap notes that 994 ports are filtered.

The interesting ports table comes next, and provides the key scan results. The columns vary depending on options used, but in this case provide the port number and protocol, state, and service protocol for each port. The service here is just a guess made by looking up the port in `nmap-services`. The service would be listed as `unknown` if any of the ports had no name registered in that file. Three of these ports are open and three are closed.

Finally, Nmap reports some basic timing stats before it exits. These stats are the number of targets specified, the number of those that the ping scan found to be up, and the total time taken.

While this simple command is often all that is needed, advanced users often go much further. In Example 4.3, the scan is modified with four options. `-p0-` asks Nmap to scan every possible TCP port, `-v` asks Nmap to be verbose about it, `-A` enables aggressive tests such as remote OS detection, service/version detection, and the Nmap Scripting Engine (NSE). Finally, `-T4` enables a more aggressive timing policy to speed up the scan.

Example 4.3. More complex: `nmap -p0- -v -A -T4 scanme.nmap.org`

```
# nmap -p0- -v -A -T4 scanme.nmap.org

Starting Nmap ( https://nmap.org )
Completed Ping Scan at 00:03, 0.01s elapsed (1 total hosts)
Scanning scanme.nmap.org (64.13.134.52) [65536 ports]
Discovered open port 22/tcp on 64.13.134.52
Discovered open port 53/tcp on 64.13.134.52
Discovered open port 80/tcp on 64.13.134.52
SYN Stealth Scan Timing: About 6.20% done; ETC: 00:11 (0:07:33 remaining)
Completed SYN Stealth Scan at 00:10, 463.55s elapsed (65536 total ports)
Completed Service scan at 00:10, 6.03s elapsed (3 services on 1 host)
Initiating OS detection (try #1) against scanme.nmap.org (64.13.134.52)
Initiating Traceroute at 00:10
64.13.134.52: guessing hop distance at 9
Completed SCRIPT ENGINE at 00:10, 4.04s elapsed
Host scanme.nmap.org (64.13.134.52) appears to be up ... good.
Nmap scan report for scanme.nmap.org (64.13.134.52)
Not shown: 65530 filtered ports
PORT    STATE  SERVICE VERSION
22/tcp  open   ssh     OpenSSH 4.3 (protocol 2.0)
25/tcp  closed smtp
53/tcp  open   domain  ISC BIND 9.3.4
70/tcp  closed gopher
80/tcp  open   http    Apache httpd 2.2.2 ((Fedora))
|_HTML title: Go ahead and ScanMe!
```

```
113/tcp closed auth
Device type: general purpose
Running: Linux 2.6.X
OS details: Linux 2.6.20-1 (Fedora Core 5)
Uptime guess: 2.457 days (since Thu Sep 18 13:13:24 2008)
TCP Sequence Prediction: Difficulty=204 (Good luck!)
IP ID Sequence Generation: All zeros

TRACEROUTE (using port 80/tcp)
HOP RTT   ADDRESS
[First eight hops cut for brevity]
9   10.36 metro0.sv.svcolo.com (208.185.168.173)
10  10.29 scanme.nmap.org (64.13.134.52)

Nmap done: 1 IP address (1 host up) scanned in 477.23 seconds
           Raw packets sent: 131432 (5.783MB) | Rcvd: 359 (14.964KB)
```

Nmap certainly provided the requested verbosity in Example 4.3! Fortunately the extra output is easy to understand. The first 13 new lines are runtime information letting the user know what is happening as she stares expectantly at the terminal, hoping for good news. What constitutes good news depends on whether she is a systems administrator who has to fix problems, a pen-tester who needs some issues to report on, or a black-hat cracker trying to exploit them. About a dozen similar lines were removed for brevity. The "discovered open port" lines provide as-it-happens notification of open ports so that she can start banging on them before the scan even finishes. The "scan timing" line provides a completion time estimate, so she knows whether to keep staring at the screen or have lunch. Since network conditions (latency, congestion, bandwidth, etc.) and packet filtering rules vary so much, the same scan options may take 30 seconds to complete against one host and 45 minutes against another. If you want the current time estimate while scanning, just press enter.

The port table shows no new ports. All the extra ports scanned are in the filtered state, raising the filtered port total from 994 to 65,530. While there are no new itemized ports, the entries have changed. A new `VERSION` column provides the application name and version details for the listening service. This comes from service detection, one of the features enabled by the `-A` option. Another feature of service detection is that all of the service protocols in the `SERVICE` column have actually been verified. In the previous scan, they were based on the relatively flimsy heuristic of an `nmap-services` port number lookup. That table lookup happened to be correct this time, but it won't always be.

Another feature added by `-A` is the Nmap Scripting Engine, which is discussed in depth in Chapter 9, *Nmap Scripting Engine*. The only script shown here is `HTML title`. Dozens of other scripts exist, but none found useful output for this machine. The traceroute results were also added by `-A`. This option is more efficient and more powerful than most traceroute programs since probes are performed in parallel and Nmap uses scan results to determine a favorable probe type (TCP packets to port 80 in this case).

Most of the remaining new lines come from OS detection (also enabled by `-A`), which is discussed in depth in Chapter 8, *Remote OS Detection*. The final line shows that all this extra info came at a price—the scan took almost 100 times longer than Example 4.2, "Simple scan: nmap scanme.nmap.org" to complete (477 seconds compared to 5).

Command-line Flags

Fuente original: https://nmap.org/book/port-scanning-options.html

While the tutorial showed how simple executing an Nmap port scan can be, dozens of command-

line flags are available to make the system more powerful and flexible. This section covers only options that relate to port scans, and often describes only the port-scanning-related functionality of those options. See Chapter 15, *Nmap Reference Guide* for a comprehensive list of option flags and everything they do.

Selecting Scan Techniques

One of the first considerations when contemplating a port scan is deciding what techniques to use. Nmap offers about a dozen such methods and this section provides a brief summary of them. Full coverage comes in the next chapter. Only one scan method may be used at a time, except that UDP scan (`-sU`) may be combined with any one of the TCP scan types. As a memory aid, port scan type options are of the form `-s<C>`, where `<C>` is a prominent character in the scan name, usually the first. The one exception to this is the deprecated FTP bounce scan (`-b`). By default, Nmap performs a SYN Scan, though it substitutes a connect scan if the user does not have proper privileges to send raw packets (requires root access on Unix) or if IPv6 targets were specified.

Port scanning methods supported by Nmap

the section called "TCP SYN (Stealth) Scan (`-sS`)" (`-sS`)

This is far and away the most popular scan type because it the fastest way to scan ports of the most popular protocol (TCP). It is stealthier than connect scan, and it works against all functional TCP stacks (unlike some special-purpose scans such as FIN scan).

the section called "TCP Connect Scan (`-sT`)" (`-sT`)

Connect scan uses the system call of the same name to scan machines, rather than relying on raw packets as most of the other methods do. It is usually used by unprivileged Unix users and against IPv6 targets because SYN scan doesn't work in those cases.

the section called "UDP Scan (`-sU`)" (`-sU`)

Don't forget UDP ports—they offer plenty of security holes too.

the section called "TCP FIN, NULL, and Xmas Scans (`-sF`, `-sN`, `-sX`)" (`-sF`, `-sX`, `-sN`)

These special purpose scan types are adept at sneaking past firewalls to explore the systems behind them. Unfortunately they rely on target behavior that some systems (particularly Windows variants) don't exhibit.

the section called "TCP ACK Scan (`-sA`)" (`-sA`)

ACK scan is commonly used to map out firewall rulesets. In particular, it helps understand whether firewall rules are stateful or not. The downside is that it cannot distinguish open from closed ports.

the section called "TCP Window Scan (`-sW`)" (`-sW`)

Window scan is like ACK scan, except that it is able to detect open versus closed ports against certain machines.

the section called "TCP Maimon Scan (`-sM`)" (`-sM`)

This obscure firewall-evading scan type is similar to a FIN scan, but includes the ACK flag as well. This allows it to get by more packet filtering firewalls, with the downside that it works against even fewer systems than FIN scan does.

the section called "TCP Idle Scan (-sI)" (-sI <*zombie host*>)

Idle scan is the stealthiest scan type of all, and can sometimes exploit trusted IP address relationships. Unfortunately, it is also slow and complex.

the section called "IP Protocol Scan (-sO)" (-sO)

Protocol scan determines which IP protocols (TCP, ICMP, IGMP, etc.) are supported by the target machine. This isn't technically a port scan, since it cycles through IP protocol numbers rather than TCP or UDP port numbers. Yet it still uses the `-p` option to select scanned protocol numbers, reports its results with the normal port table format, and even uses the same underlying scan engine as the true port scanning methods. So it is close enough to a port scan that it belongs here.

the section called "TCP FTP Bounce Scan (-b)" (-b <*FTP bounce proxy*>)

This deprecated scan type tricks FTP servers into performing port scans by proxy. Most FTP servers are now patched to prevent this, but it is a good way to sneak through restrictive firewalls when it works.

Selecting Ports to Scan

Nmap's port registration file (`nmap-services`) contains empirical data about how frequently each TCP or UDP port is found to be open. This data was collected by scanning tens of millions of Internet addresses, then combining those results with internal scan data contributed by large enterprises. By default, Nmap scans the 1,000 most popular ports of each protocol it is asked to scan. Alternatively, you can specify the `-F` (fast) option to scan only the 100 most common ports in each protocol or `--top-ports` to specify an arbitrary number of ports to scan.

When none of these canned port sets suit your needs, an arbitrary list of port numbers can be specified on the command-line with the `-p` option. The syntax of the `-p` option can be complex, and is best described with examples.

Port selection examples with the -p option

`-p 22`

Scan a single port (in this case port 22) by specifying just that number as the `-p` argument.

`-p ssh`

Port names may be specified rather than numbers. Note that a name may match multiple ports.

`-p 22,25,80`

Multiple ports may be separated with commas. Note that no protocol is specified, so these same port numbers will be used for whatever scan methods are specified on the command-line. If a TCP scan such as SYN scan (`-sS`) is specified, TCP ports 22, 25, and 80 are scanned. Those correspond to the services SSH, SMTP, and HTTP, respectively. If a UDP scan is selected (`-sU`), those three UDP ports are scanned. If both are specified, those three ports are scanned for each protocol, for a total of six scanned ports. With IP protocol scan (`-sO`), those three IP protocols (corresponding to XNS IDP, Leaf-1, and ISO-IP) are scanned.

`-p80-85,443,8000-8005,8080-8085`

Port ranges may be specified by separating the beginning and end port with a hyphen. Multiple ranges or individual ports can be specified with commas. This option scans ports 80, 81, 82, 83, 84, 85, 443, 8000, etc. Based on the port numbers, this user is probably scanning TCP and looking for web servers.

`-p-100,60000-`

You can omit the beginning of a range to imply port one, or the end to imply the last port possible (65535 for TCP and UDP, 255 for protocol scan). This example scans ports one through 100, and all ports greater or equal to 60,000.

`-p-`

Omit beginning and end numbers to scan the whole range (excluding zero).

`-pT:21,23,110,U:53,111,137,161`

Separate lists of TCP and UDP ports can be given by preceding the lists with T: (for TCP) or U:. This example scans three TCP ports (FTP, Telnet, and POP3), and four UDP services (DNS, rpcbind, NetBIOS, and SNMP). Specifying both TCP and UDP ports only matters if you also tell Nmap to do a UDP scan (`-sU`) and one of the TCP scan methods, such as `-sS`, `-sA`, or `-sF`.

`-p http*`

Wildcards may be used to match ports with similar names. This expression matches eight port numbers, including http (80), http-mgmt (280), https (443), and http-proxy (8080). Depending on your command shell, you may need to escape the asterisk so it isn't treated as a filename glob.

`-p 1-1023,[1024-]`

Enclosing a range in brackets causes those port numbers to be scanned only if they are registered in `nmap-services`. In this example, all the reserved ports (1–1,023), plus all the higher ports registered in `nmap-services`. That was Nmap's default behavior before `nmap-services` was augmented with open port frequency data for more precise selection.

Timing-related Options

Port scanning is often the most time consuming part of an Nmap scan (which might also include OS detection, version detection, and NSE scripts). While Nmap tries to be quick and efficient by default, manual optimization often helps. Nmap offers dozens of options for tailoring scan intensity and speed to match your exact needs. This section lists the most important options for optimizing port scan times. Options which take an amount of time are in seconds by default, or you may append `ms` (milliseconds), `s` (seconds), `m` (minutes), or `h` (hours) to the value. For further details on any of these options, see the section called "Timing and Performance". A much more thorough treatment, with examples and best-practices for improving Nmap performance is available in Chapter 6, *Optimizing Nmap Performance*.

Top port scan performance options

`-T0` through `-T5`

These timing templates affect many variables, offering a simple way to adjust overall Nmap speed from very slow (`-T0`) to extremely aggressive (`-T5`). A timing template may be combined with the more granular options describe below, and the most granular option takes precedence.

`--min-rtt-timeout, --max-rtt-timeout, --initial-rtt-timeout`

The minimum, maximum, and initial amount of time that Nmap will wait for a port scan probe response.

`--host-timeout`

Asks Nmap to give up on hosts that take more than the given amount of time to scan.

`--min-rate, --max-rate`

Sets the floor and ceiling, respectively, to the number of probe packets Nmap sends per second.

`--max-retries`

Specifies the maximum number of port scan probe retransmissions to a single port.

`--min-hostgroup, --max-hostgroup`

Sets the minimum and maximum number of hosts that Nmap will port scan in parallel.

`--min-parallelism, --max-parallelism`

Limits the minimum or maximum number of port scan probes (across all hosts scanned concurrently) that Nmap may have outstanding.

`--scan-delay, --max-scan-delay`

Asks Nmap to wait at least the given amount of time between sending probes to any individual host. The scan delay can grow as Nmap detects packet loss, so a maximum may be specified with `--max-scan-delay`.

Output Format and Verbosity Options

Nmap offers the ability to write its reports in its standard format, a simple line-oriented "grepable" format, or XML. These reports are enabled with the `-oN` (normal), `-oG` (grepable), and `-oX` (XML) options. Each option takes a filename, and they may be combined to output in several formats at once. Several options are also available to increase output verbosity. This section lists the most important output-related options and how they apply to port scanning. For further details on any of these options, see the section called "Output". A much more thorough treatment of output options and formats, with many examples, is available in Chapter 13, *Nmap Output Formats*.

Top Nmap output options applicable to port scans

`-v`

Increases the verbosity level, causing Nmap to print more information about the scan in progress. Open ports are shown as they are found and completion time estimates are provided when Nmap thinks a scan will take more than a few minutes. Use it twice or more for even greater verbosity.

`-d`

Increases the debugging level, causing Nmap to print out details about its operation that can be useful for tracking down bugs or simply understanding how it works. Higher levels result in massive amounts of data. Using the option once sets the debugging level to one, and it is incremented for each additional `-d`. Or you may follow the `-d` with the desired level, as in `-d5`. If you don't see enough information, try a higher level. The maximum effective level is nine. If your screen is flooded with too much debugging data, reduce the level. Reducing scan intensity, such as the number of ports or targets scanned and the features used, can also help to isolate only the debug messages you want.

`--packet-trace`

Causes Nmap to print a summary of every packet sent or received. This is often used for debugging, but is also a valuable way for new users to understand exactly what Nmap is doing under the covers. To avoid printing thousands of lines, you may want to specify a limited number of ports to scan, such as `-p20-30`.

`-oN <filename>` (normal output)

Write output in Nmap's normal format to *`<filename>`*. This format is roughly the same as the standard interactive output printed by Nmap at runtime.

`-oX <filename>` (XML output)

Write output in Nmap's XML format to *`<filename>`*. Normal (human readable) output will still be printed to stdout unless you ask for XML to be directed there by specifying `-` as *`<filename>`*. This is the preferred format for use by scripts and programs that process Nmap results.

`-oG <filename>` (grepable format output)

Write output in Nmap's so-called grepable format to *`<filename>`*. This tabular format fits the output of each host on a single line, making it easy to grep for open ports, certain operating systems, application names, or other data. Normal output will still be printed to stdout unless you ask for the grepable output to be directed there by specifying `-` as *`<filename>`*. While this format works well for parsing with simple grep and awk command-lines, significant scripts and programs should use the XML output instead. The XML format contains substantial information that grepable format has no place for, and extensibility makes XML easier to update with new information without breaking tools that rely on it.

`-oA <basename>` (output to all formats)

As a convenience, you may specify `-oA` *`<basename>`* to store scan results in normal, XML, and grepable formats at once. They are stored in *`<basename>`*.nmap, *`<basename>`*.xml, and *`<basename>`*.gnmap, respectively. As with most programs, you can prefix the filenames with a directory path, such as `~/nmaplogs/foocorp/` on Unix or `c:\hacking\sco` on Windows.

`--resume <filename>`

Resume an aborted scan by specifying the normal (`-oN`) or grepable (`-oG`) output file which was created during the ill-fated scan. Don't use any options other than `--resume`, as Nmap will use the ones specified in the output file. It then parses the file and resumes scanning (and logging to the file) at the host which the previous Nmap execution was working on when it ceased.

`--append-output`

Tells Nmap to append scan results to any output files specified (with arguments such as `-oN` or `-oX`) rather than overwriting them.

`--open`

Only show hosts that have open ports, and only show the open ports for those. Here, "open ports" are any ports that have the possibility of being open, which includes `open`, `open|filtered`, and `unfiltered`.

Firewall and IDS Evasion Options

Nmap offers many options for sneaking past IDSs undetected or evading firewall rules. For an overview, see the section called "Firewall/IDS Evasion and Spoofing". For a comprehensive look at firewall and IDS evasion techniques, along with practical examples, see Chapter 10, *Detecting and Subverting Firewalls and Intrusion Detection Systems*.

Specifying Targets

To scan a single host (or a few of them), simply add their names or IP addresses to the end of your Nmap command line. Nmap also has a structured syntax to make scanning large networks easy. You can give Nmap a file listing targets, or even ask Nmap to generate them randomly. This is all described in the section called "Specifying Target Hosts and Networks".

Miscellaneous Options

Here are some options that can be quite handy even though they don't fit into specific categories. The descriptions focus on how each option relates to port scanning. See the Chapter 15, *Nmap Reference Guide* for more comprehensive coverage of each option.

`-6`

Asks Nmap to scan the target using the IPv6 protocol. This process is described in the section called "IPv6 Scanning (`-6`)".

`-r`

Nmap randomizes the port scan order by default to make detection slightly harder. The `-r` option causes them to be scanned in numerical order instead.

`-Pn`

Tells Nmap to skip the ping test and simply scan every target host provided. Other options for controlling host discovery are described in Chapter 3, *Host Discovery ("Ping Scanning")*.

`--reason`

Adds a column to the interesting ports table which describes why Nmap classified a port as it did.

IPv6 Scanning (`-6`)

Fuente original: https://nmap.org/book/port-scanning-ipv6.html

Since 2002, Nmap has offered IPv6 support for its most popular features. In particular, ping scanning (TCP-only), connect scanning, and version detection all support IPv6. The command syntax is the same as usual except that you also add the `-6` option. Of course, you must use IPv6 syntax if you specify an address rather than a hostname. An address might look like `3ffe:7501:4819:2000:210:f3ff:fe03:14d0`, so hostnames are recommended. Example 4.4 shows a typical port scanning session. The output looks the same as it usually does, with the IPv6 address on the "interesting ports" line being the only IPv6 give away.

Example 4.4. A simple IPv6 scan

```
# nmap -6 -sV www.eurov6.org

Starting Nmap ( https://nmap.org )
Nmap scan report for ns1.euro6ix.com (2001:800:40:2a03::3)
Not shown: 996 closed ports
PORT   STATE SERVICE VERSION
21/tcp open  ftp     Pure-FTPd
22/tcp open  ssh     OpenSSH 3.5p1 (protocol 2.0)
53/tcp open  domain  ISC BIND 9.2.1
80/tcp open  http    Apache httpd

Nmap done: 1 IP address (1 host up) scanned in 56.78 seconds
```

While IPv6 hasn't exactly taken the world by storm, it gets significant use in some countries and most modern operating systems support it. To use Nmap with IPv6, both the source and target of your scan must be configured for IPv6. If your ISP (like most of them) does not allocate IPv6 addresses to you, free tunnel brokers are widely available and work fine with Nmap. I use the free IPv6 tunnel broker service at `http://www.tunnelbroker.net`. Other tunnel brokers are listed at Wikipedia. 6to4 tunnels are another popular, free approach.

Systems that support IPv6 don't always have their IPv4 and IPv6 firewall rules in sync. the section called "IPv6 Attacks" shows a real-life example of reaching ports through IPv6 that are filtered in IPv4.

SOLUTION: Scan a Large Network for a Certain Open TCP Port

Fuente original: `https://nmap.org/book/solution-find-open-port.html`

Problem

You wish to quickly find all machines on a network that have a certain TCP port open. For example, after a new Microsoft IIS vulnerability is found, you might want to scan for all machines with TCP port 80 open and ensure that they aren't running a vulnerable version of that software. Or if you investigate a compromised box and find that the attacker left a backdoor running on port 31337, scanning your whole network for that port might quickly identify other compromised systems. A full (all ports) scan would be done later.

Solution

The straightforward way is to run:

```
nmap -Pn -p<portnumber> -oG <logfilename.gnmap> <target networks>
```

Here is a concrete example of searching 4096 IPs for web servers (port 80 open):

```
nmap -Pn -p80 -oG logs/pb-port80scan-%D.gnmap 216.163.128.0/20
```

The "%D" in the filename is replaced with the numeric date on which the scan was run (e.g. "090107" on September 1, 2007). While this scan command works, a little effort choosing appropriate timing values for the network being scanned reduces scan time substantially. The scan above took 1,236 seconds, while the optimized version below provided the same results in 869 seconds:

```
nmap -T4 -Pn -p80 --max-rtt-timeout 200ms --initial-rtt-timeout 150ms
--min-hostgroup 512 -oG logs/pb-port80scan2-%D.gnmap 216.163.128.0/20
```

And much of that time is spent doing reverse-DNS resolution. Excluding that by adding `-n` to the command-line above reduces the 4096-host scan time to 193 seconds. Being patient for three minutes is far easier than for the 21 minutes taken before.

The commands above store grepable-format results in the specified file. A simple egrep command will then find the machines with port 80 open:

```
egrep '[^0-9]80/open' logs/pb-port80scan2-*.gnmap
```

The egrep pattern is preceded with [^0-9] to avoid bogus matching ports such as 3180. Of course that can't happen since we are only scanning port 80, but it is a good practice to remember for

many-port scans. If you only want the IP addresses and nothing else, pipe the egrep output to `awk '{print $2}'`.

Discussion

Sometimes a story is the best way to understand decisions, such as how I decided upon the command lines in the solution section. I was bored at home, and started exploring the network of a popular magazine named *Playboy*. Their main site includes a huge trove of images, but most are locked away behind a paid subscription authentication system. I was curious as to whether I could find any other systems on their network which offer up images for free. I figured that they might have staging or development servers which rely on obscurity rather than password authentication. While such servers could theoretically listen on any port number, the most likely is TCP port 80. So I decide to scan their whole network for that open port as quickly as possible.

The first step is determining which IP addresses to scan. I perform a whois search of the American Registry for Internet Numbers (ARIN) for organizations named Playboy. The results are shown in Example 4.5.

Example 4.5. Discovering Playboy's IP space

```
core~> whois -h whois.arin.net n playboy
[Querying whois.arin.net]
[whois.arin.net]

OrgName:    Playboy
OrgID:      PLAYBO
Address:    680 N. Lake Shore Drive
City:       Chicago
StateProv:  IL
PostalCode: 60611
Country:    US

NetRange:   216.163.128.0 - 216.163.143.255
CIDR:       216.163.128.0/20
NetName:    PLAYBOY-BLK-1
NetHandle:  NET-216-163-128-0-1
Parent:     NET-216-0-0-0-0
NetType:    Direct Assignment
NameServer: NS1-CHI.PLAYBOY.COM
NameServer: NS2-CHI.PLAYBOY.COM
[...]
```

This shows 4096 IPs (the net range 216.163.128.0/20) registered to Playboy. Using techniques discussed in the section called "Finding an Organization's IP Addresses" I could have found many more netblocks they control, but 4096 IPs are sufficient for this example.

Next I want to estimate latency to these machines, so that Nmap will know what to expect. This isn't required, but feeding Nmap appropriate timing values can speed it up. This is particularly true for single-port `-Pn` scans, such as this one. Nmap does not receive enough responses from each host to accurately estimate latency and packet drop rate, so I will help it out on the command line. My first thought is to ping their main web server, as shown in Example 4.6.

Example 4.6. Pinging Playboy's web server for a latency estimate

```
# ping -c5 www.playboy.com
PING www.phat.playboy.com (209.247.228.201) from 205.217.153.56
64 bytes from free-chi.playboy.com (209.247.228.201): icmp_seq=1 time=57.5 ms
64 bytes from free-chi.playboy.com (209.247.228.201): icmp_seq=2 time=56.7 ms
64 bytes from free-chi.playboy.com (209.247.228.201): icmp_seq=3 time=56.9 ms
```

```
64 bytes from free-chi.playboy.com (209.247.228.201): icmp_seq=4 time=57.0 ms
64 bytes from free-chi.playboy.com (209.247.228.201): icmp_seq=5 time=56.6 ms

--- www.phat.playboy.com ping statistics ---
5 packets transmitted, 5 received, 0% loss, time 4047ms
rtt min/avg/max/mdev = 56.652/57.004/57.522/0.333 ms
```

The maximum round trip time is 58 milliseconds. Unfortunately, this IP address (209.247.228.201) is not within the 216.163.128.0/20 netblock I wish to scan. I would normally add this new netblock to the target list, but have already decided to limit my scan to the original 4096 IPs. These times are probably perfectly fine to use, but finding actual values from IPs on the target network would be even better. I use dig to obtain Playboy's public DNS records from a nameserver shown in the previous whois query. The output is shown in Example 4.7.

Example 4.7. Digging through Playboy's DNS records

```
core~> dig @ns1-chi.playboy.com playboy.com. any
; <<>> DiG 8.3 <<>> @ns1-chi.playboy.com playboy.com. any
[...]
;; ANSWER SECTION:
playboy.com.            1D IN A         209.247.228.201
playboy.com.            1D IN MX        10 mx.la.playboy.com.
playboy.com.            1D IN MX        5 mx.chi.playboy.com.
playboy.com.            1D IN NS        ns15.customer.level3.net.
playboy.com.            1D IN NS        ns21.customer.level3.net.
playboy.com.            1D IN NS        ns29.customer.level3.net.
playboy.com.            1D IN NS        ns1-chi.playboy.com.
playboy.com.            1D IN NS        ns2-chi.playboy.com.
playboy.com.            1D IN SOA       ns1-chi.playboy.com. dns.playboy.com. (
                                        2004092010      ; serial
                                        12H             ; refresh
                                        2h30m           ; retry
                                        2w1d            ; expiry
                                        1D )            ; minimum

;; ADDITIONAL SECTION:
mx.chi.playboy.com.     1D IN A         216.163.143.4
mx.la.playboy.com.      1D IN A         216.163.128.15
ns1-chi.playboy.com.    1D IN A         209.247.228.135
ns2-chi.playboy.com.    1D IN A         64.202.105.36

;; Total query time: 107 msec
```

The DNS query reveals two MX (mail) servers within the target 216.163.128.0/20 netblock. Since the names `mx.chi` and `mx.la` imply that they are in different regions (Chicago and Los Angeles), I decide to test them both for latency. The **`ping`** results are shown in Example 4.8.

Example 4.8. Pinging the MX servers

```
core~> ping -c5 mx.chi.playboy.com
PING mx.chi.playboy.com (216.163.143.4) 56(84) bytes of data.

--- mx.chi.playboy.com ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4000ms

core~> ping -c5 mx.la.playboy.com
PING mx.la.playboy.com (216.163.128.15) 56(84) bytes of data.

--- mx.la.playboy.com ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4011ms
```

Well, that attempt was a miserable failure! The hosts seem to be blocking ICMP ping packets. Since they are mail servers, they must have TCP port 25 open, so I try again using hping2 to perform a TCP ping against port 25, as demonstrated in Example 4.9.

Example 4.9. TCP pinging the MX servers

```
core# hping2 --syn -p 25 -c 5 mx.chi.playboy.com
eth0 default routing interface selected (according to /proc)
HPING mx.chi.playboy.com (eth0 216.163.143.4): S set, 40 headers + 0 data bytes
46 bytes from 216.163.143.4: flags=SA seq=0 ttl=51 id=14221 rtt=56.8 ms
46 bytes from 216.163.143.4: flags=SA seq=1 ttl=51 id=14244 rtt=56.9 ms
46 bytes from 216.163.143.4: flags=SA seq=2 ttl=51 id=14274 rtt=56.9 ms
46 bytes from 216.163.143.4: flags=SA seq=3 ttl=51 id=14383 rtt=61.8 ms
46 bytes from 216.163.143.4: flags=SA seq=4 ttl=51 id=14387 rtt=57.5 ms

--- mx.chi.playboy.com hping statistic ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max = 56.8/58.0/61.8 ms

core# hping2 --syn -p 25 -c 5 mx.la.playboy.com
eth0 default routing interface selected (according to /proc)
HPING mx.la.playboy.com (eth0 216.163.128.15): S set, 40 headers + 0 data bytes
46 bytes from 216.163.128.15: flags=SA seq=0 ttl=52 id=58728 rtt=16.0 ms
46 bytes from 216.163.128.15: flags=SA seq=1 ttl=52 id=58753 rtt=15.4 ms
46 bytes from 216.163.128.15: flags=SA seq=2 ttl=52 id=58790 rtt=15.5 ms
46 bytes from 216.163.128.15: flags=SA seq=3 ttl=52 id=58870 rtt=16.4 ms
46 bytes from 216.163.128.15: flags=SA seq=4 ttl=52 id=58907 rtt=15.5 ms

--- mx.la.playboy.com hping statistic ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max = 15.4/15.8/16.4 ms
```

These are the results I was looking for. The LA host never takes more than 16 milliseconds to respond, while the Chicago one takes up to 62 milliseconds. This is not surprising, given that I am probing from a machine in California. It pays to be cautious, and latency can increase during heavy scanning, so I decide to let Nmap wait up to 200 milliseconds for responses. I'll have it start with a timeout of 150 ms. So I pass it the options `--max-rtt-timeout 200ms --initial-rtt-timeout 150ms`. To set a generally aggressive timing mode, I specify `-T4` at the beginning of the line.

Since I value minimizing completion time of the whole scan over minimizing the amount of time before the first batch of host results is returned, I specify a large scan group size. The option `--min-hostgroup 512` is specified so that at least 512 IPs will be scanned in parallel (when possible). Using an exact factor of the target network size (4096) prevents the small and less efficient 96-host block which would occur at the end if I specified `--min-hostgroup 500`. All of these timing issues are explained in much more depth in Chapter 6, *Optimizing Nmap Performance*.

There is no need to waste time with a prior ping stage, since a ping would take as long as the single-port scan itself. So `-Pn` is specified to disable that stage. Substantial time is saved by skipping reverse-DNS resolution with the `-n` argument. Otherwise, with ping scanning disabled, Nmap would try to look up all 4096 IPs. I am searching for web servers, so I request port 80 with `-p80`. Of course I will miss any HTTP servers running on non-standard ports such as 81 or 8080. SSL servers on port 443 won't be found either. One could add them to the `-p` option, but even one more port would double the scan time, which is roughly proportional to the number of ports scanned.

The final option is `-oG` followed by the filename in which I want grepable results stored. I append the target network to the command, then press enter to execute Nmap. The output is shown in Example 4.10.

Example 4.10. Launching the scan

```
# nmap -T4 -p80 -Pn --max-rtt-timeout 200ms --initial-rtt-timeout 150ms \
```

```
  --min-hostgroup 512 -n -oG pb-port80scan-%D.gnmap 216.163.128.0/20
Warning: You specified a highly aggressive --min-hostgroup.
Starting Nmap ( https://nmap.org )
Nmap scan report for 216.163.128.0
PORT   STATE    SERVICE
80/tcp filtered http

Nmap scan report for 216.163.128.1
PORT   STATE    SERVICE
80/tcp filtered http

Nmap scan report for 216.163.128.2
PORT   STATE    SERVICE
80/tcp filtered http

Nmap scan report for 216.163.128.3
PORT   STATE    SERVICE
80/tcp filtered http
[ ... ]
Nmap scan report for 216.163.143.255
PORT   STATE    SERVICE
80/tcp filtered http

Nmap done: 4096 IP addresses (4096 hosts up) scanned in 192.97 seconds
```

Nmap scans all 4096 IPs in about three minutes. The normal output shows a bunch of ports in the `filtered` state. Most of those IPs are probably not active hosts—the port simply appears filtered because Nmap receives no response to its SYN probes. I obtain the list of web servers with a simple **`egrep`** on the output file, as shown in Example 4.11.

Example 4.11. Egrep for open ports

```
# egrep '[^0-9]80/open' pb-port80scan-*.gnmap
Host: 216.163.140.20 () Ports: 80/open/tcp//http///
Host: 216.163.142.135 ()     Ports: 80/open/tcp//http///
```

After all that effort, only two accessible web servers are found out of 4096 IPs! Sometimes that happens. The first one, 216.163.140.20 (no reverse DNS name) brings me to a Microsoft Outlook Web Access (webmail) server. That might excite me if I was trying to compromise their network, but it isn't gratifying now. The next server (reverse name mirrors.playboy.com) is much better. It offers those gigabytes of free images I was hoping for! In particular it offers Linux ISO images as well as substantial FreeBSD, CPAN, and Apache archives! I download the latest Fedora Core ISOs at a respectable 6 Mbps. The abundance of bandwidth at Playboy is not surprising. Later I scan other Playboy netblocks, finding dozens more web servers, though some of their content is inappropriate for this book.

While this is an unusual reason for port scanning, single port sweeps are common for many other purposes expressed previously. The techniques described here can be easily applied to any single-port TCP sweep.

See Also

Version detection can be used to find specific applications listening on a network. For example, you could seek a certain vulnerable version of OpenSSH rather than find all hosts with port 22 open. This is also useful for single-port UDP scans, as the techniques in this solution only work well for TCP. Instructions are provided in the section called "SOLUTION: Find All Servers Running an Insecure or Nonstandard Application Version".

Chapter 6, *Optimizing Nmap Performance* looks at scan speed optimization in much more depth.

Chapter 5

Port Scanning Techniques and Algorithms

Fuente original: https://nmap.org/book/scan-methods.html

Introduction

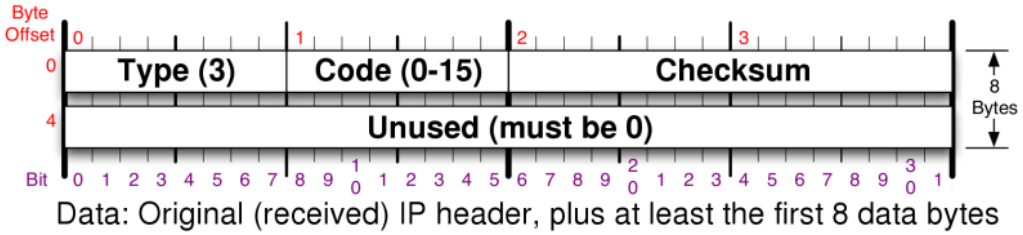
As a novice performing automotive repair, I can struggle for hours trying to fit my rudimentary tools (hammer, duct tape, wrench, etc.) to the task at hand. When I fail miserably and tow my jalopy to a real mechanic, he invariably fishes around in a huge tool chest until pulling out the perfect gizmo which makes the job seem effortless. The art of port scanning is similar. Experts understand the dozens of scan techniques and choose the appropriate one (or combination) for a given task. Inexperienced users and script kiddies, on the other hand, try to solve every problem with the default SYN scan. Since Nmap is free, the only barrier to port scanning mastery is knowledge. That certainly beats the automotive world, where it may take great skill to determine that you need a strut spring compressor, then you still have to pay thousands of dollars for it.

The previous chapter described port scanning with Nmap in general terms, including a brief summary of Nmap's supported scan types in the section called "Selecting Scan Techniques". This chapter describes each of those scan types in depth. Typical usage scenarios and instructions are given for each scan type, as are on-the-wire packet traces illustrating how they work. Then the `ultra_scan` algorithm (which most scan methods use) is discussed, with an emphasis on aspects that can be tweaked to improve performance.

Most of the scan types are only available to privileged users. This is because they send and receive raw IP packets, (or even ethernet frames) which requires root access on Unix systems. Using an administrator account on Windows is not necessary, since Nmap works for unprivileged users on that platform when Npcap has already been loaded into the OS. Requiring root privileges was a serious limitation when Nmap was released in 1997, as many users only had access to shared shell accounts. Now, the world is different. Computers are cheaper, far more people have always-on direct Internet access, and desktop Unix systems (including Linux and Mac OS X) are prevalent. A Windows version of Nmap is now available, allowing it to run on even more desktops. For all these reasons, users rarely need to run Nmap from limited shared shell accounts. This is fortunate, as the privileged options make Nmap far more powerful and flexible.

When discussing how Nmap handles probe responses, many sections discuss ICMP error messages by their type and code numbers. The type and code are each eight-bit fields in ICMP headers that describe the message's purpose. Nmap port scanning techniques are concerned only with ICMP type 3, which are destination unreachable messages. Figure 5.1 shows the ICMP header layout of such a packet (it is encapsulated in the data section of an IP packet, as shown in Figure 1, "IPv4 header").

Figure 5.1. ICMPv4 destination unreachable header layout



There are sixteen codes representing different destination unreachable messages. They are all shown in Table 5.1, though Nmap only cares about codes 0–3, 9, 10, and 13, which are marked with an asterisk.

Table 5.1. ICMP destination unreachable (type 3) code values

Code	Description
0*	Network unreachable
1*	Host unreachable
2*	Protocol unreachable
3*	Port unreachable
4	Fragmentation needed but don't-fragment bit set
5	Source route failed
6	Destination network unknown
7	Destination host unknown
8	Source host isolated (obsolete)
9*	Destination network administratively prohibited
10*	Destination host administratively prohibited
11	Network unreachable for type of service (TOS)
12	Host unreachable for TOS
13*	Communication administratively prohibited by filtering
14	Host precedence violation
15	Precedence cutoff in effect

TCP SYN (Stealth) Scan (`-sS`)

Fuente original: https://nmap.org/book/synscan.html

SYN scan is the default and most popular scan option for good reason. It can be performed quickly, scanning thousands of ports per second on a fast network not hampered by intrusive firewalls. SYN scan is relatively unobtrusive and stealthy, since it never completes TCP connections. It also works against any compliant TCP stack rather than depending on idiosyncrasies of specific platforms as Nmap's FIN/NULL/Xmas, Maimon and idle scans do. It also allows clear, reliable differentiation between `open`, `closed`, and `filtered` states.

SYN scan may be requested by passing the `-sS` option to Nmap. It requires raw-packet privileges, and is the default TCP scan when they are available. So when running Nmap as root or Administrator, `-sS` is usually omitted. This default SYN scan behavior is shown in Example 5.1, which finds a port in each of the three major states.

Example 5.1. A SYN scan showing three port states

```
krad# nmap -p22,113,139 scanme.nmap.org

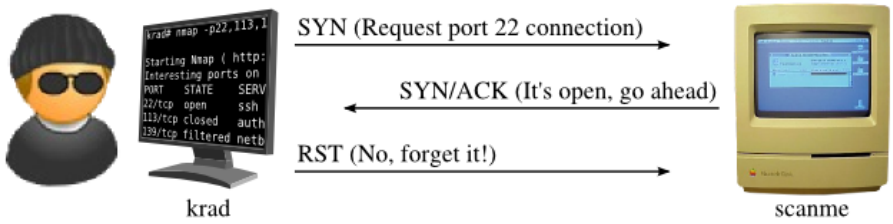
Starting Nmap ( https://nmap.org )
Nmap scan report for scanme.nmap.org (64.13.134.52)
```

```
PORT    STATE    SERVICE
22/tcp  open     ssh
113/tcp closed   auth
139/tcp filtered netbios-ssn

Nmap done: 1 IP address (1 host up) scanned in 1.35 seconds
```

While SYN scan is pretty easy to use without any low-level TCP knowledge, understanding the technique helps when interpreting unusual results. Fortunately for us, the fearsome black-hat cracker Ereet Hagiwara has taken a break from terrorizing Japanese Windows users to illustrate the Example 5.1 SYN scan for us at the packet level. First, the behavior against open port 22 is shown in Figure 5.2.

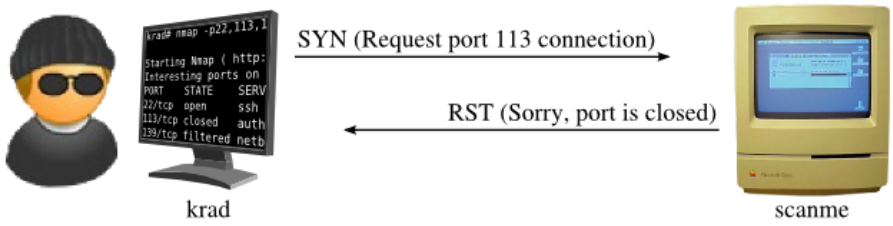
Figure 5.2. SYN scan of open port 22



As this example shows, Nmap starts by sending a TCP packet with the SYN flag set (see Figure 2, "TCP header" if you have forgotten what packet headers look like) to port 22. This is the first step in the TCP three-way handshake that any legitimate connection attempt takes. Since the target port is open, Scanme takes the second step by sending a response with the SYN and ACK flags back. In a normal connection, Ereet's machine (named krad) would complete the three-way handshake by sending an ACK packet acknowledging the SYN/ACK. Nmap does not need to do this, since the SYN/ACK response already told it that the port is open. If Nmap completed the connection, it would then have to worry about closing it. This usually involves another handshake, using FIN packets rather than SYN. So an ACK is a bad idea, yet something still has to be done. If the SYN/ACK is ignored completely, Scanme will assume it was dropped and keep re-sending it. The proper response, since we don't want to make a full connection, is a RST packet as shown in the diagram. This tells Scanme to forget about (reset) the attempted connection. Nmap could send this RST packet easily enough, but it actually doesn't need to. The OS running on krad also receives the SYN/ACK, which it doesn't expect because Nmap crafted the SYN probe itself. So the OS responds to the unexpected SYN/ACK with a RST packet. All RST packets described in this chapter also have the ACK bit set because they are always sent in response to (and acknowledge) a received packet. So that bit is not shown explicitly for RST packets. Because the three-way handshake is never completed, SYN scan is sometimes called half-open scanning.

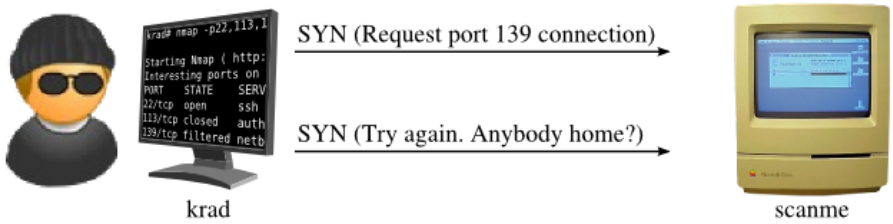
Figure 5.3 shows how Nmap determines that port 113 is closed. This is even simpler than the open case. The first step is always the same—Nmap sends the SYN probe to Scanme. But instead of receiving a SYN/ACK back, a RST is returned. That settles it—the port is closed. No more communication regarding this port is necessary.

Figure 5.3. SYN scan of closed port 113



Finally, Ereet shows us how a filtered port appears to Nmap in Figure 5.4. The initial SYN is sent first, as usual, but Nmap sees no reply. The response could simply be slow. From previous responses (or timing defaults), Nmap knows how long to wait and eventually gives up on receiving one. A non-responsive port is usually filtered (blocked by a firewall device, or perhaps the host is down), but this one test is not conclusive. Perhaps the port is open but the probe or response were simply dropped. Networks can be flaky. So Nmap tries again by resending the SYN probe. After yet another timeout period, Nmap gives up and marks the port `filtered`. In this case, only one retransmission was attempted. As described in the section called "Scan Code and Algorithms", Nmap keeps careful packet loss statistics and will attempt more retransmissions when scanning less reliable networks.

Figure 5.4. SYN scan of filtered port 139



Nmap will also consider a port `filtered` if it receives certain ICMP error messages back. Table 5.2 shows how Nmap assigns port states based on responses to a SYN probe.

Table 5.2. How Nmap interprets responses to a SYN probe

Probe Response	Assigned State
TCP SYN/ACK response	`open`
TCP RST response	`closed`
No response received (even after retransmissions)	`filtered`
ICMP unreachable error (type 3, code 1, 2, 3, 9, 10, or 13)	`filtered`

While the pretty illustrations in this section are useful when you have them, Nmap reports exactly what it is doing at the packet level when you specify the `--packet-trace` option in addition to any other desired command-line flags. This is a great way for newbies to understand Nmap's behavior when Ereet is not around to help. Even advanced users find it handy when Nmap produces results they don't expect. You may want to increase the debug level with `-d` (or even `-d5`) as well. Then scan the minimum number of ports and hosts necessary for your purpose or you could end up with literally millions of output lines. Example 5.2 repeats Ereet's three-port SYN scan with packet tracing enabled (output edited for brevity). Read the command-line, then test yourself by figuring out what packets will be sent before reading on. Then once you read the trace up to "The SYN Stealth Scan took 1.25s", you should know from the RCVD lines what the port state table will look like before continuing on to read it.

Example 5.2. Using `--packet-trace` to understand a SYN scan

```
krad# nmap -d --packet-trace -p22,113,139 scanme.nmap.org

Starting Nmap ( https://nmap.org )
SENT (0.0130s) ICMP krad > scanme echo request (type=8/code=0) ttl=52 id=1829
SENT (0.0160s) TCP krad:63541 > scanme:80 A iplen=40 seq=91911070 ack=99850910
RCVD (0.0280s) ICMP scanme > krad echo reply (type=0/code=0) iplen=28
We got a ping packet back from scanme: id = 48821 seq = 714 checksum = 16000
massping done:  num_hosts: 1  num_responses: 1
Initiating SYN Stealth Scan against scanme.nmap.org (scanme) [3 ports] at 00:53
SENT (0.1340s) TCP krad:63517 > scanme:113 S iplen=40 seq=10438635
SENT (0.1370s) TCP krad:63517 > scanme:22 S iplen=40 seq=10438635
SENT (0.1400s) TCP krad:63517 > scanme:139 S iplen=40 seq=10438635
RCVD (0.1460s) TCP scanme:113 > krad:63517 RA iplen=40 seq=0 ack=10438636
RCVD (0.1510s) TCP scanme:22 > krad:63517 SA iplen=44 seq=75897108 ack=10438636
SENT (1.2550s) TCP krad:63518 > scanme:139 S iplen=40 seq=10373098 win=3072
The SYN Stealth Scan took 1.25s to scan 3 total ports.
Nmap scan report for scanme.nmap.org (64.13.134.52)
PORT    STATE    SERVICE
22/tcp  open     ssh
113/tcp closed   auth
139/tcp filtered netbios-ssn

Nmap done: 1 IP address (1 host up) scanned in 1.40 seconds
```

SYN scan has long been called the stealth scan because it is subtler than TCP connect scan (discussed next), which was the most common scan type before Nmap was released. Despite that moniker, don't count on a default SYN scan slipping undetected through sensitive networks. Widely deployed intrusion detection systems and even personal firewalls are quite capable of detecting default SYN scans. More effective techniques for stealthy scanning are demonstrated in Chapter 10, *Detecting and Subverting Firewalls and Intrusion Detection Systems*.

TCP Connect Scan (`-sT`)

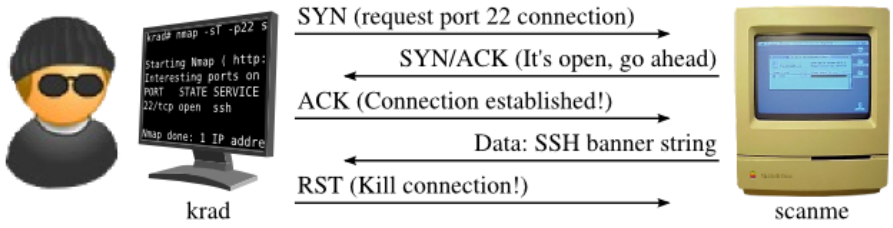
Fuente original: https://nmap.org/book/scan-methods-connect-scan.html

TCP connect scan is the default TCP scan type when SYN scan is not an option. This is the case when a user does not have raw packet privileges or is scanning IPv6 networks. Instead of writing raw packets as most other scan types do, Nmap asks the underlying operating system to establish a connection with the target machine and port by issuing the `connect` system call. This is the same high-level system call that web browsers, P2P clients, and most other network-enabled applications use to establish a connection. It is part of a programming interface known as the Berkeley Sockets API. Rather than read raw packet responses off the wire, Nmap uses this API to obtain status information on each connection attempt. This and the FTP bounce scan (the section called "TCP FTP Bounce Scan (`-b`)") are the only scan types available to unprivileged users.

When SYN scan is available, it is usually a better choice. Nmap has less control over the high level `connect` call than with raw packets, making it less efficient. The system call completes connections to open target ports rather than performing the half-open reset that SYN scan does. Not only does this take longer and require more packets to obtain the same information, but target machines are more likely to log the connection. A decent IDS will catch either, but most machines have no such alarm system. Many services on your average Unix system will add a note to syslog, and sometimes a cryptic error message, when Nmap connects and then closes the connection without sending data. Truly pathetic services crash when this happens, though that is uncommon. An administrator who sees a bunch of connection attempts in her logs from a single system should know that she has been connect scanned.

Figure 5.5 shows a connect scan in action against open port 22 of scanme.nmap.org. Recall that this only required three packets in Figure 5.2, "SYN scan of open port 22". The exact behavior against an open port depends on the platform Nmap runs on and the service listening at the other end, but this five packet example is typical.

Figure 5.5. Connect scan of open port 22



The first two steps (SYN and SYN/ACK) are exactly the same as with a SYN scan. Then, instead of aborting the half-open connection with a RST packet, krad acknowledges the SYN/ACK with its own ACK packet, completing the connection. In this case, Scanme even had time to send its SSH banner string (`SSH-1.99-OpenSSH_3.1p1\n`) through the now-open connection. As soon as Nmap hears from its host OS that the connection was successful, it terminates the connection. TCP connections usually end with another handshake involving the FIN flag, but Nmap asks the host OS to terminate the connection immediately with a RST packet.

While this connect scan example took almost twice as many packets as a SYN scan, the bandwidth differences are rarely so substantial. The vast majority of ports in a large scan will be `closed` or `filtered`. The packet traces for those are the same as described for SYN scan in Figure 5.3, "SYN scan of closed port 113" and Figure 5.4, "SYN scan of filtered port 139". Only open ports generate more network traffic.

The output of a connect scan doesn't differ significantly from a SYN scan. Example 5.3 shows a connect scan of Scanme. The `-sT` option could have been omitted since Nmap is being run from a non-privileged account so connect scan is the default type.

Example 5.3. Connect scan example

```
krad~> nmap -T4 -sT scanme.nmap.org

Starting Nmap ( https://nmap.org )
Nmap scan report for scanme.nmap.org (64.13.134.52)
Not shown: 994 filtered ports
PORT    STATE  SERVICE
22/tcp  open   ssh
25/tcp  closed smtp
53/tcp  open   domain
70/tcp  closed gopher
80/tcp  open   http
113/tcp closed auth

Nmap done: 1 IP address (1 host up) scanned in 4.74 seconds
```

UDP Scan (`-sU`)

Fuente original: https://nmap.org/book/scan-methods-udp-scan.html

While most popular services on the Internet run over the TCP protocol, UDP services are widely deployed. DNS, SNMP, and DHCP (registered ports 53, 161/162, and 67/68) are three of the most common. Because UDP scanning is generally slower and more difficult than TCP, some

security auditors ignore these ports. This is a mistake, as exploitable UDP services are quite common and attackers certainly don't ignore the whole protocol. Fortunately, Nmap can help inventory UDP ports.

UDP scan is activated with the `-sU` option. It can be combined with a TCP scan type such as SYN scan (`-sS`) to check both protocols during the same run.

UDP scan works by sending a UDP packet to every targeted port. For most ports, this packet will be empty (no payload), but for a few of the more common ports a protocol-specific payload will be sent. Based on the response, or lack thereof, the port is assigned to one of four states, as shown in Table 5.3.

Table 5.3. How Nmap interprets responses to a UDP probe

Probe Response	Assigned State
Any UDP response from target port (unusual)	`open`
No response received (even after retransmissions)	`open|filtered`
ICMP port unreachable error (type 3, code 3)	`closed`
Other ICMP unreachable errors (type 3, code 1, 2, 9, 10, or 13)	`filtered`

The most curious element of this table may be the `open|filtered` state. It is a symptom of the biggest challenges with UDP scanning: open ports rarely respond to empty probes. Those ports for which Nmap has a protocol-specific payload are more likely to get a response and be marked `open`, but for the rest, the target TCP/IP stack simply passes the empty packet up to a listening application, which usually discards it immediately as invalid. If ports in all other states would respond, then open ports could all be deduced by elimination. Unfortunately, firewalls and filtering devices are also known to drop packets without responding. So when Nmap receives no response after several attempts, it cannot determine whether the port is `open` or `filtered`. When Nmap was released, filtering devices were rare enough that Nmap could (and did) simply assume that the port was `open`. The Internet is better guarded now, so Nmap changed in 2004 (version 3.70) to report non-responsive UDP ports as `open|filtered` instead. We can see that in Example 5.4, which shows Ereet scanning a Linux box named Felix.

Example 5.4. UDP scan example

```
krad# nmap -sU -v felix

Starting Nmap ( https://nmap.org )
Nmap scan report for felix.nmap.org (192.168.0.42)
(The 997 ports scanned but not shown below are in state: closed)
PORT    STATE         SERVICE
53/udp  open|filtered domain
67/udp  open|filtered dhcpserver
111/udp open|filtered rpcbind
MAC Address: 00:02:E3:14:11:02 (Lite-on Communications)

Nmap done: 1 IP address (1 host up) scanned in 999.25 seconds
```

This scan of Felix demonstrates the `open|filtered` ambiguity issue as well as another problem: UDP scanning can be slow. Scanning a thousand ports took almost 17 minutes in this case due to ICMP response rate limiting performed by Felix and most other Linux systems. Nmap provides ways to work around both problems, as described by the following two sections.

Distinguishing Open from Filtered UDP Ports

In the case of the Felix scan, all but the three `open|filtered` ports were `closed`. So the scan was still successful in narrowing down potentially open ports to a handful. That is not always

the case. Example 5.5 shows a UDP scan against the heavily filtered site Scanme.

Example 5.5. UDP scan example

```
krad# nmap -sU -T4 scanme.nmap.org

Starting Nmap ( https://nmap.org )
All 1000 scanned ports on scanme.nmap.org (64.13.134.52) are open|filtered

Nmap done: 1 IP address (1 host up) scanned in 5.50 seconds
```

In this case, the scan didn't narrow down the open ports at all. All 1000 are `open|filtered`. A new strategy is called for.

Table 5.3, "How Nmap interprets responses to a UDP probe" shows that the `open|filtered` state occurs when Nmap fails to receive any responses from its UDP probes to a particular port. Yet it also shows that, on rare occasions, the UDP service listening on a port will respond in kind, proving that the port is open. The reason these services don't respond often is that the empty packets Nmap sends are considered invalid. Unfortunately, UDP services generally define their own packet structure rather than adhering to some common general format that Nmap could always send. An SNMP packet looks completely different than a SunRPC, DHCP, or DNS request packet.

To send the proper packet for every popular UDP service, Nmap would need a large database defining their probe formats. Fortunately, Nmap has that in the form of `nmap-service-probes`, which is part of the service and version detection subsystem described in Chapter 7, *Service and Application Version Detection*.

When version scanning is enabled with `-sV` (or `-A`), it will send UDP probes to every `open|filtered` port (as well as known `open` ones). If any of the probes elicit a response from an `open|filtered` port, the state is changed to `open`. The results of adding `-sV` to the Felix scan are shown in Example 5.6.

Example 5.6. Improving Felix's UDP scan results with version detection

```
krad# nmap -sUV -F felix.nmap.org

Starting Nmap ( https://nmap.org )
Nmap scan report for felix.nmap.org (192.168.0.42)
Not shown: 997 closed ports
PORT    STATE         SERVICE    VERSION
53/udp  open          domain     ISC BIND 9.2.1
67/udp  open|filtered dhcpserver
111/udp open          rpcbind    2 (rpc #100000)
MAC Address: 00:02:E3:14:11:02 (Lite-on Communications)

Nmap done: 1 IP address (1 host up) scanned in 1037.57 seconds
```

This new scan shows that port 111 and 53 are definitely open. The system isn't perfect though—port 67 is still `open|filtered`. In this particular case, the port is open but Nmap does not have a working version probe for DHCP. Another tough service is SNMP, which usually only responds when the correct community string is given. Many devices are configured with the community string `public`, but not all are. While these results aren't perfect, learning the true state of two out of three tested ports is still helpful.

After the success in disambiguating Felix results, Ereet turns his attention back to Scanme, which listed all ports as `open|filtered` last time. He tries again with version detection, as shown in Example 5.7.

Example 5.7. Improving Scanme's UDP scan results with version detection

```
krad# nmap -sUV -T4 scanme.nmap.org

Starting Nmap ( https://nmap.org )
Nmap scan report for scanme.nmap.org (64.13.134.52)
Not shown: 999 open|filtered ports
PORT   STATE SERVICE VERSION
53/udp open  domain  ISC BIND 9.3.4

Nmap done: 1 IP address (1 host up) scanned in 3691.89 seconds
```

Tip

While Ereet eventually found the open port, he made a mistake in not updating his Nmap version first. Nmap version 5.10BETA1 and newer have a payload system which sends proper service protocol requests to more than three dozen well known UDP ports if they are selected for port scanning or host discovery. While it isn't as comprehensive as version detection, it would have quickly identified the open port 53 in Example 5.5.

This result took an hour, versus five seconds for the previous Scanme scan, but these results are actually useful. Ereet's smile widens and eyes sparkle at this evidence of an open ISC BIND nameserver on a machine he wants to compromise. That software has a long history of security holes, so perhaps he can find a flaw in this recent version.

Ereet will focus his UDP attacks on port 53 since it is confirmed open, but he does not forget about the other 999 ports listed as `open|filtered`. As we witnessed with the dhcpserver port on Felix, certain open UDP services can hide even from Nmap version detection. He has also only scanned the default ports so far, there are 64529 others that could possibly be open. For the record, 53 is the only open UDP port on Scanme.

While this version detection technique is the only way for Nmap to automatically disambiguate `open|filtered` ports, there are a couple tricks that can be tried manually. Sometimes a specialized traceroute can help. You could do a traceroute against a known-open TCP or UDP port with Nmap or a tool such as Nping. Then try the same against the questionable UDP port. Differences in hop counts can differentiate open from filtered ports. Ereet attempts this against Scanme in Example 5.8. The first command does a UDP traceroute against known-open port 53. The second command does the same thing against presumed-closed port 54. The first few hops have been omitted to save space.

Example 5.8. Attempting to disambiguate UDP ports with TTL discrepancies

```
krad# nping --udp --traceroute -c 13 -p 53 scanme.nmap.org

Starting Nping ( https://nmap.org/nping )
SENT (7.0370s) UDP 192.168.0.21:53 > 64.13.134.52:53 ttl=8 id=4826 iplen=28
RCVD (7.1010s) ICMP 4.69.134.222 > 192.168.0.21 TTL=0 during transit (type=11/code=0) ttl=248 id=38454
↪  iplen=56
SENT (8.0400s) UDP 192.168.0.21:53 > 64.13.134.52:53 ttl=9 id=38166 iplen=28
RCVD (8.1050s) ICMP 4.68.18.204 > 192.168.0.21 TTL=0 during transit (type=11/code=0) ttl=247 id=39583
↪  iplen=56
SENT (9.0420s) UDP 192.168.0.21:53 > 64.13.134.52:53 ttl=10 id=6788 iplen=28
RCVD (9.1080s) ICMP 4.59.4.78 > 192.168.0.21 TTL=0 during transit (type=11/code=0) ttl=246 id=59897
↪  iplen=56
SENT (10.0440s) UDP 192.168.0.21:53 > 64.13.134.52:53 ttl=11 id=366 iplen=28
RCVD (10.1100s) ICMP 69.36.239.221 > 192.168.0.21 TTL=0 during transit (type=11/code=0) ttl=243 id=42710
↪  iplen=56
SENT (11.0470s) UDP 192.168.0.21:53 > 64.13.134.52:53 ttl=12 id=63478 iplen=28
SENT (12.0490s) UDP 192.168.0.21:53 > 64.13.134.52:53 ttl=13 id=56653 iplen=28

Max rtt: 73.003ms | Min rtt: 0.540ms | Avg rtt: 48.731ms
Raw packets sent: 13 (364B) | Rcvd: 10 (560B) | Lost: 3 (23.08%)
Tx time: 12.02836s | Tx bytes/s: 30.26 | Tx pkts/s: 1.08
Rx time: 13.02994s | Rx bytes/s: 42.98 | Rx pkts/s: 0.77
```

```
Nping done: 1 IP address pinged in 13.05 seconds

krad# nping --udp --traceroute -c 13 -p 54 scanme.nmap.org

Starting Nping ( https://nmap.org/nping )
SENT (7.0370s) UDP 192.168.0.21:53 > 64.13.134.52:54 ttl=8 id=56481 iplen=28
RCVD (7.1130s) ICMP 4.69.134.214 > 192.168.0.21 TTL=0 during transit (type=11/code=0) ttl=248 id=22437
↪  iplen=56
SENT (8.0400s) UDP 192.168.0.21:53 > 64.13.134.52:54 ttl=9 id=23264 iplen=28
RCVD (8.1060s) ICMP 4.68.18.76 > 192.168.0.21 TTL=0 during transit (type=11/code=0) ttl=247 id=50214
↪  iplen=56
SENT (9.0430s) UDP 192.168.0.21:53 > 64.13.134.52:54 ttl=10 id=9101 iplen=28
RCVD (9.1070s) ICMP 4.59.4.78 > 192.168.0.21 TTL=0 during transit (type=11/code=0) ttl=246 id=880
↪  iplen=56
SENT (10.0450s) UDP 192.168.0.21:53 > 64.13.134.52:54 ttl=11 id=35344 iplen=28
RCVD (10.1110s) ICMP 69.36.239.221 > 192.168.0.21 TTL=0 during transit (type=11/code=0) ttl=243 id=44617
↪  iplen=56
SENT (11.0470s) UDP 192.168.0.21:53 > 64.13.134.52:54 ttl=12 id=53857 iplen=28
SENT (12.0490s) UDP 192.168.0.21:53 > 64.13.134.52:54 ttl=13 id=986 iplen=28

Max rtt: 76.488ms | Min rtt: 0.546ms | Avg rtt: 48.480ms
Raw packets sent: 13 (364B) | Rcvd: 11 (616B) | Lost: 2 (15.38%)
Tx time: 12.02908s | Tx bytes/s: 30.26 | Tx pkts/s: 1.08
Rx time: 13.03165s | Rx bytes/s: 47.27 | Rx pkts/s: 0.84
Nping done: 1 IP address pinged in 13.05 seconds
```

In this example, Ereet was only able to reach hop eleven of both the open and closed ports. So these results can't be used to distinguish port states against this host. It was worth a try, and does work in a significant number of cases. It is more likely to work in situations where the screening firewall is at least a hop or two before the target host. Scanme, on the other hand, is running its own Linux iptables host-based firewall. So there is no difference in hop count between filtered and open ports.

Another technique is to try application-specific tools against common ports. For example, a brute force SNMP community string cracker could be tried against port 161. As Nmap's version detection probe database grows, the need to augment its results with external specialized tools is reduced. They will still be useful for special cases, such as SNMP devices with a custom community string.

Speeding Up UDP Scans

The other big challenge with UDP scanning is doing it quickly. Open and filtered ports rarely send any response, leaving Nmap to time out and then conduct retransmissions just in case the probe or response were lost. Closed ports are often an even bigger problem. They usually send back an ICMP port unreachable error. But unlike the RST packets sent by closed TCP ports in response to a SYN or connect scan, many hosts rate limit ICMP port unreachable messages by default. Linux and Solaris are particularly strict about this. For example, the Linux 2.4.20 kernel on Felix limits destination unreachable messages to one per second (in `net/ipv4/icmp.c`). This explains why the scan in Example 5.4, "UDP scan example" is so slow.

Nmap detects rate limiting and slows down accordingly to avoid flooding the network with useless packets that the target machine will drop. Unfortunately, a Linux-style limit of one packet per second makes a 65,536-port scan take more than 18 hours. Here are some suggestions for improving UDP scan performance. Also read Chapter 6, *Optimizing Nmap Performance* for more detailed discussion and general advice.

Increase host parallelism

If Nmap receives just one port unreachable error from a single target host per second, it could receive 100/second just by scanning 100 such hosts at once. Implement this by passing

a large value (such as 100) to `--min-hostgroup`.

Scan popular ports first

Very few UDP port numbers are commonly used. A scan of the most common 100 UDP ports (using the `-F` option) will finish quickly. You can then investigate those results while you launch a multi-day 65K-port sweep of the network in the background.

Add `--version-intensity 0` to version detection scans

As mentioned in the previous section, version detection (`-sV`) is often needed to differentiate open from filtered UDP ports. Version detection is relatively slow since it involves sending a large number of application protocol-specific probes to every `open` or `open|filtered` port found on the target machines. Specifying `--version-intensity 0` directs Nmap to try only the probes most likely to be effective against a given port number. It does this by using data from the `nmap-service-probes` file. The performance impact of this option is substantial, as will be demonstrated later in this section.

Scan from behind the firewall

As with TCP, packet filters can slow down scans dramatically. Many modern firewalls make setting packet rate limits easy. If you can bypass that problem by launching the scan from behind the firewall rather than across it, do so.

Use `--host-timeout` to skip slow hosts

ICMP-rate-limited hosts can take orders of magnitude more time to scan than those that respond to every probe with a quick destination unreachable packet. Specifying a maximum scan time (such as `15m` for 15 minutes) causes Nmap to give up on individual hosts if it hasn't completed scanning them in that much time. This allows you to scan all of the responsive hosts quickly. You can then work on the slow hosts in the background.

Use `-v` and chill out

With verbosity (`-v`) enabled, Nmap provides estimated time for scan completion of each host. There is no need to watch it closely. Get some sleep, head to your favorite pub, read a book, finish other work, or otherwise amuse yourself while Nmap tirelessly scans on your behalf.

A perfect example of the need to optimize UDP scans is Example 5.7, "Improving Scanme's UDP scan results with version detection". The scan obtained the desired data, but it took more than an hour to scan this one host! In Example 5.9, Ereet runs that scan again. This time he adds the `-F --version-intensity 0` options and the hour long scan is reduced to 13 seconds! Yet the same key information (an ISC Bind daemon running on port 53) is detected.

Example 5.9. Optimizing UDP Scan Time

```
krad# nmap -sUV -T4 -F --version-intensity 0 scanme.nmap.org

Starting Nmap ( https://nmap.org )
Nmap scan report for scanme.nmap.org (64.13.134.52)
Not shown: 99 open|filtered ports
PORT   STATE SERVICE VERSION
53/udp open  domain  ISC BIND 9.3.4

Nmap done: 1 IP address (1 host up) scanned in 12.92 seconds
```

TCP FIN, NULL, and Xmas Scans (`-sF`, `-sN`, `-sX`)

Fuente original: https://nmap.org/book/scan-methods-null-fin-xmas-scan.html

These three scan types (even more are possible with the `--scanflags` option described in the next section) exploit a subtle loophole in the TCP RFC to differentiate between `open` and `closed` ports. Page 65 of RFC 793 says that "if the [destination] port state is CLOSED .... an incoming segment not containing a RST causes a RST to be sent in response." Then the next page discusses packets sent to open ports without the SYN, RST, or ACK bits set, stating that: "you are unlikely to get here, but if you do, drop the segment, and return."

When scanning systems compliant with this RFC text, any packet not containing SYN, RST, or ACK bits will result in a returned RST if the port is closed and no response at all if the port is open. As long as none of those three bits are included, any combination of the other three (FIN, PSH, and URG) are OK. Nmap exploits this with three scan types:

Null scan (`-sN`)

Does not set any bits (TCP flag header is 0)

FIN scan (`-sF`)

Sets just the TCP FIN bit.

Xmas scan (`-sX`)

Sets the FIN, PSH, and URG flags, lighting the packet up like a Christmas tree.

These three scan types are exactly the same in behavior except for the TCP flags set in probe packets. Responses are treated as shown in Table 5.4.

Table 5.4. How Nmap interprets responses to a NULL, FIN, or Xmas scan probe

Probe Response	Assigned State
No response received (even after retransmissions)	`open|filtered`
TCP RST packet	`closed`
ICMP unreachable error (type 3, code 1, 2, 3, 9, 10, or 13)	`filtered`

The key advantage to these scan types is that they can sneak through certain non-stateful firewalls and packet filtering routers. Such firewalls try to prevent incoming TCP connections (while allowing outbound ones) by blocking any TCP packets with the SYN bit set and ACK cleared. This configuration is common enough that the Linux iptables firewall command offers a special `--syn` option to implement it. The NULL, FIN, and Xmas scans clear the SYN bit and thus fly right through those rules.

Another advantage is that these scan types are a little more stealthy than even a SYN scan. Don't count on this though—most modern IDS products can be configured to detect them.

The big downside is that not all systems follow RFC 793 to the letter. A number of systems send RST responses to the probes regardless of whether the port is open or not. This causes all of the ports to be labeled `closed`. Major operating systems that do this are Microsoft Windows, many Cisco devices, and IBM OS/400. This scan does work against most Unix-based systems though. Since Nmap OS detection tests for this quirk, you can learn whether the scan works against a particular type of system by examining the `nmap-os-db` file. Test T2 sends a NULL packet to an open port. So if you see a line like `T2(R=N)`, that system seems to support the RFC and one of these scans should work against it. If the T2 line is longer, the system violated the RFC by sending a response and these scans won't work. Chapter 8, *Remote OS Detection* explains OS fingerprinting in further detail.

Another downside of these scans is that they can't distinguish open ports from certain filtered ones. If the packet filter sends an ICMP destination prohibited error, Nmap knows that a port

is filtered. But most filters simply drop banned probes without any response, making the ports appear open. Since Nmap cannot be sure which is the case, it marks non-responsive ports as `open|filtered`. Adding version detection (`-sV`) can disambiguate as it does with UDP scans, but that defeats much of the stealthy nature of this scan. If you are willing and able to connect to the ports anyway, you might as well use a SYN scan.

Using these scan methods is simple. Just add the `-sN`, `-sF`, or `-sX` options to specify the scan type. Example 5.10 shows two examples. The first one, a FIN scan against Para, identifies all five open ports (as `open|filtered`). The next execution, an Xmas scan against scanme.nmap.org doesn't work so well. It detects the closed port, but is unable to differentiate the 995 filtered ports from the four open ones, all 999 are listed as `open|filtered`. This demonstrates why Nmap offers so many scan methods. No single technique is preferable in all cases. Ereet will simply have to try another method to learn more about Scanme.

Example 5.10. Example FIN and Xmas scans

```
krad# nmap -sF -T4 para

Starting Nmap ( https://nmap.org )
Nmap scan report for para (192.168.10.191)
Not shown: 995 closed ports
PORT     STATE         SERVICE
22/tcp   open|filtered ssh
53/tcp   open|filtered domain
111/tcp  open|filtered rpcbind
515/tcp  open|filtered printer
6000/tcp open|filtered X11
MAC Address: 00:60:1D:38:32:90 (Lucent Technologies)

Nmap done: 1 IP address (1 host up) scanned in 4.64 seconds

krad# nmap -sX -T4 scanme.nmap.org

Starting Nmap ( https://nmap.org )
Nmap scan report for scanme.nmap.org (64.13.134.52)
Not shown: 999 open|filtered ports
PORT    STATE  SERVICE
113/tcp closed auth

Nmap done: 1 IP address (1 host up) scanned in 23.11 seconds
```

Demonstrating the full, firewall-bypassing power of these scans requires a rather lame target firewall configuration. Unfortunately, those are easy to find. Example 5.11 shows a SYN scan of a SCO/Caldera machine named Docsrv.

Example 5.11. SYN scan of Docsrv

```
# nmap -sS -T4 docsrv.caldera.com

Starting Nmap ( https://nmap.org )
Nmap scan report for docsrv.caldera.com (216.250.128.247)
(The 997 ports scanned but not shown below are in state: filtered)
PORT    STATE  SERVICE
80/tcp  open   http
113/tcp closed auth
507/tcp open   crs

Nmap done: 1 IP address (1 host up) scanned in 28.62 seconds
```

This example looks OK. Only two ports are open and the rest (except for 113) are filtered. With a modern stateful firewall, a FIN scan should not produce any extra information. Yet Ereet tries it anyway, obtaining the output in Example 5.12.

Example 5.12. FIN scan of Docsrv

```
# nmap -sF -T4 docsrv.caldera.com

Starting Nmap ( https://nmap.org )
Nmap scan report for docsrv.caldera.com (216.250.128.247)
Not shown: 961 closed ports
PORT      STATE         SERVICE
7/tcp     open|filtered echo
9/tcp     open|filtered discard
11/tcp    open|filtered systat
13/tcp    open|filtered daytime
15/tcp    open|filtered netstat
19/tcp    open|filtered chargen
21/tcp    open|filtered ftp
22/tcp    open|filtered ssh
23/tcp    open|filtered telnet
25/tcp    open|filtered smtp
37/tcp    open|filtered time
79/tcp    open|filtered finger
80/tcp    open|filtered http
110/tcp   open|filtered pop3
111/tcp   open|filtered rpcbind
135/tcp   open|filtered msrpc
143/tcp   open|filtered imap
360/tcp   open|filtered scoi2odialog
389/tcp   open|filtered ldap
465/tcp   open|filtered smtps
507/tcp   open|filtered crs
512/tcp   open|filtered exec
513/tcp   open|filtered login
514/tcp   open|filtered shell
515/tcp   open|filtered printer
636/tcp   open|filtered ldapssl
712/tcp   open|filtered unknown
955/tcp   open|filtered unknown
993/tcp   open|filtered imaps
995/tcp   open|filtered pop3s
1434/tcp  open|filtered ms-sql-m
2000/tcp  open|filtered callbook
2766/tcp  open|filtered listen
3000/tcp  open|filtered ppp
3306/tcp  open|filtered mysql
6112/tcp  open|filtered dtspc
32770/tcp open|filtered sometimes-rpc3
32771/tcp open|filtered sometimes-rpc5
32772/tcp open|filtered sometimes-rpc7

Nmap done: 1 IP address (1 host up) scanned in 7.64 seconds
```

Wow! That is a lot of apparently open ports. Most of them are probably open, because having just these 39 filtered and the other 961 closed (sending a RST packet) would be unusual. Yet it is still possible that some or all are filtered instead of open. FIN scan cannot determine for sure. We will revisit this case and learn more about Docsrv later in this chapter.

Custom Scan Types with `--scanflags`

Fuente original: https://nmap.org/book/scan-methods-custom-scanflags.html

Truly advanced Nmap users need not limit themselves to the canned scanned types. The `--scanflags` option allows you to design your own scan by specifying arbitrary TCP flags. Let your creative juices flow, while evading intrusion detection systems whose vendors simply paged through the Nmap man page adding specific rules!

The `--scanflags` argument can be a numerical flag value such as 9 (PSH and FIN), but using symbolic names is easier. Just mash together any combination of `URG`, `ACK`, `PSH`, `RST`, `SYN`,

and FIN. For example, --scanflags URGACKPSHRSTSYNFIN sets everything, though it's not very useful for scanning. The order these are specified in is irrelevant.

In addition to specifying the desired flags, you can specify a TCP scan type (such as -sA or -sF). That base type tells Nmap how to interpret responses. For example, a SYN scan considers no-response indicative of a filtered port, while a FIN scan treats the same as open|filtered. Nmap will behave the same way it does for the base scan type, except that it will use the TCP flags you specify instead. If you don't specify a base type, SYN scan is used.

Custom SYN/FIN Scan

One interesting custom scan type is SYN/FIN. Sometimes a firewall administrator or device manufacturer will attempt to block incoming connections with a rule such as "drop any incoming packets with only the SYN flag set". They limit it to only the SYN flag because they don't want to block the SYN/ACK packets which are returned as the second step of an outgoing connection.

The problem with this approach is that most end systems will accept initial SYN packets which contain other (non-ACK) flags as well. For example, the Nmap OS fingerprinting system sends a SYN/FIN/URG/PSH packet to an open port. More than half of the fingerprints in the database respond with a SYN/ACK. Thus they allow port scanning with this packet and generally allow making a full TCP connection too. Some systems have even been known to respond with SYN/ACK to a SYN/RST packet! The TCP RFC is ambiguous as to which flags are acceptable in an initial SYN packet, though SYN/RST certainly seems bogus.

Example 5.13 shows Ereet conducting a successful SYN/FIN scan of Google. He is apparently getting bored with scanme.nmap.org.

Example 5.13. A SYN/FIN scan of Google

```
krad# nmap -sS --scanflags SYNFIN -T4 www.google.com

Starting Nmap ( https://nmap.org )
Warning: Hostname www.google.com resolves to 4 IPs. Using 74.125.19.99.
Nmap scan report for cf-in-f99.google.com (74.125.19.99)
Not shown: 996 filtered ports
PORT    STATE  SERVICE
80/tcp  open   http
113/tcp closed auth
179/tcp closed bgp
443/tcp open   https

Nmap done: 1 IP address (1 host up) scanned in 7.58 seconds
```

Similar scan types, such as SYN/URG or SYN/PSH/URG/FIN will generally work as well. If you aren't getting through, don't forget the already mentioned SYN/RST option.

PSH Scan

the section called "TCP FIN, NULL, and Xmas Scans (-sF, -sN, -sX)" noted that RFC-compliant systems allow one to scan ports using any combination of the FIN, PSH, and URG flags. While there are eight possible permutations, Nmap only offers three canned modes (NULL, FIN, and Xmas). Show some personal flair by trying a PSH/URG or FIN/PSH scan instead. Results rarely differ from the three canned modes, but there is a small chance of evading scan detection systems.

To perform such a scan, just specify your desired flags with --scanflags and specify FIN scan (-sF) as the base type (choosing NULL or Xmas would make no difference). Example 5.14

demonstrates a PSH scan against a Linux machine on a local network.

Example 5.14. A custom PSH scan

```
krad# nmap -sF --scanflags PSH  para

Starting Nmap ( https://nmap.org )
Nmap scan report for para (192.168.10.191)
(The 995 ports scanned but not shown below are in state: closed)
PORT     STATE         SERVICE
22/tcp   open|filtered ssh
53/tcp   open|filtered domain
111/tcp  open|filtered rpcbind
515/tcp  open|filtered printer
6000/tcp open|filtered X11
MAC Address: 00:60:1D:38:32:90 (Lucent Technologies)

Nmap done: 1 IP address (1 host up) scanned in 5.95 seconds
```

Because these scans all work the same way, I could keep just one of `-sF`, `-sN`, and `-sX` options, letting users emulate the others with `--scanflags`. There are no plans to do this because the shortcut options are easier to remember and use. You can still try the emulated approach to show off your Nmap skills. Execute **`nmap -sF --scanflags FINPSHURG target`** rather than the more mundane **`nmap -sX target`**.

Warning

In my experience, needlessly complex Nmap command-lines don't impress girls. They usually respond with a condescending sneer, presumably recognizing that the command is redundant.

TCP ACK Scan (`-sA`)

Fuente original: https://nmap.org/book/scan-methods-ack-scan.html

This scan is different than the others discussed so far in that it never determines `open` (or even `open|filtered`) ports. It is used to map out firewall rulesets, determining whether they are stateful or not and which ports are filtered.

ACK scan is enabled by specifying the `-sA` option. Its probe packet has only the ACK flag set (unless you use `--scanflags`). When scanning unfiltered systems, `open` and `closed` ports will both return a RST packet. Nmap then labels them as `unfiltered`, meaning that they are reachable by the ACK packet, but whether they are `open` or `closed` is undetermined. Ports that don't respond, or send certain ICMP error messages back, are labeled `filtered`. Table 5.5 provides the full details.

Table 5.5. How Nmap interprets responses to an ACK scan probe

Probe Response	Assigned State
TCP RST response	`unfiltered`
No response received (even after retransmissions)	`filtered`
ICMP unreachable error (type 3, code 1, 2, 3, 9, 10, or 13)	`filtered`

ACK scan usage is similar to most other scan types in that you simply add a single option flag, `-sA` in this case. Example 5.15 shows an ACK scan against Scanme.

Example 5.15. A typical ACK Scan

```
krad# nmap -sA -T4 scanme.nmap.org

Starting Nmap ( https://nmap.org )
Nmap scan report for scanme.nmap.org (64.13.134.52)
Not shown: 994 filtered ports
PORT    STATE      SERVICE
22/tcp  unfiltered ssh
25/tcp  unfiltered smtp
53/tcp  unfiltered domain
70/tcp  unfiltered gopher
80/tcp  unfiltered http
113/tcp unfiltered auth

Nmap done: 1 IP address (1 host up) scanned in 4.01 seconds
```

One of the most interesting uses of ACK scanning is to differentiate between stateful and stateless firewalls. See the section called "ACK Scan" for how to do this and why you would want to.

Sometimes a combination of scan types can be used to glean extra information from a system. As an example, start by reviewing the FIN scan of Docsrv in Example 5.12, "FIN scan of Docsrv". Nmap finds the closed ports in that case, but 39 of them are listed as `open|filtered` because Nmap cannot determine between those two states with a FIN scan. Now look at the ACK scan of the same host in Example 5.16, "An ACK scan of Docsrv". Two of those 39 previously unidentified ports are shown to be `filtered`. The other 37 (based on the default port line above the table) are in the state `unfiltered`. That means `open` or `closed`. If one scan type identifies a port as `open` or `filtered` and another identifies it as `open` or `closed`, logic dictates that it must be `open`. By combining both scan types, we have learned that 37 ports on Docsrv are `open`, two are `filtered`, and 961 are `closed`. While logical deduction worked well here to determine port states, that technique can't always be counted on. It assumes that different scan types always return a consistent state for the same port, which is inaccurate. Firewalls and TCP stack properties can cause different scans against the same machine to differ markedly. Against Docsrv, we have seen that a SYN scan considers the SSH port (`tcp/22`) `filtered`, while an ACK scan considers it `unfiltered`. When exploring boundary conditions and strangely configured networks, interpreting Nmap results is an art that benefits from experience and intuition.

Example 5.16. An ACK scan of Docsrv

```
# nmap -sA -T4 docsrv.caldera.com

Starting Nmap ( https://nmap.org )
Nmap scan report for docsrv.caldera.com (216.250.128.247)
Not shown: 998 unfiltered ports
PORT     STATE    SERVICE
135/tcp  filtered msrpc
1434/tcp filtered ms-sql-m

Nmap done: 1 IP address (1 host up) scanned in 7.20 seconds
```

TCP Window Scan (`-sW`)

Fuente original: https://nmap.org/book/scan-methods-window-scan.html

Window scan is exactly the same as ACK scan except that it exploits an implementation detail of certain systems to differentiate open ports from closed ones, rather than always printing `unfiltered` when a RST is returned. It does this by examining the TCP Window value of the RST packets returned. On some systems, open ports use a positive window size (even for RST

packets) while closed ones have a zero window. Window scan sends the same bare ACK probe as ACK scan, interpreting the results as shown in Table 5.6.

Table 5.6. How Nmap interprets responses to a Window scan ACK probe

Probe Response	Assigned State
TCP RST response with non-zero window field	`open`
TCP RST response with zero window field	`closed`
No response received (even after retransmissions)	`filtered`
ICMP unreachable error (type 3, code 1, 2, 3, 9, 10, or 13)	`filtered`

This scan relies on an implementation detail of a minority of systems out on the Internet, so you can't always trust it. Systems that don't support it will usually return all ports `closed`. Of course, it is possible that the machine really has no open ports. If most scanned ports are `closed` but a few common port numbers (such as 22, 25, and 53) are `open`, the system is most likely susceptible. Occasionally, systems will even show the exact opposite behavior. If your scan shows 997 open ports and three closed or filtered ports, then those three may very well be the truly open ones.

While this scan is not suited for every situation, it can be quite useful on occasion. Recall Example 5.12, "FIN scan of Docsrv", which shows many `open|filtered` ports not found in a basic SYN scan. The problem is that we can't distinguish between open and filtered ports with that FIN scan. The previous section showed that we could distinguish them by combining FIN and ACK scan results. In this case, a Window scan makes it even easier by not requiring the FIN scan results, as shown in Example 5.17.

Example 5.17. Window scan of docsrv.caldera.com

```
# nmap -sW -T4 docsrv.caldera.com

Starting Nmap ( https://nmap.org )
Nmap scan report for docsrv.caldera.com (216.250.128.247)
Not shown: 961 closed ports
PORT      STATE    SERVICE
7/tcp     open     echo
9/tcp     open     discard
11/tcp    open     systat
13/tcp    open     daytime
15/tcp    open     netstat
19/tcp    open     chargen
21/tcp    open     ftp
22/tcp    open     ssh
23/tcp    open     telnet
25/tcp    open     smtp
37/tcp    open     time
79/tcp    open     finger
80/tcp    open     http
110/tcp   open     pop3
111/tcp   open     rpcbind
135/tcp   filtered msrpc
[14 open ports omitted for brevity]
1434/tcp  filtered ms-sql-m
2000/tcp  open     callbook
2766/tcp  open     listen
3000/tcp  open     ppp
3306/tcp  open     mysql
6112/tcp  open     dtspc
32770/tcp open     sometimes-rpc3
32771/tcp open     sometimes-rpc5
32772/tcp open     sometimes-rpc7

Nmap done: 1 IP address (1 host up) scanned in 7.30 seconds
```

These results are exactly what Ereet wanted! The same 39 interesting ports are shown as with the FIN scan, but this time it distinguishes between the two filtered ports (MS-SQL and MSRPC) and the 37 that are actually open. These are the same results Ereet obtained by combining FIN and ACK scan results together in the previous section. Verifying results for consistency is another good reason for trying multiple scan types against a target network.

TCP Maimon Scan (`-sM`)

Fuente original: https://nmap.org/book/scan-methods-maimon-scan.html

The Maimon scan is named after its discoverer, Uriel Maimon. He described the technique in *Phrack* Magazine issue #49 (November 1996). Nmap, which included this technique, was released two issues later. This technique is exactly the same as NULL, FIN, and Xmas scan, except that the probe is FIN/ACK. According to RFC 793 (TCP), a RST packet should be generated in response to such a probe whether the port is open or closed. However, Uriel noticed that many BSD-derived systems simply drop the packet if the port is open. Nmap takes advantage of this to determine open ports, as shown in Table 5.7.

Table 5.7. How Nmap interprets responses to a Maimon scan probe

Probe Response	Assigned State
No response received (even after retransmissions)	`open|filtered`
TCP RST packet	`closed`
ICMP unreachable error (type 3, code 1, 2, 3, 9, 10, or 13)	`filtered`

The Nmap flag for a Maimon scan is `-sM`. While this option was quite useful in 1996, modern systems rarely exhibit this bug. They send a RST back for all ports, making every port appear closed. This result is shown in Example 5.18.

Example 5.18. A failed Maimon scan

```
# nmap -sM -T4 para

Starting Nmap ( https://nmap.org )
All 1000 scanned ports on para (192.168.10.191) are: closed
MAC Address: 00:60:1D:38:32:90 (Lucent Technologies)

Nmap done: 1 IP address (1 host up) scanned in 4.19 seconds
```

TCP Idle Scan (`-sI`)

Fuente original: https://nmap.org/book/idlescan.html

Note

Volunteers have translated this section into Spanish and Portuguese (Brazil)

In 1998, security researcher Antirez (who also wrote the hping2 tool used in parts of this book) posted to the Bugtraq mailing list an ingenious new port scanning technique. Idle scan, as it has become known, allows for completely blind port scanning. Attackers can actually scan a target without sending a single packet to the target from their own IP address! Instead, a clever side-channel attack allows for the scan to be bounced off a dumb "zombie host". Intrusion

detection system (IDS) reports will finger the innocent zombie as the attacker. Besides being extraordinarily stealthy, this scan type permits discovery of IP-based trust relationships between machines.

While idle scanning is more complex than any of the techniques discussed so far, you don't need to be a TCP/IP expert to understand it. It can be put together from these basic facts:

- One way to determine whether a TCP port is open is to send a SYN (session establishment) packet to the port. The target machine will respond with a SYN/ACK (session request acknowledgment) packet if the port is open, and RST (reset) if the port is closed. This is the basis of the previously discussed SYN scan.
- A machine that receives an unsolicited SYN/ACK packet will respond with a RST. An unsolicited RST will be ignored.
- Every IP packet on the Internet has a fragment identification number (IP ID). Since many operating systems simply increment this number for each packet they send, probing for the IPID can tell an attacker how many packets have been sent since the last probe.

By combining these traits, it is possible to scan a target network while forging your identity so that it looks like an innocent zombie machine did the scanning.

Idle Scan Step by Step

Fundamentally, an idle scan consists of three steps that are repeated for each port:

1. Probe the zombie's IP ID and record it.
2. Forge a SYN packet from the zombie and send it to the desired port on the target. Depending on the port state, the target's reaction may or may not cause the zombie's IP ID to be incremented.
3. Probe the zombie's IP ID again. The target port state is then determined by comparing this new IP ID with the one recorded in step 1.

After this process, the zombie's IP ID should have increased by either one or two. An increase of one indicates that the zombie hasn't sent out any packets, except for its reply to the attacker's probe. This lack of sent packets means that the port is not open (the target must have sent the zombie either a RST packet, which was ignored, or nothing at all). An increase of two indicates that the zombie sent out a packet between the two probes. This extra packet usually means that the port is open (the target presumably sent the zombie a SYN/ACK packet in response to the forged SYN, which induced a RST packet from the zombie). Increases larger than two usually signify a bad zombie host. It might not have predictable IP ID numbers, or might be engaged in communication unrelated to the idle scan.

Even though what happens with a closed port is slightly different from what happens with a filtered port, the attacker measures the same result in both cases, namely, an IP ID increase of 1. Therefore it is not possible for the idle scan to distinguish between closed and filtered ports. When Nmap records an IP ID increase of 1 it marks the port `closed|filtered`.

For those wanting more detail, the following three diagrams show exactly what happens in the three cases of an open, closed, and filtered port. The actors in each are:

 the attacker,  the zombie, and the target.

Figure 5.6. Idle scan of an open port

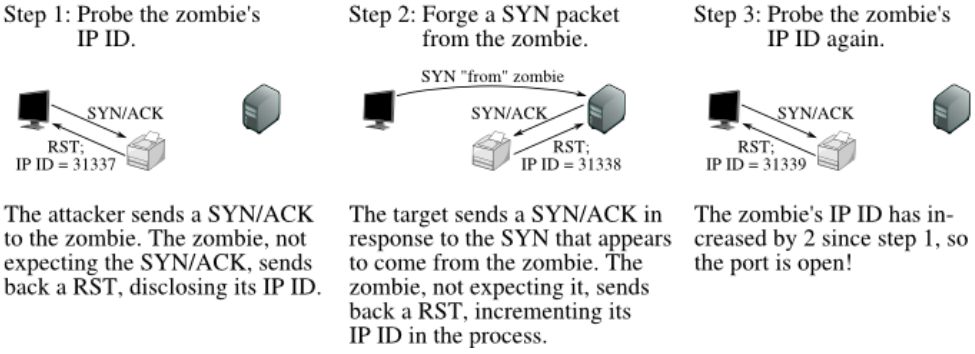
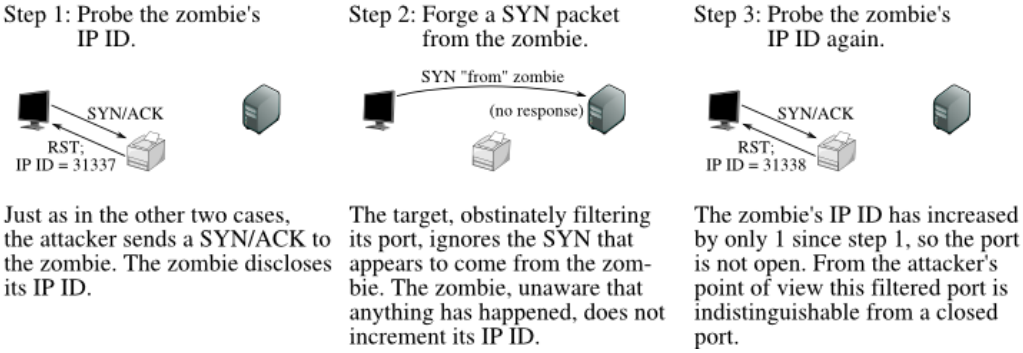


Figure 5.7. Idle scan of a closed port



Figure 5.8. Idle scan of a filtered port



Idle scan is the ultimate stealth scan. Nmap offers decoy scanning (`-D`) to help users shield their identity, but that (unlike idle scan) still requires an attacker to send some packets to the target from his real IP address in order to get scan results back. One upshot of idle scan is that intrusion detection systems will generally send alerts claiming that the zombie machine has launched a scan against them. So it can be used to frame some other party for a scan. Keep this possibility in mind when reading alerts from your IDS.

A unique advantage of idle scan is that it can be used to defeat certain packet filtering firewalls and routers. IP source address filtering is a common (though weak) security mechanism for limiting machines that may connect to a sensitive host or network. For example, a company database server might only allow connections from the public web server that accesses it. Or a home user might only allow SSH (interactive login) connections from his work machines.

A more disturbing scenario occurs when some company bigwig demands that network administrators open a firewall hole so he can access internal network resources from his home IP address. This can happen when executives are unwilling or unable to use secure VPN alternatives.

Idle scanning can sometimes be used to map out these trust relationships. The key factor is that idle scan results list open ports from the zombie host's perspective. A normal scan against the aforementioned database server might show no ports open, but performing an idle scan while using the web server's IP as the zombie could expose the trust relationship by showing the database-related service ports as open.

Mapping out these trust relationships can be very useful to attackers for prioritizing targets. The web server discussed above may seem mundane to an attacker until she notices its special database access.

A disadvantage to idle scanning is that it takes far longer than most other scan types. Despite the optimized algorithms described in the section called "Idle Scan Implementation Algorithms", A 15-second SYN scan could take 15 minutes or more as an idle scan. Another issue is that you must be able to spoof packets as if they are coming from the zombie and have them reach the target machine. Many ISPs (particularly dialup and residential broadband providers) now implement egress filtering to prevent this sort of packet spoofing. Higher end providers (such as colocation and T1 services) are much less likely to do this. If this filtering is in effect, Nmap will print a quick error message for every zombie you try. If changing ISPs is not an option, you might try using another IP on the same ISP network. Sometimes the filtering only blocks spoofing of IP addresses that are outside the range used by customers. Another challenge with idle scan is that you must find a working zombie host, as described in the next section.

Finding a Working Idle Scan Zombie Host

The first step in executing an IP ID idle scan is to find an appropriate zombie. It needs to assign IP ID packets incrementally on a global (rather than per-host it communicates with) basis. It should be idle (hence the scan name), as extraneous traffic will bump up its IP ID sequence, confusing the scan logic. The lower the latency between the attacker and the zombie, and between the zombie and the target, the faster the scan will proceed.

When an idle scan is attempted, Nmap tests the proposed zombie and reports any problems with it. If one doesn't work, try another. Enough Internet hosts are vulnerable that zombie candidates aren't hard to find. Since the hosts need to be idle, choosing a well-known host such as www.yahoo.com or google.com will almost never work.

A common approach is to simply execute a Nmap ping scan of some network. You could use Nmap's random IP selection mode (`-iR`), but that is likely to result in far away zombies with substantial latency. Choosing a network near your source address, or near the target, produces better results. You can try an idle scan using each available host from the ping scan results until you find one that works. As usual, it is best to ask permission before using someone's machines for unexpected purposes such as idle scanning.

We didn't just choose a printer icon to represent a zombie in our illustrations to be funny—simple network devices often make great zombies because they are commonly both underused (idle) and built with simple network stacks which are vulnerable to IP ID traffic detection.

Performing a port scan and OS identification (`-O`) on the zombie candidate network rather than just a ping scan helps in selecting a good zombie. As long as verbose mode (`-v`) is enabled, OS detection will usually determine the IP ID sequence generation method and print a line such as "IP ID Sequence Generation: Incremental". If the type is given as `Incremental` or `Broken little-endian incremental`, the machine is a good zombie candidate. That is still no guarantee that it will work, as Solaris and some other systems create a new IP ID sequence for each host they communicate with. The host could also be too busy. OS detection and the open port list can also help in identifying systems that are likely to be idle.

Another approach to identifying zombie candidates is the run the ipidseq NSE script against a host. This script probes a host to classify its IP ID generation method, then prints the IP ID classification much like the OS detection does. Like most NSE scripts, `ipidseq.nse` can be run against many hosts in parallel, making it another good choice when scanning entire networks looking for suitable hosts.

While identifying a suitable zombie takes some initial work, you can keep re-using the good ones.

Executing an Idle Scan

Once a suitable zombie has been found, performing a scan is easy. Simply specify the zombie hostname to the `-sI` option and Nmap does the rest. Example 5.19 shows an example of Ereet scanning the Recording Industry Association of America by bouncing an idle scan off an Adobe machine named Kiosk.

Example 5.19. An idle scan against the RIAA

```
# nmap -Pn -p- -sI kiosk.adobe.com www.riaa.com

Starting Nmap ( https://nmap.org )
Idlescan using zombie kiosk.adobe.com (192.150.13.111:80); Class: Incremental
Nmap scan report for 208.225.90.120
(The 65522 ports scanned but not shown below are in state: closed)
Port       State       Service
21/tcp     open        ftp
25/tcp     open        smtp
80/tcp     open        http
111/tcp    open        sunrpc
135/tcp    open        loc-srv
443/tcp    open        https
1027/tcp   open        IIS
1030/tcp   open        iad1
2306/tcp   open        unknown
5631/tcp   open        pcanywheredata
7937/tcp   open        unknown
7938/tcp   open        unknown
36890/tcp  open        unknown

Nmap done: 1 IP address (1 host up) scanned in 2594.47 seconds
```

From the scan above, we learn that the RIAA is not very security conscious (note the open PC Anywhere, portmapper, and Legato nsrexec ports). Since they apparently have no firewall, it is unlikely that they have an IDS. But if they do, it will show kiosk.adobe.com as the scan culprit. The `-Pn` option prevents Nmap from sending an initial ping packet to the RIAA machine. That would have disclosed Ereet's true address. The scan took a long time because `-p-` was specified to scan all 65K ports. Don't try to use kiosk for your scans, as it has already been removed.

By default, Nmap forges probes to the target from the source port 80 of the zombie. You can choose a different port by appending a colon and port number to the zombie name (e.g. `-sI kiosk.adobe.com:113`). The chosen port must not be filtered from the attacker or the target. A SYN scan of the zombie should show the port in the `open` or `closed` state.

Idle Scan Implementation Algorithms

While the section called "Idle Scan Step by Step" describes idle scan at the fundamental level, the Nmap implementation is far more complex. Key differences are parallelism for quick execution and redundancy to reduce false positives.

Parallelizing idle scan is trickier than with other scan techniques due to indirect method of deducing port states. If Nmap sends probes to many ports on the target and then checks the new IP ID value of the zombie, the number of IP ID increments will expose how many target ports are open, but not which ones. This isn't actually a major problem, as the vast majority of ports in a large scan will be `closed|filtered`. Since only open ports cause the IP ID value to increment, Nmap will see no intervening increments and can mark the whole group of ports as `closed|filtered`. Nmap can scan groups of up to 100 ports in parallel. If Nmap probes a group then finds that the zombie IP ID has increased *<N>* times, there must be *<N>* open ports among that group. Nmap then finds the open ports with a binary search. It splits the group into two and separately sends probes to each. If a subgroup shows zero open ports, that group's ports are all marked `closed|filtered`. If a subgroup shows one or more open ports, it is divided again and the process continues until those ports are identified. While this technique adds complexity, it can reduce scan times by an order of magnitude over scanning just one port at a time.

Reliability is another major idle scanning concern. If the zombie host sends packets to any unrelated machines during the scan, its IP ID increments. This causes Nmap to think it has found an open port. Fortunately, parallel scanning helps here too. If Nmap scans 100 ports in a group and the IP ID increase signals two open ports, Nmap splits the group into two fifty-port subgroups. When Nmap does an IP ID scan on both subgroups, the total zombie IP ID increase better be two again! Otherwise, Nmap will detect the inconsistency and rescan the groups. It also modifies group size and scan timing based on the detected reliability rate of the zombie. If Nmap detects too many inconsistent results, it will quit and ask the user to provide a better zombie.

Sometimes a packet trace is the best way to understand complex algorithms and techniques such as these. Once again, the Nmap `--packet-trace` makes these trivial to produce when desired. The remainder of this section provides an annotated packet trace of an actual seven port idle scan. The IP addresses have been changed to `Attacker`, `Zombie`, and `Target` and some irrelevant aspects of the trace lines (such as TCP window size) have been removed for clarity.

```
Attacker# nmap -sI Zombie -Pn -p20-25,110 -r --packet-trace -v Target
Starting Nmap ( https://nmap.org )
```

`-Pn` is necessary for stealth, otherwise ping packets would be sent to the target from Attacker's real address. Version scanning would also expose the true address, and so `-sV` is not specified. The `-r` option (turns off port randomization) is only used to make this example easier to follow.

Nmap firsts tests Zombie's IP ID sequence generation by sending six SYN/ACK packets to it and analyzing the responses. This helps Nmap immediately weed out bad zombies. It is also necessary because some systems (usually Microsoft Windows machines, though not all Windows boxes do this) increment the IP ID by 256 for each packet sent rather than by one. This happens on little-endian machines when they don't convert the IP ID to network byte order (big-endian). Nmap uses these initial probes to detect and work around this problem.

```
SENT (0.0060s) TCP Attacker:51824 > Zombie:80 SA id=35996
SENT (0.0900s) TCP Attacker:51825 > Zombie:80 SA id=25914
SENT (0.1800s) TCP Attacker:51826 > Zombie:80 SA id=39591
RCVD (0.1550s) TCP Zombie:80 > Attacker:51824 R id=15669
SENT (0.2700s) TCP Attacker:51827 > Zombie:80 SA id=43604
RCVD (0.2380s) TCP Zombie:80 > Attacker:51825 R id=15670
SENT (0.3600s) TCP Attacker:51828 > Zombie:80 SA id=34186
RCVD (0.3280s) TCP Zombie:80 > Attacker:51826 R id=15671
SENT (0.4510s) TCP Attacker:51829 > Zombie:80 SA id=27949
RCVD (0.4190s) TCP Zombie:80 > Attacker:51827 R id=15672
RCVD (0.5090s) TCP Zombie:80 > Attacker:51828 R id=15673
```

```
RCVD (0.5990s) TCP Zombie:80 > Attacker:51829 R id=15674
Idlescan using zombie Zombie (Zombie:80); Class: Incremental
```

This test demonstrates that the zombie is working fine. Every IP ID was an increase of one over the previous one. So the system appears to be idle and vulnerable to IP ID traffic detection. These promising results are still subject to the next test, in which Nmap spoofs four packets to Zombie as if they are coming from Target. Then it probes the zombie to ensure that the IP ID increased. If it hasn't, then it is likely that either the attacker's ISP is blocking the spoofed packets or the zombie uses a separate IP ID sequence counter for each host it communicates with. Both are common occurrences, so Nmap always performs this test. The last-known Zombie IP ID was 15674, as shown above.

```
SENT (0.5990s) TCP Target:51823 > Zombie:80 SA id=1390
SENT (0.6510s) TCP Target:51823 > Zombie:80 SA id=24025
SENT (0.7110s) TCP Target:51823 > Zombie:80 SA id=15046
SENT (0.7710s) TCP Target:51823 > Zombie:80 SA id=48658
SENT (1.0800s) TCP Attacker:51987 > Zombie:80 SA id=27659
RCVD (1.2290s) TCP Zombie:80 > Attacker:51987 R id=15679
```

The four spoofed packets coupled with the probe from Attacker caused the Zombie to increase its IP ID from 15674 to 15679. Perfect! Now the real scanning begins. Remember that 15679 is the latest Zombie IP ID.

```
Initiating Idlescan against Target
SENT (1.2290s) TCP Zombie:80 > Target:20 S id=13200
SENT (1.2290s) TCP Zombie:80 > Target:21 S id=3737
SENT (1.2290s) TCP Zombie:80 > Target:22 S id=65290
SENT (1.2290s) TCP Zombie:80 > Target:23 S id=10516
SENT (1.4610s) TCP Attacker:52050 > Zombie:80 SA id=33202
RCVD (1.6090s) TCP Zombie:80 > Attacker:52050 R id=15680
```

Nmap probes ports 20-23. Then it probes Zombie and finds that the new IP ID is 15680, only one higher than the previous value of 15679. There were no IP ID increments in between those two known packets, meaning ports 20-23 are probably `closed|filtered`. It is also possible that a SYN/ACK from a Target port has simply not arrived yet. In that case, Zombie has not responded with a RST and thus its IP ID has not incremented. To ensure accuracy, Nmap will try these ports again later.

```
SENT (1.8510s) TCP Attacker:51986 > Zombie:80 SA id=49278
RCVD (1.9990s) TCP Zombie:80 > Attacker:51986 R id=15681
```

Nmap probes again because four tenths of a second has gone by since the last probe it sent. The Zombie (if not truly idle) could have communicated with other hosts during this period, which would cause inaccuracies later if not detected here. Fortunately, that has not happened: the next IP ID is 15681 as expected.

```
SENT (2.0000s) TCP Zombie:80 > Target:24 S id=23928
SENT (2.0000s) TCP Zombie:80 > Target:25 S id=50425
SENT (2.0000s) TCP Zombie:80 > Target:110 S id=14207
SENT (2.2300s) TCP Attacker:52026 > Zombie:80 SA id=26941
RCVD (2.3800s) TCP Zombie:80 > Attacker:52026 R id=15684
```

Nmap probes ports 24, 25, and 110 then queries the Zombie IP ID. It has jumped from 15681 to 15684. It skipped 15682 and 15683, meaning that two of those three ports are likely open. Nmap cannot tell which two are open, and it could also be a false positive. So Nmap drills down deeper, dividing the scan into subgroups.

```
SENT (2.6210s) TCP Attacker:51867 > Zombie:80 SA id=18869
RCVD (2.7690s) TCP Zombie:80 > Attacker:51867 R id=15685
SENT (2.7690s) TCP Zombie:80 > Target:24 S id=30023
SENT (2.7690s) TCP Zombie:80 > Target:25 S id=47253
SENT (3.0000s) TCP Attacker:51979 > Zombie:80 SA id=12077
RCVD (3.1480s) TCP Zombie:80 > Attacker:51979 R id=15687
```

The first subgroup is ports 24 and 25. The IP ID jumps from 15685 to 15687, meaning that one of these two ports is most likely open. Nmap tries the divide and conquer approach again, probing each port separately.

```
SENT (3.3910s) TCP Attacker:51826 > Zombie:80 SA id=32515
RCVD (3.5390s) TCP Zombie:80 > Attacker:51826 R id=15688
SENT (3.5390s) TCP Zombie:80 > Target:24 S id=47868
SENT (3.7710s) TCP Attacker:52012 > Zombie:80 SA id=14042
RCVD (3.9190s) TCP Zombie:80 > Attacker:52012 R id=15689
```

A port 24 probe shows no jump in the IP ID. So that port is not open. From the results so far, Nmap has tentatively determined:

- Ports 20-23 are `closed|filtered`
- Two of the ports 24, 25, and 110 are `open`
- One of the ports 24 and 25 are `open`
- Port 24 is `closed|filtered`

Stare at this puzzle long enough and you'll find only one solution: ports 25 and 110 are open while the other five are `closed|filtered`. Using this logic, Nmap could cease scanning and print results now. It used to do so, but that produced too many false positive open ports when the Zombie wasn't truly idle. So Nmap continues scanning to verify its results:

```
SENT (4.1600s) TCP Attacker:51858 > Zombie:80 SA id=6225
RCVD (4.3080s) TCP Zombie:80 > Attacker:51858 R id=15690
SENT (4.3080s) TCP Zombie:80 > Target:25 S id=35713
SENT (4.5410s) TCP Attacker:51856 > Zombie:80 SA id=28118
RCVD (4.6890s) TCP Zombie:80 > Attacker:51856 R id=15692
Discovered open port 25/tcp on Target
SENT (4.6900s) TCP Zombie:80 > Target:110 S id=9943
SENT (4.9210s) TCP Attacker:51836 > Zombie:80 SA id=62254
RCVD (5.0690s) TCP Zombie:80 > Attacker:51836 R id=15694
Discovered open port 110/tcp on Target
```

Probes of ports 25 and 110 show that they are `open`, as we deduced previously.

```
SENT (5.0690s) TCP Zombie:80 > Target:20 S id=8168
SENT (5.0690s) TCP Zombie:80 > Target:21 S id=36717
SENT (5.0690s) TCP Zombie:80 > Target:22 S id=4063
SENT (5.0690s) TCP Zombie:80 > Target:23 S id=54771
SENT (5.3200s) TCP Attacker:51962 > Zombie:80 SA id=38763
RCVD (5.4690s) TCP Zombie:80 > Attacker:51962 R id=15695
SENT (5.7910s) TCP Attacker:51887 > Zombie:80 SA id=61034
RCVD (5.9390s) TCP Zombie:80 > Attacker:51887 R id=15696
```

Just to be sure, Nmap tries ports 20-23 again. A Zombie IP ID query shows no sequence jump. On the off chance that a SYN/ACK from Target to Zombie came in late, Nmap tries another IP ID query. This again shows no open ports. Nmap is now sufficiently confident with its results to print them.

```
The Idlescan took 5 seconds to scan 7 ports.
Nmap scan report for Target
PORT    STATE           SERVICE
20/tcp  closed|filtered ftp-data
21/tcp  closed|filtered ftp
22/tcp  closed|filtered ssh
23/tcp  closed|filtered telnet
24/tcp  closed|filtered priv-mail
25/tcp  open            smtp
110/tcp open            pop3

Nmap finished: 1 IP address (1 host up) scanned in 5.949 seconds
```

For complete details on the Nmap idle scan implementation, read `idle_scan.cc` from the Nmap source code distribution.

While port scanning is a clever abuse of predictable IP ID sequences, they can be exploited for many other purposes as well. Examples are peppered throughout this book, particularly in Chapter 10, *Detecting and Subverting Firewalls and Intrusion Detection Systems*.

IP Protocol Scan (`-sO`)

Fuente original: https://nmap.org/book/scan-methods-ip-protocol-scan.html

IP protocol scan allows you to determine which IP protocols (TCP, ICMP, IGMP, etc.) are supported by target machines. This isn't technically a port scan, since it cycles through IP protocol numbers rather than TCP or UDP port numbers. Yet it still uses the `-p` option to select scanned protocol numbers, reports its results within the normal port table format, and even uses the same underlying scan engine as the true port scanning methods. So it is close enough to a port scan that it belongs here.

Besides being useful in its own right, protocol scan demonstrates the power of open-source software. While the fundamental idea is pretty simple, I had not thought to add it nor received any requests for such functionality. Then in the summer of 2000, Gerhard Rieger conceived the idea, wrote an excellent patch implementing it, and sent it to the *nmap-hackers* mailing list. I incorporated that patch into the Nmap tree and released a new version the next day. Few pieces of commercial software have users enthusiastic enough to design and contribute their own improvements!

Protocol scan works in a similar fashion to UDP scan. Instead of iterating through the port number field of a UDP packet, it sends IP packet headers and iterates through the eight-bit IP protocol field. The headers are usually empty, containing no data and not even the proper header for the claimed protocol. An exception is made for certain popular protocols (including TCP, UDP, and ICMP). Proper protocol headers for those are included since some systems won't send them otherwise and because Nmap already has functions to create them. Instead of watching for ICMP port unreachable messages, protocol scan is on the lookout for ICMP protocol unreachable messages. Table 5.8 shows how responses to the IP probes are mapped to port states.

Table 5.8. How Nmap interprets responses to an IP protocol probe

Probe Response	Assigned State
Any response in any protocol from target host	`open` (for protocol used by response, not necessarily probe protocol)

ICMP protocol unreachable error (type 3, code 2)	`closed`
Other ICMP unreachable errors (type 3, code 1, 3, 9, 10, or 13)	`filtered` (though they prove ICMP is open if sent from the target machine)
No response received (even after retransmissions)	`open|filtered`

Like open ports in the TCP or UDP protocols, every open protocol is a potential exploitation vector. In addition, protocol scan results help determine the purpose of a machine and what sort of packet filtering is in place. End hosts usually have little more than TCP, UDP, ICMP, and (sometimes) IGMP open, while routers often offer much more, including routing-related protocols such as GRE and EGP. Firewalls and VPN gateways may show encryption-related protocols such as IPsec and SWIPE.

Like the ICMP port unreachable messages received during a UDP scan, ICMP protocol unreachable messages are often rate limited. For example, no more than one ICMP destination unreachable response is sent per second from a default Linux 2.4.20 box. Since there are only 256 possible protocol numbers, this is less of a problem than with a 65,536-port UDP scan. The suggestions in the section called "Speeding Up UDP Scans" apply to speeding up IP protocol scans as well.

Protocol scan is used the same way as most other scan techniques on the command line. Simply specify `-sO` in addition to whatever general Nmap options please you. The normal port (`-p`) option is used to select protocol numbers. Or you can use `-F` to scan all protocols listed in the `nmap-protocols` database. By default, Nmap scans all 256 possible values. Example 5.20 shows Ereet scanning a router in Poland followed by a typical Linux box on my local network.

Example 5.20. IP protocol scan of a router and a typical Linux 2.4 box

```
# nmap -sO 62.233.173.90 para

Starting Nmap ( https://nmap.org )
Nmap scan report for ntwklan-62-233-173-90.devs.futuro.pl (62.233.173.90)
Not shown: 240 closed ports
PROTOCOL STATE         SERVICE
1        open          icmp
4        open|filtered ip
6        open          tcp
8        open|filtered egp
9        open|filtered igp
17       filtered      udp
47       open|filtered gre
53       filtered      swipe
54       open|filtered narp
55       filtered      mobile
77       filtered      sun-nd
80       open|filtered iso-ip
88       open|filtered eigrp
89       open|filtered ospfigp
94       open|filtered ipip
103      filtered      pim

Nmap scan report for para (192.168.10.191)
Not shown: 252 closed ports
PROTOCOL STATE         SERVICE
1        open          icmp
2        open|filtered igmp
6        open          tcp
17       filtered      udp
MAC Address: 00:60:1D:38:32:90 (Lucent Technologies)

Nmap done: 2 IP addresses (2 hosts up) scanned in 458.04 seconds
```

TCP FTP Bounce Scan (-b)

Fuente original: https://nmap.org/book/scan-methods-ftp-bounce-scan.html

An interesting feature of the FTP protocol (RFC 959) is support for so-called proxy FTP connections. This allows a user to connect to one FTP server, then ask that files be sent to a third-party server. Such a feature is ripe for abuse on many levels, so most servers have ceased supporting it. One of the abuses this feature allows is causing the FTP server to port scan other hosts. Simply ask the FTP server to send a file to each interesting port of a target host in turn. The error message will describe whether the port is open or not. This is a good way to bypass firewalls because organizational FTP servers are often placed where they have more access to other internal hosts than any old Internet host would. Nmap supports FTP bounce scan with the `-b` option. It takes an argument of the form *<username>*:*<password>*@*<server>*:*<port>*. *<Server>* is the name or IP address of a vulnerable FTP server. As with a normal URL, you may omit *<username>*:*<password>*, in which case anonymous login credentials (user: `anonymous` password:`-wwwuser@`) are used. The port number (and preceding colon) may be omitted as well, in which case the default FTP port (21) on *<server>* is used.

Example 5.21 shows an attempt to scan google by bouncing off the main Microsoft FTP server.

Example 5.21. Attempting an FTP bounce scan

```
# nmap -Pn -b ftp.microsoft.com google.com

Starting Nmap ( https://nmap.org )
Your FTP bounce server doesn't allow privileged ports, skipping them.
Your FTP bounce server sucks, it won't let us feed bogus ports!
```

Frequent users of the FTP bounce scan better get used to that error message. This vulnerability was widespread in 1997 when Nmap was released, but has largely been fixed. Vulnerable servers are still around, so it is worth trying when all else fails. If bypassing a firewall is your goal, scan the target network for open port 21 (or even for any FTP services if you scan all ports with version detection), then try a bounce scan using each. Nmap will tell you whether the host is vulnerable or not. If you are just trying to cover your tracks, you don't need to (and, in fact, shouldn't) limit yourself to hosts on the target network. Before you go scanning random Internet addresses for vulnerable FTP servers, consider that sysadmins may not appreciate you abusing their servers in this way.

Example 5.22 shows a successful bounce scan against a few interesting ports on Scanme. The verbose option (`-v`) was given to provide extra detail. The given server type of "JD FTP Server" means that this is an HP JetDirect print server.

Example 5.22. Successful FTP bounce scan

```
krad~> nmap -p 22,25,135 -Pn -v -b XXX.YY.111.2 scanme.nmap.org

Starting Nmap ( https://nmap.org )
Attempting connection to ftp://anonymous:-wwwuser@@XXX.YY.111.2:21
Connected:220 JD FTP Server Ready
Login credentials accepted by ftp server!
Initiating TCP ftp bounce scan against scanme.nmap.org (64.13.134.52)
Adding open port 22/tcp
Adding open port 25/tcp
Scanned 3 ports in 12 seconds via the Bounce scan.
Nmap scan report for scanme.nmap.org (64.13.134.52)
PORT    STATE    SERVICE
22/tcp  open     ssh
25/tcp  open     smtp
```

```
135/tcp filtered msrpc

Nmap done: 1 IP address (1 host up) scanned in 21.79 seconds
```

Scan Code and Algorithms

Fuente original: https://nmap.org/book/port-scanning-algorithms.html

In 2004, Nmap's primary port scanning engine was rewritten for greater performance and accuracy. The new engine, known as `ultra_scan` after its function name, handles SYN, connect, UDP, NULL, FIN, Xmas, ACK, window, Maimon, and IP protocol scans, as well as the various host discovery scans. That leaves only idle scan and FTP bounce scan using their own engines.

While the diagrams throughout this chapter show how each scan type works, the Nmap implementation is far more complex since it has to worry about port and host parallelization, latency estimation, packet loss detection, timing profiles, abnormal network conditions, packet filters, response rate limits, and much more.

This section doesn't provide every low-level detail of the `ultra_scan` engine. If you are inquisitive enough to want that, you are better off getting it from the source. You can find `ultra_scan` and its high-level helper functions defined in `scan_engine.cc` from the Nmap tarball. Here I cover the most important algorithmic features. Understanding these helps in optimizing your scans for better performance, as described in Chapter 6, *Optimizing Nmap Performance*.

Network Condition Monitoring

Some authors brag that their scanners are faster than Nmap because of stateless operation. They simply blast out a flood of packets then listen for responses and hope for the best. While this may have value for quick surveys and other cases where speed is more important than comprehensiveness and accuracy, I don't find it appropriate for security scanning. A stateless scanner cannot detect dropped packets in order to retransmit and throttle its send rate. If a busy router half way along the network path drops 80% of the scanner's packet flood, the scanner will still consider the run successful and print results that are woefully inaccurate. Nmap, on the other hand, saves extensive state in RAM while it runs. There is usually plenty of memory available, even on a PDA. Nmap marks each probe with sequence numbers, source or destination ports, ID fields, or other aspects (depending on probe type) which allow it to recognize responses (and thus drops). It then adjusts its speed appropriately to stay as fast as the network (and given command-line options) allow without crossing the line and suffering inaccuracy or unfairly hogging a shared network. Some administrators who have not installed an IDS might not notice an Nmap SYN scan of their whole network. But you better believe the administrator will investigate if you use a brute packet flooding scanner that affects his Quake ping time!

While Nmap's congestion control algorithms are recommended for most scans, they can be overridden. The `--min-rate` option sends packets at the rate you specify (or higher) even if that exceeds Nmap's normal congestion control limits. Similarly, the `--max-retries` option controls how many times Nmap may retransmit a packet. Options such as `--min-rate 100 --max-retries 0` will emulate the behavior of simple stateless scanners. You could double that speed by specifying a rate of 200 packets per second rather than 100, but don't get too greedy—an extremely fast scan is of little value if the results are wrong or incomplete. Any use of `--min-rate` is at your own risk.

Host and Port Parallelization

Most of the diagrams in this chapter illustrate the use of a technique to determine the state of a single port. Sending a probe and receiving the response takes at least one round trip time (RTT) between the source and target machines. If your RTT is 200 ms and you are scanning 65,536 ports on a machine, handling them serially would take at least 3.6 hours. Scan a network of 20,000 machines that way and the wait balloons to more than eight years. This is clearly unacceptable so Nmap parallelizes its scans and is capable of scanning hundreds of ports on each of dozens of machines at the same time. This improves speeds by several orders of magnitude. The number of hosts and ports it scans at a time is dependent on arguments described in Chapter 6, *Optimizing Nmap Performance*, including `--min-hostgroup`, `--min-parallelism`, `-T4`, `--max-rtt-timeout`, and many others. It also depends on network conditions detected by Nmap.

When scanning multiple machines, Nmap tries to efficiently spread the load between them. If a machine appears overwhelmed (drops packets or its latency increases), Nmap slows down for that host while continuing against others at full speed.

Round Trip Time Estimation

Every time a probe response is received, Nmap calculates the microseconds elapsed since the probe was sent. We'll call this the instanceRTT, and Nmap uses it to keep a running tally of three crucial timing-related values: `srtt`, `rttvar`, and `timeout`. Nmap keeps separate values for each host and also merged values for a whole group of hosts scanned in parallel. They are calculated as follows:

`srtt`

The smoothed average round trip time. This is what Nmap uses as its most accurate RTT guess. Rather than use a true arithmetic mean, the formula favors more recent results because network conditions change frequently. The formula is:

```
newsrtt = oldsrtt + (instanceRTT - oldsrtt) / 8
```

`rttvar`

This is the observed variance or deviation in the round trip time. The idea is that if RTT values are quite consistent, Nmap can give up shortly after waiting the `srtt`. If the variance is quite high, Nmap must wait much longer than the `srtt` before giving up on a probe because relatively slow responses are common. The formula follows (ABS represents the absolute value operation):

```
newrttvar = oldrttvar + (ABS(instanceRTT - oldsrtt) - oldrttvar) / 4
```

`timeout`

This is the amount of time Nmap is willing to wait before giving up on a probe. It is calculated as:

```
timeout = newsrtt + newrttvar * 4
```

When a probe times out, Nmap may retransmit the probe or assign a port state such as `filtered` (depending on scan type). Nmap keeps some state information even after a timeout just in case a late response arrives while the overall scan is still in progress.

These simple time estimation formulas seem to work quite well. They are loosely based on similar techniques used by TCP and discussed in RFC 2988, *Computing TCP's Retransmission Timer*. We have optimized those algorithms over the years to better suit port scanning.

Congestion Control

Retransmission timers are far from the only technique Nmap gleaned from TCP. Since Nmap is most commonly used with TCP, it is only fair to follow many of the same rules. Particularly since those rules are the result of substantial research into maximizing throughput without degrading into a tragedy of the commons where everyone selfishly hogs the network. With its default options, Nmap is reasonably polite. Nmap uses three algorithms modeled after TCP to control how aggressive the scan is: a congestion window, exponential backoff, and slow start. The congestion window controls how many probes Nmap may have outstanding at once. If the window is full, Nmap won't send any more until a response is received or a probe times out. Exponential backoff causes Nmap to slow down dramatically when it detects dropped packets. The congestion window is usually reduced to one whenever drops are detected. Despite slow being in the name, slow start is a rather quick algorithm for gradually increasing the scan speed to determine the performance limits of the network.

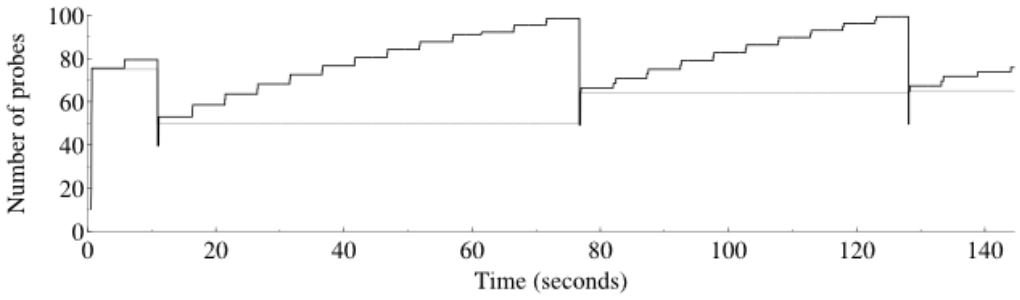
All of these techniques are described in RFC 2581, *TCP Congestion Control*. That document was written by networking gurus Richard Stevens, Vern Paxson, and Mark Allman. It is only 10 pages long and anyone interested in implementing efficient TCP stacks (or other network protocols, or port scanners) should find it fascinating.

When Nmap scans a group of targets, it maintains in memory a congestion window and threshold for each target, as well as a window and threshold for the group as a whole. The congestion window is the number of probes that may be sent at one time. The congestion threshold defines the boundary between slow start and congestion avoidance modes. During slow start, the congestion window grows rapidly in response to responses. Once the congestion window exceeds the congestion threshold, congestion avoidance mode begins, during which the congestion window increases more slowly. After a drop, both the congestion window and threshold are reduced to some fraction of their previous value.

There is an important difference between TCP streams and Nmap port scans, however. In TCP streams, it's normal to expect ACKs in response to every packet sent (or at least a large fraction of them). In fact, proper growth of the congestion window depends on this assumption. Nmap often finds itself in a different situation: facing a target with a default-deny firewall, very few sent packets will ever be responded to. The same thing happens when ping scanning a block of network addresses that contains only a few live hosts. To compensate for this, Nmap keeps track of the ratio of packets sent to responses received. Any time the group congestion window changes, the amount of the change is multiplied by this ratio. In other words, when few packets receive responses, each response carries more weight.

A graphical description of how the group congestion window and threshold vary during a typical port scan is shown in Figure 5.9. The congestion window is shown in black and the congestion threshold is in gray.

Figure 5.9. Congestion window and threshold



The congestion window starts low and the congestion threshold starts high. Slow start mode begins and the window size increases rapidly. The large "stairstep" jumps are the result of timing pings. At about 10 seconds, the congestion window has grown to 80 probes when a drop is detected. Both the congestion window and threshold are reduced. The congestion window continues to grow until about 80 seconds when another drop is detected. Then the cycle repeats, which is typical when network conditions are stable.

Drops during a scan are nothing to be afraid of. The purpose of the congestion control algorithms is to dynamically probe the network to discover its capacity. Viewed in this way, drops are valuable feedback that help Nmap determine the correct size for the congestion window.

Timing probes

Every technique discussed in this algorithms section involves (at some level) network monitoring to detect and estimate network packet loss and latency. This really is critical to obtaining fast scan times. Unfortunately, good data is often difficult to come by when scanning heavily firewalled systems. These filters often drop the overwhelming majority of packets without any response. Nmap may have to send 20,000 probes or more to find one responsive port, making it difficult to monitor network conditions.

To combat this problem, Nmap uses timing probes, also known as port scan pings. If Nmap has found at least one port responsive on a heavily filtered host, it will send a probe to that port every 1.25 seconds that it goes without receiving responses from any other ports. This allows Nmap to conduct a sufficient level of monitoring to speed up or slow down its scans as network conditions allow.

Inferred Neighbor Times

Sometimes even port scan pings won't help because no responsive ports at all have been found. The machine could be down (and scanned with `-Pn`), or every single port could be filtered. Or perhaps the target does have a couple responsive ports, but Nmap has not been lucky enough to find them yet. In these cases, Nmap uses timing values that it maintains for the whole group of machines it is scanning at the same time. As long as at least one response has been received from any machine in the group, Nmap has something to work with. Of course Nmap cannot assume that hosts in a group always share similar timing characteristics. So Nmap tracks the timing variances between responsive hosts in a group. If they differ wildly, Nmap infers long timeouts for neighboring hosts to be on the safe side.

Adaptive Retransmission

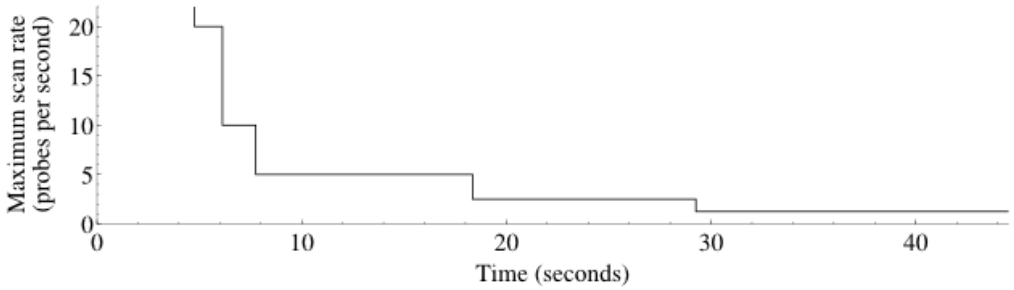
The simplest of scanners (and the stateless ones) generally don't retransmit probes at all. They simply send a probe to each port and report based on the response or lack thereof. Slightly more complex scanners will retransmit a set number of times. Nmap tries to be smarter by keeping careful packet loss statistics for each scan against a target. If no packet loss is detected, Nmap may retransmit only once when it fails to receive a probe response. When massive packet loss is evident, Nmap may retransmit ten or more times. This allows Nmap to scan hosts on fast, reliable networks quickly, while preserving accuracy (at the expense of some speed) when scanning problematic networks or machines. Even Nmap's patience isn't unlimited though. At a certain point (ten retransmissions), Nmap will print a warning and give up on further retransmissions. This prevents malicious hosts from slowing Nmap down too much with intentional packet drops, slow responses, and similar shenanigans. That technique is known as

tarpitting and is commonly used against spammers.

Scan Delay

Packet response rate limiting is perhaps the most pernicious problem faced by port scanners such as Nmap. For example, Linux 2.4 kernels limit ICMP error messages returned during a UDP (`-sU`) or IP protocol (`-sO`) scan to one per second. If Nmap counted these as normal drops, it would be continually slowing down (remember exponential backoff) but still end up having the vast majority of its probes dropped. Instead, Nmap tries to detect this situation. When a large proportion of packets are being dropped, it implements a short delay (as little as 5 milliseconds) between each probe sent to a single target. If drops continue to be a major problem, Nmap will keep doubling the delay until the drops cease or Nmap hits the maximum allowed scan delay. The effects of scan delay while UDP scanning ports 1–50 of a response rate-limited Linux host are shown in Figure 5.10. At the beginning, the scan rate is unlimited by scan delay, though of course other mechanisms such as congestion control impose their own limits. When drops are detected, the scan delay is doubled, meaning that the maximum scan rate is effectively halved. In the graph, for example, a maximum scan rate of five packets per second corresponds to a scan delay of 200 milliseconds.

Figure 5.10. Scan rate as affected by scan delay



The maximum scan delay defaults to one second between probes. The scan delay is sometimes enabled when a slow host can't keep up, even when that host has no explicit rate limiting rules. This can reduce total network traffic substantially by reducing wasted (dropped) probe packets. Unfortunately even small scan delay values can make a scan takes several times as long. Nmap is conservative by default, allowing second-long scan delays for TCP and UDP probes. If your priorities differ, you can configure maximum scan delays with `--max-scan-delay` as discussed in Chapter 5, *Port Scanning Techniques and Algorithms*.

Chapter 7

Service and Application Version Detection

Fuente original: https://nmap.org/book/vscan.html

Introduction

While Nmap does many things, its most fundamental feature is port scanning. Point Nmap at a remote machine, and it might tell you that ports `25/tcp`, `80/tcp`, and `53/udp` are open. Using its `nmap-services` database of more than 2,200 well-known services, Nmap would report that those ports probably correspond to a mail server (SMTP), web server (HTTP), and name server (DNS) respectively. This lookup is usually accurate—the vast majority of daemons listening on TCP port 25 are, in fact, mail servers. However, you should not bet your security on this! People can and do run services on strange ports. Perhaps their main web server was already on port 80, so they picked a different port for a staging or test server. Maybe they think hiding a vulnerable service on some obscure port prevents "evil hackers" from finding it. Even more common lately is that people choose ports based not on the service they want to run, but on what gets through the firewall. When ISPs blocked port 80 after major Microsoft IIS worms CodeRed and Nimda, hordes of users responded by moving their personal web servers to another port. When companies block Telnet access due to its horrific security risks, I have seen users simply run telnetd on the Secure Shell (SSH) port instead.

Even if Nmap is right, and the hypothetical server above is running SMTP, HTTP, and DNS servers, that is not a lot of information. When doing vulnerability assessments (or even simple network inventories) of your companies or clients, you really want to know which mail and DNS servers and versions are running. Having an accurate version number helps dramatically in determining which exploits a server is vulnerable to. Do keep in mind that security fixes are often back-ported to earlier versions of software, so you cannot rely solely on the version number to prove a service is vulnerable. False negatives are rarer, but can happen when silly administrators spoof the version number of a vulnerable service to make it appear patched.

Another good reason for determining the service types and version numbers is that many services share the same port number. For example, port `258/tcp` is used by both the Checkpoint Firewall-1 GUI management interface and the yak Windows chat client. This makes a guess based on the `nmap-services` table even less accurate. Anyone who has done much scanning knows that you also often find services listening on unregistered ports—these are a complete mystery without version detection. A final problem is that filtered UDP ports often look the same to a simple port scanner as open ports (see the section called "UDP Scan (`-sU`)"). But if they respond to the service-specific probes sent by Nmap version detection, you know for sure that they are open (and often exactly what is running).

Service scans sometimes reveal information about a target beyond the service type and version number. Miscellaneous information discovered about a service is collected in the "info" field. This is displayed in the `VERSION` column inside parentheses following the product name and version number. This field can include SSH protocol numbers, Apache modules, and much more.

Some services also report their configured hostnames, which differ from machines' reverse DNS hostnames surprisingly often. The hostname field is reported on a `Service Info` line following

the port table. It sounds like a minor information leak, but can have consequences. One year at the CanSecWest security conference, I was huddled up in my room with my laptop. Suddenly the tcpdump window in the corner of my screen went wild and I realized my machine was under attack. I scanned back and found an unusual high port sitting open. Upon connecting, the port spewed a bunch of binary characters, but one ASCII field in the output gave a configured domain name. The domain was for a small enough security company that I knew exactly who was responsible. I had the front desk ring his hotel room, and boy was he surprised when I asked him to stop probing my box.

Two more fields that version detection can discover are operating system and device type. These are also reported on the `Service Info` line. We use two techniques here. One is application exclusivity. If we identify a service as Microsoft Exchange, we know the operating system is Windows since Exchange doesn't run on anything else. The other technique is to persuade more portable applications to divulge the platform information. Many servers (especially web servers) require very little coaxing. This type of OS detection is intended to complement Nmap's OS detection system (`-O`) and can sometimes report differing results. For example, consider a Microsoft Exchange server hidden behind a port-forwarding Unix firewall.

The Nmap version scanning subsystem obtains all of this data by connecting to open ports and interrogating them for further information using probes that the specific services understand. This allows Nmap to give a detailed assessment of what is really running, rather than just what port numbers are open. Example 7.1 shows the actual output.

Example 7.1. Simple usage of version detection

```
# nmap -sV -T4 -F insecure.org

Starting Nmap ( https://nmap.org )
Nmap scan report for insecure.org (74.207.254.18)
Host is up (0.016s latency).
rDNS record for 74.207.254.18: web.insecure.org
Not shown: 95 filtered ports
PORT    STATE  SERVICE  VERSION
22/tcp  open   ssh      OpenSSH 4.3 (protocol 2.0)
25/tcp  open   smtp     Postfix smtpd
80/tcp  open   http     Apache httpd 2.2.3 ((CentOS))
113/tcp closed auth
443/tcp open   ssl/http Apache httpd 2.2.3 ((CentOS))
Service Info: Host:  web.insecure.org

Nmap done: 1 IP address (1 host up) scanned in 14.82 seconds
```

Nmap version detection offers the following advanced features (fully described later):

- High speed, parallel operation via non-blocking sockets and a probe/match definition grammar designed for efficient yet powerful implementation.
- Determines the application name and version number where available—not just the service protocol.
- Supports both the TCP and UDP protocols, as well as both textual ASCII and packed binary services.
- Multi-platform support, including Linux, Windows, Mac OS X, FreeBSD/NetBSD/OpenBSD, Solaris, and all the other platforms on which Nmap is known to work.
- If SSL is detected, Nmap connects using OpenSSL (if available) and tries to determine what service is listening behind that encryption layer. This allows it to discover services like HTTPS, POP3S, IMAPS, etc. as well as providing version details.

- If a SunRPC service is discovered, Nmap launches its brute-force RPC grinder to find the program number, name, and version number.
- IPv6 is supported, including TCP, UDP, and SSL over TCP.
- Common Platform Enumeration (CPE) output for interoperation with other software (some information is only included in XML output). See the section called "Common Platform Enumeration (CPE)" for more about CPE.
- Community contributions: if Nmap gets data back from a service that it does not recognize, a service fingerprint is printed along with a submission URL. This system is patterned after the extremely successful Nmap OS Detection fingerprint submission process. New probes and corrections can also be submitted.
- Comprehensive database: Nmap recognizes more than one thousand service signatures, covering more than 180 unique service protocols from ACAP, AFP, and AIM to XML-RPC, Zebedee, and Zebra.

Usage and Examples

Fuente original: https://nmap.org/book/vscan-examples.html

Before delving into the technical details of how version detection is implemented, here are some examples demonstrating its usage and capabilities. To enable version detection, just add `-sV` to whatever Nmap flags you normally use. Or use the `-A` option, which turns on version detection and other Advanced and Aggressive features later. It is really that simple, as shown in Example 7.2.

Example 7.2. Version detection against www.microsoft.com

```
# nmap -A -T4 -F www.microsoft.com

Starting Nmap ( https://nmap.org )
Nmap scan report for 80.67.68.30
(The 1208 ports scanned but not shown below are in state: closed)
PORT    STATE    SERVICE     VERSION
22/tcp  open     ssh         Akamai-I SSH (protocol 1.5)
80/tcp  open     http        AkamaiGHost (Akamai's HTTP Acceleration service)
443/tcp open     ssl/http    AkamaiGHost (Akamai's HTTP Acceleration service)
Device type: general purpose
Running: Linux 2.1.X|2.2.X
OS details: Linux 2.1.19 - 2.2.25

Nmap finished: 1 IP address (1 host up) scanned in 19.223 seconds
```

This preceding scan demonstrates a couple things. First of all, it is gratifying to see www.Microsoft.Com served off one of Akamai's Linux boxes. More relevant to this chapter is that the listed service for port 443 is `ssl/http`. That means that service detection first discovered that the port was SSL, then it loaded up OpenSSL and performed service detection again through SSL connections to discover a web server running AkamiGHost behind the encryption. Recall that `-T4` causes Nmap to go faster (more aggressive timing) and `-F` tells Nmap to scan only ports registered in `nmap-services`.

Example 7.3 is a longer and more diverse example.

Example 7.3. Complex version detection

```
# nmap -A -T4 localhost
```

```
Starting Nmap ( https://nmap.org )
Nmap scan report for felix (127.0.0.1)
(The 1640 ports scanned but not shown below are in state: closed)
PORT     STATE SERVICE    VERSION
21/tcp   open  ftp        WU-FTPD wu-2.6.1-20
22/tcp   open  ssh        OpenSSH 3.1p1 (protocol 1.99)
53/tcp   open  domain     ISC BIND 9.2.1
79/tcp   open  finger     Linux fingerd
111/tcp  open  rpcbind    2 (rpc #100000)
443/tcp  open  ssl/http   Apache httpd 2.0.39 ((Unix) mod_perl/1.99_04-dev)
515/tcp  open  printer
631/tcp  open  ipp        CUPS 1.1
953/tcp  open  rndc?
5000/tcp open  ssl/ftp    WU-FTPD wu-2.6.1-20
5001/tcp open  ssl/ssh    OpenSSH 3.1p1 (protocol 1.99)
5002/tcp open  ssl/domain ISC BIND 9.2.1
5003/tcp open  ssl/finger Linux fingerd
6000/tcp open  X11        (access denied)
8000/tcp open  http-proxy Junkbuster webproxy
8080/tcp open  http       Apache httpd 2.0.39 ((Unix) mod_perl/1.99_04-dev)
8081/tcp open  http       Apache httpd 2.0.39 ((Unix) mod_perl/1.99_04-dev)
Device type: general purpose
Running: Linux 2.4.X|2.5.X
OS details: Linux Kernel 2.4.0 - 2.5.20

Nmap finished: 1 IP address (1 host up) scanned in 42.494 seconds
```

You can see here the way RPC services are treated, with the brute-force RPC scanner being used to determine that port 111 is rpcbind version 2. You may also notice that port 515 gives the service as `printer`, but that version field is empty. Nmap determined the service name by probing, but was not able to determine anything else. On the other hand, port 953 gives the service as "`rndc?`". The question mark tells us that Nmap was not even able to determine the service name through probing. As a fallback, rndc is mentioned because that has port 953 registered in `nmap-services`. Unfortunately, none of Nmap's probes elicited any sort of response from rndc. If they had, Nmap would have printed a service fingerprint and a submission URL so that it could be recognized in the next version. As it is, Nmap requires a special probe. One might even be available by the time you read this. the section called "Community Contributions" provides details on writing your own probes.

It is also worth noting that some services provide much more information than just the version number. Examples above include whether X11 permits connections, the SSH protocol number, and the Apache module versions list. Some of the Apache modules even had to be cut from the output to fit on this page.

A few early reviewers questioned the sanity of running services such as SSH and finger over SSL. This was actually just fun with stunnel, in part to ensure that parallel SSL scans actually work.

Technique Described

Fuente original: https://nmap.org/book/vscan-technique.html

Nmap version scanning is actually rather straightforward. It was designed to be as simple as possible while still being scalable, fast, and accurate. The truly nitty-gritty details are best discovered by downloading and reviewing the source code, but a synopsis of the techniques used follows.

Nmap first does a port scan as per your instructions, and then passes all the `open` or `open|filtered` TCP and/or UDP ports to the service scanning module. Those ports are then interrogated in parallel, although a single port is described here for simplicity.

1. Nmap checks to see if the port is one of the ports to be excluded, as specified by the `Exclude` directive in `nmap-service-probes`. If it is, Nmap will not scan this port for reasons mentioned in the section called "`nmap-service-probes` File Format".

2. If the port is TCP, Nmap starts by connecting to it. If the connection succeeds and the port had been in the `open|filtered` state, it is changed to `open`. This is rare (for TCP) since people trying to be so stealthy that they use a TCP scan type which produces `open|filtered` ports (such as FIN scan) generally know better than to blow all of their stealth by performing version detection.

3. Once the TCP connection is made, Nmap listens for roughly five seconds. Many common services, including most FTP, SSH, SMTP, Telnet, POP3, and IMAP servers, identify themselves in an initial welcome banner. Nmap refers to this as the "NULL probe" because Nmap just listens for responses without sending any probe data. If any data is received, Nmap compares it to about 3,000 NULL probe signatures in its `nmap-service-probes` file (described in the section called "`nmap-service-probes` File Format"). If the service is fully identified, we are done with that port! The regular expression in signatures can includes substring matches to pick version numbers out of the response. In some cases, Nmap gets a "soft match" on the service type, but no version info. In that case, Nmap continues but only sends probes that are known to recognize the soft-matched service type.

4. This point is where Nmap starts for UDP probes, and TCP connections continue here if the NULL probe described above fails or soft-matches. Since the reality is that most ports are used by the service they are registered to in `nmap-services`, every probe has a list of port numbers that are considered to be most effective. For example, the probe called GetRequest that recognizes web servers (among many other services) lists 80-85, 8000-8010, and 8080-8085 among its probable ports. Nmap sequentially (in the order they appear in the file) executes the probe(s) that match the port number being scanned.

   Each probe includes a probe string (which can be arbitrary ASCII text or \xHH escaped binary), which is sent to the port. Responses that come back are compared to a list of signature regular expressions of the same type as discussed in the NULL probe description above. As with the NULL probe, these tests can either result in a full match (ends processing for the remote service), a soft match (limits future probes to those which match a certain service), or no match at all. The exact list of regular expressions that Nmap uses to test for a match depends on the probe fallback configuration. For instance, the data returned from the X11Probe is very unlikely to match any regular expressions crafted for the GetRequest probe. On the other hand, it is likely that results returned from a Probe such as RTSPRequest might match a regular expression crafted for GetRequest because the two protocols being tested for are closely related. So the RTSPRequest probe has a fallback to GetRequest matches. For a more comprehensive explanation, see the section called "Cheats and Fallbacks".

   If any response during version detection is ever received from a UDP port which was in the `open|filtered` state, that state is changed to `open`. This makes version detection an excellent complement to UDP scan, which is forced to label all scanned UDP ports as `open|filtered` when some common firewall rules are in effect. While combining UDP scanning with version detection can take many times as long as a plain UDP scan, it is an effective and useful technique. This method is described in the section called "Distinguishing Open from Filtered UDP Ports".

5. In most cases, the NULL probe or the probable port probe(s) (there is usually only one) described above match the service. Since the NULL probe shares its connection with the probable port probe, this allows service detection to be done with only one brief connection in most cases. With UDP only one packet is usually required. But should the

NULL probe and probable port probe(s) fail, Nmap goes through other existing probes sequentially. In the case of TCP, Nmap must make a new connection for each probe to avoid having previous probes corrupt the results. This worst-case scenario can take a bit of time, especially since Nmap must wait about five seconds for the results from each probe because of slow network connections and otherwise slowly responding services. Fortunately, Nmap utilizes several automatic techniques to speed up scans:

- Nmap makes most probes generic enough to match many services. For example, the GenericLines probe sends two blank lines ("`\r\n\r\n`") to the service. This matches daemons of many diverse service types, including FTP, ident, POP3, UUCP, Postgres, and whois. The GetRequest probe matches even more service types. Other examples include "`help\r\n`" and generic RPC and MS SMB probes.
- If a service matches a `softmatch` directive, Nmap only needs to try probes that can potentially match that service.
- All probes were not created equal! Some match many more services than others. Because of this, Nmap uses the rarity metric to avoid trying probes that are extremely unlikely to match. Experienced Nmap users can force all probes to be tried regardless or limit probe attempts even further than the default by using the `--version-intensity`, `--version-all`, and `--version-light` options discussed in the section called "Probe Selection and Rarity".

6. One of the probes tests whether the target port is running SSL. If so (and if OpenSSL is available), Nmap connects back via SSL and restarts the service scan to determine what is listening behind the encryption. A special directive allows different probable ports for normal and SSL tunneled connections. For example, Nmap should start against port 443 (HTTPS) with an SSL probe. But after SSL is detected and enabled, Nmap should try the GetRequest probe against port 443 because that port usually has a web server listening behind SSL encryption.
7. Another generic probe identifies RPC-based services. When these are found, the Nmap RPC grinder (discussed later) is initiated to brute force the RPC program number/name and supported version numbers. Similarly, an SMB post-processor for fingerprinting Windows services is available as part of the Chapter 9, *Nmap Scripting Engine*.
8. If at least one of the probes elicits some sort of response, yet Nmap is unable to recognize the service, the response content is printed to the user in the form of a fingerprint. If users know what services are actually listening, they are encouraged to submit the fingerprint to Nmap developers for integration into Nmap, as described in the section called "Submit Service Fingerprints".

Cheats and Fallbacks

Even though Nmap waits a generous amount of time for services to reply, sometimes an application is slow to respond to the NULL probe. This can occur for a number of reasons, including slow reverse DNS lookups performed by some services. Because of this, Nmap can sometimes match the results from a subsequent probe to a match line designed for the NULL probe.

For example, suppose we scan port 25 (SMTP) on a server to determine what is listening. As soon as we connect, that service may conduct a bunch of DNS blacklist lookups to determine whether we should be treated as spammers and denied service. Before it finishes that, Nmap gives up waiting for a NULL probe response and sends the next probe with port 25 registered, which is "`HELP\r\n`". When the service finally completes its anti-spam checks, it prints a greeting banner, reads the Help probe, and responds as shown in Example 7.4.

Example 7.4. NULL probe cheat example output

```
220 hcsw.org ESMTP Sendmail 8.12.3/8.12.3/Debian-7.1; Tue, [cut]
214-2.0.0 This is sendmail version 8.12.3
214-2.0.0 Topics:
214-2.0.0       HELO    EHLO    MAIL    RCPT    DATA
214-2.0.0       RSET    NOOP    QUIT    HELP    VRFY
214-2.0.0       EXPN    VERB    ETRN    DSN     AUTH
214-2.0.0       STARTTLS
214-2.0.0 For more info use "HELP <topic>".
214-2.0.0 To report bugs in the implementation send email to
214-2.0.0       sendmail-bugs@sendmail.org.
214-2.0.0 For local information send email to Postmaster at your site.
214 2.0.0 End of HELP info
```

Nmap reads this data from the socket and finds that no regular expressions from the Help probe match the data returned. This is because Nmap normally expects to receive the ESMTP banner during the NULL probe and match it there.

Because this is a relatively common scenario, Nmap "cheats" by trying to match responses to any of the NULL Probe match lines if none of the probe-specific lines match. In this case, a null match line exists which reports that the program is Sendmail, the version is 8.12.3/8.12.3/Debian-7.1, and the hostname is hcsw.org.

The NULL probe cheat is actually just a specific example of a more general Nmap feature: fallbacks. The fallback directive is described in detail in the section called "`nmap-service-probes` File Format". Essentially, any probe that is likely to encounter results that can be matched by regular expressions in other probes has a fallback directive that specifies these other probes.

For example, in some configurations of the popular Apache web server, Apache won't respond to the GetRequest ("`GET / HTTP/1.0\r\n\r\n`") probe because no virtual host name has been specified. Nmap is still able to correctly identify these servers because those servers usually respond to the HTTPOptions probe. That probe has a fallback to the GetRequest regular expressions, which are sufficiently general to recognize Apache's responses to the HTTPOptions probes.

Probe Selection and Rarity

In determining what probes to use, Nmap considers their `rarity`. This is an indication of how likely the probe is to return useful data. If a probe has a high rarity, it is considered less common and is less likely to be tried. Nmap users can specify which probes are tried by changing the intensity level of the version scan, as described below. The precise algorithm Nmap uses when determining which probes to use follows:

1. For TCP, the NULL probe is always tried first.
2. All probes that have the port being scanned listed as a probable port (see the section called "`nmap-service-probes` File Format") are tried in the order they appear in `nmap-service-probes`.
3. All other probes that have a rarity value less than or equal to the current intensity value of the scan are tried, also in the order they appear in `nmap-service-probes`.

Once a probe is found to match, the algorithm terminates and results are reported.

Because all of Nmap's probes (other than the NULL probe) have a rarity value associated with them, it is relatively easy to control how many of them are tried when performing a version

scan. Simply choose an intensity level appropriate for a scan. The higher an intensity level, the more probes will be tried. So if a very comprehensive scan is desired, a high intensity level is appropriate—even though it may take longer than a scan conducted at a lower intensity level. Nmap's default intensity level is 7 but Nmap provides the following switches for different scanning needs:

`--version-intensity <intensity level between 0 and 9>`

Sets the intensity level of a version scan to the specified value. If 0 is specified, only the NULL probe (for TCP) and probes that list the port as a probable port are tried. Example: **`nmap -sV --version-intensity 3 scanme.nmap.org`**

`--version-light`

Sets the intensity level to 2. Example: **`nmap -sV --version-light scanme.nmap.org`**

`--version-all`

Sets the intensity level to 9. Since all probes have a rarity level between 1 and 9, this tries all of the probes. Even if a soft match occurs, all additional probes will be tried. Example: **`nmap -sV --version-all scanme.nmap.org`**

Technique Demonstrated

Fuente original: https://nmap.org/book/vscan-technique-demo.html

If the English description above is not clear enough, you can see for yourself how it works by adding the `--version-trace` (and usually `-d` (debugging)) options to your Nmap command line. This shows all the connection and data read/write activity of the service scan. An annotated real-world example follows.

```
# nmap -sSV -T4 -F -d --version-trace insecure.org

Starting Nmap ( https://nmap.org )
Host insecure.org (205.217.153.53) appears to be up ... good.
Initiating SYN Stealth Scan against insecure.org (205.217.153.53) at 19:53
Initiating service scan against 4 services on 1 host at 19:53
```

The SYN scan has found 4 open ports—now we are beginning a service scan against each of them in parallel. We start with a TCP connection for the NULL probe:

```
Starting probes against new service: 205.217.153.53:22 (tcp)
NSOCK (2.0750s) TCP connection requested to 205.217.153.53:22 (IOD #1) EID 8
Starting probes against new service: 205.217.153.53:25 (tcp)
NSOCK (2.0770s) TCP connection requested to 205.217.153.53:25 (IOD #2) EID 16
Starting probes against new service: 205.217.153.53:53 (tcp)
NSOCK (2.0830s) TCP connection requested to 205.217.153.53:53 (IOD #3) EID 24
Starting probes against new service: 205.217.153.53:80 (tcp)
NSOCK (2.0860s) TCP connection requested to 205.217.153.53:80 (IOD #4) EID 32
NSOCK (2.0870s) Callback: CONNECT SUCCESS for EID 32 [205.217.153.53:80]
NSOCK (2.0870s) Read request from IOD #4 [205.217.153.53:80]
                (timeout: 5000ms) EID 42
NSOCK (2.0870s) Callback: CONNECT SUCCESS for EID 24 [205.217.153.53:53]
NSOCK (2.0870s) Read request from IOD #3 [205.217.153.53:53]
                (timeout: 5000ms) EID 50
NSOCK (2.0870s) Callback: CONNECT SUCCESS for EID 16 [205.217.153.53:25]
NSOCK (2.0870s) Read request from IOD #2 [205.217.153.53:25]
                (timeout: 5000ms) EID 58
NSOCK (2.0870s) Callback: CONNECT SUCCESS for EID 8 [205.217.153.53:22]
NSOCK (2.0870s) Read request from IOD #1 [205.217.153.53:22]
                (timeout: 5000ms) EID 66
```

At this point, NULL probe connections have successfully been made to all four services. It starts at 2 seconds because that is how long the ping and SYN scans took.

```
NSOCK (2.0880s) Callback: READ SUCCESS for EID 66 [205.217.153.53:22]
                          (23 bytes): SSH-1.99-OpenSSH_3.1p1.
Service scan match: 205.217.153.53:22 is ssh.
                    Version: |OpenSSH|3.1p1|protocol 1.99|
```

SSH was nice enough to fully identify itself immediately upon connection as OpenSSH 3.1p1. One down, three to go.

```
NSOCK (2.0880s) Callback: READ SUCCESS for EID 58 [205.217.153.53:25]
                          (27 bytes): 220 core.lnxnet.net ESMTP..
Service scan soft match: 205.217.153.53:25 is smtp
```

The mail server on port 25 also gave us a useful banner. We do not know what type of mail server it is, but starting with `220` and including the word `ESMTP` tells us it is a mail (SMTP) server. So Nmap softmatches `smtp`, meaning that only probes able to match SMTP servers are tried from now on. Note that non-printable characters are represented by dots—so the "`..`" after ESMTP is really the "`\r\n`" line termination sequence.

```
NSOCK (2.0880s) Read request from IOD #2 [205.217.153.53:25]
                (timeout: 4996ms) EID 74
NSOCK (7.0880s) Callback: READ TIMEOUT for EID 74 [205.217.153.53:25]
NSOCK (7.0880s) Write request for 6 bytes to IOD #2 EID 83
                [205.217.153.53:25]: HELP..
NSOCK (7.0880s) Read request from IOD #2 [205.217.153.53:25]
                (timeout: 5000ms) EID 90
```

Nmap listens a little longer on the SMTP connection, just in case the server has more to say. The read request times out after five seconds. Nmap then finds the next probe which is registered to port 25 and has SMTP signatures. That probe simply consists of `HELP\r\n`, which Nmap writes into the connection.

```
NSOCK (7.0880s) Callback: READ TIMEOUT for EID 50 [205.217.153.53:53]
NSOCK (7.0880s) Write request for 32 bytes to IOD #3 EID 99
                [205.217.153.53:53]: ..............version.bind.....
NSOCK (7.0880s) Read request from IOD #3 [205.217.153.53:53]
                (timeout: 5000ms) EID 106
```

The DNS server on port 53 does not return anything at all. The first probe registered to port 53 in `nmap-service-probes` is DNSVersionBindReq, which queries a DNS server for its version number. This is sent onto the wire.

```
NSOCK (7.0880s) Callback: READ TIMEOUT for EID 42 [205.217.153.53:80]
NSOCK (7.0880s) Write request for 18 bytes to IOD #4 EID 115
                [205.217.153.53:80]: GET / HTTP/1.0....
NSOCK (7.0880s) Read request from IOD #4 [205.217.153.53:80]
                (timeout: 5000ms) EID 122
```

The port 80 NULL probe also failed to return any data. An HTTP GET request is sent, since that probe is registered to port 80.

```
NSOCK (7.0920s) Callback: READ SUCCESS for EID 122
                [205.217.153.53:80] [EOF](15858 bytes)
Service scan match: insecure.org (205.217.153.53):80 is http.
                    Version: |Apache httpd|2.0.39|(Unix) mod_perl/1.99_07-dev..
```

Apache returned a huge (15KB) response, so it is not printed. That response provided detailed configuration information, which Nmap picks out of the response. There are no other probes registered for port 80. So if this had failed, Nmap would have tried the first TCP probe in `nmap-service-probes`. That probe simply sends blank lines ("`\r\n\r\n`"). A new connection would have been made in case the GET probe confused the service.

```
NSOCK (7.0920s) Callback: READ SUCCESS for EID 106 [205.217.153.53:53]
                        (50 bytes): .0.........version.bind.......9.2.1
Service scan match: insecure.org (205.217.153.53):53 is domain.
                    Version: |ISC BIND|9.2.1||
```

Port 53 responded to our DNS version request. Most of the response (as with the probe) is binary, but you can clearly see the version 9.2.1 there. If this probe had failed, the next probe registered to port 53 is a DNS server status request (14 bytes: `\0\x0C\0\0\x10\0\0\0\0\0\0\0\0\0`). Having this backup probe helps because many more servers respond to a status request than a version number request.

```
NSOCK (7.0920s) Callback: READ SUCCESS for EID 90 [205.217.153.53:25]
                (55 bytes): 214 qmail home page: http...
Service scan match: insecure.org (205.217.153.53):25 is smtp.
                    Version: |qmail smtpd|||
```

Port 25 gives a very helpful response to the Help probe. Other SMTP servers such as Postfix, Courier, and Exim can often be identified by this probe as well. If the response did not match, Nmap would have given up on this service because it had already softmatched `smtp` and there are no more SMTP probes in `nmap-service-probes`.

```
The service scan took 5 seconds to scan 4 services on 1 host.
```

This service scan run went pretty well. No service required more than one connection. It took five seconds because Qmail and Apache hit the five-second NULL probe timeout before Nmap sent the first real probes. Here is the reward for these efforts:

```
Interesting ports on insecure.org (205.217.153.53):
(The 1212 ports scanned but not shown below are in state: closed)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 3.1p1 (protocol 1.99)
25/tcp open  smtp    qmail smtpd
53/tcp open  domain  ISC BIND 9.2.1
80/tcp open  http    Apache httpd 2.0.39 ((Unix) mod_perl/1.99_07-dev)

Nmap finished: 1 IP address (1 host up) scanned in 7.104 seconds
```

Post-processors

Fuente original: https://nmap.org/book/vscan-post-processors.html

Nmap is usually finished working on a port once it has deduced the service and version information as demonstrated above. However, there are certain services for which Nmap performs additional work. The post-processors presently available are Nmap Scripting Engine integration, RPC grinding, and SSL tunneling. Windows SMB interrogation is under consideration.

Nmap Scripting Engine Integration

The regular-expression based approach of version detection is powerful, but it cannot recognize everything. Some services cannot be recognized by simply sending a standard probe and matching a pattern to the response. Some services require custom probe strings or a complex multi-step handshaking process. Others require more advanced processing than a regular expression to recognize a response. For example, the Skype v2 service was designed to be difficult to detect due to the risk that incumbent carriers (such as phone companies providing DSL lines) would consider them a competitor and degrade or block the service from their subscribers. The only way we could find to detect this service involved analyzing responses to two different probes. Similarly, we could recognize more SNMP services if we tried a few hundred different community names by brute force. Neither of these tasks are well suited to traditional Nmap version detection, but both were accomplished with the Chapter 9, *Nmap Scripting Engine*. For these reasons, version detection now calls NSE by default to handle some tricky services, as described in the section called "Version Detection Using NSE".

RPC Grinding

SunRPC (Sun Remote Procedure Call) is a common Unix protocol used to implement many services including NFS. Nmap ships with an `nmap-rpc` database of almost 600 RPC programs. Many RPC services use high-numbered ports and/or the UDP transport protocol, making them available through many poorly configured firewalls. RPC programs (and the infrastructure libraries themselves) also have a long history of serious remotely exploitable security holes. So network administrators and security auditors often wish to learn more about any RPC programs on their networks.

If the portmapper (rpcbind) service (UDP or TCP port 111) is available, RPC services can be enumerated with the Unix **`rpcinfo`** command. Example 7.5 demonstrates this against a default Solaris 9 server.

Example 7.5. Enumerating RPC services with `rpcinfo`

```
> rpcinfo -p ultra
   program vers proto   port
    100000    4   tcp    111  rpcbind
    100000    4   udp    111  rpcbind
    100232   10   udp  32777  sadmind
    100083    1   tcp  32775  ttdbserverd
    100221    1   tcp  32777  kcms_server
    100068    5   udp  32778  cmsd
    100229    1   tcp  32779  metad
    100230    1   tcp  32781  metamhd
    100242    1   tcp  32783  rpc.metamedd
    100001    4   udp  32780  rstatd
    100002    3   udp  32782  rusersd
    100002    3   tcp  32785  rusersd
    100008    1   udp  32784  walld
    100012    1   udp  32786  sprayd
    100011    1   udp  32788  rquotad
    100024    1   udp  32790  status
    100024    1   tcp  32787  status
    100133    1   udp  32790  nsm_addrand
    100133    1   tcp  32787  nsm_addrand
```

```
    [ Dozens of lines cut for brevity ]
```

This example shows that hosts frequently offer many RPC services, which increases the probability that one is exploitable. You should also notice that most of the services are on strange high-numbered ports (which may change for any number of reasons) and split between UDP and TCP transport protocols.

Because the RPC information is so sensitive, many administrators try to obscure this information by blocking the portmapper port (111). Unfortunately, this does not close the hole. Nmap can determine all of the same information by directly communicating with open RPC ports through the following three-step process.

1. The TCP and/or UDP port scan finds all of the open ports.
2. Version detection determines which of the open ports use the SunRPC protocol.
3. The RPC brute force engine determines the program identity of each RPC port by trying a null command against each of the 600 programs numbers in `nmap-rpc`. Most of the time Nmap guesses wrong and receives an error message stating that the requested program number is not listening on the port. Nmap continues trying each number in its list until success is returned for one of them. Nmap gives up in the unlikely event that it exhausts all of its known program numbers or if the port sends malformed responses that suggest it is not really RPC.

The RPC program identification probes are done in parallel, and retransmissions are handled for UDP ports. This feature is automatically activated whenever version detection finds any RPC ports. Example 7.6 demonstrates direct RPC scanning done as part of version detection.

Example 7.6. Nmap direct RPC scan

```
# nmap -F -A -sSU ultra

Starting Nmap ( https://nmap.org )
Nmap scan report for ultra.nmap.org (192.168.0.50)
(The 2171 ports scanned but not shown below are in state: closed)
PORT      STATE SERVICE            VERSION
[A whole bunch of ports cut for brevity]
32776/tcp open  kcms_server        1 (rpc #100221)
32776/udp open  sadmind            10 (rpc #100232)
32777/tcp open  kcms_server        1 (rpc #100221)
32777/udp open  sadmind            10 (rpc #100232)
32778/tcp open  metad              1 (rpc #100229)
32778/udp open  cmsd               2-5 (rpc #100068)
32779/tcp open  metad              1 (rpc #100229)
32779/udp open  rstatd             2-4 (rpc #100001)
32780/tcp open  metamhd            1 (rpc #100230)
32780/udp open  rstatd             2-4 (rpc #100001)
32786/tcp open  status             1 (rpc #100024)
32786/udp open  sprayd             1 (rpc #100012)
32787/tcp open  status             1 (rpc #100024)
32787/udp open  rquotad            1 (rpc #100011)
Device type: general purpose
Running: Sun Solaris 9
OS details: Sun Solaris 9

Nmap finished: 1 IP address (1 host up) scanned in 252.701 seconds
```

SSL Post-processor Notes

As discussed in the technique section, Nmap has the ability to detect the SSL encryption protocol and then launch an encrypted session through which it executes normal version detection. As

with the RPC grinder discussed previously, the SSL post-processor is automatically executed whenever an appropriate (SSL) port is detected. This is demonstrated by Example 7.7.

Example 7.7. Version scanning through SSL

```
$ nmap -Pn -sSV -T4 -F www.amazon.com

Starting Nmap ( https://nmap.org )
Nmap scan report for 207-171-184-16.amazon.com (207.171.184.16)
(The 1214 ports scanned but not shown below are in state: filtered)
PORT    STATE SERVICE  VERSION
80/tcp  open  http     Apache Stronghold httpd 2.4.2 (based on Apache 1.3.6)
443/tcp open  ssl/http Apache Stronghold httpd 2.4.2 (based on Apache 1.3.6)

Nmap finished: 1 IP address (1 host up) scanned in 35.038 seconds
```

Note that the version information is the same for each of the two open ports, but the service is `http` on port 80 and `ssl/http` on port 443. The common case of HTTPS on port 443 is not hard-coded—Nmap should be able to detect SSL on any port and determine the underlying protocol for any service that Nmap can detect in clear-text. If Nmap had not detected the server listening behind SSL, the service listed would be `ssl/unknown`. If Nmap had not been built with SSL support, the service listed would have simply been `ssl`. The version field would be blank in both of these cases.

The SSL support for Nmap depends on the free OpenSSL library. It is not included in the Linux RPM binaries, to avoid breaking systems which lack these libraries. The Nmap source code distribution attempts to detect OpenSSL on a system and link to it when available. See Chapter 2, *Obtaining, Compiling, Installing, and Removing Nmap* for details on customizing the build process to include or exclude OpenSSL.

`nmap-service-probes` File Format

Fuente original: https://nmap.org/book/vscan-fileformat.html

As with remote OS detection (`-O`), Nmap uses a flat file to store the version detection probes and match strings. While the version of `nmap-services` distributed with Nmap is sufficient for most users, understanding the file format allows advanced Nmap hackers to add their own services to the detection engine. Like many Unix files, `nmap-service-probes` is line-oriented. Lines starting with a hash (#) are treated as comments and ignored by the parser. Blank lines are ignored as well. Other lines must contain one of the directives described below. Some readers prefer to peek at the examples in the section called "Putting It All Together" before tackling the following dissection.

`Exclude` Directive

Syntax: `Exclude` *`<port specification>`*

Examples:

```
Exclude 53,T:9100,U:30000-40000
```

This directive excludes the specified ports from the version scan. It can only be used once and should be near the top of the file, above any Probe directives. The Exclude directive uses the

same format as the Nmap `-p` switch, so ranges and comma separated lists of ports are supported. In the `nmap-service-probes` included with Nmap the only ports excluded are TCP port 9100 through 9107. These are common ports for printers to listen on and they often print any data sent to them. So a version detection scan can cause them to print many pages full of probes that Nmap sends, such as SunRPC requests, help statements, and X11 probes.

This behavior is often undesirable, especially when a scan is meant to be stealthy. However, Nmap's default behavior of avoiding scanning this port can make it easier for a sneaky user to hide a service: simply run it on an excluded port such as 9100 and it is less likely to be identified by name. The port scan will still show it as open. Users can override the Exclude directive with the `--allports` option. This causes version detection to interrogate all open ports.

`Probe` Directive

Syntax: `Probe` *`<protocol> <probename> <probestring>`* [`no-payload`]

Examples:

```
Probe TCP GetRequest q|GET / HTTP/1.0\r\n\r\n|
Probe UDP DNSStatusRequest q|\0\0\x10\0\0\0\0\0\0\0\0\0|
Probe TCP NULL q||
Probe UDP Sqlping q|\x02| no-payload
```

The Probe directive tells Nmap what string to send to recognize various services. All of the directives discussed later operate on the most recent `Probe` statement. The arguments are as follows:

`<protocol>`

This must be either `TCP` or `UDP`. Nmap only uses probes that match the protocol of the service it is trying to scan.

`<probename>`

This is a plain English name for the probe. It is used in service fingerprints to describe which probes elicited responses.

`<probestring>`

Tells Nmap what to send. It must start with a `q`, then a delimiter character which begins and ends the string. Between the delimiter characters is the string that is actually sent. It is formatted similarly to a C or Perl string in that it allows the following standard escape characters: `\\` `\0`, `\a`, `\b`, `\f`, `\n`, `\r`, `\t`, `\v`, and `\xHH` (where `H` is any hexadecimal digit). One `Probe` line in `nmap-service-probes` has an empty probe string, as shown in the third example above. This is the TCP NULL probe which just listens for the initial banners that many services send. If your delimiter character (`|` in these examples) is needed for your probe string, you need to choose a different delimiter.

[`no-payload`]

This keyword is used to instruct Nmap not to use the given probe as a protocol-specific payload during UDP port scanning. The `Sqlping` probe, for instance, sometimes triggers IDS alerts for "MS-SQL ping attempt."

`match` Directive

Syntax: `match` *`<service> <pattern>`* [*`<versioninfo>`*]

Examples:

```
match ftp m/^220.*Welcome to .*Pure-?FTPd (\d\S+\s*)/ p/Pure-FTPd/ v/$1/ cpe:/a:pureftpd:pure-ftpd:$1/
match ssh m/^SSH-([\d.]+)-OpenSSH[_-]([\w.]+)\r?\n/i p/OpenSSH/ v/$2/ i/protocol $1/
↪  cpe:/a:openbsd:openssh:$2/
match mysql m|^\x10\0\0\x01\xff\x13\x04Bad handshake$| p/MySQL/ cpe:/a:mysql:mysql/
match chargen m|@ABCDEFGHIJKLMNOPQRSTUVWXYZ|
match uucp m|^login: login: login: $| p/NetBSD uucpd/ o/NetBSD/ cpe:/o:netbsd:netbsd/a
match printer m|^([\w-_.]+): lpd: Illegal service request\n$| p/lpd/ h/$1/
match afs m|^[\d\D]{28}\s*(OpenAFS)([\d\.]{3}[^\s\0]*)\0| p/$1/ v/$2/
```

The match directive tells Nmap how to recognize services based on responses to the string sent by the previous `Probe` directive. A single `Probe` line may be followed by dozens or hundreds of `match` statements. If the given pattern matches, an optional version specifier builds the application name, version number, and additional info for Nmap to report. The arguments to this directive follow:

<service>

This is simply the service name that the pattern matches. Examples would be `ssh`, `smtp`, `http`, or `snmp`. As a special case, you can prefix the service name with `ssl/`, as in `ssl/vmware-auth`. In that case, the service would be stored as `vmware-auth` tunneled by SSL. This is useful for services which can be fully recognized without the overhead of making an SSL connection.

<pattern>

This pattern is used to determine whether the response received matches the service given in the previous parameter. The format is like Perl, with the syntax being `m/[regex]/[opts]`. The "m" tells Nmap that a match string is beginning. The forward slash (`/`) is a delimiter, which can be substituted by almost any printable character as long as the second slash is also replaced to match. The regex is a Perl-style regular expression. This is made possible by the excellent Perl Compatible Regular Expressions (PCRE) library (`http://www.pcre.org`). The only options currently supported are '`i`', which makes a match case-insensitive and '`s`' which includes newlines in the '.' specifier. As you might expect, these two options have the same semantics as in Perl. Subexpressions to be captured (such as version numbers) are surrounded by parentheses as shown in most of the examples above.

<versioninfo>

The *<versioninfo>* section actually contains several optional fields. Each field begins with an identifying letter (such as `h` for "hostname"). Next comes a delimiter character which the signature writer chooses. The preferred delimiter is slash (`'/'`) unless that is used in the field itself. Next comes the field value, followed by the delimiter character. The following table describes the six fields:

Table 7.1. `versioninfo` field formats and values

Field format	Value description
`p/vendorproductname/`	Includes the vendor and often service name and is of the form "Sun Solaris rexecd", "ISC BIND named", or "Apache httpd".
`v/version/`	The application version "number", which may include non-numeric characters and even multiple words.

`i/info/`	Miscellaneous further information which was immediately available and might be useful. Examples include whether an X server is open to unauthenticated connections, or the protocol number of SSH servers.
`h/hostname/`	The hostname (if any) offered up by a service. This is common for protocols such as SMTP and POP3 and is useful because these hostnames may be for internal networks or otherwise differ from the straightforward reverse DNS responses.
`o/operatingsystem/`	The operating system the service is running on. This may legitimately be different than the OS reported by Nmap IP stack based OS detection. For example, the target IP might be a Linux box which uses network address translation to forward requests to an Microsoft IIS server in the DMZ. In this case, stack OS detection should report the OS as Linux, while service detection reports port 80 as being Windows.
`d/devicetype/`	The type of device the service is running on, a string like "print server" or "webcam". Some services disclose this information, and it can be inferred in many more cases. For example, the HP-ChaiServer web server only runs on printers. For a full list of device types, see the section called "Device Types".
`cpe:/cpename/[a]`	A CPE name for some aspect of the service. This may be used multiple times; it's conceivable to be able to identify not only the service (`cpe:/a` names) but also the operating system (`cpe:/o` names) and hardware platform (`cpe:/h` names) as well. The trailing slash is not part of CPE syntax but is included to match the format of other fields. See the section called "Common Platform Enumeration (CPE)" for more information about CPE.

Any of the fields can be omitted. In fact, all of the fields can be omitted if no further information on the service is available. Any of the version fields can include numbered strings such as $1 or $2, which are replaced (in a Perl-like fashion) with the corresponding parenthesized substring in the *<pattern>*. Within `cpe://` templates, such substitutions are transformed as follows: certain characters such as the colon are escaped; spaces are converted to underscores, and all characters are made lower case.

In rare cases, a *helper function* can be applied to the replacement text before insertion. The following table describes the three helper functions available:

Table 7.2. `versioninfo` helper functions

Helper function	Function description

`$P()`	Filters out unprintable characters. This is useful for converting Unicode UTF-16 encoded strings such as `W\0O\0R\0K\0G\0R\0O\0U\0P\0` into the ASCII approximation `WORKGROUP`. It can be used in any `versioninfo` field by passing it the number of the match you want to make printable, like this: `i/$P(3)/`.
`$SUBST()`	Makes substitutions in matches before they are printed. It takes three arguments. The first is the substitution number in the pattern, just as you would use in a normal replacement variable such as `$1` or `$3`. The second and third arguments specify a substring you wish to find and replace, respectively. All instances of the match string found in the substring are replaced, not just the first one. For example, the VanDyke VShell sshd gives its version number in a format such as `2_2_3_578`. We use the versioninfo field `v/$SUBST(1,"_",". ")/` to convert it to the more conventional form `2.2.3.578`.
`$I()`	Unpacks an unsigned integer from the captured bytes. Given a captured string of up to 8 bytes, it will treat them as an unsigned integer and convert it to decimal format. It takes two arguments. The first is the substitution number in the pattern. The second is the string `">"` to indicate that the bytes are in big-endian order, or `"<"` to indicate little-endian.

`softmatch` Directive

Syntax: `softmatch` *`<service>`* *`<pattern>`*

Examples:

```
softmatch ftp m/^220 [-.\w ]+ftp.*\r\n$/i
softmatch smtp m|^220 [-.\w ]+SMTP.*\r\n|
softmatch pop3 m|^\+OK [-\[\]\(\)!,/+:<>@.\w ]+\r\n$|
```

The softmatch directive is similar in format to the match directive discussed above. The main difference is that scanning continues after a softmatch, but it is limited to probes that are known to match the given service. This allows for a normal ("hard") match to be found later, which may provide useful version information. See the section called "Technique Described" for more details on how this works. Arguments are not defined here because they are the same as for `match` above, except that there is never a *`<versioninfo>`* argument. Also as with `match`, many `softmatch` statements can exist within a single `Probe` section.

`ports` and `sslports` Directives

Syntax: `ports` *`<portlist>`*

Examples:

```
ports 21,43,110,113,199,505,540,1248,5432,30444
ports 111,4045,32750-32810,38978
```

This line tells Nmap what ports the services identified by this probe are commonly found on. It should only be used once within each `Probe` section. The syntax is a slightly simplified version of that taken by the Nmap `-p` option. See the examples above. More details on how this works are in the section called "Technique Described".

Syntax: `sslports` *`<portlist>`*

Example:

```
sslports 443
```

This is the same as '`ports`' directive described above, except that these ports are often used to wrap a service in SSL. For example, the HTTP probe declares "`sslports 443`" and SMTP-detecting probes have an "`sslports 465`" line because those are the standard ports for HTTPS and SMTPS respectively. The *`<portlist>`* format is the same as with `ports`. This optional directive cannot appear more than once per `Probe`.

`totalwaitms` Directive

Syntax: `totalwaitms` *`<milliseconds>`*

Example:

```
totalwaitms 6000
```

This rarely necessary directive specifies the amount of time Nmap should wait before giving up on the most recently defined `Probe` against a particular service. The Nmap default is usually fine.

`tcpwrappedms` Directive

Syntax: `tcpwrappedms` *`<milliseconds>`*

Example:

```
tcpwrappedms 3000
```

This directive is only used for the Null probe. If a service closes the TCP connection before this timer runs out, then the service is labeled `tcpwrapped`. Otherwise, matching continues as usual.

`rarity` Directive

Syntax: `rarity` *`<value between 1 and 9>`*

Example:

```
rarity 6
```

The rarity directive roughly corresponds to how infrequently this probe can be expected to return useful results. The higher the number, the more rare the probe is considered and the less likely it is to be tried against a service. More details can be found in the section called "Probe Selection and Rarity".

`fallback` Directive

Syntax: `fallback` *<Comma separated list of probes>*

Example:

```
fallback GetRequest,GenericLines
```

This optional directive specifies which probes should be used as fallbacks for if there are no matches in the current Probe section. For more information on fallbacks see the section called "Cheats and Fallbacks". For TCP probes without a fallback directive, Nmap first tries match lines in the probe itself and then does an implicit fallback to the NULL probe. If the fallback directive is present, Nmap first tries match lines from the probe itself, then those from the probes specified in the fallback directive (from left to right). Finally, Nmap will try the NULL probe. For UDP the behavior is identical except that the NULL probe is never tried.

Putting It All Together

Here are some examples from `nmap-service-probes` which put this all together (to save space many lines have been skipped). After reading this far into the section, the following should be understood.

```
# The Exclude directive takes a comma separated list of ports.
# The format is exactly the same as the -p switch.
Exclude T:9100-9107

# This is the NULL probe that just compares any banners given to us
##############################NEXT PROBE###############################
Probe TCP NULL q||
# Wait for at least 5 seconds for data.  Otherwise an Nmap default is used.
totalwaitms 5000
# Windows 2003
match ftp m/^220[ -]Microsoft FTP Service\r\n/ p/Microsoft ftpd/
match ftp m/^220 ProFTPD (\d\S+) Server/ p/ProFTPD/ v/$1/
softmatch ftp m/^220 [-.\w ]+ftp.*\r\n$/i
match ident m|^flock\(\) on closed filehandle .*midentd| p/midentd/ i/broken/
match imap m|^\* OK Welcome to Binc IMAP v(\d[-.\w]+)| p/Binc IMAPd/ v$1/
softmatch imap m/^\* OK [-.\w ]+imap[-.\w ]+\r\n$/i
match lucent-fwadm m|^0001;2$| p/Lucent Secure Management Server/
match meetingmaker m/^\xc1,$/ p/Meeting Maker calendaring/
# lopster 1.2.0.1 on Linux 1.1
match napster m|^1$| p/Lopster Napster P2P client/

Probe UDP Help q|help\r\n\r\n|
rarity 3
ports 7,13,37
match chargen m|@ABCDEFGHIJKLMNOPQRSTUVWXYZ|
match echo m|^help\r\n\r\n$|
```

Community Contributions

Fuente original: https://nmap.org/book/vscan-community.html

No matter how technically advanced a service detection framework is, it would be nearly useless without a comprehensive database of services against which to match. This is where the open source nature of Nmap really shines. The Insecure.Org lab is pretty substantial by geek standards, but it can never hope to run more than a tiny percentage of machine types and services that are out there. Fortunately experience with OS detection fingerprints has shown that Nmap users together run all of the common stuff, plus a staggering array of bizarre equipment as well. The Nmap OS fingerprint database contains more than a thousand entries, including all sorts of switches, WAPs, VoIP phones, game consoles, Unix boxes, Windows hosts, printers, routers, PDAs, firewalls, etc. Version detection also supports user submissions. Nmap users have contributed thousands of services. There are three primary ways that the Nmap community helps to make this an exceptional database: submitting service fingerprints, database corrections, and new probes.

Submit Service Fingerprints

If a service responds to one or more of Nmap's probes and yet Nmap is unable to identify that service, Nmap prints a service fingerprint like this one:

```
SF-Port21-TCP:V=3.40PVT16%D=9/6%Time=3F5A961C%r(NULL,3F,"220\x20stage\x20F
SF:TP\x20server\x20\(Version\x202\.1WU\(1\)\+SCO-2\.6\.1\+-sec\)\x20ready\
SF:.\r\n")%r(GenericLines,81,"220\x20stage\x20FTP\x20server\x20\(Version\x
SF:202\.1WU\(1\)\+SCO-2\.6\.1\+-sec\)\x20ready\.\r\n500\x20'':\x20command\
SF:x20not\x20understood\.\r\n500\x20'':\x20command\x20not\x20understood\.\
SF:r\n");
```

If you receive such a fingerprint, and are sure you know what daemon version is running on the target host, please submit the fingerprint at the URL Nmap gives you. The whole submission process is anonymous (unless you choose to provide identifying info) and should not take more than a couple minutes. If you are feeling particularly helpful, scan the system again using `-d` (Nmap sometimes gives longer fingerprints that way) and paste both fingerprints into the fingerprint box on the submission form. Sometimes people read the file format section and submit their own working match lines. This is OK, but please submit the service fingerprint(s) as well because existing scripts make integrating and testing them relatively easy.

For those who care, the information in the fingerprint above is port number (21), protocol (TCP), Nmap version (3.40PVT16), date (September 6), Unix time in hex, and a sequence of probe responses in the form r({*<probename>*}, {*<responselength>*}, "{*<responsestring>*}").

Submit Database Corrections

This is another easy way to help improve the database. When integrating a service fingerprint submitted for "chargen on Windows XP" or "FooBar FTP server 3.9.213", it is difficult to determine how general the match is. Will it also match chargen on Solaris or FooBar FTP 2.7? Since there is no good way to tell, a very specific name is used in the hope that people will report when the match needs to be generalized. The only reason the Nmap DB is so comprehensive is that thousands of users have spent a few minutes each to submit new information. If you scan a host and the service fingerprint gives an incorrect OS, version number, application name, or even service type, please let us know as described below:

Upgrade to the latest Nmap (Optional)

Many Linux distributions and other operating systems ship with ancient versions of Nmap. The Nmap version detection database is improved with almost every release, so check your

version number by running **nmap -V** and then compare that to the latest available from https://nmap.org/download.html. The problem you are seeing may have already been corrected. Installing the newest version takes only a few minutes on most platforms, and is valuable regardless of whether the version detection flaw you are reporting still exists. But even if you don't have time to upgrade right now, submissions from older releases are still valuable.

Be absolutely certain you know what is running

Invalid "corrections" can corrupt the version detection DB. If you aren't certain exactly what is running on the remote machine, please find out before submitting.

Generate a fingerprint

Run the command **nmap -O -Pn -sSV -T4 -d --version-trace -p*<port>* *<target>***, where *<port>* is the port running the misidentified service on the *<target>* host. If the service is UDP rather than TCP, substitute **-sUV** for **-sSV**.

Send us your correction

Now simply submit your correction to us at https://insecure.org/cgi-bin/submit.cgi?corr-service. Thanks for contributing to the Nmap community and helping to make version detection even better!

Submit New Probes

Suppose Nmap fails to detect a service. If it received a response to any probes at all, it should provide a fingerprint that can be submitted as described above. But what if there is no response and thus a fingerprint is not available? Create and submit your own probe! These are very welcome. The following steps describe the process.

Steps for creating a new version detection probe

1. Download the latest version of Nmap from https://nmap.org and try again. You would feel a bit silly spending time developing a new probe just to find out that it has already been added. Make sure no fingerprint is available, as it is better to recognize services using existing probes if possible than to create too many new ones. If the service does not respond to any of the existing probes, there is no other choice.

2. Decide on a good probe string for recognizing the service. An ideal probe should elicit a response from as many instances of the service as possible, and ideally the responses should be unique enough to differentiate between them. This step is easiest if you understand the protocol very well, so consider reading the relevant RFCs and product documentation. One simple approach is to simply start a client for the given service and watch what initial handshaking is done by sniffing the network with Wireshark or tcpdump, or connecting to a listening Ncat.

3. Once you have decided on the proper string, add the appropriate new Probe line to Nmap (see the section called "Technique Described" and the section called "**nmap-service-probes** File Format"). Do not put in any match lines at first, although a **ports** directive to make this new test go first against the registered ports is OK. Then scan the service with Nmap a few times. You should get a fingerprint back showing the service's response to your new probe. Send the new probe line and the fingerprints (against different machines if possible, but even a few against the same daemon helps to note differences) to the Nmap development list at <dev@nmap.org>. It will likely then be integrated into future versions of Nmap. Any details you can provide on the nature of your probe string

is helpful as well. For custom services that only appear on your network, it is better to simply add them to your own `nmap-service-probes` rather than the global Nmap.

SOLUTION: Find All Servers Running an Insecure or Nonstandard Application Version

Fuente original: https://nmap.org/book/vscan-find-service-fast.html

Problem

A common task is scanning a range of IP addresses to find all servers of a particular version or even satisfying a particular property. This is something that Nmap's version detection excels in.

One of the most popular database application is the open-source MySQL server. MySQL can be configured to disallow all remote logins from untrusted IPs. This is a good security practice when remote logins aren't required. A case in point: in 2005 a MySQL remote code execution vulnerability was discovered and published. Fortunately, an attacker must be able to log in first—no doubt saving the Internet from yet another devastating worm. In light of problems like this and the fact that SQL logins and passwords are frequently guessable or discoverable through SQL injection attacks, intuition, and inside knowledge of the network, remote logins should be denied when possible.

The problem for a network administrator is to discover MySQL servers that needlessly allow logins from untrusted IPs and take appropriate defensive measures.

Note

This solution was contributed by Nmap developer Doug Hoyte.

Solution

Nmap's version detection comes in handy in this situation because it adds the word unauthorized to the service detection info line when the server forbids our host any access. If we want to scan the network of 10.0.0.0/24 a simple yet effective strategy is to run the following command from an untrusted source:

```
nmap -sV -p 3306 -oG 10.0.0-mysqls-032506.gnmap 10.0.0.0/24
```

Next we can use the Unix **grep** utility to find IPs that accept connections from our IP and don't disallow logins by default (**grep**'s `-v` switch specifies inverse results and only prints out lines that don't match the given pattern):

```
grep 'Ports: 3306/open/tcp//mysql' 10.0.0-mysqls-032506.gnmap | grep -v
unauthorized
```

The resulting output shows the MySQL servers that allow remote logins:

```
Host: 10.0.0.33 (foo.com) Ports: 3306/open/tcp//mysql//MySQL 4.1.11/
Host: 10.0.0.72 (bar.com) Ports: 3306/open/tcp//mysql//MySQL 4.0.24-standard/
Host: 10.0.0.99 () Ports: 3306/open/tcp//mysql//MySQL 4.1.11-Debian_4sarge2/
Host: 10.0.0.154 () Ports: 3306/open/tcp//mysql//MySQL 4.0.25-standard/
Host: 10.0.0.155 () Ports: 3306/open/tcp//mysql//MySQL 4.0.25-standard/
```

Discussion

The trick to this is understanding some MySQL protocol basics and knowing how to read the `nmap-service-probes` file. Grepping the file for `Probe` and `mysql` match lines leads to the following (line wrapped) output:

```
$ cat /usr/local/share/nmap/nmap-service-probes | egrep '^(Probe|match mysql)'
Probe TCP NULL q||
match mysql m/^.\0\0\0\xffj\x04.*Host .* is not allowed to connect to this
              MySQL server$/ p/MySQL/ i/unauthorized/
match mysql m|^.\0\0\0\xffj\x04Host hat keine Berechtigung, eine Verbindung
              zu diesem MySQL Server herzustellen\.| p/MySQL/
              i/unauthorized; German/
match mysql m/^.\0\0\0...Al sistema '[-.\w]+' non e` consentita la
              connessione a questo server MySQL$/ p/MySQL/
              i/unauthorized; Italian/
match mysql m|^.\0\0\0\xffi?\x04?Host .* is blocked because of many connection
              errors\.| p/MySQL/ i/blocked - too many connection errors/
match mysql m/^.\0\0\0.(3\.[-.\w]+)\0.*\x08\x02\0\0\0\0\0\0\0\0\0\0\0\0\0$/s
              p/MySQL/ v/$1/
match mysql m/^.\0\0\0\n(3\.[-.\w]+)\0...\0/s p/MySQL/ v/$1/
match mysql m/^.\0\0\0\n(4\.[-.\w]+)\0.../s p/MySQL/ v/$1/
match mysql m|^.\0\0\0\n(5\.[-.\w]+)\0...\0|s p/MySQL/ v/$1/
match mysql m|^.\0\0\0\xffj\x04'[\d.]+' .* MySQL|s p/MySQL/
Probe TCP GenericLines q|\r\n\r\n|
Probe TCP GetRequest q|GET / HTTP/1.0\r\n\r\n|
Probe TCP HTTPOptions q|OPTIONS / HTTP/1.0\r\n\r\n|
...
```

We see that the `mysql` match lines are designed to be triggered by the NULL probe so no custom probes are needed to determine which servers allow remote logins (for that see the section called "SOLUTION: Hack Version Detection to Suit Custom Needs, such as Open Proxy Detection"). By looking at these `mysql` match lines that we discover MySQL services that don't allow remote logins will result in an info field containing the word `unauthorized`.

In addition to service types and version numbers, there are many cases where version detection is able to gather useful information on scan targets. The probes file is full of such gems that can turn a time-consuming task of protocol research, script coding, locating test servers, and debugging into a simple Nmap command. A few interesting tidbits of information that version detection can sometimes reveal are:

- SSH protocol versions
- Whether a CVS pserver is properly configured
- The usernames used by popular peer-to-peer file sharing clients
- Whether an X server is accepting connections
- Language and other localization parameters of many services
- The wordsize of the target's CPU
- The configured botnames of popular IRC bots such as eggdrop
- Whether posting is allowed on Internet news (NNTP) servers

The version detection database is constantly growing and being refined thanks to the amazing Nmap user community and their service fingerprint submissions. This solution is a good example of how investigating the capabilities of Nmap's service detection can provide elegant, sometimes non-obvious solutions to many diverse problems.

SOLUTION: Hack Version Detection to Suit Custom Needs, such as Open Proxy Detection

Fuente original: https://nmap.org/book/vscan-hack-it.html

Problem

An important part of securing any network is identifying dangerous hosts. Nmap's service detection system is a flexible, reliable way to do this. It can help identify vulnerable versions of software, find misconfigured servers, and more. But sometimes actually trying to misuse services in ways the stock version scan doesn't dare to is the best way to determine if they are actually vulnerable.

Open proxies are servers that will blindly relay requests from untrusted hosts to servers of their choosing. Running these inside a network can be extremely dangerous for many reasons, as attackers can:

- Launch attacks that appear to come from your network
- Steal bandwidth or other network services from you
- Pretend to be an internal client to further escalate their privileges inside your organization

This provides good motivation for hacking version detection to specifically try to exploit open proxies. We could probably map out which ports are proxies by using Nmap's normal proxy match lines, but the best, and only real way to prove an application is vulnerable is to actually exploit it yourself.

Note

This solution was contributed by Nmap developer Doug Hoyte.

Solution

The first thing we do is copy the `nmap-service-probes` file so we can work on a temporary copy:

```
mkdir ~/proxydetect
cp /usr/local/share/nmap/nmap-service-probes ~/proxydetect
```

Next we want to temporarily force Nmap to use our temporary file:

```
export NMAPDIR=$HOME/proxydetect
```

Now we need to add a probe and match line to the file, so open up your favorite editor and place the following text into your copy of `nmap-service-probes`. A good place to put it is after all the match lines in the NULL probe, but immediately before the next Probe line (GenericLines).

```
Probe TCP ProxyProbe q|GET https://insecure.org/ HTTP/1.1\r\nHost: insecure.org\r\n\r\n|
rarity 1
ports 1-65535
totalwaitms 20000
match proxy m|^HTTP/1.[01] 200 OK\r?\n.*TITLE>Insecure.O|s p/Open HTTP Proxy!!/
```

Now Nmap will actually try to request an HTTP download from `insecure.org` by treating any scanned ports as proxies. We will start to see the following in scans of networks containing open proxies:

```
PORT   STATE SERVICE VERSION
80/tcp open  proxy   Open HTTP Proxy!!
```

Discussion

The placement of our probe, the low `rarity` value, and extensive `ports` range help ensure that our custom probe is tried very soon into the service scan so that other probes like GetRequest don't simply identify this as a proxy before we've had a chance to use our active probe.

We also used a `totalwaitms` directive to make Nmap wait longer for this probe to time out. This can be necessary because not only are we dealing with the latency and unreliability of the connection between us and the proxy, but also the latency and unreliability of the connection between the proxy and the server containing the page we requested (`insecure.org`).

Keep in mind that many other protocols can be proxied in addition to HTTP. Version detection will identify proxies for many of them including FTP, POP3, IMAP, and SMTP. SOCKS proxies have special match lines that determine information on the authentication options the proxy has configured. As we did in this solution, often we can use version detection to tell whether such proxies are open or not by using custom probes files. However, more complicated tests are probably best done with NSE scripts.

Chapter 8

Remote OS Detection

Fuente original: https://nmap.org/book/osdetect.html

Introduction

When exploring a network for security auditing or inventory/administration, you usually want to know more than the bare IP addresses of identified machines. Your reaction to discovering a printer may be very different than to finding a router, wireless access point, telephone PBX, game console, Windows desktop, or Unix server. Finer grained detection (such as distinguishing Mac OS X 10.4 from 10.3) is useful for determining vulnerability to specific flaws and for tailoring effective exploits for those vulnerabilities.

In part due to its value to attackers, many systems are tight-lipped about their exact nature and operating system configuration. Fortunately, Nmap includes a huge database of heuristics for identifying thousands of different systems based on how they respond to a selection of TCP/IP probes. Another system (part of version detection) interrogates open TCP or UDP ports to determine device type and OS details. Results of these two systems are reported independently so that you can identify combinations such as a Checkpoint firewall forwarding port 80 to a Windows IIS server.

While Nmap has supported OS detection since 1998, this chapter describes the 2nd generation system released in 2006.

Reasons for OS Detection

While some benefits of discovering the underlying OS and device types on a network are obvious, others are more obscure. This section lists the top reasons I hear for discovering this extra information.

Determining vulnerability of target hosts

It is sometimes very difficult to determine remotely whether an available service is susceptible or patched for a certain vulnerability. Even obtaining the application version number doesn't always help, since OS distributors often back-port security fixes without changing the version number. The surest way to verify that a vulnerability is real is to exploit it, but that risks crashing the service and can lead to wasted hours or even days of frustrating exploitation efforts if the service turns out to be patched.

OS detection can help reduce these false positives. For example, the Rwho daemon on unpatched Sun Solaris 7 through 9 may be remotely exploitable (Sun alert #57659). Remotely determining vulnerability is difficult, but you can rule it out by finding that a target system is running Solaris 10.

Taking this from the perspective of a systems administrator rather than a pen-tester, imagine you run a large Sun shop when alert #57659 comes out. Scan your whole network with OS detection to find machines which need patching before the bad guys do.

Tailoring exploits

Even after you discover a vulnerability in a target system, OS detection can be helpful in exploiting it. Buffer overflows, format-string exploits, and many other vulnerabilities often require custom-tailored shellcode with offsets and assembly payloads generated to match the target OS and hardware architecture. In some cases, you only get one try because the service crashes if you get the shellcode wrong. Use OS detection first or you may end up sending Linux shellcode to a FreeBSD server.

Network inventory and support

While it isn't as exciting as busting root through a specially crafted format string exploit, there are many administrative reasons to keep track of what is running on your network. Before you renew that IRIX support contract for another year, scan to see if anyone still uses such machines. An inventory can also be useful for IT budgeting and ensuring that all company equipment is accounted for.

Detecting unauthorized and dangerous devices

With the ubiquity of mobile devices and cheap commodity networking equipment, companies are increasingly finding that employees are extending their networks in undesirable ways. They may install a $20 wireless access point (WAP) in their cubicle without realizing (or caring) that they just opened up the protected corporate network to potential attackers in the parking lot or nearby buildings. WAPs can be so dangerous that Nmap has a special category for detecting them, as demonstrated in the section called "SOLUTION: Detect Rogue Wireless Access Points on an Enterprise Network". Users may also cause sysadmins grief by connecting insecure and/or worm-infected laptops to the corporate network. Regular scanning can detect unauthorized devices for investigation and containment.

Social engineering

Another possible use is social engineering. Lets say that you are scanning a target company and Nmap reports a "Datavoice TxPORT PRISM 3000 T1 CSU/DSU 6.22/2.06". You could call up the target pretending to be Datavoice support and discuss some issues with their PRISM 3000. Tell them you are about to announce a big security hole, but are first providing the patch to valued customers. Some naive administrators might assume that only an authorized engineer from Datavoice would know so much about their CSU/DSU. Of course the patch you send them is a Trojan horse that gives you remote access to sniff and traipse through their network. Be sure to read the rest of this chapter for detection accuracy and verification advice before trying this. If you guess the target system wrong and they call the police, that will be an embarrassing story to tell your cellmates.

Usage and Examples

Fuente original: https://nmap.org/book/osdetect-usage.html

The inner workings of OS detection are quite complex, but it is one of the easiest features to use. Simply add `-O` to your scan options. You may want to also increase the verbosity with `-v` for even more OS-related details. This is shown in Example 8.1.

Example 8.1. OS detection with verbosity (`-O -v`)

```
# nmap -O -v scanme.nmap.org

Starting Nmap ( https://nmap.org )
Nmap scan report for scanme.nmap.org (74.207.244.221)
Not shown: 994 closed ports
PORT      STATE    SERVICE
22/tcp    open     ssh
80/tcp    open     http
646/tcp   filtered ldp
1720/tcp  filtered H.323/Q.931
9929/tcp  open     nping-echo
31337/tcp open     Elite
Device type: general purpose
Running: Linux 2.6.X
OS CPE: cpe:/o:linux:linux_kernel:2.6.39
OS details: Linux 2.6.39
Uptime guess: 1.674 days (since Fri Sep  9 12:03:04 2011)
Network Distance: 10 hops
TCP Sequence Prediction: Difficulty=205 (Good luck!)
IP ID Sequence Generation: All zeros

Read data files from: /usr/local/bin/../share/nmap
Nmap done: 1 IP address (1 host up) scanned in 5.58 seconds
           Raw packets sent: 1063 (47.432KB) | Rcvd: 1031 (41.664KB)
```

Including the `-O -v` options caused Nmap to generate the following extra line items:

Device type

All fingerprints are classified with one or more high-level device types, such as `router`, `printer`, `firewall`, or (as in this case) `general purpose`. These are further described in the section called "Device and OS classification (`Class` lines)". Several device types may be shown, in which case they will be separated with the pipe symbol as in "`Device Type: router|firewall`".

Running

This field is also related to the OS classification scheme described in the section called "Device and OS classification (`Class` lines)". It shows the OS Family (`Linux` in this case) and OS generation (`2.6.X`) if available. If there are multiple OS families, they are separated by commas. When Nmap can't narrow down OS generations to one specific choice, options are separated by the pipe symbol ('|') Examples include `OpenBSD 3.X, NetBSD 3.X|4.X` and `Linux 2.4.X|2.5.X|2.6.X`.

If Nmap finds too many OS families to print concisely, it will omit this line. When there are no perfect matches, Nmap changes the field to `Running (JUST GUESSING)` and adds an accuracy percentage (100% is a perfect match) in parentheses after each candidate family name. If no fingerprints are close matches, the line is omitted.

OS CPE

This shows a Common Platform Enumeration (CPE) representation of the operating system when available. It may also have a CPE representation of the hardware type. OS CPE begins with `cpe:/o` and hardware CPE begins with `cpe:/h`. For more about CPE see the section called "Common Platform Enumeration (CPE)".

OS details

This line gives the detailed description for each fingerprint that matches. While the `Device type` and `Running` lines are from predefined enumerated lists that are easy to parse by a computer, the OS details line contains free-form data which is useful to a human reading

the report. This can include more exact version numbers, device models, and architectures specific to a given fingerprint. In this example, the only matching fingerprint was `Linux 2.6.20-1 (Fedora Core 5)`. When there are multiple exact matches, they are comma-separated. If there aren't any perfect matches, but some close guesses, the field is renamed `Aggressive OS guesses` and fingerprints are shown followed by a percentage in parentheses which specifies how close each match was.

Uptime guess

As part of OS detection, Nmap receives several SYN/ACK TCP packets in a row and checks the headers for a timestamp option. Many operating systems use a simple counter for this which starts at zero at boot time then increments at a constant rate such as twice per second. By looking at several responses, Nmap can determine the current values and rate of increase. Simple linear extrapolation determines boot time. The timestamp algorithm is used for OS detection too (see the section called "TCP timestamp option algorithm (`TS`)") since the increment rate on different systems varies from 2 Hz to 1,000 Hz.

The uptime guess is labeled a "guess" because various factors can make it completely inaccurate. Some operating systems do not start the timestamp counter at zero, but initialize it with a random value, making extrapolation to zero meaningless. Even on systems using a simple counter starting at zero, the counter eventually overflows and wraps around. With a 1,000 Hz counter increment rate, the counter resets to zero roughly every 50 days. So a host that has been up for 102 days will appear to have been up only two days. Even with these caveats, the uptime guess is accurate much of the time for most operating systems, so it is printed when available, but only in verbose mode. The uptime guess is omitted if the target gives zeros or no timestamp options in its SYN/ACK packets, or if it does not reply at all. The line is also omitted if Nmap cannot discern the timestamp increment rate or it seems suspicious (like a 30-year uptime).

Network Distance

A side effect of one of the OS detection tests allows Nmap to compute how many routers are between it and a target host. The distance is zero when you are scanning localhost, and one for a machine on the same network segment. Each additional router on the path adds one to the hop count. The `Network Distance` line is not printed in this example, since Nmap omits the line when it cannot be computed (no reply to the relevant probe).

TCP Sequence Prediction

Systems with poor TCP initial sequence number generation are vulnerable to blind TCP spoofing attacks. In other words, you can make a full connection to those systems and send (but not receive) data while spoofing a different IP address. The target's logs will show the spoofed IP, and you can take advantage of any trust relationship between them. This attack was all the rage in the mid-nineties when people commonly used rlogin to allow logins to their account without any password from trusted IP addresses. Kevin Mitnick is alleged to have used this attack to break into Tsutomu Shimomura's computers in December 1994.

The good news is that hardly anyone uses rlogin anymore, and many operating systems have been fixed to use unpredictable initial sequence numbers as proposed by RFC 1948. For these reasons, this line is only printed in verbose mode. Sadly, many vendors still ship vulnerable operating systems and devices. Even the fixed ones often vary in implementation, which leaves them valuable for OS detection purposes. The class describes the ISN generation algorithm used by the target, and difficulty is a rough estimate of how hard the system makes blind IP spoofing (0 is the easiest). The parenthesized comment is based on the difficulty index and ranges from `Trivial joke` to `Easy`, `Medium`, `Formidable`, `Worthy challenge`, and finally `Good luck!` Further details about sequence tests are provided in the section called "TCP ISN greatest common divisor (`GCD`)".

While the rlogin family is mostly a relic of the past, clever attackers can still find effective uses for blind TCP spoofing. For example, it allows for spoofed HTTP requests. You don't see the results, but just the URL (POST or GET request) can have dramatic side effects. The spoofing allows attackers to hide their identity, frame someone else, or exploit IP address restrictions.

IP ID sequence generation

Many systems unwittingly give away sensitive information about their traffic levels based on how they generate the lowly 16-bit ID field in IP packets. This can be abused to spoof a port scan against other systems and for other mischievous purposes discussed in the section called "TCP Idle Scan (`-sI`)". This field describes the ID generation algorithm that Nmap was able to discern. More information on how it classifies them is available in the section called "IP ID sequence generation algorithm (`TI`, `CI`, `II`)". Note that many systems use a different IP ID space for each host they communicate with. In that case, they may appear vulnerable (such as showing the `Incremental` class) while still being secure against attacks such as the idle scan. For this reason, and because the issue is rarely critical, the IP ID sequence generation line is only printed in verbose mode. If Nmap does not receive sufficient responses during OS detection, it will omit the whole line. The best way to test whether a host is vulnerable to being an idle scan zombie is to test it with `-sI`.

While TCP fingerprinting is a powerful method for OS detection, interrogating open ports for clues is another effective approach. Some applications, such as Microsoft IIS, only run on a single platform (thus giving it away), while many other apps divulge their platform in overly verbose banner messages. Adding the `-sV` option enables Nmap version detection, which is trained to look for these clues (among others). In Example 8.2, Nmap catches the platform details from an FTP server.

Example 8.2. Using version scan to detect the OS

```
# nmap -sV -O -v 129.128.X.XX

Starting Nmap ( https://nmap.org )
Nmap scan report for [hostname] (129.128.X.XX)
Not shown: 994 closed ports
PORT      STATE    SERVICE      VERSION
21/tcp    open     ftp          HP-UX 10.x ftpd 4.1
22/tcp    open     ssh          OpenSSH 3.7.1p1 (protocol 1.99)
111/tcp   open     rpc
445/tcp   filtered microsoft-ds
1526/tcp  open     oracle-tns   Oracle TNS Listener
32775/tcp open     rpc
No exact OS matches for host
TCP Sequence Prediction: Class=truly random
                         Difficulty=9999999 (Good luck!)
IP ID Sequence Generation: Incremental
Service Info: OS: HP-UX
```

In this example, the line "`No exact OS matches for host`" means that TCP/IP fingerprinting failed to find an exact match. Fortunately, the `Service Info` field a few lines down discloses that the OS is HP-UX. If several operating systems were detected (which can happen with NAT gateway boxes that redirect ports to several different machines), the field would be `OSs` and the values would be comma separated. The `Service Info` line can also contain hostnames and device types found during the version scan. The focus of this chapter is on TCP/IP fingerprinting though, since version detection was covered in Chapter 7, *Service and Application Version Detection*.

With two effective OS detection methods available, which one should you use? The best answer is usually both. In some cases, such as a proxy firewall forwarding to an application on another

host, the answers may legitimately differ. TCP/IP fingerprinting will identify the proxy while version scanning will generally detect the server running the proxied application. Even when no proxying or port forwarding is involved, using both techniques is beneficial. If they come out the same, that makes the results more credible. If they come out wildly different, investigate further to determine what is going on before relying on either. Since OS and version detection go together so well, the `-A` option enables them both.

OS detection is far more effective if at least one open and one closed TCP port are found. Set the `--osscan-limit` option and Nmap will not even try OS detection against hosts which do not meet this criteria. This can save substantial time, particularly on `-Pn` scans against many hosts. You still need to enable OS detection with `-O` (or `-A`) for the `--osscan-limit` option to have any effect.

Another OS detection option is `--osscan-guess`. When Nmap is unable to detect a perfect OS match, it sometimes offers up near-matches as possibilities. The match has to be very close for Nmap to do this by default. If you specify this option (or the equivalent `--fuzzy` option), Nmap will guess more aggressively. Nmap still tells you when an imperfect match is found and display its confidence level (percentage) for each guess.

When Nmap performs OS detection against a target and fails to find a perfect match, it usually repeats the attempt. By default, Nmap tries five times if conditions are favorable for OS fingerprint submission, and twice when conditions aren't so good. The `--max-os-tries` option lets you change this maximum number of OS detection tries. Lowering it (usually to 1) speeds Nmap up, though you miss out on retries which could potentially identify the OS. Alternatively, a high value may be set to allow even more retries when conditions are favorable. This is rarely done, except to generate better fingerprints for submission and integration into the Nmap OS database.

Like just about every other part of Nmap, results ultimately come from the target machine itself. While rare, systems are occasionally configured to confuse or mislead Nmap. Several programs have even been developed specifically to trick Nmap OS detection (see the section called "OS Spoofing"). Your best bet is to use numerous reconnaissance methods to explore a network, and don't trust any one of them.

TCP/IP fingerprinting requires collecting detailed information about the target's IP stack. The most commonly useful results, such as TTL information, are printed to Nmap output whenever they are obtained. Slightly less pertinent information, such as IP ID sequence generation and TCP sequence prediction difficulty, is only printed in verbose mode. But if you want all of the IP stack details that Nmap collected, you can find it in a compact form called a subject fingerprint. Nmap sometimes prints this (for user submission purposes) when it doesn't recognize a host. You can also force Nmap to print it (in normal, interactive, and XML formats) by enabling debugging with (`-d`). Then read the section called "Understanding an Nmap Fingerprint" to interpret it.

TCP/IP Fingerprinting Methods Supported by Nmap

Fuente original: https://nmap.org/book/osdetect-methods.html

Nmap OS fingerprinting works by sending up to 16 TCP, UDP, and ICMP probes to known open and closed ports of the target machine. These probes are specially designed to exploit various ambiguities in the standard protocol RFCs. Then Nmap listens for responses. Dozens of attributes in those responses are analyzed and combined to generate a fingerprint. Every probe packet is tracked and resent at least once if there is no response. All of the packets are IPv4

with a random IP ID value. Probes to an open TCP port are skipped if no such port has been found. For closed TCP or UDP ports, Nmap will first check if such a port has been found. If not, Nmap will just pick a port at random and hope for the best.

The following sections are highly technical and reveal the hidden workings of Nmap OS detection. Nmap can be used effectively without understanding this, though the material can help you better understand remote networks and also detect and explain certain anomalies. Plus, some of the techniques are pretty cool. Readers in a hurry may skip to the section called "Dealing with Misidentified and Unidentified Hosts". But for those of you who are ready for a journey through TCP explicit congestion notification, reserved UDP header bits, initial sequence numbers, bogus flags, and Christmas tree packets: read on!

Even the best of us occasionally forget byte offsets for packet header fields and flags. For quick reference, the IPv4, TCP, UDP, and ICMP header layouts can be found in the section called "TCP/IP Reference". The layout for ICMP echo request and destination unreachable packets are shown in Figure 8.1 and Figure 8.2.

Figure 8.1. ICMP echo request or reply header layout

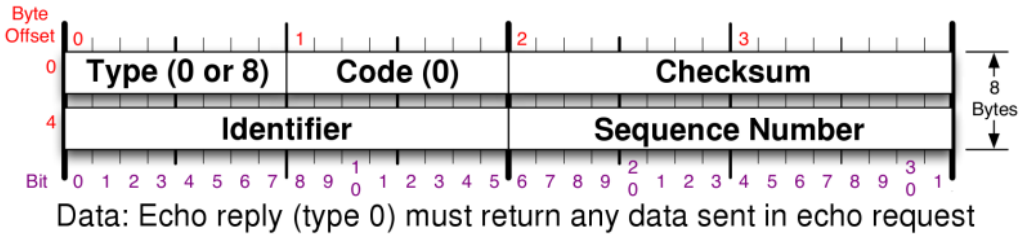
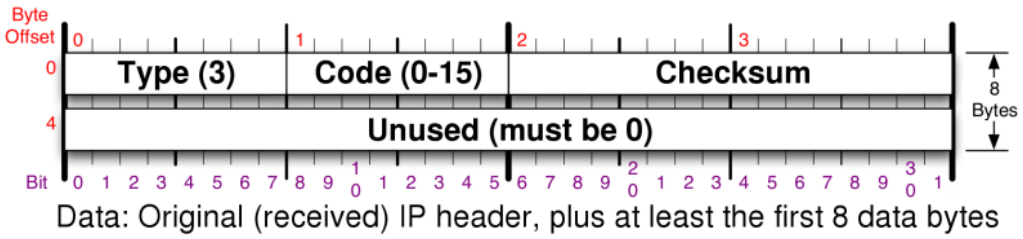


Figure 8.2. ICMP destination unreachable header layout



Probes Sent

This section describes each IP probe sent by Nmap as part of TCP/IP fingerprinting. It refers to Nmap response tests and TCP options which are explained in the following section.

Sequence generation (`SEQ`, `OPS`, `WIN`, and `T1`)

A series of six TCP probes is sent to generate these four test response lines. The probes are sent exactly 100 milliseconds apart so the total time taken is 500 ms. Exact timing is important as some of the sequence algorithms we detect (initial sequence numbers, IP IDs, and TCP timestamps) are time dependent. This timing value was chosen to take 500 ms so that we can reliably detect the common 2 Hz TCP timestamp sequences.

Each probe is a TCP SYN packet to a detected open port on the remote machine. The sequence and acknowledgment numbers are random (but saved so Nmap can differentiate responses).

Detection accuracy requires probe consistency, so there is no data payload even if the user requested one with `--data-length`.

These packets vary in the TCP options they use and the TCP window field value. The following list provides the options and values for all six packets. The listed window field values do not reflect window scaling. EOL is the end-of-options-list option, which many sniffing tools don't show by default.

- **Packet #1:** window scale (10), NOP, MSS (1460), timestamp (TSval: 0xFFFFFFFF; TSecr: 0), SACK permitted. The window field is 1.
- **Packet #2:** MSS (1400), window scale (0), SACK permitted, timestamp (TSval: 0xFFFFFFFF; TSecr: 0), EOL. The window field is 63.
- **Packet #3:** Timestamp (TSval: 0xFFFFFFFF; TSecr: 0), NOP, NOP, window scale (5), NOP, MSS (640). The window field is 4.
- **Packet #4:** SACK permitted, Timestamp (TSval: 0xFFFFFFFF; TSecr: 0), window scale (10), EOL. The window field is 4.
- **Packet #5:** MSS (536), SACK permitted, Timestamp (TSval: 0xFFFFFFFF; TSecr: 0), window scale (10), EOL. The window field is 16.
- **Packet #6:** MSS (265), SACK permitted, Timestamp (TSval: 0xFFFFFFFF; TSecr: 0). The window field is 512.

The results of these tests include four result category lines. The first, `SEQ`, contains results based on sequence analysis of the probe packets. These test results are `GCD`, `SP`, `ISR`, `TI`, `II`, `TS`, and `SS`. The next line, `OPS` contains the TCP options received for each of the probes (the test names are `O1` through `O6`). Similarly, the `WIN` line contains window sizes for the probe responses (named `W1` through `W6`). The final line related to these probes, `T1`, contains various test values for packet #1. Those results are for the `R`, `DF`, `T`, `TG`, `W`, `S`, `A`, `F`, `O`, `RD`, and `Q` tests. These tests are only reported for the first probe since they are almost always the same for each probe.

ICMP echo (`IE`)

The `IE` test involves sending two ICMP echo request packets to the target. The first one has the IP DF bit set, a type-of-service (TOS) byte value of zero, a code of nine (even though it should be zero), the sequence number 295, a random IP ID and ICMP request identifier, and 120 bytes of 0x00 for the data payload.

The second ping query is similar, except a TOS of four (`IP_TOS_RELIABILITY`) is used, the code is zero, 150 bytes of data is sent, and the ICMP request ID and sequence numbers are incremented by one from the previous query values.

The results of both of these probes are combined into a `IE` line containing the `R`, `DFI`, `T`, `TG`, and `CD` tests. The `R` value is only true (Y) if both probes elicit responses. The `T`, and `CD` values are for the response to the first probe only, since they are highly unlikely to differ. `DFI` is a custom test for this special dual-probe ICMP case.

These ICMP probes follow immediately after the TCP sequence probes to ensure valid results of the shared IP ID sequence number test (see the section called "Shared IP ID sequence Boolean (`SS`)").

TCP explicit congestion notification (ECN)

This probe tests for explicit congestion notification (ECN) support in the target TCP stack. ECN is a method for improving Internet performance by allowing routers to signal congestion problems before they start having to drop packets. It is documented in RFC 3168. Nmap tests this by sending a SYN packet which also has the ECN CWR and ECE congestion control flags set. For an unrelated (to ECN) test, the urgent field value of 0xF7F5 is used even though the urgent flag is not set. The acknowledgment number is zero, sequence number is random, window size field is three, and the reserved bit which immediately precedes the CWR bit is set. TCP options are WScale (10), NOP, MSS (1460), SACK permitted, NOP, NOP. The probe is sent to an open port.

If a response is received, the `R`, `DF`, `T`, `TG`, `W`, `O`, `CC`, and `Q` tests are performed and recorded.

TCP (T2–T7)

The six `T2` through `T7` tests each send one TCP probe packet. With one exception, the TCP options data in each case is (in hex) `03030A0102040109080AFFFFFFFF000000000402`. Those 20 bytes correspond to window scale (10), NOP, MSS (265), Timestamp (TSval: 0xFFFFFFFF; TSecr: 0), then SACK permitted. The exception is that `T7` uses a Window scale value of 15 rather than 10. The variable characteristics of each probe are described below:

- **T2** sends a TCP null (no flags set) packet with the IP DF bit set and a window field of 128 to an open port.
- **T3** sends a TCP packet with the SYN, FIN, URG, and PSH flags set and a window field of 256 to an open port. The IP DF bit is not set.
- **T4** sends a TCP ACK packet with IP DF and a window field of 1024 to an open port.
- **T5** sends a TCP SYN packet without IP DF and a window field of 31337 to a closed port.
- **T6** sends a TCP ACK packet with IP DF and a window field of 32768 to a closed port.
- **T7** sends a TCP packet with the FIN, PSH, and URG flags set and a window field of 65535 to a closed port. The IP DF bit is not set.

In each of these cases, a line is added to the fingerprint with results for the `R`, `DF`, `T`, `TG`, `W`, `S`, `A`, `F`, `O`, `RD`, and `Q` tests.

UDP (U1)

This probe is a UDP packet sent to a closed port. The character 'C' (0x43) is repeated 300 times for the data field. The IP ID value is set to 0x1042 for operating systems which allow us to set this. If the port is truly closed and there is no firewall in place, Nmap expects to receive an ICMP port unreachable message in return. That response is then subjected to the `R`, `DF`, `T`, `TG`, `IPL`, `UN`, `RIPL`, `RID`, `RIPCK`, `RUCK`, and `RUD` tests.

Response Tests

The previous section describes probes sent by Nmap, and this one completes the puzzle by describing the barrage of tests performed on responses. The short names (such as `DF`, `R`, and `RIPCK`) are those used in the `nmap-os-db` fingerprint database to save space. All numerical test

values are given in hexadecimal notation, without leading zeros, unless noted otherwise. The tests are documented in roughly the order they appear in fingerprints.

TCP ISN greatest common divisor (GCD)

The SEQ test sends six TCP SYN packets to an open port of the target machine and collects SYN/ACK packets back. Each of these SYN/ACK packets contains a 32-bit initial sequence number (ISN). This test attempts to determine the smallest number by which the target host increments these values. For example, many hosts (especially old ones) always increment the ISN in multiples of 64,000.

The first step in calculating this is creating an array of differences between probe responses. The first element is the difference between the 1st and 2nd probe response ISNs. The second element is the difference between the 2nd and 3rd responses. There are five elements if Nmap receives responses to all six probes. Since the next couple of sections reference this array, we will call it diff1. If an ISN is lower than the previous one, Nmap looks at both the number of values it would have to subtract from the first value to obtain the second, and the number of values it would have to count up (including wrapping the 32-bit counter back to zero). The smaller of those two values is stored in diff1. So the difference between 0x20000 followed by 0x15000 is 0xB000. The difference between 0xFFFFFF00 and 0xC000 is 0xC0FF. This test value then records the greatest common divisor of all those elements. This GCD is also used for calculating the SP result.

TCP ISN counter rate (ISR)

This value reports the average rate of increase for the returned TCP initial sequence number. Recall that a difference is taken between each two consecutive probe responses and stored in the previously discussed diff1 array. Those differences are each divided by the amount of time elapsed (in seconds—will generally be about 0.1) between sending the two probes which generated them. The result is an array, which we'll call seq_rates containing the rates of ISN counter increases per second. The array has one element for each diff1 value. An average is taken of the array values. If that average is less than one (e.g. a constant ISN is used), ISR is zero. Otherwise ISR is eight times the binary logarithm (log base-2) of that average value, rounded to the nearest integer.

TCP ISN sequence predictability index (SP)

While the ISR test measures the average rate of initial sequence number increments, this value measures the ISN variability. It roughly estimates how difficult it would be to predict the next ISN from the known sequence of six probe responses. The calculation uses the difference array (seq_rates) and GCD values discussed in the previous section.

This test is only performed if at least four responses were seen. If the previously computed GCD value is greater than nine, the elements of the previously computed seq_rates array are divided by that value. We don't do the division for smaller GCD values because those are usually caused by chance. A standard deviation of the array of the resultant values is then taken. If the result is one or less, SP is zero. Otherwise the binary logarithm of the result is computed, then it is multiplied by eight, rounded to the nearest integer, and stored as SP.

Please keep in mind that this test is only done for OS detection purposes and is not a full-blown audit of the target ISN generator. There are many algorithm weaknesses that lead to easy predictability even with a high SP value.

IP ID sequence generation algorithm (TI, CI, II)

There are three tests that examine the IP header ID field of responses. TI is based on responses to the TCP SEQ probes. CI is from the responses to the three TCP probes sent to a closed port: T5, T6, and T7. II comes from the ICMP responses to the two IE ping probes. For TI, at least three responses must be received for the test to be included; for CI, at least two responses are required; and for II, both ICMP responses must be received.

For each of these tests, the target's IP ID generation algorithm is classified based on the algorithm below. Minor differences between tests are noted. Note that difference values assume that the counter can wrap. So the difference between an IP ID of 65,100 followed by a value of 700 is 1,136. The difference between 2,000 followed by 1,100 is 64,636. Here are the calculation details:

1. If all of the ID numbers are zero, the value of the test is Z.
2. If the IP ID sequence ever increases by at least 20,000, the value is RD (random). This result isn't possible for II because there are not enough samples to support it.
3. If all of the IP IDs are identical, the test is set to that value in hex.
4. If any of the differences between two consecutive IDs exceeds 1,000, and is not evenly divisible by 256, the test's value is RI (random positive increments). If the difference is evenly divisible by 256, it must be at least 256,000 to cause this RI result.
5. If all of the differences are divisible by 256 and no greater than 5,120, the test is set to BI (broken increment). This happens on systems like Microsoft Windows where the IP ID is sent in host byte order rather than network byte order. It works fine and isn't any sort of RFC violation, though it does give away host architecture details which can be useful to attackers.
6. If all of the differences are less than ten, the value is I (incremental). We allow difference up to ten here (rather than requiring sequential ordering) because traffic from other hosts can cause sequence gaps.
7. If none of the previous steps identify the generation algorithm, the test is omitted from the fingerprint.

Shared IP ID sequence Boolean (SS)

This Boolean value records whether the target shares its IP ID sequence between the TCP and ICMP protocols. If our six TCP IP ID values are 117, 118, 119, 120, 121, and 122, then our ICMP results are 123 and 124, it is clear that not only are both sequences incremental, but they are both part of the same sequence. If, on the other hand, the TCP IP ID values are 117–122 but the ICMP values are 32,917 and 32,918, two different sequences are being used.

This test is only included if II is RI, BI, or I and TI is the same. If SS is included, the result is S if the sequence is shared and O (other) if it is not. That determination is made by the following algorithm:

Let avg be the final TCP sequence response IP ID minus the first TCP sequence response IP ID, divided by the difference in probe numbers. If probe #1 returns an IP ID of 10,000 and probe #6 returns 20,000, avg would be (20,000 - 10,000) / (6 - 1), which equals 2,000.

If the first ICMP echo response IP ID is less than the final TCP sequence response IP ID plus three times avg, the SS result is S. Otherwise it is O.

TCP timestamp option algorithm (TS)

TS is another test which attempts to determine target OS characteristics based on how it generates a series of numbers. This one looks at the TCP timestamp option (if any) in responses to the SEQ probes. It examines the TSval (first four bytes of the option) rather than the echoed TSecr (last four bytes) value. It takes the difference between each consecutive TSval and divides that by the amount of time elapsed between Nmap sending the two probes which generated those responses. The resultant value gives a rate of timestamp increments per second. Nmap computes the average increments per second over all consecutive probes and then calculates the TS as follows:

1. If any of the responses have no timestamp option, TS is set to U (unsupported).
2. If any of the timestamp values are zero, TS is set to 0.
3. If the average increments per second falls within the ranges 0-5.66, 70-150, or 150-350, TS is set to 1, 7, or 8, respectively. These three ranges get special treatment because they correspond to the 2 Hz, 100 Hz, and 200 Hz frequencies used by many hosts.
4. In all other cases, Nmap records the binary logarithm of the average increments per second, rounded to the nearest integer. Since most hosts use 1,000 Hz frequencies, A is a common result.

TCP options (O, O1-O6)

This test records the TCP header options in a packet. It preserves the original ordering and also provides some information about option values. Because RFC 793 doesn't require any particular ordering, implementations often come up with unique orderings. Some platforms don't implement all options (they are, of course, optional). When you combine all of those permutations with the number of different option values that implementations use, this test provides a veritable trove of information. The value for this test is a string of characters representing the options being used. Several options take arguments that come immediately after the character. Supported options and arguments are all shown in Table 8.1.

Table 8.1. O test values

Option Name	Character	Argument (if any)
End of Options List (EOL)	L	
No operation (NOP)	N	
Maximum Segment Size (MSS)	M	The value is appended. Many systems echo the value used in the corresponding probe.
Window Scale (WS)	W	The actual value is appended.
Timestamp (TS)	T	The T is followed by two binary characters representing the TSval and TSecr values respectively. The characters are 0 if the field is zero and 1 otherwise.
Selective ACK permitted (SACK)	S	

As an example, the string M5B4NW3NNT11 means the packet includes the MSS option (value 0x5B4) followed by a NOP. Next comes a window scale option with a value of three, then two more NOPs. The final option is a timestamp, and neither of its two fields were zero. If there are no TCP options in a response, the test will exist but the value string will be empty. If no probe was returned, the test is omitted.

While this test is generally named O, the six probes sent for sequence generation purposes are a special case. Those are inserted into the special OPS test line and take the names O1 through O6 to distinguish which probe packet they relate to. The "O" stands for "options". Despite the different names, each test O1 through O6 is processed exactly the same way as the other O tests.

TCP initial window size (W, W1–W6)

This test simply records the 16-bit TCP window size of the received packet. It is quite effective, since there are more than 80 values that at least one OS is known to send. A down side is that some operating systems have more than a dozen possible values by themselves. This leads to false negative results until we collect all of the possible window sizes used by an operating system.

While this test is generally named W, the six probes sent for sequence generation purposes are a special case. Those are inserted into a special WIN test line and take the names W1 through W6. The window size is recorded for all of the sequence number probes because they differ in TCP MSS option values, which causes some operating systems to advertise a different window size. Despite the different names, each test is processed exactly the same way.

Responsiveness (R)

This test simply records whether the target responded to a given probe. Possible values are Y and N. If there is no reply, remaining fields for the test are omitted.

A risk with this test involves probes that are dropped by a firewall. This leads to R=N in the subject fingerprint. Yet the reference fingerprint in nmap-os-db may have R=Y if the target OS usually replies. Thus the firewall could prevent proper OS detection. To reduce this problem, reference fingerprints generally omit the R=Y test from the IE and U1 probes, which are the ones most likely to be dropped. In addition, if Nmap is missing a closed TCP port for a target, it will not set R=N for the T5, T6, or T7 tests even if the port it tries is non-responsive. After all, the lack of a closed port may be because they are all filtered.

IP don't fragment bit (DF)

The IP header contains a single bit which forbids routers from fragmenting a packet. If the packet is too large for routers to handle, they will just have to drop it (and ideally return a "destination unreachable, fragmentation needed" response). This test records Y if the bit is set, and N if it isn't.

Don't fragment (ICMP) (DFI)

This is simply a modified version of the DF test that is used for the special IE probes. It compares results of the don't fragment bit for the two ICMP echo request probes sent. It has four possible values, which are enumerated in Table 8.2.

Table 8.2. DFI test values

Value	Description
N	Neither of the ping responses have the DF bit set.
S	Both responses echo the DF value of the probe.
Y	Both of the response DF bits are set.

O	The one remaining other combination—both responses have the DF bit toggled.

IP initial time-to-live (T)

IP packets contain a field named time-to-live (TTL) which is decremented every time they traverse a router. If the field reaches zero, the packet must be discarded. This prevents packets from looping endlessly. Because operating systems differ on which TTL they start with, it can be used for OS detection. Nmap determines how many hops away it is from the target by examining the ICMP port unreachable response to the U1 probe. That response includes the original IP packet, including the already-decremented TTL field, received by the target. By subtracting that value from our as-sent TTL, we learn how many hops away the machine is. Nmap then adds that hop distance to the probe response TTL to determine what the initial TTL was when that ICMP probe response packet was sent. That initial TTL value is stored in the fingerprint as the T result.

Even though an eight-bit field like TTL can never hold values greater than 0xFF, this test occasionally results in values of 0x100 or higher. This occurs when a system (could be the source, a target, or a system in between) corrupts or otherwise fails to correctly decrement the TTL. It can also occur due to asymmetric routes.

Nmap can also learn from the system interface and routing tables when the hop distance is zero (localhost scan) or one (on the same network segment). This value is used when Nmap prints the hop distance for the user, but it is not used for T result computation.

IP initial time-to-live guess (TG)

It is not uncommon for Nmap to receive no response to the U1 probe, which prevents Nmap from learning how many hops away a target is. Firewalls and NAT devices love to block unsolicited UDP packets. But since common TTL values are spread well apart and targets are rarely more than 20 hops away, Nmap can make a pretty good guess anyway. Most systems send packets with an initial TTL of 32, 60, 64, 128, or 255. So the TTL value received in the response is rounded up to the next value out of 32, 64, 128, or 255. 60 is not in that list because it cannot be reliably distinguished from 64. It is rarely seen anyway. The resulting guess is stored in the TG field. This TTL guess field is not printed in a subject fingerprint if the actual TTL (T) value was discovered.

Explicit congestion notification (CC)

This test is only used for the ECN probe. That probe is a SYN packet which includes the CWR and ECE congestion control flags. When the response SYN/ACK is received, those flags are examined to set the CC (congestion control) test value as described in Table 8.3.

Table 8.3. CC test values

Value	Description
Y	Only the ECE bit is set (not CWR). This host supports ECN.
N	Neither of these two bits is set. The target does not support ECN.

S	Both bits are set. The target does not support ECN, but it echoes back what it thinks is a reserved bit.
O	The one remaining combination of these two bits (other).

TCP miscellaneous quirks (`Q`)

This tests for two quirks that a few implementations have in their TCP stack. The first is that the reserved field in the TCP header (right after the header length) is nonzero. This is particularly likely to happen in response to the `ECN` test as that one sets a reserved bit in the probe. If this is seen in a packet, an "R" is recorded in the `Q` string.

The other quirk Nmap tests for is a nonzero urgent pointer field value when the URG flag is not set. This is also particularly likely to be seen in response to the `ECN` probe, which sets a non-zero urgent field. A "U" is appended to the `Q` string when this is seen.

The `Q` string must always be generated in alphabetical order. If no quirks are present, the `Q` test is empty but still shown.

TCP sequence number (`S`)

This test examines the 32-bit sequence number field in the TCP header. Rather than record the field value as some other tests do, this one examines how it compares to the TCP acknowledgment number from the probe that elicited the response. It then records the appropriate value as shown in Table 8.4.

Table 8.4. `S` test values

Value	Description
Z	Sequence number is zero.
A	Sequence number is the same as the acknowledgment number in the probe.
A+	Sequence number is the same as the acknowledgment number in the probe plus one.
O	Sequence number is something else (other).

TCP acknowledgment number (`A`)

This test is the same as `S` except that it tests how the acknowledgment number in the response compares to the sequence number in the respective probe. The four possible values are given in Table 8.5.

Table 8.5. `A` test values

Value	Description
Z	Acknowledgment number is zero.
S	Acknowledgment number is the same as the sequence number in the probe.
S+	Acknowledgment number is the same as the sequence number in the probe plus one.
O	Acknowledgment number is something else (other).

TCP flags (F)

This field records the TCP flags in the response. Each letter represents one flag, and they occur in the same order as in a TCP packet (from high-bit on the left, to the low ones). So the value AS represents the ACK and SYN bits set, while the value SA is illegal (wrong order). The possible flags are shown in Table 8.6.

Table 8.6. F test values

Character	Flag name	Flag byte value
E	ECN Echo (ECE)	64
U	Urgent Data (URG)	32
A	Acknowledgment (ACK)	16
P	Push (PSH)	8
R	Reset (RST)	4
S	Synchronize (SYN)	2
F	Final (FIN)	1

TCP RST data checksum (RD)

Some operating systems return ASCII data such as error messages in reset packets. This is explicitly allowed by section 4.2.2.12 of RFC 1122. When Nmap encounters such data, it performs a CRC32 checksum and reports the results. When there is no data, RD is set to zero. Some of the few operating systems that may return data in their reset packets are HP-UX and versions of Mac OS prior to Mac OS X.

IP total length (IPL)

This test records the total length (in octets) of an IP packet. It is only used for the port unreachable response elicited by the U1 test. That length varies by implementation because they are allowed to choose how much data from the original probe to include, as long as they meet the minimum RFC 1122 requirement. That requirement is to include the original IP header and at least eight bytes of data.

Unused port unreachable field nonzero (UN)

An ICMP port unreachable message header is eight bytes long, but only the first four are used. RFC 792 states that the last four bytes must be zero. A few implementations (mostly ethernet switches and some specialized embedded devices) set it anyway. The value of those last four bytes is recorded in this field.

Returned probe IP total length value (RIPL)

ICMP port unreachable messages (as are sent in response to the U1 probe) are required to include the IP header which generated them. This header should be returned just as they received it, but some implementations send back a corrupted version due to changes they made during IP processing. This test simply records the returned IP total length value. If the correct value of 0x148 (328) is returned, the value G (for good) is stored instead of the actual value.

Returned probe IP ID value (RID)

The U1 probe has a static IP ID value of 0x1042. If that value is returned in the port unreachable message, the value G is stored for this test. Otherwise the exact value returned is stored. Some systems, such as Solaris, manipulate IP ID values for raw IP packets that Nmap sends. In such cases, this test is skipped. We have found that some systems, particularly HP and Xerox printers, flip the bytes and return 0x4210 instead.

Integrity of returned probe IP checksum value (RIPCK)

The IP checksum is one value that we don't expect to remain the same when returned in a port unreachable message. After all, each network hop during transit changes the checksum as the TTL is decremented. However, the checksum we receive should match the enclosing IP packet. If it does, the value G (good) is stored for this test. If the returned value is zero, then Z is stored. Otherwise the result is I (invalid).

Integrity of returned probe UDP checksum (RUCK)

The UDP header checksum value should be returned exactly as it was sent. If it is, G is recorded for this test. Otherwise the value actually returned is recorded.

Integrity of returned UDP data (RUD)

This test checks the integrity of the (possibly truncated) returned UDP payload. If all the payload bytes are the expected 'C' (0x43), or if the payload was truncated to zero length, G is recorded; otherwise, I (invalid) is recorded.

ICMP response code (CD)

The code value of an ICMP echo reply (type zero) packet is supposed to be zero. But some implementations wrongly send other values, particularly if the echo request has a nonzero code (as one of the IE tests does). The response code values for the two probes are combined into a CD value as described in Table 8.7.

Table 8.7. CD test values

Value	Description
Z	Both code values are zero.
S	Both code values are the same as in the corresponding probe.
<NN>	When they both use the same non-zero number, it is shown here.
O	Any other combination.

IPv6 fingerprinting

Fuente original: https://nmap.org/book/osdetect-ipv6-methods.html

Nmap has a similar but separate OS detection engine specialized for IPv6. At a high level, the technique is the same: send probes, collect responses, and match the set of responses against a

database. The differences are in the specific probes used, and in the way they are matched.

IPv6 OS detection is used just like IPv4. Just use the `-6` and `-O` options together. For example, `nmap -6 -O <target>`.

Probes Sent

IPv6 OS detection uses many of the same probes that IPv4 OS detection does. Most of the power to distinguish operating systems comes from higher-layer protocols like TCP, though there are a few new IPv6-specific detection features.

In all cases, the IPv6 flow label is 0x12345, on platforms that allow us to set it. On platforms that do not (which includes non-Linux Unix platforms when not using Ethernet to send), the flow label will be 0. Because this can affect the responses, the value of the flow label is recorded in the `EXTRA` field of OS fingerprints. Except for the `NS` probe, hop limits are set randomly.

In all, up to 18 probes may be sent. They are sent in the following order.

Sequence generation (`S1`–`S6`)

These are the same six probes as the `T1` collection of probes sent in IPv4 detection. See the section called "Sequence generation (`SEQ`, `OPS`, `WIN`, and `T1`)" for documentation of the packet contents. These six probes are sent 100 ms apart for timing measurements.

The `S1`–`S6` probes are skipped if the target lacks an open port.

ICMPv6 echo (`IE1`)

This is more or less an ordinary ICMPv6 echo request. The type is 128 (Echo Request) and the code is 9, though it should be 0. The ICMPv6 ID is 0xabcd and the sequence number is 0. The data payload is 120 zero bytes. There is one Hop-By-Hop extension header containing only padding.

ICMPv6 echo (`IE2`)

This is an echo request with a type of 128 (Echo Request) and a code of 0. The ICMPv6 ID is 0xabcd and the sequence is 1. There is no data payload.

What makes this probe interesting are the erroneous extension headers it includes. There are four of them in all, in this order:

Hop-By-Hop
Destination Options
Routing
Hop-By-Hop

These headers are erroneous: no header other than Destination Options is supposed to appear more than once, and Hop-by-hop options are only supposed to appear in the first position. In our tests, no operating systems treat this as a legitimate echo request. They do, however, respond with different ICMPv6 errors.

Node Information Query (NI)

RFC 4620 defines ICMPv6 messages called Node Information Queries that allow asking a target for its hostnames, IPv4 addresses, and IPv6 addresses. The NI probe has type 139 (ICMP Node Information Query) and code 0 (indicating that the subject is an IPv6 address). The qtype is 4 (IPv4 Addresses). The A flag (return all unicast addresses) flag is set, and no others. The nonce is set to the fixed string "\x01\x02\x03\x04\x05\x06\x07\x0a".

Despite being asked for IPv4 addresses, some operating systems return a DNS name instead.

Neighbor Solicitation (NS)

The NS probe sends a Neighbor Solicitation query, as if asking for the target's hardware address. The type is 135 and the code is 0. The hop limit is always set to 255, no matter the setting of --ttl; RFC 2461 forbids hosts to reply otherwise. All flags are set to 0.

This probe is only sent to hosts on the same subnet.

UDP (U1)

A UDP packet is sent to a a closed port, if available. The data payload is set to 300 'C' (0x43) bytes. This probe is designed to elicit an ICMPv6 Port Unreachable message.

TCP explicit congestion notification (TECN)

This is the same as the ECN probe from IPv4. It is a SYN packet to an open port, that also has the ECE and CWR flags set. The urgent field value of 0xF7F5 is used even though the urgent flag is not set. The acknowledgment number is zero, sequence number is random, and the window size field is three. TCP options are WScale (10), NOP, MSS (1460), SACK permitted, NOP, NOP.

TCP (T2–T7)

These correspond to the T2–T7 probes from IPv4 detection, described in the section called "TCP (T2–T7)". The numbering starts at 2 rather than 1 because the six sequencing probes are collectively known as "T1" in IPv4 (they were renamed to S1–S6 for IPv6).

Feature extraction

After responses are received, various pieces of data are extracted from them. In machine learning literature these pieces of data are known as "features". Examples of features are: IPv6 hop limit, ICMPv6 type and code, and code of first TCP option. (In Nmap's terminology, these are known as IPV6_HOPLIMIT, ICMPV6_TYPE, and TCP_OPT_0 respectively.) Some features are simply extracted directly from response packets, and some are the result of doing a calculation over several packets (like TCP_ISR, the TCP initial sequence number counter rate).

Any features whose value cannot be determined (for example, features from a response that was never received) are set to -1. The features are put in a big one-dimensional feature vector. Then each is scaled and translated to put it approximately into the range [0, 1], using scale parameters estimated from our training data.

List of all features

`TCP_ISR`

TCP ISN counter rate. This is derived from the `S1`–`S6` sequence probes, which are sent 100 ms apart. The differences between consecutive sequence responses are added up, then this sum is divided by the time elapsed between the first and last probe.

The following features are repeated for each response, so for example a fully qualified feature name might be `S1.PLEN`.

`PLEN`

IPv6 Payload Length field

`TC`

IPv6 Traffic Class field

`HLIM`

A guess at the original value of the IPv6 Hop Limit field

The following features are repeated for each TCP response. A full feature name might be `T2.TCP_WINDOW`.

`TCP_WINDOW`

TCP window size

`TCP_FLAG_F`, `TCP_FLAG_S`, `TCP_FLAG_R`, `TCP_FLAG_P`, `TCP_FLAG_A`, `TCP_FLAG_U`, `TCP_FLAG_E`, `TCP_FLAG_C`

TCP flags. Each flag becomes a feature with the value 0 or 1.

`TCP_FLAG_RES8`, `TCP_FLAG_RES9`, `TCP_FLAG_RES10`, `TCP_FLAG_RES11`

These are the four bits of the reserved part of the TCP header. RFC 3540 defines `TCP_FLAG_RES8` as the nonce sum (NS) bit.

`TCP_OPT_0`, `<...>`, `TCP_OPT_16`

Type codes for the first 16 TCP options.

`TCP_OPTLEN_0`, `<...>`, `TCP_OPTLEN_16`

Lengths of the first 16 TCP options.

`TCP_MSS`

Value of the first MSS option, if present.

`TCP_SACKOK`

1 if the SACK-permitted option is present, 0 otherwise.

`TCP_WSCALE`

Value of the first Window Scale option, if present.

Differences from IPv4

IPv6 fingerprints look somewhat different from IPv4 fingerprints. Instead of a broken-down list of packet features, they consist of a hex dump of packet contents along with send and receive times. See the section called “Understanding an Nmap Fingerprint” for details.

The IPv6 matching algorithm is quite different. It uses a machine learning algorithm called logistic regression rather than simple comparison against a list of fingerprints. the section called "IPv6 matching" has a description of the algorithm.

Fingerprinting Methods Avoided by Nmap

Fuente original: https://nmap.org/book/osdetect-other-methods.html

Nmap supports many more OS detection techniques than any other program, and we are always interested in hearing about new ideas. Please send them to the Nmap development list (*nmap-dev*) for discussion. However there are some methods that just aren't a good fit. This section details some of the most interesting ones. While they aren't supported by Nmap, some are useful in combination with Nmap to verify findings or learn further details.

Passive Fingerprinting

Passive fingerprinting uses most of the same techniques as the active fingerprinting performed by Nmap. The difference is that a passive system simply sniffs the network, opportunistically classifying hosts as it observes their traffic. This is more difficult than active fingerprinting, since you have to accept whatever communication happens rather than designing your own custom probes. It is a valuable technique, but doesn't belong in a fundamentally active tool such as Nmap. Fortunately, Michal Zalewski has written the excellent p0f passive OS fingerprinting tool. He also devised a couple of the current Nmap OS fingerprinting tests. Another option is SinFP by GomoR, which supports both active and passive fingerprinting.

Exploit Chronology

TCP/IP fingerprinting works well for distinguishing different operating systems, but detecting different versions of the same operating system can be troublesome. The company must change their stack in some way we can differentiate. Fortunately, many OS vendors regularly update their systems to comply with the latest standards. But what about those who don't? Most of them at least get around to fixing exploitable stack bugs eventually. And those fixes are easy to detect remotely. First send the exploit payload, be it a land attack, teardrop, ping of death, SYN flood, or WinNuke. Send one attack at a time, then immediately try to contact the system again. If it is suddenly non-responsive, you have narrowed down the OS to versions which didn't ship with the fix.

Warning

If you use denial of service (DoS) exploits as part of your OS detection suite, remember to perform those tests last.

Retransmission Times

TCP implementations have significant leeway in exactly how long they wait before retransmitting packets. The proof-of-concept tools Ring and Cron-OS are available to exploit this. They send a SYN packet to an open port, then ignore the SYN/ACK they receive rather than acknowledging it with an ACK (to complete the connection) or a RST (to kill it). The target host will resend the SYN/ACK several more times, and these tools track every subsecond of the wait. While

some information can indeed be gleaned from this technique, there are several reasons that I haven't incorporated the patch into Nmap:

- It usually requires modifying the source host firewall rules to prevent your system from replying with a RST packet to the SYN/ACK it receives. That is hard to do in a portable way. And even if it was easy, many users don't appreciate applications mucking with their firewall rules.
- It can be very slow. The retransmissions can go on for several minutes. That is a long time to wait for a test that doesn't give all that much information in the first place.
- It can be inaccurate because packet drops and latency (which you have to expect in real-world environments) can lead to bogus results.

I have enumerated these reasons here because they also apply to some other proposed OS detection methods. I would love to add new tests, but they must be quick and require few packets. Messing with host firewall is unacceptable. I try to avoid making full TCP connections for stack fingerprinting, though that is done for OS detection as part of the version scanning system.

IP Fragmentation

IP fragmentation is a complex system and implementations are riddled with bugs and inconsistencies. Possible tests could examine how overlapping fragments are assembled or time the defragmentation timeouts. These tests are avoided for Nmap because many firewalls and other inline devices defragment traffic at gateways. Thus Nmap may end up fingerprinting the firewall rather than the true destination host. In addition, fragments are difficult to send on some operating systems. Linux 2.6 kernels have a tendency to queue the fragments you are trying to send and assemble them itself before transmission.

Open Port Patterns

The target host OS can often be guessed simply by looking at the ports which are open. Microsoft Windows machines often have TCP ports 135 and 139 open. Windows 2000 and newer also listen on port 445. Meanwhile, a machine running services on port 22 (ssh) and 631 (Internet Printing Protocol) is likely running Unix.

While this heuristic is often useful, it just isn't reliable enough for Nmap. Combinations of ports can be obscured by firewall rules, and most mainstream protocols are available on multiple platforms. OpenSSH servers can be run on Windows, and the "Windows SMB" ports can be serviced by Samba running on a Unix machine. Port forwarding clouds the issue even further. A machine which appears to be running Microsoft IIS might be a Unix firewall simply forwarding port 80 to a Windows machine.

For these reasons, Nmap does not consider open port numbers during TCP/IP stack fingerprinting. However, Nmap can use version detection information (see Chapter 7, *Service and Application Version Detection*) to separately discover operating system and device type information. By keeping the OS detection results discovered by OS detection and version detection separate, Nmap can gracefully handle a Checkpoint firewall which uses TCP port forwarding to a Windows web server. The stack fingerprinting results should be "Checkpoint Firewall-1" while version detection should suggest that the OS is Windows. Keep in mind that only a small fraction of version detection signatures include OS and device type information—we can only

populate these fields when the application divulges the information or when it only runs on one OS or device type.

Retired Tests

There are some tests that were once performed by Nmap, but which have been retired because they were found not to help in distinguishing operating systems and only took up space in the database. Two tests in the `IE` line were removed: `DLI` checked the length of the data payload in returned packets and `SI` checked the ICMP sequence numbers. They were never found to vary from the values sent. In the `U1` line, the `RUL` test checked the length of the UDP packet returned. It was different from what was sent in only one case out of more than 1,700. These tests were removed in March 2009.

Other tests were removed because they were too discriminating; they caused false differences to be measured that harmed detection accuracy. Both of these had to do with the TOS (type of service) field in responses. `TOS` did this for the `U1` probe and `TOSI` did it for `IE`. Although the test values did legitimately differ between operating systems, often false differences were recorded because the TOS was modified by an intermediate host. Better results were had overall when these tests were removed in October 2008.

Understanding an Nmap Fingerprint

Fuente original: https://nmap.org/book/osdetect-fingerprint-format.html

When Nmap stores a fingerprint in memory, Nmap uses a tree of attributes and values in data structures that users need not even be aware of. But there is also a special ASCII-encoded version which Nmap can print for users when a machine is unidentified. Thousands of these serialized fingerprints are also read back every time Nmap runs (with OS detection enabled) from the `nmap-os-db` database. The fingerprint format is a compromise between human comprehension and brevity. The format is so terse that it looks like line noise to many inexperienced users, but those who read this document should be able to decipher fingerprints with ease. There are actually two types of fingerprints, though they have the same general structure. The fingerprints of known operating systems that Nmap reads in are called reference fingerprints, while the fingerprint Nmap displays after scanning a system is a subject fingerprint. The reference fingerprints are a bit more complex since they can be tailored to match a whole class of operating systems by adding leeway to (or omitting) tests that aren't so reliable while allowing only a single possible value for other tests. The reference fingerprints also have OS details and classifications. Since the subject tests are simpler, we describe them first.

Decoding the Subject Fingerprint Format

If Nmap performs OS fingerprinting on a host and doesn't get a perfect OS matches despite promising conditions (such as finding both open and closed ports accessible on the target), Nmap prints a subject fingerprint that shows all of the test results that Nmap deems relevant, then asks the user to submit the data to Nmap.Org. Tests aren't shown when Nmap has no useful results, such as when the relevant probe responses weren't received. A special line named `SCAN` gives extra details about the scan (such as Nmap version number) that provide useful context for integrating fingerprint submissions into `nmap-os-db`. A typical subject fingerprint is shown in Example 8.3.

Example 8.3. A typical subject fingerprint

```
OS:SCAN(V=5.05BETA1%D=8/23%OT=22%CT=1%CU=42341%PV=N%DS=0%DC=L%G=Y%TM=4A91CB
OS:90%P=i686-pc-linux-gnu)SEQ(SP=C9%GCD=1%ISR=CF%TI=Z%CI=Z%II=I%TS=A)OPS(O1
OS:=M400CST11NW5%O2=M400CST11NW5%O3=M400CNNT11NW5%O4=M400CST11NW5%O5=M400CS
OS:T11NW5%O6=M400CST11)WIN(W1=8000%W2=8000%W3=8000%W4=8000%W5=8000%W6=8000)
OS:ECN(R=Y%DF=Y%T=40%W=8018%O=M400CNNSNW5%CC=N%Q=)T1(R=Y%DF=Y%T=40%S=O%A=S+
OS:%F=AS%RD=0%Q=)T2(R=N)T3(R=Y%DF=Y%T=40%W=8000%S=O%A=S+%F=AS%O=M400CST11NW
OS:5%RD=0%Q=)T4(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T5(R=Y%DF=Y%T=40%W
OS:=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)
OS:T7(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%U
OS:N=0%RIPL=G%RID=G%RIPCK=G%RUCK=G%RUD=G)IE(R=Y%DFI=N%T=40%CD=S)
```

Now you may look at this fingerprint and immediately understand what everything means. If so, you can simply skip this section. But I have never seen such a reaction. Many people probably think some sort of buffer overflow or unterminated string error is causing Nmap to spew garbage data at them. This section helps you decode the information so you can immediately tell that blind TCP sequence prediction attacks against this machine are moderately hard, but it may make a good idle scan (`-sI`) zombie. The first step in understanding this fingerprint is to fix the line wrapping. The tests are all squished together, with each line wrapped at 71 characters. Then `OS:` is prepended to each line, raising the length to 74 characters. This makes fingerprints easy to cut and paste into the Nmap fingerprint submission form (see the section called "When Nmap Fails to Find a Match and Prints a Fingerprint"). Removing the prefix and fixing the word wrapping (each line should end with a right parenthesis) leads to the cleaned-up version in Example 8.4.

Example 8.4. A cleaned-up subject fingerprint

```
SCAN(V=5.05BETA1%D=8/23%OT=22%CT=1%CU=42341%PV=N%DS=0%DC=L%G=Y%TM=4A91CB90%
     P=i686-pc-linux-gnu)
SEQ(SP=C9%GCD=1%ISR=CF%TI=Z%CI=Z%II=I%TS=A)
OPS(O1=M400CST11NW5%O2=M400CST11NW5%O3=M400CNNT11NW5%
    O4=M400CST11NW5%O5=M400CST11NW5%O6=M400CST11)
WIN(W1=8000%W2=8000%W3=8000%W4=8000%W5=8000%W6=8000)
ECN(R=Y%DF=Y%T=40%W=8018%O=M400CNNSNW5%CC=N%Q=)
T1(R=Y%DF=Y%T=40%S=O%A=S+%F=AS%RD=0%Q=)
T2(R=N)
T3(R=Y%DF=Y%T=40%W=8000%S=O%A=S+%F=AS%O=M400CST11NW5%RD=0%Q=)
T4(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)
T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)
T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)
T7(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)
U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK=G%RUD=G)
IE(R=Y%DFI=N%T=40%CD=S)
```

While this still isn't the world's most intuitive format (we had to keep it short), the format is much clearer now. Every line is a category, such as `SEQ` for the sequence generation tests, `T3` for the results from that particular TCP probe, and `IE` for tests related to the two ICMP echo probes.

Following each test name is a pair of parentheses which enclose results for individual tests. The tests take the format *<testname>=<value>*. All of the possible categories, tests, and values are described in the section called "TCP/IP Fingerprinting Methods Supported by Nmap". Each pair of tests are separated by a percentage symbol (%). Tests values can be empty, leading to a percentage symbol or category-terminating right-parenthesis immediately following the equal sign. The string "`O=%RD=0%Q=)`" in `T4` of our example shows two of these empty tests. A blank test value must match another blank value, so this empty TCP quirks `Q` value wouldn't match a fingerprint with `Q` set to `RU`.

In some cases, a whole test is missing rather than just its value. For example, `T2` of our sample fingerprint has no `W` (TCP window), `S` (sequence number), `A` (acknowledgment number), `T` (TTL),

or TG (TTL guess) tests. This is because the one test and value it does include, R=N, means that no response was returned for the T2 probe. So including a window value or sequence number would make little sense. Similarly, tests which aren't well supported on the system running Nmap are skipped. An example is the RID (IP ID field returned in ICMP packet) test, which doesn't work well on Solaris because that system tends to corrupt the ID field Nmap sends out. Tests which are inconclusive (such as failing to detect the IP ID sequence for the TI, CI, and II tests) are also omitted.

Decoding the SCAN line of a subject fingerprint

The SCAN line is a special case in a subject fingerprint. Rather than describe the target system, these tests describe various conditions of the scan. These help us integrate fingerprints submitted to Nmap.Org. The tests in this line are:

- Nmap version number (V).
- Date of scan (D) in the form month/day.
- Open and closed TCP ports (on target) used for scan (OT and CT). Unlike most tests, these are printed in decimal format. If Nmap was unable to find an open or a closed port, the test is included with an empty value (even when Nmap guesses a possibly closed port and sends a probe there).
- Closed UDP port (CU). This is the same as CT, but for UDP. Since the majority of scans don't include UDP, this test's value is usually empty.
- Private IP space (PV) is Y if the target is on the 10.0.0.0/8, 172.16.0.0/12, or 192.168.0.0/16 private networks (RFC 1918). Otherwise it is N.
- Network distance (DS) is the network hop distance from the target. It is 0 if the target is localhost, 1 if directly connected on an ethernet network, or the exact distance if discovered by Nmap. If the distance is unknown, this test is omitted.
- The distance calculation method (DC) indicates how the network distance (DS) was calculated. It can take on these values: L for localhost (DS=0); D for a direct subnet connection (DS=1); I for a TTL calculation based on an ICMP response to the U1 OS detection probe; and T for a count of traceroute hops. This test exists because it is possible for the ICMP TTL calculation to be incorrect when intermediate machines change the TTL; it distinguishes between a host that is truly directly connected and what may be just a miscalculation.
- Good results (G) is Y if conditions and results seem good enough to submit this fingerprint to Nmap.Org. It is N otherwise. Unless you force them by enabling debugging (-d) or extreme verbosity (-vv), G=N fingerprints aren't printed by Nmap.
- Target MAC prefix (M) is the first six hex digits of the target MAC address, which correspond to the vendor name. Leading zeros are not included. This field is omitted unless the target is on the same ethernet network (DS=1).
- The OS scan time (TM) is provided in Unix time_t format (in hexadecimal).
- The platform Nmap was compiled for is given in the P field.

Decoding the Reference Fingerprint Format

When Nmap scans a target to create a subject fingerprint, it then tries to match that data against the thousands of reference fingerprints in the nmap-os-db database. Reference fingerprints are

initially formed from one or more subject fingerprints and thus have much in common. They do have a bit of extra information to facilitate matching and of course to describe the operating systems they represent. For example, the subject fingerprint we just looked at might form the basis for the reference fingerprint in Example 8.5.

Example 8.5. A typical reference fingerprint

```
Fingerprint Apple Mac OS X Server 10.2.8 (Jaguar) (Darwin 6.8, PowerPC)
Class Apple | Mac OS X | 10.2.X | general purpose
CPE cpe:/o:apple:mac_os_x:10.2.8
SEQ(SP=FB-111%GCD=1-6%ISR=104-10E%TI=I%II=I%SS=S%TS=1)
OPS(O1=M5B4NW0NNT11%O2=M5B4NW0NNT11%O3=M5B4NW0NNT11%O4=M5B4NW0NNT11%O5=M5B4NW0NNT11%O6=M5B4NNT11)
WIN(W1=8218%W2=8220%W3=8204%W4=80E8%W5=80F4%W6=807A)
ECN(R=Y%DF=Y%T=3B-45%TG=40%W=832C%O=M5B4NW0%CC=N%Q=)
T1(R=Y%DF=Y%T=3B-45%TG=40%S=O%A=S+%F=AS%RD=0%Q=)
T2(R=N)
T3(R=Y%DF=Y%T=3B-45%TG=40%W=807A%S=O%A=S+%F=AS%O=M5B4NW0NNT11%RD=0%Q=)
T4(R=Y%DF=Y%T=3B-45%TG=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)
T5(R=Y%DF=N%T=3B-45%TG=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)
T6(R=Y%DF=Y%T=3B-45%TG=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)
T7(R=Y%DF=N%T=3B-45%TG=40%W=0%S=Z%A=S%F=AR%O=%RD=0%Q=)
U1(DF=N%T=3B-45%TG=40%IPL=38%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK=0%RUD=G)
IE(DFI=S%T=3B-45%TG=40%CD=S)
```

Some differences are immediately obvious. Line wrapping is not done because that is only important for the submission process. The `SCAN` line is also removed, since that information describes a specific scan instance rather than general target OS characteristics.

You probably also noticed the new lines, `Fingerprint`, `Class`, and `CPE`, which are new to this reference fingerprint. A more subtle change is that some of the individual test results have been removed while others have been enhanced with logical expressions.

Free-form OS description (`Fingerprint` line)

The `Fingerprint` line first serves as a token so Nmap knows to start loading a new fingerprint. Each fingerprint only has one such line. Immediately after the `Fingerprint` token (and a space) comes a textual description of the operating system(s) represented by this fingerprint. These are in free-form English text, designed for human interpretation rather than a machine parser. Nevertheless, Nmap tries to stick with a consistent format including the vendor, product name, and then version number. Version number ranges and comma-separated alternatives discussed previously can be found in this field. Here are some examples:

```
Fingerprint HP LaserJet printer (4050, 4100, 4200, or 8150)
Fingerprint Sun Solaris 9 or 10 (SPARC)
Fingerprint Linux 2.6.22 - 2.6.24
Fingerprint Microsoft Windows Server 2003 SP1
Fingerprint Microsoft Windows XP Professional SP1
Fingerprint Minolta Di550 laser printer
```

In an ideal world, every different OS would correspond to exactly one unique fingerprint. Unfortunately, OS vendors don't make life so easy for us. The same OS release may fingerprint differently based on what network drivers are in use, user-configurable options, patch levels, processor architecture, amount of RAM available, firewall settings, and more. Sometimes the fingerprints differ for no discernible reason. While the reference fingerprint format has an expression syntax for coping with slight variations, creating multiple fingerprints for the same OS is often preferable when major differences are discovered.

Just as multiple fingerprints are often needed for one OS, sometimes a single fingerprint describes several systems. If two systems give the exact same results for every single test, Nmap has little

choice but to offer up both as possibilities. This commonly occurs for several reasons. One is that vendors may release a new version of their OS without any significant changes to their IP stack. Maybe they made important changes elsewhere in the system, or perhaps they did little but want to make a bunch of money selling "upgrades". In these cases, Nmap often prints a range such as `Apple Mac OS X 10.4.8 - 10.4.11` or `Sun Solaris 9 or 10`.

Another cause of duplicate fingerprints is embedded devices which share a common OS. For example, a printer from one vendor and an ethernet switch from another may actually share an embedded OS from a third vendor. In many cases, subtle differences between the devices still allow them to be distinguished. But sometimes Nmap must simply list a group of possibilities such as `Cisco 1200-series WAP, HP ProCurve 2650 switch, or Xerox Phaser 7400N or 8550DT printer`.

There are also cases where numerous vendors private label the exact same OEM device with their own brand name and model number. Here again, Nmap must simply list the possibilities. But distinguishing these is less important because they are all fundamentally the same device.

Tip

If the description printed by Nmap (which comes from the `Fingerprint` line) isn't informative enough for you, more detailed information may be available in comments above the fingerprint itself in `nmap-os-db`. You can find it installed on your system as described in Chapter 14, *Understanding and Customizing Nmap Data Files*, or look up the latest version at https://svn.nmap.org/nmap/nmap-os-db. Search for the exact OS description that Nmap gives you. Keep in mind that there may be several `Fingerprint` lines with exactly the same description, so you may have to examine them all. Or use the Nmap XML output, which shows the line number of each match.

Device and OS classification (`Class` lines)

While the `Fingerprint` description works great for analysts reading Nmap output directly, many people run Nmap from other scripts and applications. Those applications might use the OS information to check for OS-specific vulnerabilities or just create a pretty graph or report.

A more structured OS classification system exists for these purposes. It is also useful when there are multiple matches. If you only get a partial fingerprint (maybe no open ports were found on the target so many tests had to be skipped), it might match dozens of different fingerprints in the `nmap-os-db` database. Printing the details for all of those fingerprints would be a mess. But thanks to OS classification, Nmap can find commonality. If all of the matches are classified as Linux, Nmap will simply print that the target is a Linux box.

Every fingerprint has one or more `Class` lines. Each contains four well-defined fields: vendor, OS family, OS generation, and device type. The fields are separated by the pipe symbol (`|`).

The vendor is the company which makes an OS or device. Examples are `Apple`, `Cisco`, `Microsoft`, and `Linksys`. For community projects such as OpenBSD and Linux without a controlling vendor, the OS family name is repeated for the vendor column.

OS family includes products such as `Windows`, `Linux`, `IOS` (for Cisco routers), `Solaris`, and `OpenBSD`. There are also hundreds of devices such as switches, broadband routers, and printers which use undisclosed operating systems. When the underlying OS isn't clear, `embedded` is used.

OS generation is a more granular description of the OS. Generations of Linux include `2.4.X` and `2.6.X`, while Windows generations include `95`, `98`, `Me`, `2000`, `XP`, and `Vista`. FreeBSD uses generations such as `4.X` and `5.X`. For obscure operating systems which we haven't subdivided into generations (or whenever the OS is listed simply as `embedded`), this field is left blank.

The device type is a broad classification such as `router`, `printer`, or `game console` and was discussed previously in this chapter. General-purpose operating systems such as `Linux` and `Windows` which can be used for just about anything are classified as `general purpose`.

Each field may contain just one value. When a fingerprint represents more than one possible combination of these four fields, multiple `Class` lines are used. Example 8.6 provides some example `Fingerprint` lines followed by their corresponding classifications.

Example 8.6. Some typical fingerprint descriptions and corresponding classifications

```
Fingerprint D-Link DSL-500G ADSL router
Class D-Link | embedded || broadband router

Fingerprint Linksys WRT54GC or TRENDnet TEW-431BRP WAP
Class Linksys | embedded || WAP
Class TRENDnet | embedded || WAP

Fingerprint Apple Mac OS X 10.3.9 (Panther) - 10.4.7 (Tiger)
Class Apple | Mac OS X | 10.3.X | general purpose
Class Apple | Mac OS X | 10.4.X | general purpose

Fingerprint Sony PlayStation 3 game console
Class Sony | embedded || game console
```

If these examples aren't enough, a listing of classifications recognized by the latest version of Nmap is maintained at https://nmap.org/data/os-classes.txt.

CPE name (`CPE` lines)

`CPE` lines give Common Platform Enumeration equivalents of `Class` lines. Each `Class` may be followed by several `CPE` lines (`CPE` lines always "belong" to the `Class` line that immediately precedes them). It's common for one CPE name to describe the operating system and another to describe the hardware platform. A description of CPE syntax and meaning can be found in the section called "Common Platform Enumeration (CPE)".

Example 8.7. Typical CPE classifications

```
Fingerprint 3Com NBX 100 VoIP gateway (VxWorks)
Class 3Com | VxWorks || VoIP adapter
CPE cpe:/h:3com:nbx_100
CPE cpe:/o:3com:vxworks auto

Fingerprint Linux 2.0.33
Class Linux | Linux | 2.0.X | general purpose
CPE cpe:/o:linux:linux_kernel:2.0.33
```

The `auto` flag that follows some CPE names is not part of CPE; it is only used internally by maintenance scripts to indicate that a CPE name was automatically generated from other information rather than inserted manually.

Test expressions

The test expressions don't have to change between a subject and reference fingerprint, but they almost always do. The reference fingerprint often needs to be generalized a little bit to match all instances of a particular OS, rather than just the machine you are scanning. For example, some Windows XP machines return a Window size of `F424` to the `T1` probe, while others return `FAF0`. This may be due to the particular ethernet device driver in use, or maybe how much memory

is available. In any case, we would like to detect Windows XP no matter which window size is used.

One way to generalize a fingerprint is to simply remove tests that produce inconsistent results. Remove all of the window size tests from a reference fingerprint, and systems will match that print no matter what size they use. The downside is that you can lose a lot of important information this way. If the only Window sizes that a particular system ever sends are `F424` and `FAF0`, you really only want to allow those two values, not all 65,536 possibilities.

While removing tests is overkill in some situations, it is useful in others. The `R=Y` test value, meaning there was a response, is usually removed from the `U1` and `IE` tests before they are added to `nmap-os-db`. These probes are often blocked by a firewall, so the lack of a response should not count against the OS match.

When removing tests is undesirable, Nmap offers an expression syntax for allowing a test to match multiple values. For example, `W=F424|FAF0` would allow those two Windows XP window values without allowing any others. Table 8.8 shows the permitted operators in test values.

Table 8.8. Reference fingerprint test expression operators

Op Name	Symbol	Example	Description
Or	`|`	`O=|ME|MNNTNW`	Matches if the corresponding subject fingerprint test takes the value of any of the clauses. In this example, the initial pipe symbol means that an empty options list will match too.
Range	`-`	`SP=7-A`	Matches if the subject fingerprint's corresponding test produces a numeric value which falls within the range specified.
Greater than	`>`	`SP=>8`	Matches if the subject fingerprint's corresponding test produces a numeric value which is greater than the one specified.
Less than	`<`	`GCD=<5`	Matches if the subject fingerprint's corresponding test produces a numeric value which is less than the one specified.

Expressions can combine operators, as in `GCD=1-6|64|256|>1024`, which matches if the `GCD` is between one and six, exactly 64, exactly 256, or greater than 1024.

IPv6 fingerprints

Because the IPv6 classification engine works differently, it has different fingerprints. There are no reference fingerprints; instead a set of previously identified training examples is run through a training algorithm, which outputs a large matrix of coefficients, one for each feature and OS class. Subject fingerprints use the same ASCII-armored format as IPv4, as shown in Example 8.8, "An IPv6 fingerprint".

Example 8.8. An IPv6 fingerprint

```
OS:SCAN(V=5.61TEST1%E=6%D=9/27%OT=22%CT=443%CU=42192%PV=N%DS=5%DC=T%G=Y%TM
OS:=4E82908D%P=x86_64-unknown-linux-gnu)S1(P=6000{4}28063cXX{32}0016c1b002
OS:bbd213c57562f5a01212e0f8880000020404c40402080a5be177f2ff{4}01030307%ST=
OS:0.021271%RT=0.041661)S2(P=6000{4}28063cXX{32}0016c1b108d7da47c57562f6a0
OS:1212e0e9d20000020404c40402080a5be17856ff{4}01030307%ST=0.121251%RT=0.14
OS:4586)S3(P=6000{4}28063cXX{32}0016c1b21029efebc57562f7a01212e0cf63000002
OS:0404c40101080a5be178ceff{4}01030307%ST=0.221232%RT=0.268086)S4(P=6000{4
OS:}28063cXX{32}0016c1b31553d32dc57562f8a01212e0e3a40000020404c40402080a5b
OS:e1791eff{4}01030307%ST=0.321237%RT=0.340261)S5(P=6000{4}28063cXX{32}001
OS:6c1b41ae90087c57562f9a01212e0b04f0000020404c40402080a5be17982ff{4}01030
OS:307%ST=0.421246%RT=0.441253)S6(P=6000{4}24063cXX{32}0016c1b5207baa83c57
OS:562fa901212e014690000020404c40402080a5be179e6ff{4}%ST=0.521245%RT=0.541
OS:755)IE1(P=6000{4}803a3cXX{32}810927cbabcd00{122}%ST=0.565533%RT=0.59350
OS:5)U1(P=6000{3}01643a3cXX{32}0104be5300{4}6001234501341131XX{32}c1a9a4d0
OS:013482ff43{300}%ST=0.713832%RT=0.734263)TECN(P=6000{4}20063cXX{32}0016c
OS:1b62f0c74d8c57562fb80121310241c0000020404c40101040201030307%ST=0.763567
OS:%RT=0.784838)T2(P=6000{4}583a3cXX{32}0101ca5600{4}6001234500280632XX{32
OS:}c1b70016c57562fb85549cefa00000808c0d000003030a0102040109080aff{4}00{4}
OS:0402%ST=0.813012%RT=0.833344)T3(P=6000{4}583a3cXX{32}0101ca6000{4}60012
OS:34500280628XX{32}c1b80016c57562fc2b8e3db7a02b0100445f000003030a01020401
OS:09080aff{4}00{4}0402%ST=0.863293%RT=0.881198)T4(P=6000{4}14063cXX{32}00
OS:16c1b93a67fc8a00{4}500400000c7c0000%ST=0.912394%RT=0.93247)T5(P=6000{4}
OS:14063cXX{32}01bbc1ba00{4}c57562ff50140000194300000%ST=0.96164%RT=0.98347
OS:5)T6(P=6000{4}14063cXX{32}01bbc1bba9e336d500{4}50040000610e0000%ST=1.01
OS:164%RT=1.03554)T7(P=6000{4}583a3cXX{32}0101ca5800{4}6001234500280630XX{
OS:32}c1bc01bbc5756300095eb241a029ffffec59000003030f0102040109080aff{4}00{
OS:4}0402%ST=1.06173%RT=1.07961)EXTRA(FL=12345)
```

Example 8.9, "A cleaned-up IPv6 fingerprint" shows what this fingerprint looks like unwrapped. Most of the probes are omitted because they all have the same format.

Example 8.9. A cleaned-up IPv6 fingerprint

```
SCAN(V=5.61TEST1%E=6%D=9/27%OT=22%CT=443%CU=42192%PV=N%DS=5%DC=T%G=Y%TM4E82908D%
     P=x86_64-unknown-linux-gnu)
S1(P=6000{4}28063cXX{32}0016c1b002bbd213c57562f5a01212e0f8880000020404c4040208
     0a5be177f2ff{4}01030307%ST=0.021271%RT=0.041661)
EXTRA(FL=12345)
```

The `SCAN` line has the same meaning as in IPv4 fingerprints. The pseudo-test `E=6` indicates that this is an IPv6 fingerprint.

Then there is one line for each probe that received a response. Within each of these there are three keys:

`P`

The contents of the packet, hex- and run-length–encoded. Whenever two digits are followed by a number in curly brackets, it means to repeat that byte the given number of times. For example, `00{4}` is short for `00000000`. The characters `XX` are put in place of source and destination addresses, which are private and anyway not useful for training the classifier.

`ST`

The time when the packet was sent, in seconds, relative to when OS detection began.

`RT`

The time when the packet was received.

The `EXTRA` line stores any other information that can't be determined purely from the packet contents. The `FL` key stores the flow label that was sent during the scan. This would always be

12345, except that some operating systems don't allow setting the flow label and always send 00000 instead.

When a fingerprint like the one above is processed, it is converted to an internal representation as shown below. Each value is one element of the feature vector that is passed to the classifier. They correspond to the features listed in the section called "List of all features".

```
        40  S1.PLEN
         0  S1.TC
        40  S2.PLEN
         0  S2.TC
        40  S3.PLEN
         0  S3.TC
        40  S4.PLEN
         0  S4.TC
        40  S5.PLEN
         0  S5.TC
        36  S6.PLEN
         0  S6.TC
       128  IE1.PLEN
         0  IE1.TC
   UNKNOWN  IE2.PLEN
   UNKNOWN  IE2.TC
   UNKNOWN  NS.PLEN
   UNKNOWN  NS.TC
       356  U1.PLEN
         0  U1.TC
<Many more lines omitted.>
```

UNKNOWN values usually mean that no response was received to the relevant probe. UNKNOWN values are mapped to -1 before classification.

Device Types

Fuente original: https://nmap.org/book/osdetect-device-types.html

As stated in the section called "Device and OS classification (Class lines)", every reference fingerprint is classified with one or more device types. This list contains the device types used by Nmap and the criteria for classifying a device as each type. These same rules are used to classify the device types in version detection; see the description of the d// field in the section called "match Directive".

general purpose

This category contains general-purpose operating systems like Linux and Windows. In the nmap-service-probes file this class is indicated by a lack of a d// field.

bridge

A bridge combines two or more subnetworks into one. With a bridge this happens at a lower level than with a router. This category also includes things like Ethernet-to-serial bridges.

broadband router

Devices in this category connect a network to the Internet via cable, ADSL, fiber optics, etc. Some of these devices provide network address translation, a firewall, port forwarding, or other services.

firewall

A firewall controls what traffic is allowed into or out of a network. Some also have additional capabilities. This category doesn't include general-purpose operating systems that happen

to come with a firewall, but it does include OS distributions purpose-built to work only as a firewall.

game console

A video game console like the Xbox or PlayStation.

hub

A hub joins network segments by re-broadcasting all traffic. Hubs are distinct from switches, which selectively transmit packets only to relevant destinations.

load balancer

A device that distributes inbound traffic to multiple devices to ease the load on those devices.

media device

This category includes all kinds of audiovisual equipment, including portable music players, home audio systems, TVs, and projectors.

PBX

A private branch exchange, or PBX, routes telephone calls within a private organization and connects them to the public telephone network or VoIP.

PDA

A handheld computer. Devices that are also telephones go in the "phone" category.

phone

A network-capable telephone that is not a VoIP phone. Devices in this category are typically mobile phones.

power-device

Miscellaneous power devices like uninterruptable power supplies and surge protectors.

printer

Network-enabled printers, including printers with an embedded print server.

print server

A print server connects a printer to a network. Printers that contain their own print server go in the "printer" category instead.

proxy server

Any kind of proxy, including web proxies and other servers that cache data or understand high-level protocols.

remote management

Devices that allow servers or other equipment to be monitored or managed remotely.

router

Routers connect multiple networks. They are distinct from hubs and switches because they route packets between different networks as opposed to extending one network.

security-misc

Any security device that doesn't fall into the "firewall" category belongs in this category. This includes intrusion detection and prevention systems.

specialized

The catch-all category. If a device doesn't fall into one of the other categories, it is specialized. Examples in this category are diverse and include such things as clocks, oscilloscopes, climate sensors, and more.

storage-misc

Data storage devices like tape decks and network-attached storage appliances.

switch

A device that extends a network by selectively re-broadcasting packets. Switches are distinct from hubs, which broadcast all packets.

telecom-misc

Devices used by telephone systems that aren't PBXs, like voicemail and ISDN systems.

terminal

A device with a keyboard and monitor with the primary purpose of communicating directly with a terminal server or mainframe.

terminal server

A device providing terminal facilities to clients over a network.

VoIP adapter

A device that converts between voice over IP (VoIP) protocols and normal telephone traffic. Also may convert different VoIP protocols.

VoIP phone

A phone capable of a VoIP protocol.

WAP

Wireless access points offer a wireless connection to a network. Most work with radio technology like 802.11b but some use infra-red or something else. Devices that could also be put in another category, like wireless broadband routers, are put in the WAP category because WAPs require special network considerations.

webcam

Any kind of camera that stores or transmits pictures or video. This includes everything from consumer webcams to security system cameras.

OS Matching Algorithms

Fuente original: https://nmap.org/book/osdetect-guess.html

IPv4 matching

Nmap's algorithm for detecting matches is relatively simple. It takes a subject fingerprint and tests it against every single reference fingerprint in `nmap-os-db`.

When testing against a reference fingerprint, Nmap looks at each probe category line from the subject fingerprint (such as `SEQ` or `T1`) in turn. Any probe lines which do not exist in the

reference fingerprint are skipped. When the reference fingerprint does have a matching line, they are compared.

For a probe line comparison, Nmap examines every individual test (`R`, `DF`, `W`, etc.) from the subject category line in turn. Any tests which do not exist in the reference line are skipped. Whenever a matching test is found, Nmap increments the `PossiblePoints` accumulator by the number of points assigned to this test. Then the test values are compared. If the reference test has an empty value, the subject test only matches if its value is empty too. If the reference test is just a plain string or number (no operators), the subject test must match it exactly. If the reference string contains operators (`|`, `-`, `>`, or `<`), the subject must match as described in the section called "Test expressions". If a test matches, the `NumMatchPoints` accumulator is incremented by the test's point value.

Once all of the probe lines are tested for a fingerprint, Nmap divides `NumMatchPoints` by `PossiblePoints`. The result is a confidence factor describing the probability that the subject fingerprint matches that particular reference fingerprint. For example, `1.00` is a perfect match while `0.95` is very close (95%).

Test point values are assigned by a special `MatchPoints` entry (which may only appear once) in `nmap-os-db`. This entry looks much like a normal fingerprint, but instead of providing results for each test, it provides point values (non-negative integers) for each test. Tests listed in the `MatchPoints` structure only apply when found in the same test they are listed in. So a value given for the `W` (Window size) test in `T1` doesn't affect the `W` test in `T3`. A test can be effectively disabled by assigning it a point value of 0. An example `MatchPoints` structure is given in Example 8.10.

Example 8.10. The `MatchPoints` structure

```
MatchPoints
SEQ(SP=25%GCD=75%ISR=25%TI=100%CI=50%II=100%SS=80%TS=100)
OPS(O1=20%O2=20%O3=20%O4=20%O5=20%O6=20)
WIN(W1=15%W2=15%W3=15%W4=15%W5=15%W6=15)
ECN(R=100%DF=20%T=15%TG=15%W=15%O=15%CC=100%Q=20)
T1(R=100%DF=20%T=15%TG=15%S=20%A=20%F=30%RD=20%Q=20)
T2(R=80%DF=20%T=15%TG=15%W=25%S=20%A=20%F=30%O=10%RD=20%Q=20)
T3(R=80%DF=20%T=15%TG=15%W=25%S=20%A=20%F=30%O=10%RD=20%Q=20)
T4(R=100%DF=20%T=15%TG=15%W=25%S=20%A=20%F=30%O=10%RD=20%Q=20)
T5(R=100%DF=20%T=15%TG=15%W=25%S=20%A=20%F=30%O=10%RD=20%Q=20)
T6(R=100%DF=20%T=15%TG=15%W=25%S=20%A=20%F=30%O=10%RD=20%Q=20)
T7(R=80%DF=20%T=15%TG=15%W=25%S=20%A=20%F=30%O=10%RD=20%Q=20)
U1(R=50%DF=20%T=15%TG=15%TOS=0%IPL=100%UN=100%RIPL=100%RID=100%RIPCK=100%RUCK=100%RUL=100%RUD=100)
IE(R=50%DFI=40%T=15%TG=15%TOSI=0%CD=100%SI=100%DLI=100)
```

Once all of the reference fingerprints have been evaluated, Nmap orders them and prints the perfect matches (if there aren't too many). If there are no perfect matches, but some are very close, Nmap may print those. Guesses are more likely to be printed if the `--osscan-guess` option is given.

IPv6 matching

IPv6 OS classification uses a machine learning technique called logistic regression. Nmap uses the LIBLINEAR library to do this classification. The process starts with a large corpus of training examples, which are fingerprints submitted by Nmap users and carefully labeled with their OS. Each training example is represented by a feature vector, which can be thought of as the "coordinates" of that OS in a multi-dimensional space. The training algorithm calculates an optimal boundary between members of each OS class and members of every other class. It then encodes each of these boundaries as a vector. There is a different vector for each OS class.

When matching, the engine takes each of these boundary vectors in turn and calculates a dot product between it and the feature vector. The result is a single real number. The higher (more positive) the number, the more likely the match. Negative numbers are unlikely matches. A number x is mapped from the range [-∞, ∞] to [0, 100] using the logistic formula 100 / (1 + e^x). (This is the source of the name "logistic regression".)

In general, the OS class with the highest score is the most likely match, but in the case of a never-before-seen operating system, it's possible to have a very high score but an inaccurate match nevertheless. Therefore a second "novelty detection" algorithm checks whether the observed fingerprint is very unlike the other representatives of the class. The algorithm finds the Euclidean distance from the observed feature vector to the mean of the feature vectors of the members of the class, scaled in each dimension by the inverse of that feature's variance. Feature vectors similar to those already seen will have low novelty, and those that are different will have high novelty.

The OS class with the highest score is reported as a match, but only if the novelty is below 15. Also, if the two highest OS classes have scores that differ by less than 10%, the classification is considered ambiguous and not a successful match. Sample logistic and novelty scores from a run against Mac OS X 10.6.8 are shown in Table 8.9, "OS guesses against Mac OS X".

Table 8.9. OS guesses against Mac OS X

Score	Novelty	OS class
61.05%	1.00	Apple Mac OS X 10.6.8 - 10.7.0 (Snow Leopard - Lion) (Darwin 10.8.0 - 11.0.0)
10.08%	18.04	Apple Mac OS X 10.7 (Lion) (Darwin 11.1.0)
9.97%	24.06	Apple Mac OS X 10.6.8 (Snow Leopard) (Darwin 10.8.0)
e)		
9.43%	19.26	Apple Mac OS X 10.7.2 (Lion) (Darwin 11.2.0)
5.99%	23.63	Apple Mac OS X 10.4.11 (Tiger) (Darwin 8.11.1)
2.28%	34.67	Apple iPhone mobile phone (iOS 4.2.1)
2.19%	35.07	Apple Mac OS X 10.4.7 (Panther) (Apple TV 3.0.2)
2.19%	57.63	HP ProCurve 2520G switch
2.04%	37.03	Apple Mac OS X 10.6.8 (Snow Leopard) (Darwin 10.8.0)
2.03%	68.55	Apple Mac OS X 10.6.8 (Snow Leopard) (Darwin 10.8.0)
0.59%	79.61	FreeBSD 6.1-RELEASE

There isn't a separate data file containing the IPv6 OS database as there is with IPv4. The database is stored in C++ source code file `FPModel.cc`. This file contains scaling constants (used to put feature values roughly into the range [0, 1]), and the boundary vectors described above.

Dealing with Misidentified and Unidentified Hosts

Fuente original: https://nmap.org/book/osdetect-unidentified.html

While Nmap has a huge database, it cannot detect everything. Nmap has no chance to detect

most toasters, refrigerators, chairs, or automobiles because they have no IP stack. Yet I wouldn't rule any of these out, given the ever-expanding list of connected devices. The Nmap fingerprint DB includes plenty of game consoles, phones, thermometers, cameras, interactive toys, and media players.

Having an IP address is necessary but not sufficient to guarantee a proper fingerprint. Nmap may still guess wrong or fail to produce any guess at all. Here are some suggestions for improving your results:

Upgrade to the latest Nmap

Many Linux distributions and other operating systems ship with ancient versions of Nmap. The Nmap OS database is improved with almost every release, so check your version number by running `nmap -V` and then compare that to the latest available from https://nmap.org/download.html. Installing the newest version takes only a few minutes on most platforms.

Scan all ports

When Nmap detects OS detection problems against a certain host, it will issue warnings. One of the most common is: "Warning: OS detection will be MUCH less reliable because we did not find at least 1 open and 1 closed TCP port". It is possible that such ports really are unavailable on the machine, but retrying your scan with `-p-` to scan all ports may find some that are responsive for OS detection. Doing a UDP scan (`-sU`) too can help even more, though it will slow the scan substantially.

Try a more aggressive guess

If Nmap says there are no matches close enough to print, something is probably wrong. Maybe a firewall or NAT box in the way is modifying the probe or response packets. This can cause a hybrid situation where one group of tests look like they are from one OS, while another set look completely different. Adding the `--osscan-guess` may give more clues as to what is running.

Scan from a different location

The more network hops your packet has to go through to reach its target, the greater the chances that a network device will modify (or drop) the probe or response. NAT gateways, firewalls, and especially port forwarding can confuse OS detection. If you are scanning the IP of a load balancing device which simply redirects packets to a diverse network of servers, it isn't even clear what the "correct" OS detection result would be.

Many ISPs filter traffic to "bad" ports, and others use transparent proxies to redirect certain ports to their own servers. The port 25 or 80 you think are open on your target may actually be spoofed from your ISP to connect to ISP proxy servers. Another behavior which can confuse OS detection is when firewalls spoof TCP reset packets as if they are coming from the destination host. This is particularly common from port 113 (identd). Both the reset spoofing and transparent proxies can often be detected by noticing that every machine on a target network seems to exhibit the behavior—even those which otherwise seem to be down. If you detect any such nonsense, be sure to exclude these ports from your scan so they don't taint your results. You may also want to try from a completely different network location. The closer you are to the target, the more accurate the results will be. In a perfect case, you would always scan the target from the same network segment it resides on.

When Nmap Guesses Wrong

Occasionally Nmap will report an OS guess which you know is wrong. The errors are usually minor (such as reporting a machine running Linux 2.4.16 as "Linux kernel 2.4.8 - 2.4.15"), but

there have been reports of Nmap being completely off (such as reporting your web server as an AppleWriter printer). When you encounter such problems (minor or major), please report them so everyone can benefit. The only reason the Nmap DB is so comprehensive is that thousands of users have spent a few minutes each to submit new information. Please follow these instructions:

Have a recent version of Nmap

Run `nmap -V` to determine which version of Nmap you have. You don't need to be running the absolute latest version of Nmap (though that would be ideal), but make sure your version is 4.20 or higher because we only need second generation OS fingerprints, not the old style produced by previous versions. You can determine the latest available version of Nmap by visiting https://nmap.org/download.html. If you upgrade, you might find that the mis-identification has already been fixed.

Be absolutely certain you know what is running

Invalid "corrections" can corrupt the OS DB. If you aren't certain exactly what is running on the remote machine, please find out before submitting.

Generate a fingerprint

Run the command `nmap -O -sV -T4 -d <target>`, where *<target>* is the misidentified system in question. Look at the OS detection results to ensure that the misidentification is still present. If you are scanning the target system over IPv6, add the `-6` option as well.

If the Nmap output for the host OS results says `(JUST GUESSING)`, it is expected that results may be a little off. Don't submit a correction in this case.

Otherwise, the map command should have produced results including the line `OS fingerprint:`. Below that is the fingerprint (a series of lines which each start with `OS:`).

Check that OS detection works against other hosts

Try scanning a couple other hosts on the target network which you know have a different OS. If they aren't detected properly, maybe there is some network obstruction between the systems which is corrupting the packets.

If you have gotten this far and are still able to submit, good for you! Please submit the information at https://insecure.org/cgi-bin/submit.cgi?corr-os

When Nmap Fails to Find a Match and Prints a Fingerprint

When Nmap detects that OS detection conditions seem ideal and yet it finds no exact matches, it will print out a message like this:

```
No OS matches for host (If you know what OS is running on it, see
https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=5.05BETA1%D=8/23%OT=22%CT=1%CU=42341%PV=N%DS=0%DC=L%G=Y%TM=4A91CB
OS:90%P=i686-pc-linux-gnu)SEQ(SP=C9%GCD=1%ISR=CF%TI=Z%CI=Z%II=I%TS=A)OPS(O1
OS:=M400CST11NW5%O2=M400CST11NW5%O3=M400CNNT11NW5%O4=M400CST11NW5%O5=M400CS
OS:T11NW5%O6=M400CST11)WIN(W1=8000%W2=8000%W3=8000%W4=8000%W5=8000%W6=8000)
OS:ECN(R=Y%DF=Y%T=40%W=8018%O=M400CNNSNW5%CC=N%Q=)T1(R=Y%DF=Y%T=40%S=O%A=S+
OS:%F=AS%RD=0%Q=)T2(R=N)T3(R=Y%DF=Y%T=40%W=8000%S=O%A=S+%F=AS%O=M400CST11NW
OS:5%RD=0%Q=)T4(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T5(R=Y%DF=Y%T=40%W
OS:=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)
OS:T7(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%U
OS:N=0%RIPL=G%RID=G%RIPCK=G%RUCK=G%RUD=G)IE(R=Y%DFI=N%T=40%CD=S)
```

Please consider submitting the fingerprint so that all Nmap users can benefit. It only takes a minute or two and it may mean you don't need to see that ugly message again when you scan the host with the next Nmap version! Simply visit the URL Nmap provides for instructions.

If Nmap finds no matches and yet prints no fingerprint, conditions were not ideal. Even if you obtain the fingerprint through debug mode or XML output, please don't submit it unless Nmap asks you to (as in the previous example).

Modifying the `nmap-os-db` Database Yourself

People often ask about integrating a fingerprint themselves rather than (or in addition to) submitting it to Nmap.Org. While we don't offer detailed instructions or scripts for this, it is certainly possible after you become intimately familiar with the section called "Understanding an Nmap Fingerprint". I hope this is useful for your purposes, but there is no need to send your own reference fingerprint creations to us. We can only integrate raw subject fingerprint submissions from the web form.

SOLUTION: Detect Rogue Wireless Access Points on an Enterprise Network

Fuente original: https://nmap.org/book/osdetect-find-rogue-ap.html

Problem

With the ubiquity of mobile devices and cheap commodity networking equipment, companies are increasingly finding that employees are extending their networks in undesirable ways. Among the most dangerous devices are 802.11 wireless access points (WAPs). Users may install a $20 WAP in their cubicle so they can work from the break room, without realizing (or caring) that they just opened the protected corporate network to potential attackers in the parking lot or nearby buildings.

Some WAP installations are even worse than those installed by naive users. Breaching a building's security is much riskier for an attacker than accessing corporate data from far away through a network. It carries the risk of being arrested on the spot. So attackers have been known to install compact WAPs so they can then intrude on the network at will from the relative safety of a car down the street. A WAP taped under a desk or otherwise hidden is unlikely to be noticed for a while.

While the focus of this solution is finding WAPs, the same strategy can be used to find just about anything. You might need to locate all Cisco routers to apply a new patch or Solaris boxes to determine whether you have enough systems to warrant paying for support.

One way to find unauthorized wireless devices is to sweep the area with a wireless sniffer such as Kismet or NetStumbler. Another approach is to scan the wired side with Nmap. Not surprisingly, this solution focuses exclusively on the latter approach. Each technique can miss certain WAPs, so the best approach is to do both and merge the results.

Solution

Scan your whole address space using the `-A` option. You can speed it up by limiting scanned ports to 1–85, 113, 443, and 8080–8100. Those should find both an open and closed port on most WAPs, which improves OS detection accuracy. If your network spans multiple ethernet segments, scan each segment from a designated machine on the same segment. This speeds up the scan (especially since you can do them in parallel), and also gives you the MAC address of each device. Scanning from the same segment also allows you to spot stealth devices. Even a WAP with all ports filtered will generally respond to an ARP request. Results should be saved in at least normal and XML formats, so you might as well use `-oA`. Consider all of the performance-enhancing options described in Chapter 6, *Optimizing Nmap Performance*. A good and relatively safe start for performance options is `-T4 --min-hostgroup 50 --max-rtt-timeout 1000ms --initial-rtt-timeout 300ms --max-retries 3 --host-timeout 20m --max-scan-delay 1000ms`. Put this all together for a command like:

```
nmap -A -oA ~/nmap-logs/wapscan -p 1-85,113,443,8080-8100 -T4 --min-hostgroup
50 --max-rtt-timeout 1000ms --initial-rtt-timeout 300ms --max-retries 3
--host-timeout 20m --max-scan-delay 1000ms <target_network>
```

When the scan completes, search for WAP characteristics. On a network of fewer than a couple hundred live hosts, your best bet is to look at each one individually. For larger networks, you will likely need to automate the task. Searching for individual characteristics can be done with grep, though a Perl script which analyzes the XML output is preferable. This is pretty easy thanks to existing modules, such as Nmap::Scanner and Nmap::Parser, for parsing Nmap XML output. See the section called "Manipulating XML Output with Perl" for examples.

Once you determine a list of candidates, it is probably best to open the normal Nmap output file and examine each one to eliminate false positives. For example, a Linksys device may be flagged as a possible WAP even though it could be one of their plain switches without any wireless functionality.

Once you find the WAPs, it is time to track them down. This can usually be done by querying the switch they connect to for their physical ethernet port number.

WAP Characteristics

Now it is time to discuss the WAP characteristics to look for. Understanding these is useful for manual inspections or for modifying the WAP finder script to search for something else. You will probably see many of them immediately by looking at the scan of a typical WAP in Example 8.11.

Example 8.11. Scan results against a consumer WAP

```
# nmap -A -v wap.nmap.org

Starting Nmap ( https://nmap.org )
Nmap scan report for wap.nmap.org (192.168.0.6)
Not shown: 999 closed ports
PORT   STATE SERVICE VERSION
80/tcp open  http    Netgear MR-series WAP (MR814; Embedded HTTPD 1.00)
MAC Address: 00:09:5B:3F:7D:5E (Netgear)
Device type: WAP
Running: Compaq embedded, Netgear embedded
OS details: WAP: Compaq iPAQ Connection Point or Netgear MR814
Service Info: Device: WAP

Nmap done: 1 IP address (1 host up) scanned in 10.90 seconds
```

```
          Raw packets sent: 1703 (75.706KB) | Rcvd: 1686 (77.552KB)
```

This device shows many obvious clues to being a WAP (`Device type: WAP` is pretty blatant) and some more subtle ones. But WAPs aren't always so easy to discover. This section provides a list of WAP characteristics, starting with the most powerful and ending with heuristics that are long shots or more likely to produce false positives. Each characteristic listed is accompanied by an XPath expression that shows where to find it in Nmap XML output. Since this is security related, I suggest trying all of them and removing false positives manually.

TCP/IP fingerprinting device type

As described in the section called "Device and OS classification (`Class` lines)", every reference fingerprint has at least one classification (which includes device type) associated with it. Because WAPs are so controversial, we try to use that (or give two classifications) when multiple types would fit. So devices like the D-Link DI-624 wireless broadband router is classified as `WAP` rather than `switch` or `router`. The device type can be found in XML output using the XPath expression `/nmaprun/host/os/osclass/@type`. (That is, the `type` attribute of the `osclass` element of the `os` element of any of the `host` elements inside the root `nmaprun` element).

TCP/IP fingerprinting details

While devices with Wireless capability should be classified as device type `WAP`, it is worth searching the detailed OS description for terms such as `wireless` or `wap` just to be sure. The description is in `/nmaprun/host/os/osmatch/@name` in XML output.

Version detection device type

Version detection also tries to determine device types, but by fingerprinting the target's running services rather than its IP stack. Check whether the XML `devicetype` attribute located at `/nmaprun/host/ports/port/service/@devicetype` is `WAP`. To be completely safe, checking the `/nmaprun/host/ports/port/service/@extrainfo` field for the substrings `wap` or `wireless` is worthwhile.

Vendor (from MAC address, TCP/IP fingerprinting, and version detection)

Certain vendors specialize in producing the low-cost consumer networking devices which are most likely to covertly find their way onto office networks. Examples are Linksys, Netgear, Belkin, SMC, D-Link, Motorola, Trendnet, Zyxel, and Gateway. You can check for these vendors based on the MAC address lookup (which is at `/nmaprun/host/address/@vendor` in XML output), OS detection (`/nmaprun/host/os/osclass/@vendor` in XML output), or version detection (`/nmaprun/host/ports/port/service/@product` in XML output) results. Be sure to search for the vendor as a substring of the fields, since the field may contain incorporation type (e.g. Inc.) or other information.

This test may lead to many false positives. If you use a vendor heavily for authorized devices, such as putting Netgear NICs in your desktop machines, you may have to remove that vendor and rerun the script.

Hostname

It doesn't hurt to check hostnames (reverse DNS resolution) for terms such as `wap`, `wireless`, or `airport`. These can be found at `/nmaprun/host/hostnames/hostname/@name` in XML output. Non-administrative employees rarely change DNS names, but this can be useful for pen-testers, new administrators, and others who may be scanning a new network looking for authorized access points.